



An Incomplete Introduction to Artificial Intelligence

Prof. Dr. Karl Stroetmann

October 5, 2026

These lecture notes, their $\text{\LaTeX}$ sources, and the programs discussed in these lecture notes are all available at

<https://github.com/karlstroetmann/Artificial-Intelligence>.

In particular, the lecture notes are found in the directory `Lecture-Notes` in the file `artificial-intelligence.pdf`. The lecture notes are subject to continuous change. Provided the program `git` is installed on your computer, the command

```
git clone https://github.com/karlstroetmann/Artificial-Intelligence.git
```

clones the repository containing the lecture notes and stores them on your local drive. Once you have cloned the repository, the command

```
git pull
```

can be used to load the current version of these lecture notes from `GitHub`. As artificial intelligence is a very active area of research, these lecture notes will always be incomplete and hence will change from time to time. If you find any typos, errors, or inconsistencies, please contact me via `Discord` or, if that is not possible, email me at:

karl.stroetmann@dhbw.de

You are also welcome to send a `pull request` on `GitHub`.

Contents

Contents	1
1 Introduction	4
1.1 Overview	4
1.2 Literature	5
2 Search	7
2.1 The Sliding Puzzle	11
2.2 Breadth First Search	15
2.2.1 A Queue Based Implementation of Breadth First Search	18
2.3 Depth First Search	19
2.3.1 A Recursive Implementation of Depth First Search	20
2.4 Iterative Deepening	22
2.5 Bidirectional Breadth First Search	24
2.6 Best First Search	27
2.7 A* Search	32
2.8 Bidirectional A* Search	34
2.9 Iterative Deepening A* Search	35
3 Solving Constraint Satisfaction Problems	42
3.1 Formal Definition of CSPs	43
3.1.1 Example: Map Colouring	43
3.1.2 Example: The Eight Queens Puzzle	45
3.1.3 Example: The Zebra Problem	48
3.1.4 Applications	51
3.2 Brute Force Search	51
3.3 Backtracking Search	53
3.4 Constraint Propagation	57
3.5 Consistency Checking*	62
3.6 Local Search*	69
3.7 Z3	73
3.7.1 A Simple Text Problem	73
3.7.2 The Knight's Tour	77
3.7.3 Literature	81
4 Playing Games	82
4.1 Basic Definitions	82
4.2 Tic-Tac-Toe	83
4.2.1 A Naive Implementation of Tic-Tac-Toe	84

4.2.2	A Bitboard-Based Implementation of Tic-Tac-Toe	86
4.3	The Minimax Algorithm	87
4.3.1	Memoization	89
4.4	Alpha-Beta Pruning	92
4.4.1	Alpha-Beta Pruning with Memoization	97
4.5	Progressive Deepening	100
5	Equational Theorem Proving	104
5.1	Equational Proofs	105
5.1.1	A Calculus for Equality	109
5.1.2	Term Rewriting	110
5.1.3	Proofs via Term Rewriting	111
5.2	Confluence	113
5.3	The Knuth-Bendix Order	117
5.4	Unification	120
5.5	The Knuth-Bendix Algorithm	124
5.6	Literature	129
6	Linear Regression	130
6.1	Simple Linear Regression	130
6.1.1	Assessing the Quality of Linear Regression	131
6.1.2	Putting the Theory to the Test	133
6.2	General Linear Regression	137
6.2.1	Some Useful Gradients	139
6.2.2	Deriving the Normal Equation	140
6.2.3	Implementation	141
6.3	Polynomial Regression	143
6.3.1	Case Study: German Civil Servant Salaries	143
6.4	Overfitting and Underfitting in Linear Regression	147
6.4.1	Data Preprocessing	148
6.4.2	Feature Selection by Importance	149
6.4.3	The Experiment	150
6.4.4	Results and Interpretation	150
7	Classification	153
7.1	Introduction	153
7.1.1	Notation	155
7.1.2	Applications of Classification	156
7.2	Digression: The Method of Gradient Ascent	157
7.3	Logistic Regression	160
7.3.1	The Sigmoid Function	161
7.3.2	The Model of Logistic Regression	163
7.3.3	Implementing Logistic Regression	165
7.3.4	Logistic Regression with SciKit-Learn	171
7.4	Polynomial Logistic Regression	176
7.5	Naive Bayes Classifiers	181
7.5.1	Example: Spam Detection	185
7.5.2	Naive Bayes Classifier with Numerical Features	189
7.6	Support Vector Machines	190

7.6.1	Non-Optimality of Logistic Regression	190
7.7	Digression: The Method of Lagrange Multipliers	193
7.7.1	Geometric Intuition	193
7.7.2	Solving the Example	194
7.7.3	The Mathematical Theory of Support Vector Machines	195
7.8	Support Vector Machines on Medical Data	199
7.8.1	Data Preprocessing: Standardization	199
7.8.2	From Hard Margin to Soft Margin	200
7.8.3	Implementation	202
8	Neural Networks	206
8.1	Feed Forward Neural Networks	207
8.2	Backpropagation	211
8.2.1	Definition of some Auxiliary Variables	212
8.2.2	The Hadamard Product	212
8.2.3	Backpropagation: The Equations	213
8.2.4	A Proof of the Backpropagation Equations	214
8.3	Stochastic Gradient Descent	217
8.4	Implementation	218
9	Automatic Differentiation	226
9.1	The Library <code>autograd</code>	233
	Bibliography	236
	List of Figures	238
	Index	242

Chapter 1

Introduction

Artificial Intelligence has evolved through two primary approaches. The first, known as [symbolic AI](#), focuses on [symbolic logic](#). This approach led to the creation of automatic theorem provers, [symbolic integration](#) systems, and chess-playing programs like [Deep Blue](#). Initially, symbolic AI was the predominant paradigm in the field.

The second approach, [machine learning](#), was defined by Arthur Samuel as “the field of study that enables computers to learn without explicit programming” [[Sam59](#)]. This approach has primarily fueled the recent hype in AI.

1.1 Overview

This lecture discusses only symbolic AI, as machine learning is part of the module [data mining](#). It emphasizes [declarative programming](#). The core principle of declarative programming involves starting with a [formal problem specification](#), a succinct description of the issue at hand. This specification is then processed by a [problem solver](#) to produce a solution. Originally, [declarative programming](#) adopted a broad approach to problem-solving, where problems were framed as logical formulas and tackled using [automated theorem provers](#). The programming language [Prolog](#) is based on this paradigm. However, this approach has proven to be less effective as a universal problem-solving framework for two reasons:

1. It is often challenging to fully articulate practical problems within a logical framework.
2. In cases where it is possible to completely define a problem using logical formulas, automatic theorem proving generally lacks the capability to autonomously find solutions.

Despite these limitations, declarative programming has proven valuable in several domains, which we will explore, demonstrating its application in solving various types of problems:

1. [Search problems](#), where the objective is to find a path within a graph. A classic instance is the [fifteen puzzle](#). We will examine several advanced algorithms designed to resolve such search problems.
2. [Constraint satisfaction problems](#) hold significant practical relevance. Currently, highly efficient constraint solvers exist, capable of addressing various practical constraint satisfaction problems. We will delve into different strategies for solving these problems and discuss [Z3](#), a leading-edge automatic theorem prover and constraint solver developed by [Microsoft](#).
3. [Games](#), such as [chess](#) or [checkers](#), can be defined using a declarative approach. We will cover several techniques enabling computers to devise optimal strategies for these adversarial games.

4. We discuss [automatic theorem proving](#). Having previously covered [resolution theorem proving](#) in our lecture on [logic](#), we will now turn our attention to [equational theorem proving](#) in the final chapter of this first part.
5. Next, we turn to the topic of [machine learning](#). We start by discussing [linear regression](#), since this is one of the most fundamental machine learning algorithms and is also the foundation for more advanced forms of machine learning like [logistic regression](#) and [neural networks](#).

The goal of linear regression is to predict the values of unknown variables from the values of known variables. For example, given the weight and the engine displacement of a car, we want to predict its fuel consumption.

6. The following chapter discusses [classification](#). A good example of classification is [spam detection](#). In particular, we will discuss [logistic regression](#), since this is needed later when discussing neural networks.
7. Then we discuss [artificial neural networks](#).
8. Finally, we discuss [automatic differentiation](#), which is a technique for computing the gradient of a function that does neither rely on numeric approximation nor does it force us to compute symbolic derivatives manually. This technique is the basis for all modern libraries for neural networks, i.e. [TensorFlow](#) and [PyTorch](#) rely heavily on automatic differentiation.

1.2 Literature

My main sources for these lecture notes were the following:

1. A specialized course on artificial intelligence available through the EDX platform. All relevant course materials can be accessed at <http://ai.berkeley.edu/home.html>.
2. The book titled *Artificial Intelligence: A Modern Approach*, authored by Stuart Russell and Peter Norvig [RN20].
3. The *Intro to Artificial Intelligence* course provided by the Udacity platform.
4. For the second part of the lecture I can recommend the book *Introduction to Machine Learning* by Ethem Alpaydın [Alp20].

For exam preparation, a thorough understanding of the material covered in these lecture notes should suffice. Therefore, purchasing additional books or enrolling in other courses is certainly not necessary.

A personal remark: Nowadays, rather than reading voluminous books, I usually consult [Gemini](#) regarding topics I wish to explore. Proficiency with modern LLMs, such as Gemini or [DeepSeek](#), is a distinguishing asset for those entering the workforce. This proficiency does not come overnight. To achieve it, you have to use these tools for an extended period in order to judge their limits and capabilities. Currently, Google provides a [special offer](#) for students, granting free access to Gemini for the first 12 months.

Remark: The programs presented in these lecture notes are expected to run with the *Python* version 3.13. Unfortunately, there is a bug in Python 3.14 that prevents us from using this most recent version of Python. I have created the Python environment that I am using for these lecture notes via the shell commands shown in Figure 1.1 on page 6.

```
conda create -n ai
conda activate ai
conda install -y -c conda-forge python=3.13 jupyter nbclassic
conda install -y -c conda-forge graphviz
conda install -y -c conda-forge python-graphviz
conda install -y -c conda-forge numpy matplotlib seaborn
conda install -y -c conda-forge scikit-learn
conda install -y -c conda-forge ipycanvas
pip install ply
pip install mypy
pip install nb-mypy
pip install z3-solver
pip install git+https://github.com/reclinarka/problem_visuals
pip install git+https://github.com/reclinarka/chess-problem-visuals
```

Figure 1.1: Bash commands to set up an Anaconda environment for Python.

When starting *Jupyter notebooks* you should take care to use the command

```
jupyter nbclassic
```

instead of the command `jupyter notebook`. The command `jupyter nbclassic` uses the classic version of *Jupyter notebooks*. There are lots of incompatibilities with the new *Jupyter notebooks* of version 7.x and I have found that they do not work for me.

Chapter 2

Search

This chapter is the first of three chapters where we will solve problems by making use of [declarative programming](#). The idea of declarative programming is that rather than developing a program to solve a specific problem, we implement an algorithm that can solve a whole class of problems. Then, in order to solve a problem that falls within this class, we just have to specify the problem, which is usually much easier than writing a program that solves the given problem. In this chapter, this idea is illustrated via [search problems](#). First, we define the notion of a [search problem](#) formally. This notion is then illustrated with two examples. We start with the [missionaries and cannibals puzzle](#). Next, we use the [sliding puzzle](#) as our running example. After that, we introduce various algorithms for solving search problems. In particular, we present

1. [breadth first search](#),
2. [depth first search](#),
3. [iterative deepening](#),
4. [bidirectional breadth first search](#).

The algorithms mentioned work on any search problem. If we have a [heuristic](#) that estimates how many steps it takes to solve the given search problem, then a solution can be found much faster. The following algorithms make use of a heuristic:

5. [A* search](#) and [bidirectional A* search](#),
6. [iterative deepening A* search](#), and
7. [A*-IDA* search](#).

We proceed to define the notion of a search problem.

Definition 1 (Search Problem) A [search problem](#) is a tuple of the form

$$\mathcal{P} = \langle Q, \text{next_states}, \text{start}, \text{goal} \rangle$$

where

1. Q is the set of [states](#), also known as the [state space](#).
2. `next_states` is a function taking a state as input and returning the set of those states that can be reached from the given state in one step, i.e. we have

$$\text{next_states} : Q \rightarrow 2^Q.$$

The function `next_states` gives rise to the **transition relation** R , which is a binary relation on Q , i.e. we have $R \subseteq Q \times Q$. This relation is defined as follows:

$$R := \{ \langle s_1, s_2 \rangle \in Q \times Q \mid s_2 \in \text{next_states}(s_1) \}.$$

If either $\langle s_1, s_2 \rangle \in R$ or $\langle s_2, s_1 \rangle \in R$, then s_1 and s_2 are called **neighboring states**.

3. `start` is the **start state**, hence `start` $\in Q$.
4. `goal` is the **goal state**, hence `goal` $\in Q$.

Sometimes, instead of a single state `goal` there is a set of states `Goals`.

A **path** is a list $[s_1, \dots, s_n]$ such that $s_{i+1} \in \text{next_states}(s_i)$ for all $i \in \{1, \dots, n-1\}$. The **length** of this path is defined as the length of this list minus 1, i.e. the path $[s_1, \dots, s_n]$ has length $n-1$. The reason for defining the length of this path as $n-1$ and not n is that the path consists of $n-1$ **edges** of the form $\langle s_i, s_{i+1} \rangle$ where $i \in \{1, \dots, n-1\}$. A path $[s_1, \dots, s_n]$ is a **solution** to the search problem $\mathcal{P}$ iff the following conditions are satisfied:

1. $s_1 = \text{start}$, i.e. the first element of the path is the start state.
2. $s_n = \text{goal}$, i.e. the last element of the path is the goal state.

If instead of a single goal we have a set of `Goals`, then the last condition is changed into

$$s_n \in \text{Goals}.$$

A path $p = [s_1, \dots, s_n]$ is a **minimal solution** to the search problem $\mathcal{P}$ iff it is a solution and, furthermore, the length of p is minimal among all other solutions. $\diamond$

Remark: In the literature, a **state** is often called a **node**. In these lecture notes, I will also sometimes refer to states as nodes. $\diamond$

Example: We illustrate the notion of a search problem with the following example, which is also known as the **missionaries and cannibals puzzle**: Three missionaries and three infidels have to cross a river that runs from the north to the south. Initially, both the missionaries and the infidels are on the western shore. There is just one small boat that can carry at most two passengers. Both the missionaries and the infidels can steer the boat. However, if at any time the missionaries are confronted with a majority of infidels on either shore of the river, then the missionaries have a problem. Figure 2.1 shows an artist's rendition¹ of the problem.

Figure 2.2 shows a formalization of the missionaries and cannibals puzzle as a search problem. We discuss this formalization line by line.

1. Line 1 defines the auxiliary function `no_problem`.

If m is the number of missionaries on the western shore and i is the number of infidels on that shore, then the expression `no_problem( $m, i$ )` is `True`, if there is no problem for the missionaries on either shore. There are three cases where there is no problem:

- (a) all missionaries are on the left shore, i.e. $m = 3$, or
- (b) all missionaries are on the right shore, i.e. $m = 0$, or
- (c) the number of missionaries is the same as the number of infidels, i.e. $m = i$.

¹Thanks to Marcel Vilas for providing this beautiful painting as well as an animation of this problem.

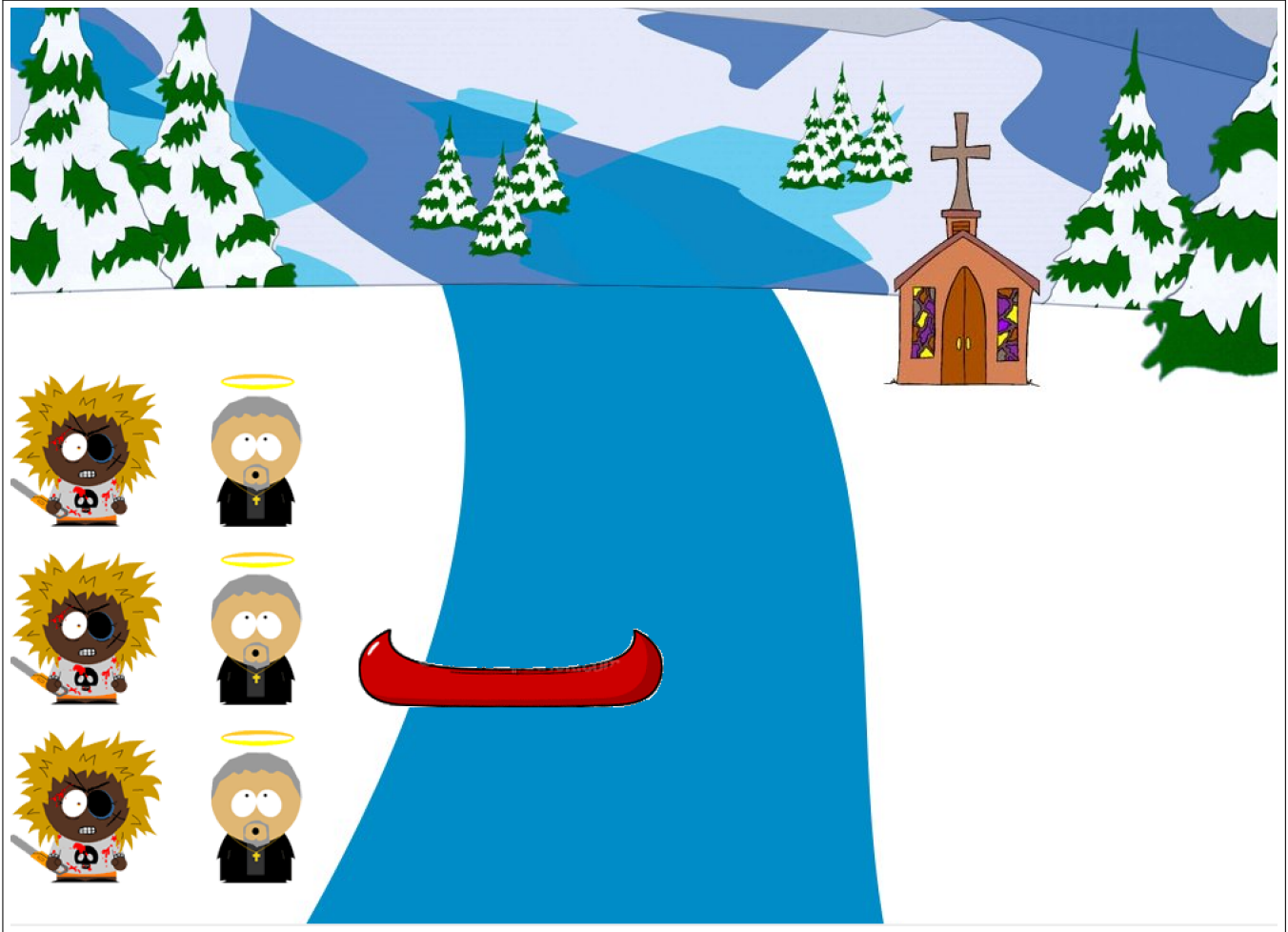


Figure 2.1: Start state of the [missionaries-and-infidels problem](#).

2. Lines 4 to 17 define the function `next_states`. A state s is represented as a triple of the form

$$s = (m, i, b) \quad \text{where } m \in \{0, 1, 2, 3\}, i \in \{0, 1, 2, 3\}, \text{ and } b \in \{0, 1\}.$$

Here m , i , and b are, respectively, the number of missionaries, the number of infidels, and the number of boats on the western shore.

- (a) Line 5 extracts the components m , i , and b from the state s .
- (b) Line 6 checks whether the boat is on the western shore.
- (c) If this is the case, then the states reachable from the given state s are those states where mb missionaries and ib infidels cross the river. After mb missionaries and ib infidels have crossed the river and reached the eastern shore, $m - mb$ missionaries and $i - ib$ infidels remain on the western shore. Of course, after the crossing, the boat is no longer on the western shore. Therefore, the new state has the form

$$(m - mb, i - ib, 0).$$

This explains line 10.

- (d) Since the number mb of missionaries leaving the western shore can not be greater than the number m of all missionaries on the western shore, we have the condition

```

1  def no_problem(m: int, i: int) -> bool:
2      return m == 0 or m == 3 or m == i
3
4  def next_states(state):
5      m, i, b = state
6      if b == 1:
7          return { (m - mb, i - ib, 0) for mb in range(m+1)
8                  for ib in range(i+1)
9                  if 1 <= mb + ib <= 2 and
10                     no_problem(m - mb, i - ib)
11                  }
12      else:
13          return { (m + mb, i + ib, 1) for mb in range(3-m+1)
14                  for ib in range(3-i+1)
15                  if 1 <= mb + ib <= 2 and
16                     no_problem(m + mb, i + ib)
17                  }
18
19  start = (3, 3, 1)
20  goal  = (0, 0, 0)

```

Figure 2.2: The missionary and cannibals problem coded as a search problem.

$$mb \in \{0, \dots, m\},$$

which is implemented by the line

```
for mb in range(m+1).
```

There is a similar condition for the number of infidels crossing:

$$ib \in \{0, \dots, i\}$$

which is implemented by

```
for ib in range(i+1).
```

- (e) Furthermore, we have to check that the number of persons crossing the river is at least 1 and at most 2. This explains the condition

$$1 \leq mb + ib \leq 2.$$

- (f) Finally, there should be no problem in the new state on either shore. This is checked using the expression

```
noProblem(m - mb, i - ib).
```

3. If the boat is on the eastern shore instead, then the missionaries and the infidels will be crossing the river from the eastern shore to the western shore. Therefore, the number of missionaries and infidels on the western shore is now increased. Hence, in this case the new state has the form

$$(m + mb, i + ib, 1).$$

Here, mb is the number of missionaries arriving on the western shore and ib is the number of arriving infidels. As the number of missionaries on the eastern shore is $3 - m$ and the number

of infidels on the eastern shore is $3 - i$, `mb` has to be a member of the set $\{0, \dots, 3 - m\}$, while `ib` has to be a member of the set $\{0, \dots, 3 - i\}$.

4. Finally, the start state and the goal state are defined in line 22 and line 23.

The code in Figure 2.2 does not define the set of states Q of the search problem. The reason is that, in order to solve the problem, we do not need to define this set. If we wanted to, we could define the set of states as follows:

```
States = { (m, i, b) for m in {0, 1, 2, 3}
           for i in {0, 1, 2, 3}
           for b in {0, 1}
           if no_problem(m, i)
         }
```

However, in general the set of states is not needed by the algorithms solving search problems and in many cases this set is so big that it would be impossible to store it. Hence, in practice the set of states is only an abstract notion that is needed in order to specify the function `next_states`, but it is not implemented.

Figure 2.3 shows a graphical representation of the transition relation of the missionaries and cannibals puzzle. In that figure, for every state, both the western and the eastern shore are shown. The start state is covered with a blue ellipse, while the goal state is covered with a green ellipse. The figure clearly shows that the problem is solvable and that there is a solution involving just 11 crossings of the river. $\diamond$

Exercise 1: The *Three Thieves Puzzle* is similar to the *Missionaries and Cannibals Puzzle*. Three greedy thieves have to cross a river. Each of the thieves has a bag of gold coins.

1. Ariel has 1,000 gold coins.
2. Benjamin has 700 gold coins.
3. Claude has 300 gold coins.

There is a boat available that can carry either two people or one person along with a bag of gold coins. The boat can transport two entities at a time, meaning either two thieves or one thief and a bag can cross together. A problem arises if a thief, or a pair of thieves, is left with a quantity of gold greater than their own, since then they will take the money and run away with it.

Write a program that formulates this puzzle as a search problem. You should do this by augmenting the notebook [Three-Thieves.ipynb](#) that can be found in the github repository

<http://github.com/karlstroetmann/Artificial-Intelligence>

in the directory [Python/Chapter-01/02-Three-Thieves.ipynb](#). $\diamond$

2.1 The Sliding Puzzle

The missionaries and cannibals puzzle is rather small and therefore it is not useful when we want to compare the efficiency of various algorithms for solving search problems. Therefore, we will now present a problem that has a greater complexity: The 3×3 sliding puzzle uses a square board, where each side has a length of 3. This board is subdivided into $3 \times 3 = 9$ squares of length 1. Of these 9 squares, 8 are occupied with square tiles that are numbered from 1 to 8. One square remains empty.

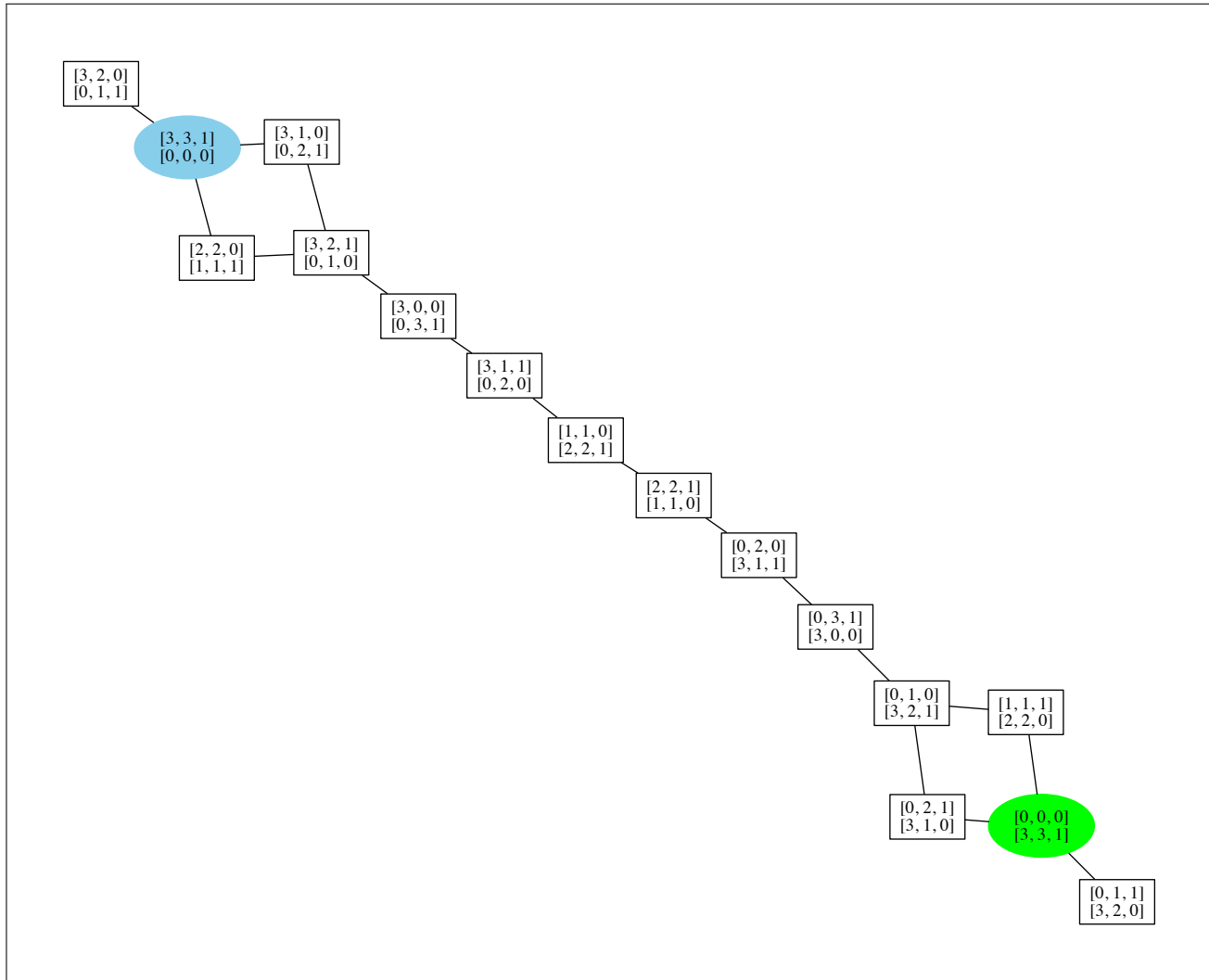
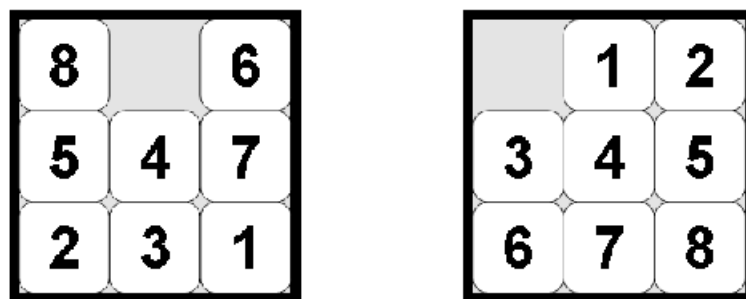


Figure 2.3: A graphical representation of the missionaries and cannibals puzzle.

Figure 2.4 on page 12 shows two possible states of this sliding puzzle. The 4×4 sliding puzzle is similar to the 3×3 sliding puzzle, but uses a square board of size 4 instead. The 4×4 sliding puzzle is also known as the 15 puzzle, while the 3×3 puzzle is called the 8 puzzle.

Figure 2.4: The 3×3 sliding puzzle.

In order to **solve** the 3×3 sliding puzzle shown in Figure 2.4 we have to transform the state shown on the left of Figure 2.4 into the state shown on the right of this figure. The following operations are permitted when transforming a state of the sliding puzzle:

1. If a tile is to the left of the free square, this tile can be moved to the right.
2. If a tile is to the right of the free square, this tile can be moved to the left.
3. If a tile is above the free square, this tile can be moved down.
4. If a tile is below the free square, this tile can be moved up.

In order to get a feeling for the complexity of the sliding puzzle, you can check the page

<https://www.helpfulgames.com/subjects/brain-training/sliding-puzzle.html>.

The sliding puzzle is much more complex than the missionaries and cannibals puzzle because the state space is much larger. For the case of the 3×3 sliding puzzle, there are 9 squares that can be positioned in $9!$ different ways. It turns out that only half of these positions are reachable from a given start state. Therefore, the effective number of states for the 3×3 sliding puzzle is

$$9!/2 = 181,440.$$

This is already a big number, but 181,440 states can easily be stored on a modern computer. However, the 4×4 sliding puzzle has

$$16!/2 = 10,461,394,944,000$$

different states reachable from a given start state. If a state is represented as a matrix containing 16 numbers and we store every number using just 4 bits, we still need $16 \cdot 4 = 64$ bits or 8 bytes for every state. Hence, we would need a total of

$$(16!/2) \cdot 8 = 83,691,159,552,000$$

bytes to store every state. We would thus need about 84 terabytes to store the set of all states. As few computers are equipped with this kind of memory, it is obvious that we won't be able to store the entire state space in memory.

Figure 2.5 shows how the 3×3 sliding puzzle can be formulated as a search problem. In order to discuss the program, we first have to understand that states are represented as tuples of tuples. For example, the state shown above on the left side in Figure 2.4 is represented as the tuple:

```
( (8, 0, 6),
  (5, 4, 7),
  (2, 3, 1)
)
```

Here, we have represented the empty tile as 0. If states are represented as tuples of tuples, given a state s , the expression $s[r][c]$ returns the tile in the row r and column c , where the counting of rows and columns starts from 0. We have to represent states as tuples of tuples rather than lists of lists since tuples are immutable while lists are mutable and we need to store states in sets later. In *Python*, sets can only store immutable objects. However, we also have to manipulate the states. To this end, we have to first transform the states to lists of lists, which can be manipulated. After the manipulation, these lists of lists have to be transformed back to tuples of tuples. We proceed to discuss the program shown in Figure 2.5 line by line.

1. The function `to_list` transforms a tuple of tuples into a list of lists.

```

1  def to_list(State):
2      return [list(row) for row in State]
3  def to_tuple(State):
4      return tuple(tuple(row) for row in State)
5  def find_tile(tile, State):
6      n = len(State)
7      for row in range(n):
8          for col in range(n):
9              if State[row][col] == tile:
10                 return row, col
11
12  def move_dir(State, row, col, dx, dy):
13      State = to_list(State)
14      State[row][col] = State[row + dx][col + dy]
15      State[row + dx][col + dy] = 0
16      return to_tuple(State)
17
18  def next_states(State):
19      n = len(State)
20      row, col = find_tile(0, State)
21      New_States = set()
22      Directions = [ (1, 0), (-1, 0), (0, 1), (0, -1) ]
23      for dx, dy in Directions:
24          if row + dx in range(n) and col + dy in range(n):
25              New_States.add(move_dir(State, row, col, dx, dy))
26      return New_States
27
28  start = ( (8, 0, 6),
29            (5, 4, 7),
30            (2, 3, 1)
31            )
32
33  goal = ( (0, 1, 2),
34           (3, 4, 5),
35           (6, 7, 8)
36           )

```

Figure 2.5: The 3×3 sliding puzzle.

2. The function `to_tuple` transforms a list of lists into a tuple of tuples.
3. `find_tile` is an auxiliary function that is needed to implement the function `next_states`. It is called with a `number` and a `State` and returns the row and column where the tile labelled with `number` can be found.
4. `move_dir` takes a `State`, the row and the column where to find the empty square and a direction in which the empty square should be moved. This direction is specified via the two variables `dx` and `dy`. The tile at the position $\langle \text{row} + \text{dx}, \text{col} + \text{dy} \rangle$ is moved into the position $\langle \text{row}, \text{col} \rangle$, while

the tile at position $\langle \text{row} + \text{dx}, \text{col} + \text{dy} \rangle$ becomes empty.

5. Given a **State**, the function **next_states** computes the set of all states that can be reached in one step from **State**. The basic idea is to find the position of the empty tile and then try to move the empty tile in all possible directions. If the empty tile is found at position $\langle \text{row}, \text{col} \rangle$ and the direction of the movement is given as $\langle \text{dx}, \text{dy} \rangle$, then in order to ensure that the empty tile can be moved to the position $\langle \text{row} + \text{dx}, \text{col} + \text{dy} \rangle$, we have to ensure that both

$$\text{row} + \text{dx} \in \{0, \dots, n-1\} \quad \text{and} \quad \text{col} + \text{dy} \in \{0, \dots, n-1\}$$

hold, where n is the size of the board. ◇

Next, we want to develop an algorithm that can solve puzzles of the kind described so far. The most basic algorithm to solve search problems is **breadth first search**. We discuss this algorithm next.

2.2 Breadth First Search

Informally, breadth first search, abbreviated as BFS, works as follows:

1. Given a search problem $\langle Q, \text{next_states}, \text{start}, \text{goal} \rangle$, we initialize a set **Frontier** to contain the state **start**.

In general, **Frontier** contains those states that have just been discovered and whose successors have not yet been seen.

2. As long as the set **Frontier** does not contain the state **goal**, we recompute this set by adding all states to it that can be reached in one step from a state in **Frontier**. Then, the states that had been previously present in **Frontier** are removed. These old states are then added to the set **Visited**.

In order to avoid going around in circles, an implementation of breadth first search keeps track of those states that have been visited in the set **Visited**. Once a state has been added to the set **Visited**, it will never be revisited again. Furthermore, in order to keep track of the path leading to the goal, we utilize a dictionary called **Parent**. For every state s that is in **Frontier**, **Parent**[s] is the state that has caused s to be added to the set **Frontier**, i.e. for all states $s \in \text{Frontier}$ we have

$$s \in \text{next_states}(\text{Parent}[s]).$$

Figure 2.6 on page 16 shows an implementation of breadth first search in *Python*. The function **search** takes three arguments to solve a **search problem**:

- (a) **start** is the **start state** of the search problem,
- (b) **goal** is the **goal state** of the search problem, and
- (c) **next_states** is a function with signature

$$\text{next_states} : Q \rightarrow 2^Q,$$

where Q is the set of states. For every state $s \in Q$, **next_states**(s) is the set of states that can be reached from s in one step.

If successful, **search** returns a path from **start** to **goal** that is a solution of the search problem

$$\langle Q, \text{next_states}, \text{start}, \text{goal} \rangle.$$

Next, we discuss the implementation of the function **search**:

```

1  def search(start, goal, next_states):
2      Frontier = { start }
3      Visited = set()
4      Parent = { start: start }
5      while Frontier:
6          NewFrontier = set()
7          for s in Frontier:
8              for ns in next_states(s):
9                  if ns not in Visited:
10                     NewFrontier.add(ns)
11                     Parent[ns] = s
12                     if ns == goal:
13                         return path_to(goal, Parent)
14             Visited |= Frontier
15             Frontier = NewFrontier

```

Figure 2.6: Breadth first search.

1. **Frontier** is the set of all those states that have been encountered but whose neighbours have not yet been explored. Initially, it contains the state **start**.
After the n^{th} iteration of the **while** loop, every state s in the set **Frontier** has a distance of n from the node **start**, i.e. there is a path of length n leading from **start** to s .
2. **Visited** is the set of all those states, all of whose neighbours have already been added to the set **Frontier** in the last iteration of the **while** loop. In order to avoid infinite loops, these states must not be visited again.
3. **Parent** is a dictionary keeping track of the predecessors of the state that have been reached. The only state with no real predecessor is the state **start**. By convention, **start** is its own predecessor.
4. As long as the set **Frontier** is not empty, we add all neighbours of states in **Frontier** that have not yet been visited to the set **NewFrontier**. When doing this, we keep track of the path leading to a new state **ns** by storing its parent in the dictionary **Parent**.
5. If the new state happens to be the state **goal**, we return a path leading from **start** to **goal** by calling the function **path_to**. This function is shown in Figure 2.7 on page 17.
6. After we have collected all successors of states in **Frontier**, the states in the set **Frontier** have been visited and are therefore added to the set **Visited**, while the set **Frontier** is updated to **NewFrontier**.

The function call **path_to(state, Parent)** constructs a path reaching from **start** to **state** in reverse by looking up the parent states. It uses the fact that only the start state is its own parent.

If we try breadth first search to solve the missionaries and cannibals puzzle, we obtain the solution shown in Figure 2.8. 15 nodes had to be expanded to find this solution. To keep this in perspective, we note that Figure 2.3 shows that the entire state space contains 16 states. Therefore, with the exception of one state, we have inspected all the states. If the search problem is difficult, then this is a typical behaviour of breadth first search.

```

1  def path_to(state, Parent):
2      p = Parent[state]
3      if p == state:
4          return [state]
5      return path_to(p, Parent) + [state]

```

Figure 2.7: The function `path_to`.

1	MMM	KKK	B	~~~~~			
2				> KK >			
3	MMM	K		~~~~~		KK	B
4				< K <			
5	MMM	KK	B	~~~~~		K	
6				> KK >			
7	MMM			~~~~~		KKK	B
8				< K <			
9	MMM	K	B	~~~~~		KK	
10				> MM >			
11	M	K		~~~~~	MM	KK	B
12				< M K <			
13	MM	KK	B	~~~~~	M	K	
14				> MM >			
15		KK		~~~~~	MMM	K	B
16				< K <			
17		KKK	B	~~~~~	MMM		
18				> KK >			
19		K		~~~~~	MMM	KK	B
20				< K <			
21		KK	B	~~~~~	MMM	K	
22				> KK >			
23				~~~~~	MMM	KKK	B

Figure 2.8: A solution of the missionaries and cannibals puzzle.

Next, let us try to solve the 3×3 sliding puzzle. It takes about 1.2 seconds to solve this problem on my computer², while 181,439 states are touched. Again, we see that breadth first search touches nearly all the states reachable from the start state. If we measure the memory consumption, we discover that the program uses about 90 megabytes of memory.

Breadth first search has two important properties:

- (a) Breadth first search is **complete**: If there is a solution to the given search problem, then breadth first search is going to find it.
- (b) The solution found by breadth first search is **optimal**, i.e. it is one of the shortest possible solutions.

Proof: Both of these claims can be shown simultaneously. Consider the implementation of breadth

²My computer is a Mac Studio from 2022. This machine is equipped with 64 Gigabytes of main memory and an Apple M1 Max processor.

first search shown in Figure 2.6 on page 16. We prove by induction on the number of iterations of the **while** loop that after n iterations of the **while** loop, the set **Frontier** contains exactly those states that have a distance of n to the state **start**.

Base Case: $n = 0$.

After 0 iterations of the **while** loop, i.e. before the first iteration of this loop, the set **Frontier** only contains the state **start**. As this is the only state that has a distance of 0 to the state **start**, the claim is true in this case.

Induction Step: $n \mapsto n + 1$.

In the induction step we assume the claim is true after n iterations. Then, in the next iteration all states that can be reached in one step from a state in **Frontier** are added to the new **Frontier**, provided there is no shorter path to them. By induction hypothesis, there is a shorter path to a state if this state is already a member of the set **Visited**. In this case, the state would not be added to **NewFrontier**. Otherwise, the shortest path to a state that is reached in iteration $n + 1$ has the length $n + 1$ and the state is added to **NewFrontier**. Hence, the claim is true after $n + 1$ iterations also.

Now, if there is a path from **start** to **goal**, there must also be a shortest path. Assume this path has a length of k . Then, **goal** is reached in the k^{th} iteration and the shortest path is returned. $\square$

The fact that breadth first search is both complete and the path returned is optimal is rather satisfying. However, breadth first search still has a big downside that makes it unusable for many problems: If the **goal** is far from the **start**, breadth first search will use a lot of memory because it will store a large part of the state space in the set **Visited**. In many cases, the state space is so big that this is impossible. For example, it is impossible to solve the more interesting cases of the 4×4 sliding puzzle using breadth first search.

2.2.1 A Queue Based Implementation of Breadth First Search

In the literature, for example in Figure 3.9 of Russell & Norvig [RN20], breadth first search is often implemented using a **queue** data structure.

```

1  from collections import deque
2
3  def search(start, goal, next_states):
4      Frontier = deque([start])
5      Parent   = { start: start }
6      while Frontier:
7          state = Frontier.popleft()
8          if state == goal:
9              return path_to(state, Parent)
10         for ns in next_states(state):
11             if ns not in Parent:
12                 Parent[ns] = state
13                 Frontier.append(ns)

```

Figure 2.9: A queue based implementation of breadth first search.

Figure 2.9 on page 18 shows an implementation of breadth first search that uses a queue to store the set **Frontier**. Here we use the module **deque** from the package **collections**. This module implements a **double-ended queue**, which is implemented as a **doubly linked list**. Besides the constructor, our implementation uses two methods from the class **deque**:

1. Line 4 initializes the **Frontier** as a double-ended queue that contains the state **start**.
2. In line 7 we remove the oldest element in the queue **Frontier**, which is supposed to be at the left end of the queue. This is achieved via the method **popleft**.
3. In line 14 we add the states that have not been encountered previously at the right end of the queue **Frontier** using the method **append**.

Additionally, we have used the fact that the information contained in the set **Visited** is already available in the dictionary **Parent**, because when we visit a state s , we add an entry for **Parent**[s]. As a result, this implementation of breadth first search is slightly faster than our previous implementation. Furthermore, only 76 megabytes of memory are used for the computation.

2.3 Depth First Search

To overcome the memory limitations of breadth first search, the **depth first search** algorithm has been developed. Depth first search is abbreviated as DFS. While BFS ensures that every state is visited by implementing the **Frontier** as a queue, DFS replaces this queue by a **stack**. This way, DFS tries to get as far away from the state **start** as early as possible. In order to prevent the search from looping, we still have the parent dictionary.

```

1  def search(start, goal, next_states):
2      Stack = [start]
3      Parent = { start: start }
4      while Stack:
5          state = Stack.pop()
6          for ns in next_states(state):
7              if ns not in Parent:
8                  Parent[ns] = state
9                  if ns == goal:
10                     return path_to(goal, Parent)
11             Stack.append(ns)
12
13  def path_to(state, Parent):
14      Path = [state]
15      while state != Parent[state]:
16          state = Parent[state]
17          Path = [ state ] + Path
18  return Path

```

Figure 2.10: The depth first search algorithm.

Since a stack can be implemented as an ordinary *Python* list, we don't need the module **deque** anymore. The idea is that the top of the stack is at the end of this list. Therefore, when we **pop** an

element from the stack, it is removed from the end of the list, while we can push an element onto the stack by using the method `append`. The resulting algorithm is shown in Figure 2.10 on page 19. Basically, in this implementation, a path is searched to its end before trying an alternative. This way, we might be able to find a `goal` that is far away from `start` without exploring the whole state space.

The implementation of `search` works as follows:

1. Any states that are encountered during the search are placed on top of the stack `Stack`.
2. In order to record the information how a state has been added to the `Stack`, we have the dictionary `Parent`. For every state `s` that is on `Stack`, `Parent[s]` returns a state `p` such that $s \in \text{next_states}(p)$, i.e. `p` is the state that immediately precedes `s` on the path that leads from `start` to `s`.
3. Initially, `Stack` only contains the state `start`.
4. As long as `Stack` is not empty, the `state` on top of `Stack` is replaced by all states that can be reached in one step from `state`. However, in order to prevent depth first search from running in circles, only those states `ns` from the set `next_states(state)` are appended to `Stack` that have not been encountered previously. This is checked by testing whether `ns` is in the domain of `Parent`.
5. When the `goal` is reached, a path leading from `start` to `goal` is returned.
6. We have reimplemented the function `path_to` using a `while` loop. The reason is that the recursive implementation that we had used before is not viable when the path gets too long because the recursion limit in *Python* is set to 3000 and hence the previous implementation of `path_to` does not work if the path exceeds a length of 3000.

When we test the implementation shown above with the 3×3 sliding puzzle, it takes 264 milliseconds on my computer to find a solution. This is an improvement compared to breadth first search. The memory consumption is reduced to 3 megabytes. This is still a lot and is due to the fact that we still have to maintain the dictionary `Parent`. Fortunately, we will be able to get rid of the dictionary `Parent` when we develop a recursive implementation of depth first search in the following subsection.

However, there is also bad news: the solution that is found has a length of 17,510 steps. As the shortest path from `start` to `goal` has only 31 steps, the solution found by depth first search is very far from being optimal.

2.3.1 A Recursive Implementation of Depth First Search

Sometimes, the depth first search algorithm is presented as a recursive algorithm, since this leads to an implementation that is slightly shorter and also easier to understand. What is more, we no longer need the dictionary `Parent` to record the parent of each node. The resulting implementation is shown in Figure 2.11 on page 21.

The only purpose of the function `search` is to call the function `dfs`, which needs two additional arguments. These arguments are called `Path` and `PathSet`. The idea is that `Path` is a path leading from the state `start` to the current `state` that is the first argument of the function `dfs`, while `PathSet` is a set containing all the elements of the path `Path`. The argument `PathSet` is only used for efficiency reasons: In order to avoid infinite loops, when we discover a node we have to check that this node does not occur already in `Path`. However, checking whether an element occurs in the list `Path` is much slower than checking whether the element occurs in the corresponding set `PathSet`. On the first invocation of `dfs`, the parameter `state` is equal to `start` and therefore `Path` is initialized as the list containing only `start`.

The implementation of `dfs` works as follows:

```

1 def search(start, goal, next_states):
2     return dfs(start, goal, next_states, [start], { start })
3
4 def dfs(state, goal, next_states, Path, PathSet):
5     if state == goal:
6         return Path
7     for ns in next_states(state):
8         if ns not in PathSet:
9             Result = dfs(ns, goal, next_states, Path + [ns], PathSet | {ns})
10            if Result:
11                return Result

```

Figure 2.11: A recursive implementation of depth first search.

1. If `state` is equal to `goal`, our search is successful. Since by assumption the list `Path` is a path connecting `start` and `state` and we have checked that `state` is equal to `goal`, we can return `Path` as our solution.
2. Otherwise, `next_states(state)` is the set of states that are reachable from `state` in one step. Any of the states `ns` in this set could be the next state on a path that leads to `goal`. Therefore, we try recursively to reach `goal` from every state `ns`. Note that we have to change `Path` to the list

`Path + [ ns ]`

when we call the function `dfs` recursively. This way, we retain the invariant of `dfs` that the list `Path` is a path connecting `start` with `state`.

3. In the same spirit we have to change `PathSet` to the set

`PathSet | { ns }`

since we have to maintain the invariant that `PathSet` is the set of all nodes in `Path`.

4. We still have to avoid running in circles. In the recursive version of depth first search, this is achieved by checking that the state `ns` is not already a member of the set `PathSet`. In the non-recursive version of depth first search, we had used the dictionary `Parent` instead. The current implementation no longer has a need for the dictionary `Parent`. This is very fortunate since it reduces the memory requirements of depth first search considerably.
5. If one of the recursive calls of `dfs` returns a list, this list is a solution to our search problem and hence it is returned. However, if instead `None` is returned, the `for` loop needs to carry on and test the other successors of `state`.
6. Note that the recursive invocation of `dfs` returns `None` if the end of the `for` loop is reached and no solution has been returned so far.

Unfortunately, due to a bug in *Python* 3.12, the *Python* kernel just dies when trying to solve the 3×3 sliding puzzle. This is due to the fact that the path gets very long and the garbage collector is not reclaiming the memory.

2.4 Iterative Deepening

The fact that the stack-based version of depth first search took less than one second to find a solution is very impressive, but the fact that this solution has a length of more than ten thousand steps is disappointing. The question is, whether it might be possible to force depth first search to find the shortest solution. The answer to this question leads to an algorithm that is known as **iterative deepening**. The main idea behind iterative deepening is to run depth first with a **depth limit** d . This limit enforces that a solution has at most a length of d . If no solution is found at a depth of d , the new depth $d + 1$ can be tried next and the process can be continued until a solution is found. The program shown in Figure 2.12 on page 22 implements this strategy. There is one simplification that we can apply: As the search will always find the shortest path, there is no need to keep the dictionary `PathSet` around. Instead of checking whether a node is a member of `PathSet`, we can just check whether it is a member of the list `Path`. This works, because searching in a small list does not take much more time than searching in a small set. We proceed to discuss the details of the implementation.

```

1 def search(start, goal, next_states):
2     limit = 1
3     while True:
4         Path = dls(start, goal, next_states, [start], limit)
5         if Path is not None:
6             return Path
7         limit += 1
8
9 def dls(state, goal, next_states, Path, limit):
10     if state == goal:
11         return Path
12     if len(Path) == limit:
13         return None
14     for ns in next_states(state):
15         if ns not in Path:
16             Result = dls(ns, goal, next_states, Path + [ns], limit)
17             if Result:
18                 return Result

```

Figure 2.12: Iterative deepening implemented in *Python*.

1. The function `search` initializes the variable `limit` to 1 and tries to find a solution to the search problem that has a length that is less than or equal to `limit`. If a solution is found, it is returned. Otherwise, the variable `limit` is incremented by one and a new instance of depth first search is started. This process continues until either a solution is found or the sun rises in the west.
2. The function `dls` implements a recursive version of depth first search but takes care to compute only those paths that have a length of at most `limit`. The name `dls` is short for **depth limited search**. If the `Path` has reached a length of `limit` but does not end in `goal`, the function returns `None` instead of trying to extend this `Path`. Otherwise, the implementation is the same as the recursive implementation of depth first search that was shown in Figure 2.11 on page 21 and that has been discussed in the previous section. The only difference is that we no longer need to use the set `PathSet`.

The nice thing about the program presented in this section is the fact that it uses only 136 kilobytes of memory. The reason is that the `Path` can never have a size that is longer than `limit`. However, when we run this program to solve the 3×3 sliding puzzle, the algorithm takes about 7 minutes. There are two reasons for the long computation time:

1. First, it is quite wasteful to run the search for a depth limit of 1, 2, 3, $\dots$ all the way up to 32. Essentially, all the computations done with a limit less than 32 are wasted. However, this process is not as wasteful as we might first expect. To see this, assume that the number of states reached is doubled³ after every iteration. Then the number of states to explore when searching with a depth limit of d is roughly 2^d . Hence, when we run depth limited search up to depth d , the number of states visited is

$$1 + 2^1 + 2^2 + \dots + 2^d = \sum_{i=0}^d 2^i = 2^{d+1} - 1.$$

Therefore, if the solution is found at a depth of $d + 1$, we will explore at most 2^{d+1} states to find the solution if we would do depth first search with a depth limit of $d + 1$. If, instead, we use iterative deepening, we have wastefully explored an additional number of $2^{d+1} - 1$ states. Hence, we will visit only about twice the number of states with iterative deepening than we would have visited with depth limited search with the correct depth limit.

2. Given a state s that is reachable from the `start`, there often are a huge number of different paths that lead from `start` to s . The version of iterative deepening presented in this section tries all of these paths and hence needs a large amount of time.

To check what is really going on, we can change the initial value of `limit` that is set to 1 in line 2 of Figure 2.12 on page 22. If we set this value to 31, which is one less than the value that is needed, the program needs about 5 minutes to compute the solution. However, if this value is set to 32, then the program is able to find the solution in less than two minutes. The reason is that in the case that `limit` has the value 31, the program has to check all possible lists `Path` that have a length of at most 31. Unfortunately, there is no such list, so all possible states that have a distance of at most 30 from `start` have to be explored. However, if `limit` has the value 32, it is sufficient to find any `Path` of length 32 that leads to the `goal` and if that `Path` has been found, the program can return immediately. The following exercise digs deeper into this observation.

Exercise 2: Assume the set of states Q is defined as

$$Q := \{\langle a, b \rangle \mid a \in \mathbb{N} \wedge b \in \mathbb{N}\}.$$

Furthermore, the states `start` and `goal` are defined as

$$\text{start} := \langle 0, 0 \rangle \quad \text{and} \quad \text{goal} := \langle n, n \rangle \text{ where } n \in \mathbb{N}.$$

Next, the function `next_states` is defined as

$$\text{next_states}(\langle a, b \rangle) := \{\langle a + 1, b \rangle, \langle a, b + 1 \rangle\}.$$

Finally, the search problem $\mathcal{P}$ is defined as

$$\mathcal{P} := \langle Q, \text{next_states}, \text{start}, \text{goal} \rangle.$$

Given a natural number n , compute the number of different solutions of this search problem and prove your claim. The Figure 2.13 on page 24 shows possible solutions in a graph.

³When we run breadth first search for the sliding puzzle, we can observe that at least at the beginning of the search, the number of states is roughly doubled after each step. This observation holds true for the first 16 steps.

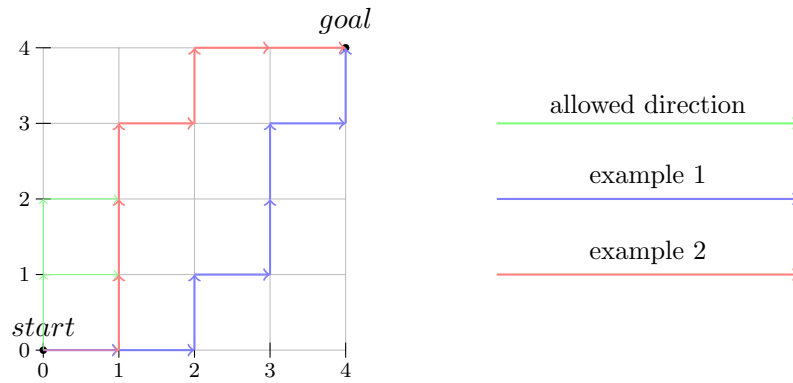


Figure 2.13: Example for possible paths in a graph

Hint: The expression giving the number of different solutions contains factorials. In order to get a better feeling for the asymptotic growth of this expression we can use [Stirling's approximation](#) of the factorial. Stirling's approximation of $n!$ is given as follows:

$$n! \sim \sqrt{2 \cdot \pi \cdot n} \cdot \left(\frac{n}{e}\right)^n. \quad \diamond$$

Exercise 3: If there is no solution, the implementation of iterative deepening that is shown in [Figure 2.12](#) does not terminate. The reason is that the function `dls` does not distinguish between the following two reasons for failure:

- (a) It can fail to find a solution because the depth limit is reached.
- (b) It can also fail because it has exhausted all possible paths without hitting the depth limit.

Improve the implementation of iterative deepening so that it will always terminate eventually, provided the state space is finite. $\diamond$

2.5 Bidirectional Breadth First Search

Breadth first search first visits all states that have a distance of 1 from start, then all states that have a distance of 2, then of 3 and so on until finally the goal is found. If the length of the shortest path from **start** to **goal** is d , then all states that have a distance of at most d will be visited. In many search problems, the number of states grows exponentially with the distance, i.e. there is a [branching factor](#) b such that the set of all states that have a distance of at most d from **start** is roughly

$$1 + b + b^2 + b^3 + \dots + b^d = \frac{b^{d+1} - 1}{b - 1} = \mathcal{O}(b^d).$$

At least this is true in the beginning of the search. As the size of the memory that is needed is the most constraining factor when searching, it is important to cut down this size. If the search problem is [symmetrical](#), i.e. if we have

$$x \in \text{next_states}(y) \Leftrightarrow y \in \text{next_states}(x),$$

then a simple idea is to start searching both from the node **start** and the node **goal** simultaneously. This approach is known as [bidirectional search](#). All of the search problems that we have encountered so far are symmetrical.

The justification for bidirectional search is that the path starting from **start** and the path starting from **goal** will meet in the middle and hence they will both have a size of approximately $d/2$. If this is the case, only

$$2 \cdot (1 + b + \dots + b^{\frac{d}{2}}) = 2 \cdot \frac{b^{\frac{d}{2}+1} - 1}{b - 1}$$

nodes need to be explored and even for modest values of b this number is much smaller than

$$\frac{b^{d+1} - 1}{b - 1}$$

which is the number of nodes expanded in breadth first search. For example, assume that the branching factor $b = 2$ and that the length of the shortest path leading from **start** to **goal** is $d = 40$. Then we need to explore

$$2^{41} - 1 = 2,199,023,255,551$$

states in breadth first search, while we only have to explore

$$2 \cdot (2^{\frac{40}{2}+1} - 1) = 4,194,302$$

states with bidirectional breadth first search. While it is certainly feasible to keep four million states in memory, keeping two trillion states in memory is impossible on most devices. The Figure 2.14 on page 25 demonstrates that the conventional search algorithm has to use a lot more space than the bidirectional approach.

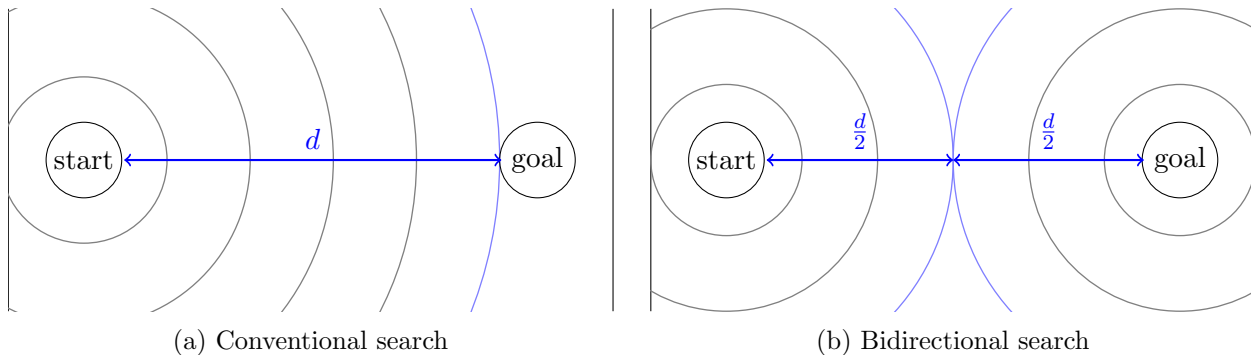


Figure 2.14: Example of space usage of conventional and bidirectional-BFS

```

1 def search(start, goal, next_states):
2     FrontierA = { start }
3     ParentA   = { start: start }
4     FrontierB = { goal }
5     ParentB   = { goal: goal }
6     while FrontierA and FrontierB:
7         if Path := bfs_one_step(FrontierA, ParentA, ParentB, next_states):
8             return Path
9         if Path := bfs_one_step(FrontierB, ParentB, ParentA, next_states):
10            return Path[::-1]

```

Figure 2.15: Bidirectional breadth first search.

Figure 2.15 on page 25 shows the implementation of bidirectional breadth first search. Essentially, we have two copies of the breadth first search program shown in Figure 2.6. However, since the information that was stored in the set `Visited` in the implementation of BFS shown in Figure 2.6 is also available in the dictionary `Parent`, we have removed the variable `Visited` in our implementation of bidirectional breadth first search.

Let us discuss the details of the implementation.

1. The variable `FrontierA` is the frontier that starts from the state `start`, while `FrontierB` is the frontier that starts from the state `goal`.
2. For every state s that is in `FrontierA`, `ParentA[s]` is the state that caused s to be added to the set `FrontierA`. Similarly, for every state s that is in `FrontierB`, `ParentB[s]` is the state that caused s to be added to the set `FrontierB`.
3. The bidirectional search keeps running for as long as both sets `FrontierA` and `FrontierB` are non-empty and a path has not yet been found.
4. In line 7, the function `bfs_one_step` tries to compute a path that connects `start` with `goal`. If such a path is found, this path which is then returned in line 8.

The details of the function `bfs_one_step` are discussed below.

5. Similarly, the function `bfs_one_step` in line 9 tries to find a path that connects `goal` with `start` by trying to expand states in `FrontierB`.

```

1 def bfs_one_step(Frontier, ParentA, ParentB, next_states):
2     NewFrontier = set()
3     for s in Frontier:
4         for ns in next_states(s):
5             if ns not in ParentA:
6                 NewFrontier |= { ns }
7                 ParentA[ns] = s
8                 if ns in ParentB:
9                     return combinePaths(ns, ParentA, ParentB)
10    Frontier.clear()
11    Frontier.update(NewFrontier)

```

Figure 2.16: The function `bfs_one_step`.

The function `bfs_one_step` is shown in Figure 2.16 on page 26.

1. The function `bfs_one_step` takes four arguments:
 - (a) `Frontier` is the frontier of states that result from a breadth first search that originates in a state p where p is either equal to `start` or to `goal`.
 - (b) `ParentA` is a dictionary. For every state q that is discovered in the breadth first search originating in p , `ParentA[q]` is a state that satisfies

$$q \in \text{next_states}(\text{ParentA}[q]),$$

i.e. `ParentA[q]` is the state that lead to the state q .

- (c) `ParentB` is a dictionary that is similar to `ParentA` but that instead contains as keys the states from the opposite search, i.e. if $p = \text{start}$, then `ParentB` contains the states as keys that have been found while searching from `goal` and if instead $p = \text{goal}$, then `ParentB` contains the states as keys that have been found while searching from `start`.
- (d) For every state s , we have that `next_states(s)` is the set of states that can be reached in one step from s .

The function `bfs_one_step` either returns a path or `None`. In the latter case, the function just updates the set `Frontier` to contain those states that can be reached in one step from the previous version of the set `Frontier`. Hence, the function `bfs_one_step` performs one iteration of breadth first search.

2. The set `NewFrontier` is initialized as the empty set in line 2.
3. Next, we iterate over all states s in the set `Frontier`.
4. For every state ns that is reachable from the state s in one step and that has not already been visited, we add ns to the set `NewFrontier` in line 6 and record its parent in line 7.
5. If the state ns has already been reached in the search starting from `goal` and hence has a parent node in `ParentB`, we have found a path from `start` to `goal`. Hence, we combine the path that leads from `start` to ns with the path leading from `goal` to ns in line 9.
6. It is important to note that the function `bfs_one_step` does not only return a path, it also has a side effect: If no path has been found, then the set `Frontier` is updated to contain those states that have been found in the current iteration.

Finally, Figure 2.17 on page 27 show the function `combinePaths` that takes a `state` that is reachable from both `start` and `goal`. It computes the path from `start` to the node `state` in line 2, the path from `goal` to the node `state` in line 3 and then combines these paths by first reversing the second path and appending it to the first path. When combining the paths, we have to take care to remove the last state from the first path `Path1`, since otherwise the node `state` would occur twice in the resulting path.

```

1  def combinePaths(state, ParentA, ParentB):
2      Path1 = path_to(state, ParentA)
3      Path2 = path_to(state, ParentB)
4      return Path1[:-1] + Path2[::-1] # Path2 is reversed

```

Figure 2.17: Combining two paths.

On my computer, bidirectional breadth first search solves the 3×3 sliding puzzle in 81 milliseconds and uses 4 megabytes. However, bidirectional breadth first search is still not able to solve the more interesting cases of the 4×4 sliding puzzle since the portion of the search space that needs to be computed is still too big to fit into memory.

2.6 Best First Search

Up to now, all the search algorithms we have discussed have been essentially blind. Given a state s and all of its neighbours, they had no idea which of the neighbours they should pick because they had

no conception which of these neighbours might be more promising than the other neighbours. Search algorithms that know nothing about the distance of a state to the goal are called **blind**. Russell and Norvig [RN20] use the name **uninformed search** instead of blind search.

If a human tries to solve a search problem, she will usually develop an intuition that certain states are more favourable than other states because they seem to be closer to the solution. In order to formalise this procedure, we next define the notion of a **search heuristic**.

Definition 2 (Search Heuristic) Given a search problem

$$\mathcal{P} = \langle Q, \text{next_states}, \text{start}, \text{goal} \rangle,$$

a **search heuristic** or simply **heuristic** is a function

$$h : Q \rightarrow \mathbb{R}$$

that computes an approximation of the distance of a given state s to the state **goal**. The heuristic is **admissible** if it never **overestimates** the true distance, i.e. if the function

$$d : Q \rightarrow \mathbb{N}$$

computes the **true distance** from a state s to the goal, then we must have

$$h(s) \leq d(s) \quad \text{for all } s \in Q.$$

Hence, the heuristic is admissible iff it is **optimistic**: Although it never overestimates the distance to the goal, it is free to underestimate this distance.

Finally, the heuristic h is called **consistent** iff we have

$$h(\text{goal}) = 0 \quad \text{and} \quad h(s_1) \leq 1 + h(s_2) \quad \text{for all } s_2 \in \text{next_states}(s_1). \quad \diamond$$

Let us explain the idea behind the notion of **consistency**. First, if we are already at the goal, the heuristic should notice this fact and therefore we need to have $h(\text{goal}) = 0$. Secondly, assume we are at the state s_1 and s_2 is a neighbour of s_1 , i.e. we have that

$$s_2 \in \text{next_states}(s_1).$$

Now if our heuristic h assumes that the distance of s_2 from the **goal** is $h(s_2)$, then the distance of s_1 from the **goal** can be at most $1 + h(s_2)$ because starting from s_1 we can first go to s_2 in one step and then from s_2 to **goal** in $h(s_2)$ steps for a total of $1 + h(s_2)$ steps. Of course, it is possible that there exists a shorter path from s_1 leading to the **goal** than the one that visits s_2 first. Hence, we have the inequality

$$h(s_1) \leq 1 + h(s_2).$$

The Figure 2.18 on page 28 demonstrates this inequality.

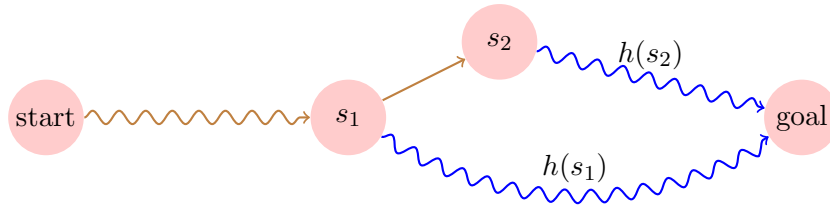


Figure 2.18: Explanation of the inequality $h(s_1) \leq 1 + h(s_2)$.

Theorem 3 Every consistent heuristic is an admissible heuristic.

Proof: Assume that the heuristic h is consistent. Assume further that $s \in Q$ is some state such that there is a shortest path P from s to the **goal**. Assume this path has the form

$$P = [s_n, s_{n-1}, \dots, s_1, s_0], \quad \text{where } s_n = s \text{ and } s_0 = \text{goal}.$$

Then the length of the path p is n and we have to show that $h(s) \leq n$. In order to prove this claim, we show that we have

$$h(s_k) \leq k \quad \text{for all } k \in \{0, 1, \dots, n\}.$$

This claim is shown by induction on k .

B.C.: $k = 0$.

We have $h(s_0) = h(\text{goal}) = 0 \leq 0$, because the fact that h is consistent implies $h(\text{goal}) = 0$.

I.S.: $k \mapsto k + 1$.

We have to show that $h(s_{k+1}) \leq k + 1$ holds. This is shown as follows:

$$\begin{aligned} h(s_{k+1}) &\leq 1 + h(s_k) && \text{because } s_k \in \text{next_states}(s_{k+1}) \text{ and } h \text{ is consistent,} \\ &\leq 1 + k && \text{because } h(s_k) \leq k \text{ by induction hypotheses.} \end{aligned}$$

We have shown $h(s_k) \leq k$ and since this also holds for $k = n$ we know that $h(s) = h(s_n) \leq n$. Since P is a shortest path we know that the state s has the distance n from the state **goal**. Hence the heuristic h underestimates this distance and is therefore admissible. $\square$

It is natural to ask whether the last theorem can be reversed, i.e. whether every admissible heuristic is also consistent. The answer to this question is negative since there are some *contorted* heuristics that are admissible but that fail to be consistent. However, in practice it turns out that most admissible heuristics are also consistent. Therefore, when we construct consistent heuristics later, we will start with admissible heuristics, since these are easy to find. We will then have to check that these heuristics are also consistent.

Examples: In the following, we will discuss several heuristics for the sliding puzzle.

1. The simplest heuristic that is admissible is the function $h(s) := 0$. Since we have

$$0 \leq 1 + 0,$$

this heuristic is obviously consistent, but when we use this heuristic, we are back to blind search.

2. The next heuristic is the **number of misplaced tiles** heuristic. For a state s , this heuristic counts the number of tiles in s that are not in their final position, i.e. that are not in the same position as the corresponding tile in **goal**. For example, in Figure 2.4 on page 12 in the state depicted to the left, only the tile with the label 4 is in the same position as in the state depicted to the right. Hence, there are 7 misplaced tiles.

As every misplaced tile must be moved at least once and every step in the sliding puzzle moves at most one tile, it is obvious that this heuristic is admissible. It is also consistent. First, the **goal** has no misplaced tiles, hence its heuristic is 0. Second, in every step of the sliding puzzle only one tile is moved. Therefore the number of misplaced tiles in two neighbouring states can differ by at most one and hence the inequality

$$h(s_1) \leq 1 + h(s_2)$$

is satisfied for any neighbouring states s_1 and s_2 . Unfortunately, the number of misplaced tiles heuristic is very crude and therefore not particularly useful.

3. The **Manhattan heuristic** improves on the previous heuristic. For two points $\langle x_1, y_1 \rangle, \langle x_2, y_2 \rangle \in \mathbb{R}^2$ the **Manhattan distance** of these points is defined as

$$d_1(\langle x_1, y_1 \rangle, \langle x_2, y_2 \rangle) := |x_2 - x_1| + |y_2 - y_1|.$$

The Manhattan distance is also called the **L_1 norm** of the difference vector $\langle x_2 - x_1, y_2 - y_1 \rangle$. If we associate **Cartesian coordinates** with the tiles of the sliding puzzle such that the tile in the upper left corner has coordinates $\langle 1, 1 \rangle$ and the coordinates of the tile in the lower right corner are $\langle 3, 3 \rangle$, then the Manhattan distance of two positions measures how many steps it takes to move a tile from the first position to the second position if we are allowed to move the tile horizontally or vertically regardless of the fact that the intermediate positions might be blocked by other tiles. To compute the Manhattan heuristic for a state s with respect to the **goal**, we first define the function $\text{pos}(t, s)$ for all tiles $t \in \{1, \dots, 8\}$ in a given state s as follows:

$$\text{pos}(t, s) = \langle \text{row}, \text{col} \rangle \stackrel{\text{def}}{\iff} s[\text{row}][\text{col}] = t,$$

i.e. given a state s , the expression $\text{pos}(t, s)$ computes the Cartesian coordinates of the tile t with respect to the state s . Then we can define the Manhattan heuristic h for the 3×3 puzzle as follows:

$$h(s) := \sum_{t=1}^8 d_1(\text{pos}(t, s), \text{pos}(t, \text{goal})).$$

The Manhattan heuristic measures the number of moves that would be needed if we wanted to put every tile of s into its final position and if we were allowed to slide tiles over each other. Figure 2.19 on page 30 shows how the Manhattan distance can be computed. The code given in that figure works for a general $n \times n$ sliding puzzle. It takes two states **stateA** and **stateB** and computes the Manhattan distance between these states.

```
def manhattan(stateA, stateB):
    n = len(stateA)
    result = 0
    for rowA in range(n):
        for colA in range(n):
            tile = stateA[rowA][colA]
            if tile != 0:
                rowB, colB = find_tile(tile, stateB)
                result += abs(rowA - rowB) + abs(colA - colB)
    return result
```

Figure 2.19: The Manhattan distance between two states.

- (a) First, the size **n** of the puzzle is computed by checking the number of rows of **stateA**.
- (b) Next, the **for** loops iterates over all rows and columns of **stateA** that do not contain a blank tile. Remember that the blank tile is coded using the number 0. The tile at position $\langle \text{rowA}, \text{colA} \rangle$ in **stateA** is computed using the expression **stateA**[**rowA**][**colA**] and the corresponding position $\langle \text{rowB}, \text{colB} \rangle$ of this tile in state **stateB** is computed using the function **find_tile**.
- (c) Finally, the Manhattan distance between the two positions $\langle \text{rowA}, \text{colA} \rangle$ and $\langle \text{rowB}, \text{colB} \rangle$ is added to the **result**.

The Manhattan heuristic is admissible. The reason is that if $s_2 \in \text{next_states}(s_1)$, then there can be only one tile t such that the position of t in s_1 is different from the position of t in s_2 . Furthermore, this position differs by either one row or one column. Therefore,

$$|h(s_1) - h(s_2)| = 1$$

and hence $h(s_1) \leq 1 + h(s_2)$. $\square$

Now we are ready to present **best first search**. This algorithm is derived from the stack based version of depth first search. However, instead of using a stack, the algorithm uses a **priority queue**. In this priority queue, the paths are ordered with respect to the estimated distance of the state at the end of the path from the **goal**. We always expand the path next that seems to be closest to the goal.

In *Python* the module **heapq** provides **priority queues** that are implemented as **heaps**. Technically, these heaps are just lists. In order to use them as priority queues, the entries of these lists will be pairs of the form (p, o) , where p is the priority of the object o . Usually, the priorities are numbers and, contra-intuitively, high priorities correspond to **small** numbers, that is (p_1, o_1) has a higher priority than (p_2, o_2) if and only if $p_1 < p_2$. We need only two functions from the module **heapq**:

1. Given a heap H , the function `heapq.heappop( $H$ )` returns and removes the pair from H that has the highest priority.
2. Given a heap H , the function `heapq.heappush( $H, (p, o)$ )` pushes the pair (p, o) onto the heap H . This method does not return a value. Instead, the heap H is changed in place.

```
def search(start, goal, next_states, heuristic):
    Visited = set()
    PrioQueue = [ (heuristic(start, goal), [start]) ]
    while PrioQueue:
        _, Path = heapq.heappop(PrioQueue)
        state = Path[-1]
        if state == goal:
            return Path
        if state in Visited:
            continue
        for ns in next_states(state):
            if ns not in Visited:
                prio = heuristic(ns, goal)
                heapq.heappush(PrioQueue, (prio, Path + [ns]))
        Visited.add(state)
```

Figure 2.20: The best first search algorithm.

The function `search` shown in Figure 2.20 on page 31 takes four parameters. The first three of these parameters are the same as in the previous search algorithms. The last parameter `heuristic` is a function that takes two states and then estimates the distance between these states. Later, when solving the sliding puzzle, we will use the Manhattan distance to serve as the parameter `heuristic`. The details of the implementation are as follows:

1. The Variable `Visited` collects all states that have been **visited**. A node n counts as **visited** when all of its neighbours have been inspected in line 11.

2. The variable `PrioQueue` serves as a priority queue. This priority queue is initialized as a list containing the pair $\langle d, [\text{start}] \rangle$, where d is the estimated distance of a path leading from `start` to `goal`. In `PrioQueue` we store the paths in pairs of the form

$\langle \text{estimate}, \text{Path} \rangle$,

where `Path` is a list of states starting from the node `start`. If the last node on this list is called `state`, then we have

`estimate = heuristic(state, goal)`,

i.e. `estimate` is the estimated distance between this `state` and `goal` and hence an estimate of the number of steps needed to complete `Path` into a solution. This ensures, that the path whose last state is nearest to the `goal` is at the beginning of the set `PrioQueue`.

3. As long as `PrioQueue` is not empty, we take the `Path` from the top of this priority queue and remove it from the queue. This is then the path with the highest priority. If the state at the end of `Path` is named `state`, then `state` is, according to our heuristic, the state nearest to the `goal` among all states in the priority queue.

The function `heappop(PrioQueue)` returns the smallest pair from `PrioQueue` and, furthermore, this pair is removed from `PrioQueue`.

4. If `state` is the `goal`, a solution has been found and is returned.
5. Next, we inspect all neighbouring states `ns` of `state` that have not already been visited. The paths leading to these nodes are pushed on the priority queue.
6. Finally, we mark `state` as `visited` by adding it to the set `Visited`.

Best first search solves the instance of the 3×3 puzzle shown in Figure 2.4 on page 12 in less than 3 millisecond and visits only 87 different states while solving the puzzle. However, the solution that is found takes 49 steps. While the length of this solution is not as ridiculous as the length of the solution found by depth first search, this length is far from being optimal. Best first search is able to solve the 4×4 puzzle shown in Figure 2.23 on page 36 in less than a second. It visits 8224 different states in order to find the solution. Unfortunately, the solution that is found has a length of 492 steps, while the optimal solution only needs 36 steps.

It should be noted that the fact that the Manhattan distance is a `consistent` heuristic is of no consequence for best first search. Only the A* algorithm, which is presented next, makes use of this fact.

2.7 A* Search

We have observed that best-first search can be remarkably efficient. However, most of the times the solution it provides is not optimal. This limitation arises because when best-first search considers a state s , it only takes into account the distance from s to the `goal` state. Crucially, it neglects the distance from the `start` state to s . In contrast, the `A* search algorithm` incorporates both the distance from `start` to s and the estimated distance from s to the `goal`. As a result, when the heuristic employed by the A* search algorithm is admissible, it is assured to find the shortest path.

Specifically, in the context of a given state s , the function $g(s)$ calculates the length of the path from `start` to s , and $h(s)$ provides a heuristic estimate of the distance from s to the `goal`. Consequently, the `priority` utilized by the A* search algorithm is expressed as

$$f(s) := g(s) + h(s).$$

The intricacies of the A* algorithm are detailed in Figure 2.21 on page 33. When we compare this implementation with the implementation of best first search shown in Figure 2.20 on page 31 we realize that these figure differ only in line 13 where the priority of a path is computed. With A* search, the priority of a path P ending in state s is the length of the path plus the estimated distance of the state s to the goal. The fact that we have to add 1 to `len(Path)` is due to the fact that `Path` only leads to `state` and we need the length of the path that leads to `ns`.

```
def search(start, goal, next_states, heuristic):
    Visited = set()
    PrioQueue = [ (heuristic(start, goal), [start]) ]
    while PrioQueue:
        _, Path = heapq.heappop(PrioQueue)
        state = Path[-1]
        if state in Visited:
            continue
        if state == goal:
            return Path
        for ns in next_states(state):
            if ns not in Visited:
                prio = len(Path) + heuristic(ns, goal)
                heapq.heappush(PrioQueue, (prio, Path + [ns]))
        Visited.add(state)
```

Figure 2.21: The A* search algorithm.

The A* search algorithm has been discovered by Hart, Nilsson, and Raphael and was first published in 1968 [HNR68]. However, there was a subtle bug in the first publication which was corrected in 1972 [HNR72].

When we run A* search on the 3×3 sliding puzzle, it takes about 0.1 seconds to solve the instance shown in Figure 2.4 on page 12 and visits 6614 states. Furthermore, the good news about A* search is that, provided the heuristic is admissible, the path which is found is optimal [HNR72].

Theorem 4 (Completeness and Optimality of A* Search)

If $\mathcal{P} = \langle Q, \text{next_states}, \text{start}, \text{goal} \rangle$ is a search problem and $h : Q \rightarrow \mathbb{R}$ is a admissible heuristic for $\mathcal{P}$, then A* search is both complete and optimal, i.e. if there is a path from `start` to `goal`, then the search is successful and, furthermore, the solution that is computed is a shortest path leading from `start` to `goal`.

Proof: To simplify the notation of the proof we agree to use the following notation. If P is a path that is an element of `PrioQueue` and s is the last state of the path P , i.e. $P[-1] = s$, then we say that $s \in \text{PrioQueue}$, although it really is the path P that is an element of `PrioQueue`. Furthermore, we denote the length of P with $g(s)$. Finally, we define

$$f(s) := g(s) + h(s) \quad \text{for all the states } s \in \text{PrioQueue}.$$

Therefore, $f(s)$ computes the priority of a state $s \in \text{PrioQueue}$.

Next, the proof of our claim proceeds indirect. We assume that the path $P_1 = [s_0, s_1, \dots, s_n]$ that is computed by A* search is not the shortest path. Then there is a path $P_2 = [t_0, t_1, \dots, t_m]$ leading from `start` to `goal` such that P_2 is shorter than P_1 , i.e. we must have $m < n$. We claim that

$$f(t_i) \leq m \quad \text{for all } i \in \{0, \dots, m\}.$$

The reasoning is as follows:

$$\begin{aligned}
 f(t_i) &= g(t_i) + h(t_i) \\
 &= i + h(t_i) && \text{since } g(t_i) = \text{len}([t_0, \dots, t_i]) = i \\
 &\leq i + (m - i) && \text{since } h(t_i) \leq \text{len}([t_i, \dots, t_m]) = m - i \\
 &= m
 \end{aligned}$$

However, for the path P_1 we know that

$$f(s_n) = g(s_n) + h(s_n) = n + 0 = n > m.$$

Since `PrioQueue` is a priority queue, we only remove the path P_1 from `PrioQueue` when all paths with a higher priority have already been removed and the corresponding end nodes have been expanded. But as $n > m$ this means that all paths

$$[t_0], [t_0, t_1], \dots, [t_0, \dots, t_m]$$

have already been removed from `PrioQueue` before P_1 is removed. But then A^* search would have already found the shortest path from `start` to `goal` and hence the path P_1 would never be removed from `PrioQueue`. This shows that A^* search can't return a path that is not a shortest path. $\square$

2.8 Bidirectional A* Search

When we refined breadth first search into bidirectional breadth first search we were able to increase the power of breadth first search. We can try to do something similar with the A^* algorithm and develop a bidirectional variant of this algorithm. Figure 2.22 on page 35 shows the resulting program. This program relates to the A^* algorithm shown in Figure 2.21 on page 33 as the algorithm for bidirectional search shown in Figure 2.15 on page 25 relates to breadth first search shown in Figure 2.6 on page 16. The only new idea is that we alternate between the A^* search starting from `start` and the A^* search starting from `goal` depending on the estimated total distance:

- (a) As long as the search starting from `start` is more promising, we remove states from `FrontierA`.
- (b) Once the total estimated distance of a path starting from `goal` is less than the best total estimated distance of a path starting from `start`, we switch and remove states from `FrontierB`.

There is one more twist, as the computation of the priority is a bit more involved. This is necessary to guarantee that the shortest path is computed.

When we run bidirectional A^* search for the 3×3 sliding puzzle shown in Figure 2.4 on page 12, the program takes 150 milliseconds and uses 10,554 states. I have also used bidirectional A^* search to solve the 4×4 sliding puzzle shown in Figure 2.23 on page 36. A solution of 36 steps was found in 2 seconds. A total 77,870 states had to be processed to compute this solution. This shows that, counter-intuitively, bidirectional A^* search uses more memory than unidirectional A^* search. Hence, in general, it not worth the trouble and we should stick with unidirectional A^* search.

```

def search(start, goal, next_states, heuristic):
    VisitedA = {}
    VisitedB = {}
    PrioQueueA = [ (heuristic(start, goal), [start]) ]
    PrioQueueB = [ (heuristic(goal, start), [goal]) ]
    while PrioQueueA and PrioQueueB:
        a, PathA = PrioQueueA[0]
        b, PathB = PrioQueueB[0]
        if a <= b:
            heapq.heappop(PrioQueueA)
            for Result in search_os(PrioQueueA, PathA, goal,
                                   VisitedA, VisitedB, next_states, heuristic):
                return Result
        else:
            heapq.heappop(PrioQueueB)
            for Result in search_os(PrioQueueB, PathB, start,
                                   VisitedB, VisitedA, next_states, heuristic):
                return Result[::-1]

def search_os(PQ, Path, goal, VisitedA, VisitedB, next_states, heuristic):
    state = Path[-1]
    if state in VisitedA:
        return None
    if state in VisitedB:
        return Path[:-1] + VisitedB[state][::-1]
    for ns in next_states(state):
        if ns not in VisitedA:
            prio1 = len(Path) + heuristic(ns, goal)
            prio2 = 2 * len(Path)
            prio = max(prio1, prio2)
            heapq.heappush(PQ, (prio, Path + [ns]))
    VisitedA[state] = Path

```

Figure 2.22: Bidirectional A* search.

2.9 Iterative Deepening A* Search

So far, we have combined A* search with bidirectional search and achieved good results. When memory space is too limited for bidirectional A* search to be possible, we can instead combine A* search with [iterative deepening](#). The resulting search technique is known as [iterative deepening A* search](#) and is commonly abbreviated as IDA search. It has been invented by Richard Korf [\[Kor85\]](#). [Figure 2.24](#) on [page 36](#) shows an implementation of IDA search. We proceed to discuss this program.

1. As in the A* search algorithm, the function `search` takes four parameters.
 - (a) `start` is a state. This state represents the start state of the search problem.
 - (b) `goal` is the goal state.

```

1  start = ( ( 0, 1, 2, 3 ),
2           ( 4, 5, 6, 8 ),
3           ( 14, 7, 11, 10 ),
4           ( 9, 15, 12, 13 )
5           )
6  goal  = ( ( 0, 1, 2, 3 ),
7           ( 4, 5, 6, 7 ),
8           ( 8, 9, 10, 11 ),
9           ( 12, 13, 14, 15 )
10          )

```

Figure 2.23: A start state and a goal state for the 4×4 sliding puzzle.

```

def search(start, goal, next_states, heuristic):
    limit = heuristic(start, goal)
    while True:
        Path = dl_search([start], goal, next_states, limit, heuristic)
        if isinstance(Path, list):
            return Path
        limit = Path

def dl_search(Path, goal, next_states, limit, heuristic):
    state = Path[-1]
    distance = len(Path) - 1
    total = distance + heuristic(state, goal)
    if total > limit:
        return total
    if state == goal:
        return Path
    smallest = float("Inf") # infinity
    for ns in next_states(state):
        if ns not in Path:
            Solution = dl_search(Path+[ns], goal, next_states, limit, heuristic)
            if isinstance(Solution, list):
                return Solution
            smallest = min(smallest, Solution)
    return smallest

```

Figure 2.24: Iterative deepening A* search.

- (c) `next_states` is a function that takes a state s as a parameter and computes the set of all those states that can be reached from s in a single step.
- (d) `heuristic` is a function that takes two parameters s_1 and s_2 , where s_1 and s_2 are states. The expression

$$\text{heuristic}(s_1, s_2)$$

computes an estimate of the distance between s_1 and s_2 . In IDA search it is required that

this estimate is optimistic, i.e. the `heuristic` has to be [admissible](#).

2. The function `search` initializes `limit` to be an estimate of the distance between `start` and `goal`. As we assume that the function `heuristic` is optimistic, we know that there is no path from `start` to `goal` that is shorter than `limit`. Hence, we start our search by assuming that we might find a path that has a length of `limit`.
3. Next, we start a `while` loop. In this loop, we call the function `dl_search` ([depth limited search](#)) to compute a path from `start` to `goal` that has a length of at most `limit`. The function `dl_search` is described in detail below. When the function `dl_search` returns, there are two cases:
 - (a) `dl_search` does find a path. In this case, this path is returned in the variable `Path` and the value of this variable is a list. This list is returned as the solution to the search problem.
 - (b) `dl_search` is not able to find a path within the given `limit`. In this case, `dl_search` will not return a list representing a path, but instead it will return a number. This number will specify the minimal length that any path leading from `start` to `goal` needs to have. This number is then used to update the `limit` which is used for the next invocation of `dl_search`.

Note that the fact that `dl_search` is able to compute this new `limit` is a significant enhancement over iterative deepening. While we had to test every single possible length in iterative deepening, now the fact that we can intelligently update the `limit` results in a considerable saving of computation time.

We proceed to discuss the function `dl_search`. This function takes 5 parameters, which we describe next.

1. `Path` is a list of states. This list starts with the state `start`. If `state` is the last state on this list, then `Path` represents a path leading from `start` to `state`.
2. `goal` is another state. The purpose of the recursive invocations of `dl_search` is to find a path from `state` to `goal`, where `state` is the last element of the list `Path`.
3. `next_states` is a function that takes a state s as input and computes the set of states that are reachable from s in one step.
4. `limit` is the upper bound for the length of the path from `start` to `goal`. If the function `dl_search` is not able to find a path from `start` to `goal` that has a length of at most `limit`, then the search is unsuccessful. In that case, instead of a path the function `dl_search` returns a new estimate for the distance between `start` and `goal`. Of course, this new estimate will be bigger than `limit`.
5. `heuristic` is a function taking two states as arguments. The invocation `heuristic( $s_1, s_2$ )` computes an [estimate](#) of the distance between the states s_1 and s_2 . It is assumed that this estimate is optimistic, i.e. the value returned by `heuristic( $s_1, s_2$ )` is less than or equal to the true distance between s_1 and s_2 .

We proceed to describe the implementation of the function `dl_search`

1. The variable `state` is assigned the last element of `Path`. Hence, `Path` connects `start` and `state`.
2. The length of the path connecting `start` and `state` is stored in `distance`.

3. Since `heuristic` is assumed to be optimistic, if we want to extend `Path`, then the best we can hope for is to find a path from `start` to `goal` that has a length of

`distance + heuristic(state, goal).`

This length is computed and saved in the variable `total`.

4. If `total` is bigger than `limit`, it is not possible to find a path from `start` to `goal` passing through `state` that has a length of at most `limit`. Hence, in this case we return `total` to communicate that the limit needs to be increased to have at least a value of `total`.
5. If we are lucky and `state` is equal to `goal`, the search is successful and `Path` is returned.
6. Otherwise, we iterate over all nodes `ns` reachable from `state` that have not already been visited by `Path`. If `ns` is a node of this kind, we extend the `Path` so that this node is visited next. The resulting path has the form

`Path + [ns].`

Next, we recursively start a new search starting from the node `ns`. If this search is successful, the resulting path is returned. Otherwise, the search returns the minimum distance that is needed to reach the state `goal` from the state `start` on a path via the state `ns`. If this distance is smaller than the distance `smallest` that is returned from visiting previous neighbouring nodes, the variable `smallest` is updated accordingly. This way, if the `for` loop is not able to return a path leading to `goal`, the variable `smallest` contains a lower bound for the distance that is needed to reach `goal` by a path that extends the given `Path`.

Note: At this point, a natural question is to ask whether the `for` loop should collect all paths leading to `goal` and then only return the path that is shortest. However, this is not necessary: Every time the function `dl_search` is invoked it is already guaranteed that there is no path that is shorter than the parameter `limit`. Therefore, if `dl_search` is able to find a path that has a length of at most `limit`, this path is known to be optimal.

Iterative deepening A* is a complete search algorithm that does find an optimal path, provided that the employed heuristic is optimistic. On the instance of the 3×3 sliding puzzle shown on Figure 2.4 on page 12, this algorithm takes about 1.2 seconds to solve the puzzle. Only about 170 kilobytes of memory are necessary for this search. For the 4×4 sliding puzzle shown in Figure 2.23, the algorithm takes about 1.6 seconds and uses 184 kilobytes. Although this is more than the time needed by bidirectional A* search, the good news is that the IDA* algorithm does not need much memory since basically only the path discovered so far is stored in memory. Hence, IDA* is a viable alternative if the available memory is not sufficient to support the bidirectional A* algorithm.

Exercise 4: The [eight queens puzzle](#) is the problem of placing eight chess queens on a chessboard so that no two queens can attack each other. In [chess](#), a [queen](#) can attack by moving horizontally, vertically, or diagonally.

- (a) Reformulate the [eight queens puzzle](#) as a search problem.
- (b) Compute an upper bound for the number of states.
- (c) Which of the algorithms we have discussed are suitable to solve this problem?
- (d) Compute all 92 solutions of the eight queens puzzle.

Hint: It is easiest to encode states as lists. For example, the solution of the eight queens puzzle that is shown in Figure 2.25 would be represented as the list

[6, 4, 7, 1, 8, 2, 5, 3]

because the queen in the first row is positioned in column 6, the queen in the second row is positioned in column 4, and so on. The start state would then be the empty list and given a state L , all states from the set `nextState( $L$ )` would be lists of the form $L + [c]$. If $\#L = k$, then the state $l + [c]$ has $k + 1$ queens, where the queen in row $k + 1$ has been placed in column c . A frame for solving this problem is available at [Artificial-Intelligence/blob/master/Python/Chapter-01/14-Eight-Queens-Puzzle.ipynb](https://artificialintelligence.blob/master/Python/Chapter-01/14-Eight-Queens-Puzzle.ipynb).

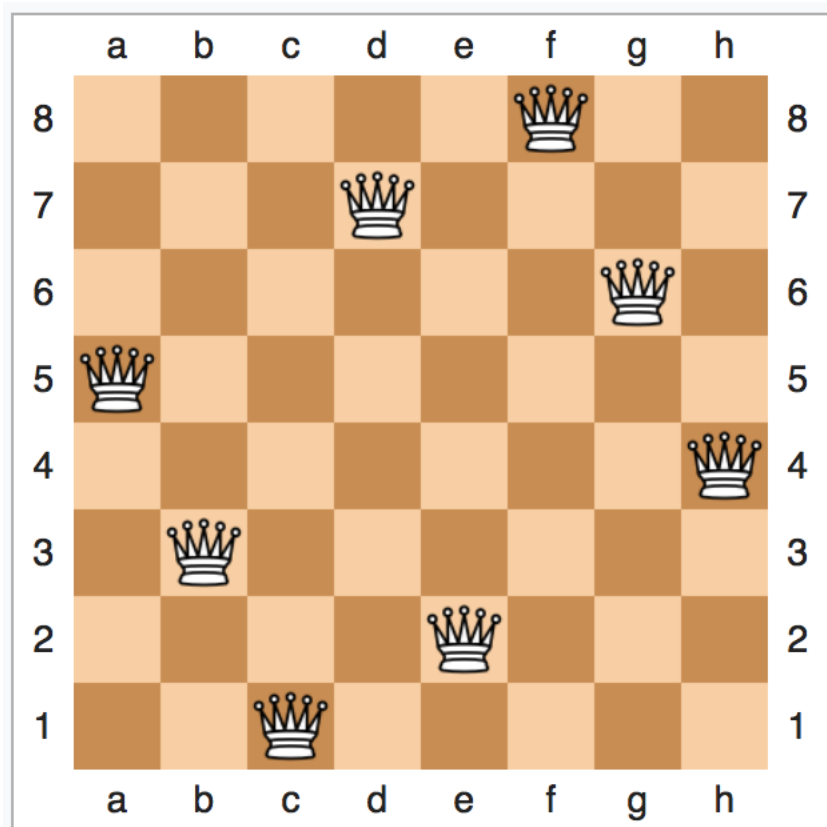


Figure 2.25: A solution of the eight queens puzzle.

Exercise 5: The founder of [Taoism](#), the Chinese philosopher [Laozi](#) once said:

“A journey of a thousand miles begins but with a single step”.

This proverb is the foundation of [taoistic search](#). The idea is, instead of trying to reach the goal directly, we rather define some intermediate states which are easier to reach than the goal state and that are nearer to the goal than the start state. To make this idea more precise, consider the following instance of the 15-puzzle, where the states **Start** and **Goal** are given as follows:

Start :=	+---+---+---+---+	Goal :=	+---+---+---+---+
	15 14 8		1 2 3
	+---+---+---+---+		+---+---+---+---+
	12 10 11 13		4 5 6 7
	+---+---+---+---+		+---+---+---+---+
	9 6 2 5		8 9 10 11
	+---+---+---+---+		+---+---+---+---+
	1 3 4 7		12 13 14 15
	+---+---+---+---+		+---+---+---+---+

In order to solve this instance of the 15-puzzle, we could try to first move the tiles numbered with 14 and 15 into the lower right corner. The resulting state would have the following form:

```
+---+---+---+---+
| * | * | * | * |
+---+---+---+---+
| * | * | * | * |
+---+---+---+---+
| * | * | * | * |
+---+---+---+---+
| * | * | 14 | 15 |
+---+---+---+---+
```

Here, the character “*” is used as a wildcard character, i.e. we do not care about the actual character in the state, for we only want to ensure that the first two tiles are positioned correctly. Once we have reached a state specified by the pattern given above, we could then proceed to reach a state that is described by the following pattern:

```
+---+---+---+---+
| * | * | * | * |
+---+---+---+---+
| * | * | * | * |
+---+---+---+---+
| * | * | * | * |
+---+---+---+---+
| 12 | 13 | 14 | 15 |
+---+---+---+---+
```

We have now solved the bottom line of the puzzle. In a similar way, we can try to solve the line above the bottom line. After that, the next step would be to reach a goal of the form

```

+---+---+---+---+
| * | * | 2 | 3 |
+---+---+---+---+
| * | * | 6 | 7 |
+---+---+---+---+
| 8 | 9 |10 |11 |
+---+---+---+---+
|12 |13 |14 |15 |
+---+---+---+---+

```

The final step would then solve the puzzle. I have prepared a framework for taoistic search. The file

[Python/Chapter-01/15-Taoistic-Search.ipynb](#)

from my github repository at <https://github.com/karlstroetmann/Artificial-Intelligence> contains a framework to solve the sliding puzzle using taoistic search where some functions are left unimplemented. Your task is to implement the missing functions in this file and thereby solve the puzzle. ◇

Chapter 3

Solving Constraint Satisfaction Problems

In this chapter, we delve into a variety of algorithms designed for solving **constraint satisfaction problems**. In a **constraint satisfaction problem**, we are presented with a set of **formulas**, and our objective is to find **values** that can be assigned to the **variables** in these formulas, ensuring all formulas evaluate to true. Constraint satisfaction problems represent a more refined version of the search problems addressed in the previous chapter. Unlike search problems, where states are abstract and lack exploitable structure for guiding the search, **constraint satisfaction problems** feature structured states in the form of **variable assignments**. This structure can be leveraged to direct the search process more effectively.

This chapter is organized as follows:

- (a) The initial section introduces the concept of a constraint satisfaction problem. To elucidate this concept, we explore two examples: **map colouring** and the **eight queens puzzle**. Subsequently, we examine various applications of constraint satisfaction problems.
- (b) The most basic algorithm for addressing a constraint satisfaction problem is **brute force search**. The principle of *brute force search* involves examining every possible **variable assignment**.
- (c) Often, the search space is so vast that enumerating all variable assignments becomes impractical. **Backtracking search** enhances brute force search by intertwining the generation of variable assignments with the evaluation of constraints, significantly boosting the efficiency of the search.
- (d) Backtracking search can be further refined through the integration of **constraint propagation** and the application of the **most restricted variable** heuristic.
- (e) Additionally, verifying the **consistency** of values assigned to different variables can substantially reduce the search space.
- (f) **Local search** presents an alternative methodology for solving constraint satisfaction problems, particularly beneficial for large but uncomplicated problems.
- (g) Lastly, we discuss the **Z3** theorem prover, an industrial-grade **constraint solver**. Here, a *constraint solver* is defined as software that accepts a *constraint satisfaction problem* as input and produces a *solution* for the problem.

Upon concluding our exploration of constraint satisfaction problems, we will have developed a constraint solver capable of resolving the most challenging **Sudoku** puzzles in mere seconds.

3.1 Formal Definition of Constraint Satisfaction Problems

Formally, a **constraint satisfaction problem** is defined as a triple:

$$\mathcal{P} := \langle \text{Vars}, \text{Values}, \text{Constraints} \rangle$$

where

- (a) **Vars** represents a set of strings, functioning as **variables**.
- (b) **Values** denotes a set of **values** that can be assigned to the variables in **Vars**.
- (c) **Constraints** is a collection of formulas derived from **first order logic**, each termed a **constraint** of $\mathcal{P}$. To evaluate these formulas, an **interpretation** of the function and predicate symbols appearing in these constraints is essential. To avoid excessive formalization, we presume these interpretations are implicitly understood. In the provided examples, these interpretations will be given through functions implemented in *Python*.

In subsequent sections, the abbreviation **CSP** refers to **constraint satisfaction problem**. Given a CSP

$$\mathcal{P} = \langle \text{Vars}, \text{Values}, \text{Constraints} \rangle,$$

a **variable assignment** for $\mathcal{P}$ is a function

$$A : \text{Vars} \rightarrow \text{Values}$$

that maps variables to values. A variable assignment A is a **solution** to $\mathcal{P}$ if, under A , all constraints are fulfilled, that is:

$$\text{eval}(f, A) = \text{true} \quad \text{for every } f \in \text{Constraints}.$$

Moreover, a **partial variable assignment** B for $\mathcal{P}$ is a function

$$B : \text{Vars} \rightarrow \text{Values} \cup \{\Omega\}, \text{ with } \Omega \text{ symbolizing the undefined value.}$$

Therefore, a partial variable assignment does not assign values to all variables, but only to a subset of **Vars**. The **domain** $\text{dom}(B)$ of a partial variable assignment B is defined as the set of variables assigned a value different from Ω , namely:

$$\text{dom}(B) := \{x \in \text{Vars} \mid B(x) \neq \Omega\}.$$

The concepts delineated thus far will be explained through three examples.

3.1.1 Example: Map Colouring

In **map colouring** a map showing different states and their borders is given and the task is to colour the different states such that no two states that have a common border share the same colour. Figure 3.1 on page 44 shows a map of **Australia**. There are seven different states in Australia:

1. **Western Australia**, abbreviated as WA,
2. **Northern Territory**, abbreviated as NT,
3. **South Australia**, abbreviated as SA,
4. **Queensland**, abbreviated as Q,
5. **New South Wales**, abbreviated as NSW,
6. **Victoria**, abbreviated as V, and

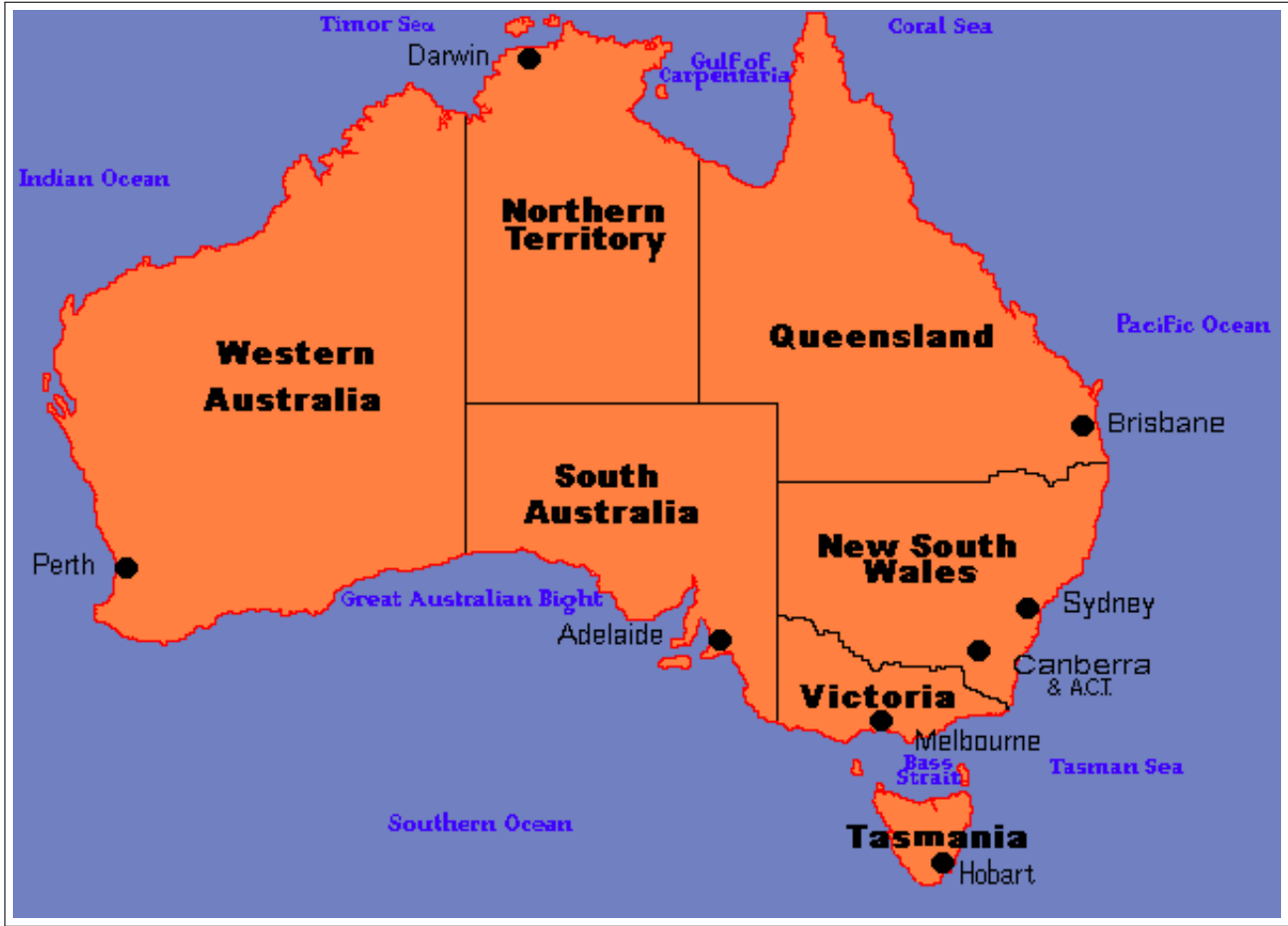


Figure 3.1: A map of Australia.

7. **Tasmania**, abbreviated as T.

Figure 3.1 would certainly look better if different states that share a common border had been coloured with different colours. For the purpose of this example let us assume that we only have the three colours **red**, **green**, and **blue** available. The task is then to colour the different states in a way that no two neighbouring states share the same colour. This task can be formalized as a constraint satisfaction problem. To this end we define:

1. **Vars** := {WA, NT, SA, Q, NSW, V, T},

2. **Values** := {**red**, **green**, **blue**},

3. **Constraints** :=

$$\{WA \neq NT, WA \neq SA, SA \neq Q, NT \neq Q, SA \neq Q, SA \neq NSW, SA \neq V, Q \neq NSW, NSW \neq V\}.$$

Then $\mathcal{P} := \langle \text{Vars}, \text{Values}, \text{Constraints} \rangle$ is a constraint satisfaction problem. If we define the assignment A such that

(a) $A(WA) = \text{blue}$,

(b) $A(NT) = \text{red}$,

(c) $A(SA) = \text{green}$,

- (d) $A(Q) = \text{blue}$,
- (e) $A(NSW) = \text{red}$,
- (f) $A(V) = \text{blue}$,
- (g) $A(T) = \text{red}$,

then it is straightforward to check that this assignment is indeed a solution to the constraint satisfaction problem $\mathcal{P}$.

3.1.2 Example: The Eight Queens Puzzle

The **eight queens puzzle** asks to put 8 queens on a chessboard such that no queen can attack another queen. In **chess**, a queen can attack all pieces that are either in the same row, the same column, or the same diagonal. If we want to put 8 queens on a chessboard such that no two queens can attack each other, we have to put exactly one queen in every row: If we would put more than one queen in a row, the queens in that row could attack each other. If we would leave a row empty, then, given that the other rows contain at most one queen, there would be less than 8 queens on the board. Therefore, in order to model the eight queens problem as a constraint satisfaction problem, we will use the following set of variables:

$$\text{Vars} := \{V_1, V_2, V_3, V_4, V_5, V_6, V_7, V_8\},$$

where for $i \in \{1, \dots, 8\}$ the variable V_i specifies the column of the queen that is placed in row i . As the column numbers run from 1 up to 8, we define the set **Values** as

$$\text{Values} := \{1, 2, 3, 4, 5, 6, 7, 8\}.$$

Next, let us define the constraints. There are two different types of constraints.

1. We need constraints that express that no two queens that are positioned in different rows share the same column. To capture these constraints, we define

$$\text{DifferentCols} := \{V_i \neq V_j \mid i \in \{1, \dots, 8\} \wedge j \in \{1, \dots, 8\} \wedge j < i\}.$$

Here the condition $j < i$ ensures that, for example, while we have the constraint $V_2 \neq V_1$ we do not also have the constraint $V_1 \neq V_2$, as the latter constraint would be redundant if the former constraint had already been established.

2. We need constraints that express that no two queens positioned in different rows share the same diagonal. The queens in row i and row j share the same diagonal iff the equation

$$|i - j| = |V_i - V_j|$$

holds. The expression $|i - j|$ is the absolute value of the difference of the rows of the queens in row i and row j , while the expression $|V_i - V_j|$ is the absolute value of the difference of the columns of these queens. To capture these constraints, we define

$$\text{DifferentDiags} := \{|i - j| \neq |V_i - V_j| \mid i \in \{1, \dots, 8\} \wedge j \in \{1, \dots, 8\} \wedge j < i\}.$$

For a fixed pair of values $\langle j, V_j \rangle$ the equations

$$V_i = V_j - j + i \quad \text{and} \quad V_i = V_j + j - i$$

are the linear equations for the straight lines with slope 1 and -1 that pass through $\langle j, V_j \rangle$.

Then, the set of constraints is defined as

$\text{Constraints} := \text{DifferentCols} \cup \text{DifferentDiags}$

and the eight queens problem can be stated as the constraint satisfaction problem

$\mathcal{P} := \langle \text{Vars}, \text{Values}, \text{Constraints} \rangle$.

If we define the assignment A such that

$A(V_1) := 4, A(V_2) := 7, A(V_3) := 5, A(V_4) := 2, A(V_5) := 6, A(V_6) := 1,$
 $A(V_7) := 3, A(V_8) := 8,$

then it is easy to see that this assignment is a solution of the eight queens problem. This solution is shown in Figure 3.2 on page 46. In this figure, we have numbered the rows from bottom to top, i.e. the topmost row is row number 8 and therefore the column of the queen in the first row is determined by the variable V_8 .

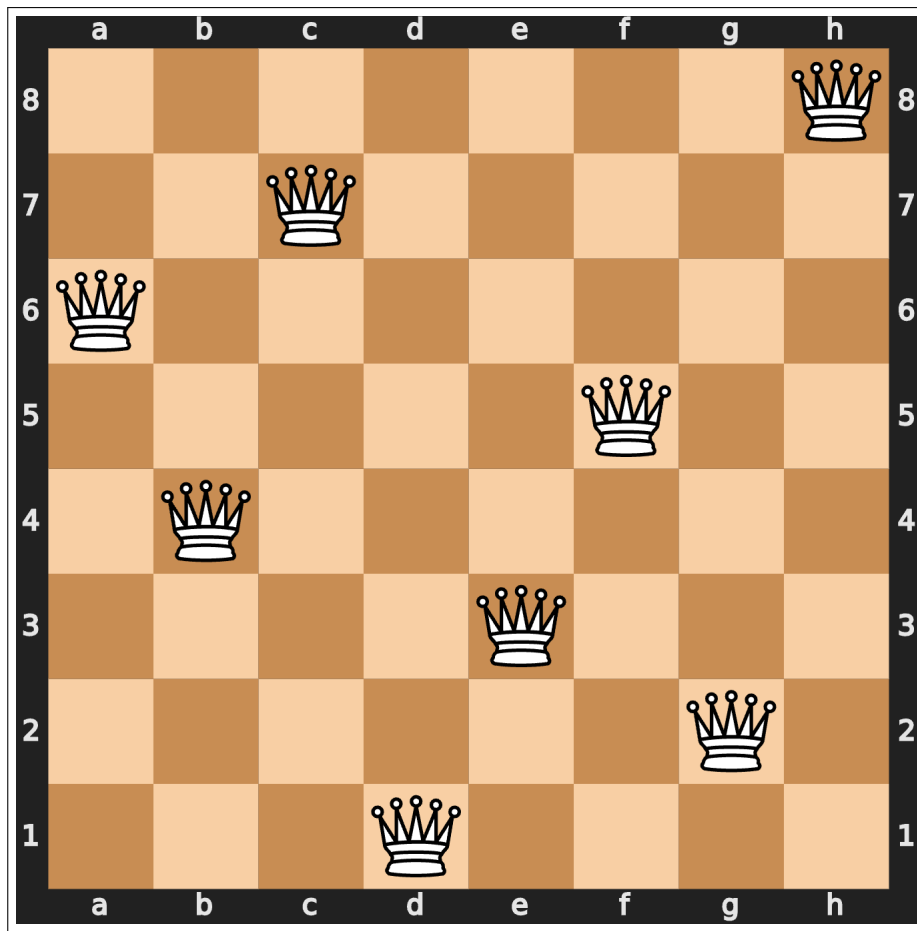


Figure 3.2: A solution of the eight queens puzzle.

Later, when we develop algorithms to solve CSPs, we will represent variable assignments and partial variable assignments as *Python dictionaries*. For example, A would then be represented as the dictionary

$A := \{V_1 : 4, V_2 : 7, V_3 : 5, V_4 : 2, V_5 : 6, V_6 : 1, V_7 : 3, V_8 : 8\}$.

If we define

$B := \{V_1 : 4, V_2 : 7, V_3 : 3\}$,

then B is a [partial](#) variable assignment and $\text{dom}(B) = \{V_1, V_2, V_3\}$. This partial variable assignment is shown in Figure 3.3 on page 47. Note that the bottom-most row is the row number 1.

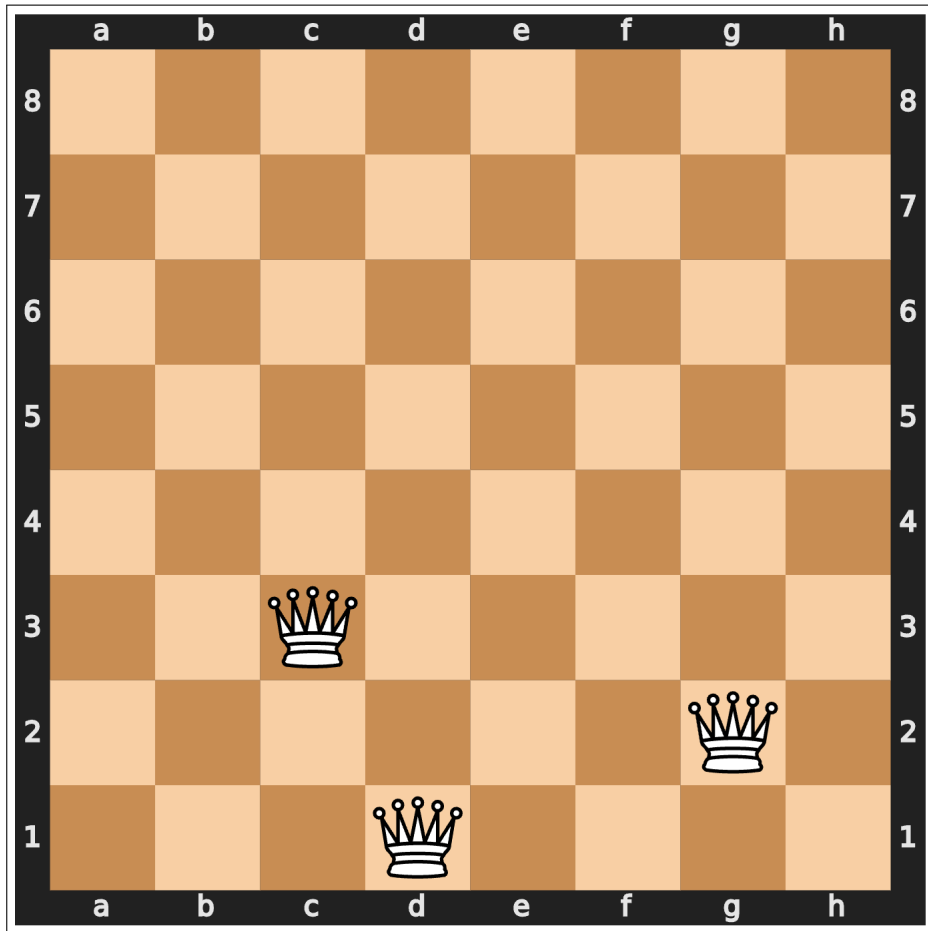


Figure 3.3: A partial solution of the eight queens puzzle.

```
def create_csp(n):
    S = range(1, n+1)
    Variables = { f'V{i}' for i in S }
    Values = set(S)
    DifferentCols = { f'V{i} != V{j}' for i in S
                      for j in S
                      if i < j
                    }
    DifferentDiags = { f'abs(V{j} - V{i}) != {j - i}' for i in S
                      for j in S
                      if i < j
                    }
    return Variables, Values, DifferentCols | DifferentDiags
```

Figure 3.4: The n queens problem formulated as a CSP.

Figure 3.4 on page 47 shows a *Python* program that can be used to create a CSP that encodes the eight queens puzzle. The code shown in this figure is more general than necessary. Given a natural number n , the function call `create_csp(n)` creates a constraint satisfaction problem $\mathcal{P}$ that generalizes the eight queens problem to the problem of placing n queens on a board of size n times n such that no queen can capture another queen. The fact that the n -queen problem is parameterized by the number of queens n gives us the ability to check how the running time of the algorithms for solving CSPs scales with the size of the problem.

The beauty of **constraint programming** is the fact that we will be able to develop a so called **constraint solver** that takes as input a CSP like the one produced by the program shown in Figure 3.4 and that is capable of computing a solution automatically. In effect, this enables us to use **declarative programming**: Instead of developing an algorithm that solves a given problem we confine ourselves to specifying the problem precisely and then let a general purpose problem solver do the job of computing the solution. This approach of declarative programming was one of the main ideas incorporated in the programming language **Prolog**. While **Prolog** could not live up to its promises as a viable general purpose programming language, constraint programming has proved to be very useful in a number of domains.

3.1.3 Example: The Zebra Problem

The following puzzle is known as the **Zebra Puzzle** and was featured in the magazine *Life International* on December 17, 1962. It presents a series of clues pertaining to the occupants of five distinct houses, challenging the solver to deduce specific details about them. The puzzle is structured as follows:

1. There are five houses.
2. The Englishman lives in the red house.
3. The Spaniard owns the dog.
4. Coffee is drunk in the green house.
5. The Ukrainian drinks tea.
6. The green house is immediately to the right of the ivory house.
7. The Old Gold smoker owns snails.
8. Kools are smoked in the yellow house.
9. Milk is drunk in the middle house.
10. The Norwegian lives in the first house.
11. The man who smokes Chesterfields lives in the house next to the man with the fox.
12. Kools are smoked in the house next to the house where the horse is kept.
13. The Lucky Strike smoker drinks orange juice.
14. The Japanese smokes Parliaments.
15. The Norwegian lives next to the blue house.
16. Each of the five houses is painted a unique color.

17. The residents of the five houses each have a distinct nationality.
18. A different pet is kept in each house.
19. The beverages consumed in the various houses are all unique.
20. A distinct brand of cigarettes is smoked in each house.

The objective of the Zebra Puzzle is to answer the following two questions:

1. Who drinks water?
2. Who owns the zebra?

Next, we formulate the zebra puzzle as a constraint satisfaction problem. To this end, we first define the set of variables:

```
Nations = { 'English', 'Spanish', 'Ukrainian', 'Norwegian', 'Japanese' }
Drinks  = { 'Coffee', 'Tea', 'Milk', 'OrangeJuice', 'Water' }
Pets    = { 'Dog', 'Snails', 'Horse', 'Fox', 'Zebra' }
Brands  = { 'LuckyStrike', 'Parliaments', 'Kools', 'Chesterfields', 'OldGold' }
Colours = { 'Red', 'Green', 'Ivory', 'Yellow', 'Blue' }
Variables = Nations | Drinks | Pets | Brands | Colours
```

Then, we define the set of values as:

```
Values = { 1, 2, 3, 4, 5 }
```

The interpretation of the variables and their values should be obvious: For example, if the variable `English` has the value 1, then this would imply that the Englishman lives in the first house.

In order to write down the last 5 constraints we implement the auxiliary function `allDifferent(V)` which takes a set of variables V as input and returns a set of formulas expressing that the values of the variables in V are all different.

```
def allDifferent(Variables: set[str]) -> set[str]:
    return { f'{x} != {y}' for x in Variables
            for y in Variables
            if x < y }
}
```

For example, using the function `allDifferent` we can express the fact that all five houses are painted in a different color via the formula

```
allDifferent(Nations).
```

Using the function `allDifferent`, we can define the set of all constraints as follows:

```

Constraints = { 'English      == Red',
                'Spanish      == Dog',
                'Coffee       == Green',
                'Ukrainian    == Tea',
                'Green        == Ivory + 1',
                'OldGold     == Snails',
                'Kools        == Yellow',
                'Milk         == 3',
                'Norwegian    == 1',
                'abs(Chesterfields - Fox) == 1',
                'abs(Kools - Horse) == 1',
                'LuckyStrike  == OrangeJuice',
                'Japanese     == Parliaments',
                'abs(Norwegian - Blue) == 1'
              }

Constraints |= allDifferent(Nations)
Constraints |= allDifferent(Drinks)
Constraints |= allDifferent(Pets)
Constraints |= allDifferent(Brands)
Constraints |= allDifferent(Colours)

```

We will soon develop a solver that is able to solve the resulting constraint satisfaction problem.

Exercise 6: In an oncology ward, five patients are lying in adjacent rooms. Except for one of the patients, each has smoked exactly one brand of cigarette. The patient who did not smoke cigarettes smoked a pipe. Each patient drives exactly one car and is diagnosed with exactly one type of cancer. Additionally, we have the following information:

1. In the room next to Michael, Camel is being smoked.
2. The Trabant driver smokes Harvest 23 and is in the room next to the tongue cancer patient.
3. Rolf is in the last room and has laryngeal cancer.
4. The West smoker is in the first room.
5. The Mazda driver has tongue cancer and is next to the Trabant driver.
6. The Nissan driver is next to the tongue cancer patient.
7. Rudolf is desperately begging for euthanasia and his room is between the room of the Camel smoker and the room of the Trabant driver.
8. Tomorrow is the last birthday of the Seat driver.
9. The Luckies smoker is next to the patient with lung cancer.
10. The Camel smoker is next to the patient with intestinal cancer.
11. The Nissan driver is next to the Mazda driver.
12. The Mercedes driver smokes a pipe and is next to the Camel smoker.

13. Jens is next to the Luckies smoker.
14. Yesterday, the patient with testicular cancer flushed his balls down the toilet.

Given this information, the task is to answer the following questions:

1. What does the intestinal cancer patient smoke?
2. What car does Kurt drive?

Your task is to formulate this puzzle as a constraint satisfaction problem. ◇

3.1.4 Applications

Besides the toy problems discussed so far, there are a number of industrial applications of constraint satisfaction problems. The most important applications seem to be variants of **scheduling problems**. A simple example of a scheduling problem is the problem of generating a timetable for a school. A school has various teachers, each of which can teach some subjects but not others. Furthermore, there are a number of classes that must be taught in different subjects. The problem is then to assign teachers to classes and to create a timetable. A special case of scheduling problems is **crew scheduling**. For example, airlines have to solve a crew scheduling problem in order to efficiently assign crews of pilots and crews of stewards to their aircraft. Stewards and pilots work in different crews as they have different required resting times.

3.2 Brute Force Search

The most straightforward algorithm to solve a CSP is to test all possible combinations of assigning values to variables. This approach is known as **brute-force search**. If there are n different values that can be assigned to k variables, this approach amounts to checking at most n^k different variable assignments. For example, for the eight queens problem there are 8 variables and 8 possible values and hence there are at most

$$8^8 = 2^{24} = 16,777,216$$

different assignments that need to be tested. Given the clock rate of modern computers, checking a million assignments per second is plausible. Hence, this approach is able to solve the eight queens problem in about 30 seconds. An implementation of brute force search is shown in Figure 3.5 on page 52.

The function `solve` takes a constraint satisfaction problem P as its input. This CSP is given as a triple of the form

$$P = (\text{Variables}, \text{Values}, \text{Constraints}).$$

The sole purpose of the function `solve` is to call the function `brute_force_search`, which needs an additional argument. This argument is a **partial variable assignment** that is initially empty. Every recursive iteration of the function `brute_force_search` assigns one additional variable.

1. **Assignment** is a partial variable assignment. Initially, this assignment will be the empty dictionary. Every recursive call of `brute_force_search` adds the assignment of one variable to the given assignment.
2. `csp` is a triple of the form

$$\text{csp} = (\text{Variables}, \text{Values}, \text{Constraints}).$$

```

def solve(P):
    return brute_force_search({}, P)

def brute_force_search(Assignment, csp):
    Variables, Values, Constraints = csp
    if len(Assignment) == len(Variables): # all variables have been assigned
        if check_all_constraints(Assignment, Constraints):
            return Assignment
        else:
            return None
    var = arb(Variables - set(Assignment.keys()))
    for value in Values:
        NewAss = Assignment.copy()
        NewAss[var] = value
        result = brute_force_search(NewAss, csp)
        if result != None:
            return result
    return None

```

Figure 3.5: Solving a CSP via brute force search.

Here, `Constraints` is a set of Boolean expressions that are given as strings. These strings have to follow the syntax of *Python* so that they can be evaluated using the *Python* function `eval`.

The implementation of `brute_force_search` works as follows:

1. If all variables have been assigned a value, the dictionary `Assignment` will have the same number of entries as the set `Variables` has elements. Hence, in that case `Assignment` is a complete assignment of all variables and we now have to test whether all constraints are satisfied. This is done using the auxiliary function `check_all_constraints` that is shown in Figure 3.6 on page 53. If the current `Assignment` does indeed satisfy all constraints, it is a solution to the given CSP and is therefore returned.

If, instead, some constraint is violated, then `brute_force_search` returns the value `None`.

2. If the assignment is not yet complete, we arbitrarily pick a variable `var` from the set of those `Variables` that still have no value assigned. Then, for every possible `value` in the set `Values`, we extend the current partial `Assignment` to a new assignment `NewAss` that satisfies

`NewAss[var] = value.`

Next, the algorithm recursively tries to find a solution for this new partial assignment. If this recursive call succeeds, the solution it has computed is returned. Otherwise, the next value for the given variable `var` is tried.

3. If none of the values work for `var`, the function returns `None`.

The function `check_all_constraints` takes a complete variable `Assignment` as its first input. The second input is the set `Constraints` which is a set of *Python* expressions. For all expressions `f` from the set `Constraints`, the function `check_all_constraints` checks whether `f` yields `True` under the given variable assignment. This check is done using the function `eval`, which is a predefined function. This function takes two arguments:

```
def check_all_constraints(Assignment, Constraints):
    A = Assignment.copy()
    return all(eval(f, A) for f in Constraints)
```

Figure 3.6: Auxiliary functions for brute force search.

- (a) The first argument is a *Python* expression `f`.
- (b) The second argument is a variable assignment `A`, that is represented as a dictionary.

The function `eval` evaluates the expression `f`. In order to do this, any variables occurring in `f` are assigned values according to the variable assignment `A`. As a side effect, the function `eval` changes the dictionary `A` that is used as its second argument. This is the reason we have to make a copy of the `Assignment` that is given as the first argument of the function `check_all_constraints`.

When I tested the program discussed above with the eight queens problem, it took about 30 seconds to compute a solution. In contrast, the seven queens problem took about 1.7 seconds. As we have

$$\frac{8^8}{7^7} \approx 20 \quad \text{and} \quad 30/1.7 \approx 18$$

this shows that the computation time does indeed roughly grow with the number of possible assignments that have to be checked. However, the correspondence is not exact. The reason is that we stop our search as soon as a solution is found. If we are lucky and the given CSP is easy to solve, this might happen when we have checked only a small portion of the set of all possible assignments.

3.3 Backtracking Search

For the n queens problem the number of possible variable assignments grows as fast as n^n . This growth is super-exponential and this is what usually happens when we scale a CSP up. The reason is that the number of all variable assignments is given as

$$\text{card}(\text{Values})^{\text{card}(\text{Vars})},$$

where for a set M , the expression `card( $M$ )` returns the number of elements of M . For this reason, [brute force search](#) is only viable for small problems. One approach to solve a CSP that is both conceptually simple and at least more efficient than brute force search is [backtracking](#). The idea is to try to evaluate constraints as soon as possible: If C is a constraint and B is a partial assignment such that all the variables occurring in C have already been assigned a value in B and the evaluation of C fails, then there is no point in trying to complete the variable assignment B . Hence, in backtracking we evaluate a constraint C as soon as all of its variables have been assigned a value. If C is not valid, we discard the current partial variable assignment. This approach can result in huge time savings when compared to the baseline of brute force search.

Figure 3.7 on page 54 shows a simple CSP solver that employs the backtracking strategy. We discuss this program next. The function `solve` takes a constraint satisfaction problem `P` as input and tries to find a solution.

1. First, the CSP `P` is split into its components.
2. Next, for every constraint `f` of the given CSP, we compute the set of variables that are used in `f`. This is done using the function `collect_variables` that is shown in Figure 3.10 on page 56.

```
def solve(P):
    Variables, Values, Constraints = P
    csp = (Variables, Values, [(f, collect_variables(f)) for f in Constraints])
    try:
        return backtrack_search({}, csp)
    except Backtrack:
        return None
```

Figure 3.7: A backtracking CSP solver.

These variables are then stored together with the constraint `f` and the correspondingly modified data structure is stored in the variable `csp` and is called an **augmented CSP**.

The reason to compute and store these variables is efficiency: When we later check whether a constraint `f` is satisfied for a partial variable assignment `Assignment` where `Assignment` is stored as a dictionary, we only need to check the constraint `f` iff all of the variables occurring in `f` are elements of the domain of `Assignment`. It would be wasteful to compute these variables every time that a partial variable assignment is extended.

3. Next, we call the function `backtrack_search` to compute a solution of CSP. This function is enclosed in a `try-except`-block that catches exceptions of class `Backtrack`. This class is defined as follows:

```
class Backtrack(Exception):
    pass
```

Its only purpose is to create a name for the special kind of exceptions used to administer backtracking. The reason for enclosing the call to `backtrack_search` in a `try-except`-block is that the function `backtrack_search` either returns a solution or, if it is not able to find a solution, it raises an exception of class `Backtrack`. The `try-except`-block ensures that this exception is silently discarded.

Next, we discuss the implementation of the function `backtrack_search` that is shown in Figure 3.8 on page 55. This function receives a partial assignment `Assignment` as input together with an augmented CSP `P`. This partial assignment is **consistent** with `P`: If `f` is a constraint of CSP such that all the variables occurring in `f` are members of `dom(Assignment)`, then evaluating `f` using `Assignment` yields `true`. Initially, this partial assignment is empty and hence trivially consistent. The idea is to extend this partial assignment until it is a complete variable assignment. We take care to ensure that this partial variable assignment remains consistent when it is extended. This way, once this assignment is complete it has to satisfy all the constraints of the given CSP.

1. First, the augmented CSP `P` is split into its components.
2. Next, if `Assignment` is already a complete variable assignment, i.e. if the dictionary `Assignment` has as many elements as there are variables, then it must be a solution of the CSP and, therefore, it is returned. The reason is that the function `backtrack_search` is only called with a **consistent** partial assignment.
3. Otherwise, we have to extend the partial `Assignment`. In order to do so, we first have to select a variable `var` that has not yet been assigned a value in `Assignment` so far. This is done in line 5 using the function `arb` that selects an arbitrary variable from its input set.

```

def backtrack_search(Assignment, P):
    Variables, Values, Constraints = P
    if len(Assignment) == len(Variables):
        return Assignment
    var = arb(Variables - Assignment.keys())
    for value in Values:
        try:
            if is_consistent(var, value, Assignment, Constraints):
                NewAss = Assignment.copy()
                NewAss[var] = value
                return backtrack_search(NewAss, P)
        except Backtrack:
            continue
    raise Backtrack()

```

Figure 3.8: A backtracking CSP solver: The function `backtrack_search`.

4. Next, we try to assign a `value` to the selected variable `var`. After assigning a `value` to `var`, we immediately check whether this assignment would be consistent with the constraints using the function `is_consistent`. If the partial `Assignment` turns out to be consistent, the partial variable `Assignment` is extended to the new partial assignment `NewAss` that satisfies

`NewAss[var] = value.`

Then, the function `backtrack_search` is called recursively to complete this new partial assignment. If this is successful, the resulting assignment is a solution that is returned. Otherwise, the recursive call of `backtrack_search` will raise an exception. This exception is muted by the `try-except`-block that surrounds the call to `backtrack_search`. In that case, the `for`-loop generates a new `value` that can be assigned to the variable `var`. If all possible values have been tried and none was successful, the `for`-loop ends and we have to `backtrack`, i.e. we have to reassign one of the variables that have been assigned earlier. This is done by raising a `Backtrack` exception. This exception is then caught by one of the prior invocations of `backtrack_search`. If all variable assignments have been tried and none is successful, then the `Backtrack` exception propagates back to the function `solve`, which will return `None` in that case.

```

def is_consistent(var, value, Assignment, Constraints):
    NewAssign = Assignment.copy()
    NewAssign[var] = value
    return all(eval(f, NewAssign) for (f, Vs) in Constraints
              if var in Vs and Vs <= NewAssign.keys()
              )

```

Figure 3.9: The definition of the function `is_consistent`.

We still need to discuss the implementation of the auxiliary function `is_consistent` shown in Figure 3.9. This function takes a variable `var`, a `value`, a partial `Assignment` and a set of `Constraints` as arguments. It is assumed that `Assignment` is `partially consistent` with respect to the set `Constraints`,

i.e. for every formula f occurring in `Constraints` such that

$$\text{vars}(f) \subseteq \text{dom}(\text{Assignment})$$

holds, the formula f evaluates to `True` given the `Assignment`. The purpose of `is_consistent` is to check, whether the extended assignment

$$\text{NewAssign} := \text{Assignment} \cup \{\text{var} \mapsto \text{value}\}$$

that assigns `value` to the variable `var` is still partially consistent with `Constraints`. To this end, the `for`-loop iterates over all formulas f in `Constraints`. However, we only have to check those formulas f that contain the variable `var` and, furthermore, have the property that

$$\text{vars}(f) \subseteq \text{dom}(\text{NewAssign}),$$

i.e. all variables occurring in the formula f need to have a value assigned in `NewAssign`. The reasoning is as follows:

1. If `var` does not occur in the formula f , then adding `var` to `Assignment` cannot change the result of evaluating f and as `Assignment` is assumed to be partially consistent with respect to f , `NewAssign` is also partially consistent with respect to f .
2. If $\text{dom}(\text{NewAssign}) \not\subseteq \text{vars}(f)$, then f can not be evaluated anyway.

Note that the domain of a variable assignment `A` can be computed with the expression `A.keys()` since `A` is represented as a dictionary in *Python*.

```
import extractVariables as ev

def collect_variables(expr):
    return { var for var in ev.extractVars(expr)
            if var not in dir(__builtins__)
            if var not in ['and', 'or', 'not']
            }
    
```

Figure 3.10: The function `collectVars`.

Finally, let us discuss the function `collect_variables` that is shown in Figure 3.10 on page 56. This function uses the module `extractVariables` that provides the function `extractVars(e)`. This function takes a string e that can be interpreted as a *Python* expression as its argument and returns the set of all variables and function symbols occurring in the expression e . As we only want to keep the variable names, the function `collect_variables` takes care to eliminate the function symbols. This is done by making use of the fact that all function symbols that have been defined are members of the list `dir(__builtins__)`. It turns out that the keyword “and”, “or”, and “not” also need to be removed since they might also be members of the set returned by `extractVars(expr)`.

If we use the program discussed in this section, we can solve the 8 queens problem in about 22 milliseconds. Hence, for the eight queens problem backtracking is more than a thousand times faster than brute force search.

Exercise 7: There are many different versions of the *zebra puzzle*. The version below is taken from *Wikipedia*. The puzzle reads as follows:

- (a) There are five houses.

- (b) The Englishman lives in the red house.
- (c) The Spaniard owns the dog.
- (d) Coffee is drunk in the green house.
- (e) The Ukrainian drinks tea.
- (f) The green house is immediately to the right of the ivory house.
- (g) The Old Gold smoker owns snails.
- (h) Kools are smoked in the yellow house.
- (i) Milk is drunk in the middle house.
- (j) The Norwegian lives in the first house.
- (k) The man who smokes Chesterfields lives in the house next to the man with the fox.
- (l) Kools are smoked in the house next to the house where the horse is kept.
- (m) The Lucky Strike smoker drinks orange juice.
- (n) The Japanese smokes Parliaments.
- (o) The Norwegian lives next to the blue house.
- (p) Who drinks water?
- (q) Who owns the zebra?

In order to solve the puzzle, we also have to know the following facts:

- Each of the five houses is painted in a **different** colour.
- The inhabitants of the five houses are of **different** nationalities, and
- they own **different** pets, drink **different** beverages, and smoke **different** brands of cigarettes.

Formulate the zebra puzzle as a constraint satisfaction problem and solve the puzzle using the program discussed in this section. ◇

3.4 Constraint Propagation

Once we have chosen a value for a variable, this choice influences the values that are still available for other variables. For example, suppose that in order to solve the n queens problem we place the queen in row one in the second column, then no other queen can be placed in that column. Furthermore, due to the constraints on diagonals, the queen in row two can not be placed in any of the first three columns. Abstractly, constraint propagation works as follows.

1. Before the search is started, we create a dictionary **ValuesPerVar**. Initially, for every variable x , the set

ValuesPerVar[x]

contains all values v from the set **Values**. As soon as we discover that assigning a value v to the variable x is inconsistent with the variable assignments that have already taken place for other variables, the value v will be removed from the set **ValuesPerVar**[x].

2. As long as the given CSP is not solved, we choose a variable x that has not been assigned a value yet. This variable is chosen using the [most constrained variable](#) heuristic: We choose a variable x such that the number of values in the set

`ValuesPerVar[x]`

is minimal. This is done because we have to find values for all variables. If the current partial variable assignment can not be completed into a solution, then we want to find out this fact as soon as possible. Therefore, we try to find the values for the most difficult variables first. A variable is more difficult to get right if it has only a few values left that can be used to instantiate it.

3. Once we have picked a variable x , we next iterate over all values v in `ValuesPerVar[x]`. Once we have assigned a value v to the variable x , we [propagate](#) the consequences of this assignment:
 - (a) For every constraint f that mentions only the variable x and one other variable y that has not yet been instantiated, we compute the set `Legal` of those values from `ValuesPerVar[y]` that can be assigned to y without violating the constraint f .
 - (b) Then, the set `ValuesPerVar[y]` is updated to the set `Legal` and we go back to step 2.

It turns out that elaborating the idea outlined above can enhance the performance of backtracking search considerably. Figure 3.11 on page 58 shows an implementation of [constraint propagation](#). In addition to the ideas described above, this implementation takes care of [unary constraints](#), i.e. constraints that contain only a single variable, as these constraints can be solved prior to the other constraints without backtracking.

```
def solve(P):
    Variables, Values, Constraints = P
    Annotated = { (f, collect_variables(f)) for f in Constraints }
    ValuesPerVar = { v: Values for v in Variables }
    UnaryConstrs = { (f, V) for f, V in Annotated if len(V) == 1 }
    OtherConstrs = { (f, V) for f, V in Annotated if len(V) >= 2 }
    try:
        for f, V in UnaryConstrs:
            var = arb(V)
            ValuesPerVar[var] = solve_unary(f, var, ValuesPerVar[var])
        return backtrack_search({}, ValuesPerVar, OtherConstrs)
    except Backtrack:
        return None
```

Figure 3.11: Constraint Propagation.

In order to implement constraint propagation, it is necessary to administer the values that can be used to instantiate the different variables separately, i.e. for every variable x we need to know which values are admissible for x . To this end, we need a dictionary `ValuesPerVar` that contains the set of possible values for every variable x . Initially, this dictionary assigns the set `Values` to every variable. Next, we take care of the unary constraints and shrink these sets so that the unary constraints are satisfied. Then, whenever we assign a value to a variable x , we inspect those constraints that mention the variable x and exactly one other yet unassigned variable y and shrink the set of values `ValuesPerVar[y]` that can be assigned to this variables y . This process is called [constraint propagation](#) and is described in more detail below when we discuss the function `propagate`.

1. The function `solve` receives a CSP `P` as its argument. The CSP `P` is first split into its three components and the constraints are annotated with the sets of variables occurring in them. These `annotated constraints` are stored in the set `Annotated`.
2. The most important data structure maintained by the function `solve` is the dictionary

`ValuesPerVar`.

Given a variable `v`, this dictionary assigns the set of values that can be used to instantiate this variable. Initially, this set is the same for all variables and is equal to `Values`.

3. In order to solve the unary constraints we first have to find them. The set `UnaryConstrs` contains all those pairs (f, V) from the set of annotated constraints such that the set of variables `V` occurring in `f` only contains a single variable.
4. Similarly, the set `OtherConstrs` contains those constraints that involve two or more variables.
5. In order to solve the unary constraints, we iterate over these constraints and shrink the set of values associated with the variable occurring in the constraint as dictated by the constraint. This is done using the function `solve_unary`.
6. Then, we start backtracking search using the function `backtrack_search`. Besides backtracking, the implementation of `backtrack_search` that we present below implements the `most constrained variable` heuristic and `constraint propagation`.

```
def solve_unary(f, x, Values):
    Legal = { value for value in Values if eval(f, { x: value }) }
    if not Legal:
        raise Backtrack()
    return Legal
```

Figure 3.12: Implementation of `solve_unary`.

The function `solve_unary` shown in Figure 3.12 on page 59 takes a unary constraint `f`, the variable `x` occurring in `f` and the set of values `Values` that can be assigned to this variable. It returns the subset of values that can be substituted for the variable `x` without violating the given constraint `f`. If this set is empty, a `Backtrack` exception is raised since in that case the given CSP is unsolvable.

The function `backtrack_search` shown in Figure 3.13 on page 60 is called with a partial variable `Assignment` that is guaranteed to be consistent, a dictionary `ValuesPerVar` associating every variable with the set of values that might be substituted for this variable, and a set of annotated `Constraints`. It tries to complete `Assignment` and thereby computes a solution of the given CSP.

1. If the partial `Assignment` is already complete, i.e. if it assigns a value to every variable, then a solution to the given CSP has been found and this solution is returned. As the dictionary `ValuesPerVar` has an entry for every variable, its size is the same as the number of variables. Therefore, `Assignment` is complete iff it has the same size as `ValuesPerVar`.
2. Otherwise, we choose a variable `x` such that the number of values that can still be used to instantiate `x` is minimal. This strategy is known as the `most constrained variable heuristic`. The variable `x` is computed using the function `most_constrained_variable` that is shown in Figure 3.14 on page 60.

```

def backtrack_search(Assignment, ValuesPerVar, Constraints):
    if len(Assignment) == len(ValuesPerVar):
        return Assignment
    x = most_constrained_variable(Assignment, ValuesPerVar)
    for v in ValuesPerVar[x]:
        try:
            NewValues = propagate(x, v, Assignment, Constraints, ValuesPerVar)
            NewAssign = Assignment.copy()
            NewAssign[x] = v
            return backtrack_search(NewAssign, NewValues, Constraints, lcv)
        except Backtrack:
            continue
    raise Backtrack()

```

Figure 3.13: Implementation of `backtrack_search`.

The logic behind choosing a maximally constrained variables is that these variables are the most difficult to get right. If we have a partial assignment that is inconsistent, then we will discover this fact earlier if we try the most difficult variables first. This might save us a lot of unnecessary backtracking later.

3. Next, we try to find a value that can be assigned to the variable `x`. To this end we iterate over all values in `ValuesPerVar[x]`. Note that since `ValuesPerVar[x]` is, in general, smaller than the set `Values` of all values of the CSP, the `for`-loop in this version of backtracking search is more efficient than the corresponding `for`-loop in backtracking search discussed in the previous section.
4. If assigning the value `v` to the variable `x` is consistent, we propagate the consequences of this assignment using the function `propagate` shown in Figure 3.15 on page 61. This function updates the dictionary `ValuesPerVar` for all variables that are still unassigned.
5. Finally, the partial variable `Assignment` is updated to include the assignment of `v` to `x` and the recursive call to `backtrack_search` tries to complete this new assignment and thereby compute a solution to the given CSP.

```

def most_constrained_variable(Assignment, ValuesPerVar):
    Unassigned = { (x, len(U)) for x, U in ValuesPerVar.items()
                    if x not in Assignment
                  }
    minSize = min(lenU for _, lenU in Unassigned)
    return arb({ x for x, lenU in Unassigned if lenU == minSize })

```

Figure 3.14: Finding a most constrained variable.

Figure 3.14 on page 60 shows the implementation of the function `most_constrained_variable`. The function `most_constrained_variable` takes a partial `Assignment` and a dictionary `ValuesPerVar` returning for all variables `x` the set of values `ValuesPerVar[x]` that can be assigned to `x`.

1. First, this function computes the set of **Unassigned** variables. For every variable x that has not yet been assigned a value in **Assignment** this set contains the pair $(x, \text{len}(U))$, where U is the set of values that still might be tried for the variable x .
2. Next, **minSize** is the minimum size of the sets **ValuesPerVar**[x] for all unassigned variables.
3. Finally, an arbitrary variable x that has only **minSize** values available is returned.

```
def propagate(x, v, Assignment, Constraints, ValuesPerVar):
    ValuesDict = ValuesPerVar.copy()
    ValuesDict[x] = { v }
    BoundVars = set(Assignment.keys())
    for f, Vars in Constraints:
        if x in Vars:
            UnboundVars = Vars - BoundVars - { x }
            if len(UnboundVars) == 1:
                y = arb(UnboundVars)
                Legal = set()
                for w in ValuesDict[y]:
                    NewAssign = Assignment.copy()
                    NewAssign[x] = v
                    NewAssign[y] = w
                    if eval(f, NewAssign):
                        Legal.add(w)
                if len(Legal) == 0:
                    raise Backtrack()
                ValuesDict[y] = Legal
    return ValuesDict
```

Figure 3.15: Constraint Propagation.

The function **propagate** shown in Figure 3.15 on page 61 implements **constraint propagation**. It takes the following inputs:

- (a) x is a variable and v is a value that is assigned to the variable x .
- (b) **Assignment** is a partial assignment that contains assignments for those variables that are different from the variable x .
- (c) **Constraints** is a set of annotated constraints, i.e. this set contains pairs of the form (f, Vars) , where f is a constraint and **Vars** is the set of variables occurring in f .
- (d) **ValuesPerVar** is a dictionary assigning sets of possible values to all variables.

The purpose of the function **propagate** is to restrict the values of variables different from the variable x by propagating the consequences of setting x to v . To this end the function **propagate** updates the dictionary **ValuesPerVar** by taking into account the consequences of assigning the value v to the variable x . The implementation of **propagate** proceeds as follows.

1. Initially, we copy the Dictionary **ValuesPerVar** to the dictionary **ValuesDict**

2. As x is assigned the value v , the corresponding entry in the dictionary `ValuesDict` is changed accordingly.
3. `BoundVars` is the set of those variable that already have a value assigned.
4. Next, `propagate` iterates over all constraints f such that the variable x occurs in f .
5. `UnboundVars` is the set of those variables occurring in f that are different from x and that do not yet have a value assigned.
6. If there is exactly one unbound variable y in the constraint f , then we can test those values that satisfy f and recompute the set `ValuesDict[x]`.
7. As the set `UnboundVars` contains just a single variable in line 9, the function `arb` returns this variable.
8. In order to recompute the set `ValuesDict[y]`, all values w in `ValuesDict[y]` are tested. The set `Legal` contains all values w that can be assigned to the variable y without violating the constraint f .
9. If it turns out that `Legal` is the empty set, then this means that the constraint f is inconsistent with assigning the value v to the variable x . Hence, in this case the search has to [backtrack](#).
10. Otherwise, the set of admissible values for y is updated to be the set `Legal`.
11. Finally, the dictionary `ValuesDict` is returned.

I have tested the program described in this section using the eight queens puzzle. It takes about 18 milliseconds to find a solution. I have also tested it with the Zebra Puzzle described in a previous exercise. It solves this puzzle in 21 milliseconds. To compare, the backtracking algorithm shown in the previous section takes roughly 10 seconds to solve this puzzle.

3.5 Consistency Checking*

So far, the constraints in the constraints satisfaction problems discussed are either [unary constraints](#) or [binary constraints](#): A [unary](#) constraint is a constraint f such that the formula f contains only one variable, while a [binary](#) constraint contains two variables. If we have a constraint satisfaction problem that involves also constraints that mention more than two variables, then the constraint propagation shown in the previous section is not as effective as it is only used for a constraint f if all but one variable of f have been assigned. For example, consider the [cryptarithmic puzzle](#) shown in Figure 3.16 on page 63. The idea is that the letters “S”, “E”, “N”, “D”, “M”, “O”, “R”, “Y” are interpreted as variables ranging over the set of decimal digits, i.e. these variables can take values in the set $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$. Then, the string “SEND” is interpreted as a decimal number, i.e. it is interpreted as the number

$$S \cdot 10^3 + E \cdot 10^2 + N \cdot 10^1 + D \cdot 10^0.$$

The strings “MORE” and “MONEY” are interpreted similarly. To make the problem interesting, the assumption is that different variables have different values. Furthermore, the digits at the beginning of a number should be different from 0.

A naïve approach to solve this problem would be to code it as a constraint satisfaction problem that has, among others, the following constraint:

$$(S \cdot 10^3 + E \cdot 10^2 + N \cdot 10 + D) + (M \cdot 10^3 + O \cdot 10^2 + R \cdot 10 + E) = M \cdot 10^4 + O \cdot 10^3 + N \cdot 10^2 + E \cdot 10 + Y.$$

$$\begin{array}{r}
 \text{S E N D} \\
 + \text{M O R E} \\
 \hline
 \text{M O N E Y}
 \end{array}$$

Figure 3.16: A cryptarithmic puzzle

The problem with this constraint is that it involves far too many variables. As this constraint can only be checked when all the variables have values assigned to them, the backtracking search would essentially boil down to a mere brute force search. We would have 8 variables that each could take 10 different values and hence we would have to test 10^8 possible assignments. In order to do better, we have to perform the addition shown in Figure 3.16 column by column, just as it is taught in elementary school. Figure 3.17 on page 63 shows how this can be implemented in *Python*.

```

def crypto_csp():
    Digits      = { 'S', 'E', 'N', 'D', 'M', 'O', 'R', 'Y' }
    Variables    = Digits | { 'C1', 'C2', 'C3' }
    Values       = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 }
    Constraints   = allDifferent(Digits)
    Constraints |= { '(D + E) % 10 == Y', '(D + E) // 10 == C1',
                    '(N + R + C1) % 10 == E', '(N + R + C1) // 10 == C2',
                    '(E + O + C2) % 10 == N', '(E + O + C2) // 10 == C3',
                    '(S + M + C3) % 10 == O', '(S + M + C3) // 10 == M'
                  }
    Constraints |= { 'S != 0', 'M != 0' }
    Constraints |= { 'C1 < 2', 'C2 < 2', 'C3 < 2' }
    return Variables, Values, Constraints

def allDifferent(Variables):
    return { f'{x} != {y}' for x in Variables
            for y in Variables
            if x < y
          }

```

Figure 3.17: Formulating “SEND + MORE = MONEY” as a CSP.

Notice that we have introduced three additional variables “C1”, “C2”, “C3”. These variables serve as the **carry digits**. For example, “C1” is the carry digit that we get when we add the final digits of the two numbers, i.e. we have

$$D + E = C1 \cdot 10 + Y.$$

This equation still contains four variables. We can split this equation into two smaller equations that each involve only three variables with the help of the **modulo operator** “%” and the operator for **integer division** “//” as follows:

$$(D + E) \% 10 = Y \quad \text{and} \quad (D + E) // 10 = C1.$$

If we solve the cryptarithmic puzzle as coded in Figure 3.17 on page 63 using the constraint solver developed, then solving the puzzle takes about a second on my computer. The reason is that most constraints involve either three or four variables and therefore the effects of constraint propagation kick only in when many variables have already been initialized. However, we can solve the problem in less than 50 milliseconds if we add the following constraints for the variables “C1”, “C2”, “C3”:

"C1 < 2", "C2 < 2", "C3 < 2".

Although these constraints are certainly true, the problem with this approach is that we would prefer our constraint solver to figure out these constraints by itself. After all, since D and E are both less than 10, their sum is obviously less than 20 and hence the carry C1 has to be less than 2. This line of reasoning is known as **consistency maintenance**: Assume that the formula f is a constraint and the set of variables occurring in f has the form

$$\text{Var}(f) = \{x\} \cup R \quad \text{where } x \notin R,$$

i.e. the variable x occurs in the constraint f and, furthermore, $R = \{y_1, \dots, y_n\}$ is the set of all variables occurring in f that are different from x . In addition, assume that we have a dictionary **ValuesPerVar** such that for every variable y , the dictionary entry **ValuesPerVar**[y] is the set of values that can be substituted for the variable y . The formal definition follows.

Definition 5 (Consistent Value for a Variable) A value v is **consistent** for the variable x with respect to the constraint f iff the partial assignment $\{x \mapsto v\}$ can be extended to an assignment A satisfying the constraint f , i.e. for every variable y_i that is different from x we have to find a value $w_i \in \text{ValuesPerVar}[y_i]$ such that the resulting assignment $A = \{x \mapsto v, y_1 \mapsto w_1, \dots, y_n \mapsto w_n\}$ satisfies the equations

$$\text{eval}(f, A) = \text{True}.$$

Here, the function **eval** takes a formula f and a variable assignment A and evaluates f using this assignment. ◇

Given a CSP $\mathcal{P} = \langle \text{Vars}, \text{Values}, \text{Constraints} \rangle$, the algorithm for **consistency maintenance** is shown below.

1. The dictionary **ValuesPerVar** is initialized as follows:

$$\text{ValuesPerVar}[x] := \text{Values} \quad \text{for all } x \in \text{Variables},$$

i.e. initially every variable x can take any value from the set of **Values**.

2. Next, the set **UncheckedVariables** is initialized to the set of all **Variables**:

$$\text{UncheckedVariables} := \text{Variables}.$$

3. As long as the set **UncheckedVariables** is not empty, we remove one variable x from this set:

$$x := \text{UncheckedVariables.pop}()$$

4. We iterate over all constraints f such that x occurs in f .

- (a) For every value $v \in \text{ValuesPerVar}[x]$ we check whether v is consistent with f .
- (b) If the value v is not consistent with f , then v is removed from **ValuesPerVar**[x]. Furthermore, all variables **connected** to x are added to the set of **UncheckedVariables**. Here we define a variable $y \neq x$ to be connected to x if there is some constraint f such that both x and y occur in f . The reason is that some of their values might have become inconsistent by removing the value v from **ValuesPerVar**[x].

5. Once the set `UncheckedVariables` is empty, the algorithm terminates. Otherwise, we jump back to step 3 and remove the next variable from the set `UncheckedVariables`.

The algorithm terminates as every iteration removes either a variable from the set `UncheckedVariables` or it removes a value from one of the sets `ValuesPerVar[y]` for some variable y . Although the set `UncheckedVariables` can grow during the algorithm, the union

$$\bigcup_{x \in \text{Vars}} \text{ValuesPerVar}[x]$$

can never grow: Every time the set `UncheckedVariables` grows, for some variable x the set

$$\text{ValuesPerVar}[x]$$

shrinks. As the sets `ValuesPerVar[x]` are finite for all variables x , the set `UncheckedVariables` can only grow a finite number of times. Once the set `UncheckedVariables` does not grow any more, every iteration of the algorithm removes one variable from this set and hence the algorithm terminates eventually.

```
def enforce_consistency(ValuesPerVar, Var2Formulas, Annotated, Connected):
    UncheckedVars = set(Var2Formulas.keys())
    while UncheckedVars:
        variable = UncheckedVars.pop()
        RemovedVals = set()
        for f in Var2Formulas[variable]:
            OtherVars = Annotated[f] - { variable }
            for value in ValuesPerVar[variable]:
                if not exists_values(variable, value, f, OtherVars, ValuesPerVar):
                    RemovedVals |= { value }
                    UncheckedVars |= Connected[variable]
        ValuesPerVar[variable] -= RemovedVals
        if len(ValuesPerVar[variable]) == 0: # the problem is unsolvable
            raise Backtrack()
```

Figure 3.18: Consistency maintenance in *Python*.

Figure 3.18 on page 65 shows how consistency maintenance can be implemented in *Python*. The function `enforce_consistency` takes four arguments.

- (a) `ValuesPerVar` is a dictionary associating the set of possible values with each variable.
- (b) `Var2Formulas` is a dictionary. For every variable x , `Var2Formulas[x]` is the set of those constraints f such that x occurs in f .
- (c) `Annotated` is a dictionary mapping constraints to the set of variables occurring in them, i.e. if f is a constraint, then `Annotated[f]` is the set of variables occurring in f .
- (d) `Connected` is a dictionary that takes a variable x and returns the set of all variables that are *connected* to x via a common constraint f , i.e. we have $y \in \text{Connected}[x]$ iff there exists a constraint f such that both x and y occur in f and, furthermore, $x \neq y$.

The function `enforce_consistency` modifies the dictionary `ValuesPerVar` so that once the function has terminated, for every variable x the values in the set `ValuesPerVar[x]` are consistent with the constraints for x . The implementation works as follows:

1. Initially, all variables need to be checked for consistency. Therefore, `UncheckedVars` is defined to be the set of all variables that occur in any of the constraints.
2. The `while`-loop iterates as long as there are still variables x left in `UncheckedVars` such that the consistency of `ValuesPerVar[x]` has not been established.
3. Next, a variable `variable` is selected and removed from `UncheckedVars`.
4. `RemovedVals` is the subset of those values that are found to be `inconsistent` with some constraint for `variable`.
5. We iterate over all constraints $f \in \text{Var2Formulas}[\text{variable}]$.
6. `OtherVars` is the set of variables occurring in f that are different from the chosen variable `variable`.
7. We iterate over all $\text{value} \in \text{ValuesPerVar}[\text{variable}]$ that can be substituted for the variable `variable` and check whether `value` is consistent with f . To this end, we need to find values that can be assigned to the variables in the set `OtherVars` such that f evaluates as `True`. This is checked using the function `exists_values`.
8. If we do not find such values, then `value` is inconsistent for the variable `variable` w.r.t. f and needs to be removed from the set `ValuesPerVar[variable]`. Furthermore, all variables that are connected to `variable` have to be added to the set `UncheckedVars`. The reason is that once a value is removed for the variable `var`, the value assigned to another variable y occurring in a constraint that mentions both `var` and y might now become inconsistent.
9. The set of values that are known to be consistent for `variable` is stored as `ValuesPerVar[variable]`.
10. If there are no consistent values for `variable` left, the problem is unsolvable and an exception is raised.

Figure 3.19 on page 67 shows the implementation of the function `exists_values` that is used in the implementation of `enforce_consistency`. This function is called with five arguments.

- (a) `var` is variable.
- (b) `val` is a value that is to be assigned to `var`.
- (c) f is a constraint such that the variable `var` occurs in f
- (d) `Vars` is the set of all those other variables occurring in f , i.e. the set of those variables that occur in f but that are different from `var`.
- (e) `ValuesPerVar` is a dictionary associating the set of possible values with each variable.

The function checks whether the partial assignment $\{\text{var} \mapsto \text{val}\}$ can be extended so that the constraint f is satisfied. To this end it needs to create the set of all possible assignments. This set is generated using the function `all_assignments`. This function gets a set of variables `Vars` and a dictionary that assigns to every variable `var` in `Vars` the set of values that might be assigned to `var`. It returns a list containing all possible variable assignments. The implementation proceeds as follows:

1. As the argument `Variables` is a `frozenset` but we need to modify this set for the recursive call of `all_assignments`, we transform the `frozenset` into a `set`.

```

def exists_values(var, val, f, Vars, ValuesPerVar):
    Assignments = all_assignments(Vars, ValuesPerVar)
    return any(eval(f, extend(A, var, val)) for A in Assignments)

def extend(A, x, v):
    B = A.copy()
    B[x] = v
    return B

def all_assignments(Variables, ValuesPerVar):
    Variables = set(Variables) # turn frozenset into a set
    if not Variables:
        return [ {} ] # list containing empty assignment
    var = Variables.pop()
    Assignments = all_assignments(Variables, ValuesPerVar)
    return [ extend(A, var, val) for A in Assignments
            for val in ValuesPerVar[var]
            ]

```

Figure 3.19: The implementation of `exists_value`.

2. If the set of variables `Vars` is empty, the empty dictionary can serve as a mapping that assigns a value to every variable in `Vars`.
3. Otherwise, we remove a variable `var` from `Vars` and get the set of `Values` that can be assigned to `var`.
4. Recursively, we create the set of all `Assignments` that associate values with the remaining variables.
5. Finally, the set of all possible assignments is the set of all combinations of assigning a value `val` $\in$ `Values` to `var` and assigning the remaining variables according to an assignment `A` $\in$ `Assignments`. Here we have to make use of the function `extend` that takes a dictionary `A`, a key `x` not occurring in `A` and a value `v` and returns a new dictionary that maps `x` to `v` and otherwise coincides with `A`.

On one hand, consistency checking is a pre-processing step that creates a lot of overhead.¹ Therefore, it might actually slow down the solution of some constraint satisfaction problems that are easy to solve using just backtracking and constraint propagation. On the other hand, many difficult constraint satisfaction problems can not be solved without consistency checking.

Figure 3.20 on page 68 shows how consistency checking is integrated into a constraint solver as a pre-processing step. The procedure `solve(P)` takes a [constraint satisfaction problem](#) *P* as input. The function `solve` converts the CSP *P* into an [augmented](#) CSP where every constraint *f* is annotated with the variables occurring in *f*. Furthermore, the function `solve` maintains the following data structures:

1. `VarsInConstrs` is the set of all variables occurring in any constraint.

¹To be fair, the implementation shown in this section is far from optimal. In particular, by remembering which combinations of variables and values work for a given formula, the overhead can be reduced significantly. I have refrained from implementing this optimization because I did not want the code to get too complex.

```

def solve(P):
    Variables, Values, Constraints = P
    VarsInConstrs = union([ collect_variables(f) for f in Constraints ])
    MisspelledVars = (VarsInConstrs - Variables) | (Variables - VarsInConstrs)
    if MisspelledVars:
        print("Did you misspell any of the following Variables?")
        for v in MisspelledVars:
            print(v)
    ValuesPerVar = { x: Values.copy() for x in Variables }
    Annotated = { f: collect_variables(f) for f in Constraints }
    UnaryConstrs = { (f, V) for f, V in Annotated.items()
                     if len(V) == 1
                     }
    OtherConstrs = { (f, V) for f, V in Annotated.items()
                    if len(V) >= 2
                    }
    Connected = {}
    Var2Formulas = variables_2_formulas(OtherConstrs)
    for x in Variables:
        Connected[x] = union([ V for f, V in Annotated.items()
                               if x in V
                               ]) - { x }

    try:
        for f, V in UnaryConstrs:
            var = arb(V)
            ValuesPerVar[var] = solve_unary(f, var, ValuesPerVar[var])
            enforce_consistency(ValuesPerVar, Var2Formulas, Annotated, Connected)
        for x, Values in ValuesPerVar.items():
            print(f'{x}: {Values}')
        return backtrack_search({}, ValuesPerVar, OtherConstrs)
    except Backtrack:
        return None

```

Figure 3.20: A constraint solver with consistency checking as a preprocessing step.

2. **ValuesPerVar** is a dictionary mapping variables to sets of values. For every variable x occurring in a constraint of P , the expression $\text{ValuesPerVar}(x)$ is the set of values that can be used to instantiate the variable x . Initially, $\text{ValuesPerVar}(x)$ is set to **Values**, but as the search for a solution proceeds, the sets $\text{ValuesPerVar}(x)$ are reduced by removing any values that cannot be part of a solution.
3. **Annotated** is a dictionary. For every constraint f we have that $\text{Annotated}[f]$ is the set of all variables occurring in f .
4. **UnaryConstrs** is a set of pairs of the form (f, V) where f is a constraint containing only a single variable and V is the set containing just this variable.
5. **OtherConstrs** is a set of pairs of the form (f, V) where f is a constraint containing more than one variable and V is the set of all variables occurring in f .

6. **Connected** is a dictionary mapping variables to sets of variables. If x is a variable, then **Connected**[x] is the set of those variables y such that there is a constraint f that mentions both the variable x and the variable y .
7. **Var2Formulas** is a dictionary mapping variables to sets of formulas. For every variable x , **Var2Formulas**[x] is the set of all those non-unary constraints f such that x occurs in f .

After initializing these data structures, the unary constraints are immediately solved. Then the function **enforce_consistency** performs **consistency maintenance**: Formally, we define: A value v is **consistent** for x with respect to the constraint f iff the partial assignment $\{x \mapsto v\}$ can be extended to an assignment A satisfying the constraint f , i.e. for every variable y_i occurring in f there is a value $w_i \in \text{ValuesPerVar}[y_i]$ such that

$$\text{evaluate}(f, \{x \mapsto v, y_1 \mapsto w_1, \dots, y_n \mapsto w_n\}) = \text{True}.$$

The call to **enforce_consistency** shrinks the sets **ValuesPerVars**[x] until all values in **ValuesPerVars**[x] are consistent with respect to all constraints.

Finally, **backtrack_search** is called to solve the remaining constraint satisfaction problem by the means of both **backtracking** and **constraint propagation**.

3.6 Local Search*

There is another approach to solve constraint satisfaction problems. This approach is known as **local search**. The basic idea is simple: Given a constraint satisfaction problem $\mathcal{C}$ of the form

$$\mathcal{P} := \langle \text{Variables}, \text{Values}, \text{Constraints} \rangle,$$

local search works as follows:

1. Use consistency checking as an optional pre-processing step.
2. Initialize the values of the variables in **Variables** randomly.
3. If all **Constraints** are satisfied, return the solution.
4. For every $x \in \text{Variables}$, count the number of **unsatisfied** constraints that involve the variable x .
5. Set **maxNum** to be the maximum of these numbers, i.e. **maxNum** is the maximal number of unsatisfied constraints for any variable.
6. Compute the set **maxVars** of those variables that have **maxNum** unsatisfied constraints.
7. Randomly choose a variable x from the set **maxVars**.
8. Find a value $d \in \text{Values}$ such that by assigning d to the variable x , the number of unsatisfied constraints for the variable x is minimized.
If there is more than one value d with this property, choose the value d randomly from those values that minimize the number of unsatisfied constraints.
9. Rinse and repeat until a solution is found.

```

def solve(P):
    Variables, Values, Constraints = P
    VarsInConstrs = union([ collect_variables(f) for f in Constraints ])
    MisspelledVars = (VarsInConstrs - Variables) | (Variables - VarsInConstrs)
    if MisspelledVars:
        print("Did you misspell any of the following Variables?")
        for v in MisspelledVars:
            print(v)
    ValuesPerVar = { x: Values for x in Variables }
    Annotated = { f: collect_variables(f) for f in Constraints }
    Connected = {}
    Var2Formulas = variables_2_formulas(Annotated)
    for x in Variables:
        Connected[x] = union([V for f, V in Annotated.items() if x in V]) - {x}
    try:
        enforce_consistency(ValuesPerVar, Var2Formulas, Annotated, Connected)
    except Failure:
        return None
    return local_search(Variables, ValuesPerVar, Annotated)

```

Figure 3.21: A constraint solver using local search.

Figure 3.21 on page 70 shows the preprocessing step. The function `solve` takes a constraint satisfaction problem `P` as its argument and performs consistency checking similar to the algorithm discussed in the previous section. Following the preprocessing it calls the function `local_search` that solves the given CSP.

Figure 3.22 on page 71 shows an implementation of `local_search` in *Python*. We proceed to discuss this program line by line.

1. The function `local_search` takes three parameters.
 - (a) `Variables` is the set of all variables occurring in the given CSP.
 - (b) `ValuesPerVar` is a dictionary. For every variable `x`, `ValuesPerVar[x]` is the set of values that can be used to instantiate `x`.
 - (c) `Annotated` is a dictionary. For every constraint `f`, `Annotated[f]` is the set of variables occurring in `f`.

If the computation is successful, `local_search` returns a dictionary that encodes a solution of the given CSP by mapping variables to values.

2. The set `Variables` is turned into a list. This is necessary because the function

```
random.choice(L)
```

that is used to select a random element from `L` expects its argument `L` to be indexable, i.e. for a number $k \in \{0, \dots, \text{len}(L) - 1\}$ the expression `L[k]` needs to be defined.

3. `Assign` is a dictionary mapping all variables from the set `Variables` to values from the set `Values`. Initially the values are assigned randomly.


```

def local_search(Variables, ValuesPerVar, Annotated):
    Variables = list(Variables)
    Assign = { x: random.choice(list(ValuesPerVar[x])) for x in Variables }
    iteration = 0
    lastVar = arb(Variables)
    while True:
        Conflicts = [(numConflicts(x, Assign, Annotated), x) for x in Variables
                     if x != lastVar]

        maxNum, _ = Set.last(cast_to_Set(Conflicts))
        if maxNum == 0 and numConflicts(lastVar, Assign, Annotated) == 0:
            print(f'Number of iterations: {iteration}')
            return Assign
        if iteration % 11 == 0:      # avoid infinite loop
            x = random.choice(Variables)
        else:      # choose var with max number of conflicts
            FaultyVars = [ var for (num, var) in Conflicts if num == maxNum ]
            x = random.choice(FaultyVars)
        if iteration % 13 == 0:      # avoid infinite loop
            newVal = random.choice(list(ValuesPerVar[x]))
        else:
            Conflicts = [ (numConflicts(x, extend(Assign, x, v), Annotated), v)
                          for v in ValuesPerVar[x] ]
            minNum, _ = Set.first(cast_to_Set(Conflicts))
            ValuesForX = [ val for (n, val) in Conflicts if n == minNum ]
            newVal = random.choice(ValuesForX)
        Assign[x] = newVal
        lastVar = x
        iteration += 1

```

Figure 3.22: Implementation of local search.

4. The variable `iteration` counts the number of times that we have changed the assignment `Assign` by reassigning a variable.
5. If we have reassigned a variable x in the last iteration of the loop, then we do not want to reassign it again in the next step since otherwise the program could get stuck in an infinite loop. Therefore, the variable `lastVar` stores the variable that has been reassigned in the previous iteration. We will ensure that in the next iteration step, another variable is chosen for reassignment.
6. At the beginning of the `while` loop, we count the number of conflicts for all variables, i.e. if x is a variable that is different from the variable that has been reassigned in the last iteration, then we count the number of `conflicts` that x causes. This number is defined as the number of constraints f such that
 - (a) x occurs in f and
 - (b) f is not satisfied.

This is done using the function `numConflicts` shown in Figure 3.23 on page 72. The list `Conflicts` defined in line 7 contains pairs of the form (n, x) where x is a variable and n is the number of conflicts that this variable is involved in.

7. In line 10 the list `Conflicts` is turned into a set that is represented as an ordered binary set. This set is effectively a priority queue that is ordered by the number of conflicts. We pick the variable with the most conflicts from this set and store the number of conflicts in `maxNum`, i.e. `maxNum` is the maximum number of conflicts that any variable is involved in.
8. Now if `maxNum` is 0 and additionally the variable `lastVar` that is excluded from the computation of the set `Conflicts` has no conflicts, then the given CSP has been solved and the solution is returned.
9. Otherwise, the list `FaultyVars` defined in line 17 collects those variables that have a maximal number of conflicts.
10. In line 18 we choose a random variable `x` from this list as the variable to be reassigned. However, this is only done ten out of eleven times. In order to avoid running into an infinite loop where we keep changing the same variables, every 11th iteration chooses `x` randomly. This is controlled by the test `iteration % 11 == 0` in line 16.
11. Line 22 computes a list `Conflicts` that this time contains pairs of the form (n, v) where n is the number of conflicts that the variable `x` would cause if we would assign the value v to `x`.
12. Line 25 casts the list `Conflicts` into a set that is represented as an ordered binary tree. This ordered binary tree is used as a priority queue that is ordered by the number of conflicts. We pick the smallest number of conflicts that any value v causes when `x` is assigned to v .
13. `ValuesForX` is the list of those values that cause only `minNum` conflicts when assigned to `x`.
14. `newVal` is a random element from this list that is then assigned to `x`. Again, this is only done twelve out of thirteen times. The 13th time a random value is assigned to `x` instead.
15. In line 29 we remember that we have reassigned `x` in this iteration so that we don't reassign `x` in the next iteration again.

```
def numConflicts(x, Assign, Annotated):
    NewAssign = Assign.copy()
    return len([ (f, V) for (f, V) in Annotated
                  if x in V and not eval(f, NewAssign)
                ])

```

Figure 3.23: The function `numConflicts`.

The function `numConflicts` is shown in Figure 3.23 on page 72. If x is a variable, `Assign` is a variable assignment and `Annotated` is a list of pairs of the form (f, V) where f is a constraint and V is the set of variables occurring in f , then `numConflicts(x, Assign, Annotated)` is the number of conflicts caused by the variable x .

Using the program discussed in this section, the n queens problem can be solved for a $n = 1000$ in 30 minutes. As the memory requirements for local search are small, even much higher problem sizes can be tackled if sufficient time is available. It is a fact that often large problems, which are

not inherently difficult, can be solved much faster with local search than with any other algorithm. However, we have to note that local search is **incomplete**: If a constraint satisfaction problem $\mathcal{P}$ has no solution, then local search loops forever. Therefore, in practise a **dual approach** is used to solve a constraint satisfaction problem. The constraint solver starts two threads: The first search does local search, the second thread tries to solve the problem via some refinement of backtracking. The first thread that terminates wins. The resulting algorithm is complete and, for a solvable problem, will have a performance that is similar to the performance of local search. If the problem is unsolvable, this will **eventually** be discovered by backtracking. Note, however, that the constraint satisfaction problem is **NP-complete**. Hence, it is unlikely that there is an efficient algorithm that works **always**. However, today many practically relevant constraint satisfaction problems can be solved in a reasonably short time.

3.7 Z3

We conclude this chapter with a discussion of the solver **Z3**. Z3 implements most of the state-of-the-art constraint solving algorithms and is exceptionally powerful. We introduce Z3 via a series of examples.

3.7.1 A Simple Text Problem

The following is a simple text problem from my old 8th grade math book.

- *I have as many brothers as I have sisters.*
- *My oldest sister has twice as many brothers as she has sisters.*
- *How many children does my father have?*

However, in order to solve this puzzle we need two additional assumptions.

1. My father has no illegitimate children.
2. All of my fathers children identify themselves as either male or female.

Strangely, in my old math book these assumptions are not mentioned.

In order to infer the number of children we first have to determine whether I am male or female. If I were female, I would have as many brothers as my sister has. Now if my sister would have twice as many brother, this could only be true if I had no brothers. But then I would not have any sisters either and this contradicts the fact that I have an oldest sister. This contradiction shows that I have to be male.

If we denote the number of **boys** with the variable b and the number of **girls** with g , the problem statements are equivalent to the following two equations:

- (a) $b - 1 = g$, since I am not my own brother.
- (b) $2 \cdot (g - 1) = b$ as my sister is not my own sister.

Before we can start to solve this problem, we have to install Z3 via **pip** using the following command in the shell:

```
pip install z3-solver
```

Once we have done this and we have added the directory

```
export PATH="$~/opt/anaconda3/envs/ai/bin/"
```

to the environment variable `PATH`, we can use the file shown in Figure 3.24 to solve the problem. The command to invoke Z3 has the form

```
z3 file.z3
```

where `file.z3` is the name of the file that stores the Z3 specification of the problem.

```
(declare-const b Int)
(declare-const g Int)

(assert (= (- b 1) g))
(assert (= (* 2 (- g 1)) b))

(check-sat)
(get-model)
```

Figure 3.24: Solving a simple text problem with Z3.

- (a) Line 1 and 2 declare the variables `b` and `g` as integer variables.

With Z3 we are not confined to use a finite set of values. Instead we can use integer variables and floating point variables.

The syntax of Z3 files is similar to the syntax of the programming language `lisp`. Later, we will only use the *Python* API of Z3. Therefore, you do not need to worry about this syntax.

- (b) Line 4 specifies the equation $b - 1 = g$ as a constraint.

Note that we have to use prefix notation for all operators.

- (c) Similarly, line 5 specifies the equation $2 * (g - 1) = b$ as a constraint.

- (d) Line 7 asks Z3 to check whether the problem is solvable.

- (e) Line 8 prints the solution of the problem.

If we run this command with the specification shown in Figure 3.24, then we get the output shown below:

```
sat
(
  (define-fun b () Int 4)
  (define-fun g () Int 3)
)
```

The string “`sat`” tells us that the problem is solvable and the following lines show that $b = 4$ and $g = 3$ is the solution, i.e. there are 4 boys and 3 girls.

Instead of using the command line to solve CSPs we will utilize the *Python* interface of Z3. There are two reasons why this is more convenient:

```
import z3

boys = z3.Int('boys')
girls = z3.Int('girls')

S = z3.Solver()

S.add(boys - 1 == girls)
S.add(2 * (girls - 1) == boys)
S.check()
solution = S.model()

b = solution[boys].as_long()
g = solution[girls].as_long()

print(f'My father has {b + g} children.')
```

Figure 3.25: Solving a simple text problem.

1. In an interesting CSP there can easily be hundreds of variables and thousands of constraints. It would be very inconvenient if we had to write these variables and constraints manually into a file.
2. The *Python* interface allows us to extract the solution that has been computed so that we can then proceed to use the values of the solution in our own programs.

The Python program shown in Figure 3.25 solves the text problem given above via the *Python* API of Z3.

1. In line 1 we import the module `z3` so that we can use the Python API of Z3. The documentation of this API is available at the following address:

<https://ericpony.github.io/z3py-tutorial/guide-examples.htm>

2. Lines 3 and 4 creates the Z3 variables `boys` and `girls` as integer valued variables. The function `Int` takes one argument, which has to be a string. This string is the name of the variable. We store these variables in Python variables of the same name. It would be possible to use different names for the Python variables, but that would be very confusing.
3. Line 6 creates an object of the class `Solver`. This is the constraint solver provided by Z3.
4. Lines 8 and 9 add the constraints expressing that the number of girls is one less than the number of boys and that my sister has twice as many brothers as she has sisters as constraints to the solver `S`.
5. In line 10 the method `check` examines whether the given set of constraints is satisfiable. In general, this method returns one of the following results:
 - (a) `sat` is returned if the problem is solvable, (`sat` is short for *satisfiable*)
 - (b) `unsat` is returned if the problem is unsolvable,

- (c) `unknown` is returned if Z3 is not powerful enough to solve the given problem.
6. Since in our case the method `check` returns `sat`, we can extract the solution that is computed via the method `model` in line 11.
 7. In order to extract the values that have been computed by Z3 for the variables `boys` and `girls`, we can use dictionary syntax and write `solution[boys]` and `solution[girls]` to extract these values. However, these values are not stored as integers but rather as objects of the class `IntNumRef`, which is some internal class of Z3 to store integers. This class provides the method `as_long` that converts its object into an integer number.

Exercise 8: Solve the following text problem using Z3.

- (a) A Japanese deli offers both *penguins* and *parrots*.
- (b) A parrot and a penguin together cost 666 bucks.
- (c) The penguin costs 600 bucks more than the parrot.
- (d) **What is the price of the parrot?**

You may assume that the prices of these delicacies are integer valued. ◇

Exercise 9: Solve the following text problem using Z3.

- (a) A train travels at a uniform speed for 360 miles.
- (b) The train would have taken 48 minutes less to travel the same distance if it had been faster by 5 miles per hour.
- (c) **Find the speed of the train!**

Hints:

1. As the speed is a real number you should declare this variable via the Z3 function `Real` instead of using the function `Int`.
2. 48 minutes are four fifth of an hour. The fraction $\frac{4}{5}$ can be represented in Z3 by the expression `Q(4, 5)`.
3. When you formulate the information given above, you will get a system of **non-linear** equations, which is equivalent to a quadratic equation. This quadratic equation has two different solutions. One of these solutions is negative. In order to exclude the negative solution you need to add a constraint stating that the speed of the train has to be greater than zero.
4. The solution will be some real number which is represented internally as an object of type `RatNumRef`. If `o` is an object of this type, then this object can be converted to a string as follows:

```
o.as_decimal(17)
```

Here, 17 is the number of digits following the decimal point. This string can be then converted to a float by using the function `float`. ◇

3.7.2 The Knight's Tour

In this subsection we will solve the puzzle [The Knight's Tour](#) using Z3. This puzzle asks whether it is possible for a knight to visit all 64 squares of the board and return to its starting square in 64 moves. The tour starts in one of the corners of the board.

In order to model this puzzle as a constraint satisfaction problem we first have to decide on the variables that we want to use. The idea is to have 65 variables that describe the position of the knight after its i^{th} move where $i = 0, 1, \dots, 64$. However, it turns out that it is best to split the values of these positions up into a row and a column. If we do this, we end up with 130 variables of the form

$$R_i \text{ and } C_i \quad \text{for } i \in \{0, 1, \dots, 64\}.$$

Here R_i denotes the row of the knight after its i^{th} move, while C_i denotes the corresponding column.

Next, we have to formulate the constraints. In this case, there are two kinds of constraints:

1. We have to specify that the move from the position $\langle R_i, C_i \rangle$ to the position $\langle R_{i+1}, C_{i+1} \rangle$ is legal move for a knight. In chess, there are two ways for a knight to move:
 - (a) The knight can move two squares horizontally left or right followed by moving vertically one square up or down, or
 - (b) the knight can move two squares vertically up or down followed by moving one square left or right.

Figure [3.26](#) shows all legal moves of a knight that is positioned in the square e4. Therefore, a formula that expresses that the i^{th} move is a legal move of the knight is a disjunction of the following eight formulas that each describe one possible way for the knight to move:

- (a) $R_{i+1} = R_i + 2 \wedge C_{i+1} = C_i + 1,$
- (b) $R_{i+1} = R_i + 2 \wedge C_{i+1} = C_i - 1,$
- (c) $R_{i+1} = R_i - 2 \wedge C_{i+1} = C_i + 1,$
- (d) $R_{i+1} = R_i - 2 \wedge C_{i+1} = C_i - 1,$
- (e) $R_{i+1} = R_i + 1 \wedge C_{i+1} = C_i + 2,$
- (f) $R_{i+1} = R_i + 1 \wedge C_{i+1} = C_i - 2,$
- (g) $R_{i+1} = R_i - 1 \wedge C_{i+1} = C_i + 2,$
- (h) $R_{i+1} = R_i - 1 \wedge C_{i+1} = C_i - 2.$

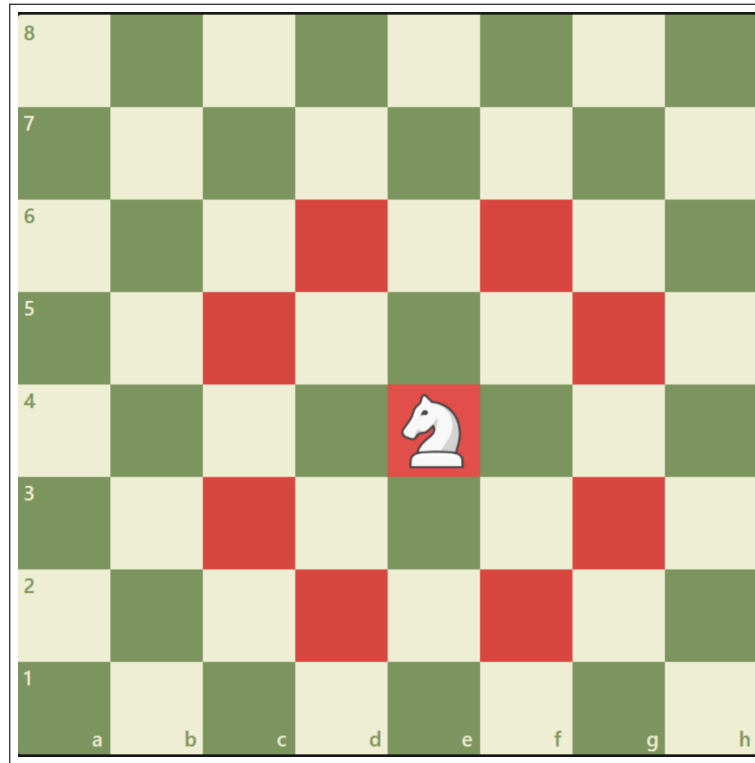
2. Furthermore, we have to specify that the position $\langle R_i, C_i \rangle$ is different from the position $\langle R_j, C_j \rangle$ if $i \neq j$.

Figure [3.27](#) shows how we can formulate the puzzle using Z3.

1. In line 1 we import everything from the library `z3` so that we can write, e.g. `And(x, y)` instead of having to write `z3.And(x, y)`.

The expression `z3.And(x, y)` computes the conjunction of x and y .

2. It is not convenient to declare all of the 130 variables R_i and C_i for $i = 0, 1, \dots, 64$ explicitly. Instead, we will write a function that creates and declares these variables. To implement this function, we define the auxiliary functions `row` and `col` in line 2 and 3. Given a natural number i , the expression `row(i)` returns the string `'Ri'` and `col(i)` returns the string `'Ci'`. These strings in turn represent the variables R_i and C_i .

Figure 3.26: The moves of a knight, courtesy of chess.com.

3. The function `all_variables` returns a set of all variable names.
4. The function `is_knight_move` checks whether the move from position i specified as $\langle R_i, C_i \rangle$ to the position $\langle R_{i+1}, C_{i+1} \rangle$ is a legal move for a knight.
5. The function `all_different` computes a set of formulas that state that the positions $\langle R_i, C_i \rangle$ for $i = 0, 1, \dots, 63$ are all different from each other.
6. The function `all_constraints` computes the set of all constraints. In addition to the constraints already discussed this function specifies that the knight starts its tour at the leftmost topmost corner of the board and that the tour also ends in this corner.

Additionally there are constraints that the variables R_i and C_i are all non-negative. These constraints are needed as we will model the variables with bit vectors of length 4. These bit vectors store integers in [two's complement](#) representation. In two's complement representation of a bit vector of length 4 we can model integers from the set $\{-8, \dots, 7\}$. If we add the number 1 to a 4-bit bit vector v that represents the number 7, then an overflow will occur and the result will be -8 instead of 8. This could happen in the additions that are performed in the formulas computed by the function `is_knight_move`. We can exclude these cases by adding the constraints that all variables are non-negative.

Finally, the function `solve` that is shown in Figure 3.28 on page 80 can be used to solve the puzzle. This function takes two arguments:

- (a) `Constraints` is a set of strings that are interpreted as Z3 constraints.
- (b) `Variables` is a set of strings that are interpreted as variables.

```

from z3 import *
def row(i): return f'R{i}'
def col(i): return f'C{i}'

def all_variables():
    Variables = set()
    for i in range(64+1):
        Variables.add(row(i))
        Variables.add(col(i))
    return Variables

def is_knight_move(i):
    r = row(i)
    c = col(i)
    rX = row(i+1)
    cX = col(i+1)
    Formulas = set()
    for delta_r, delta_c in [(1, 2), (2, 1)]:
        Formulas.add(f'And({rX} == {r} + {delta_r}, {cX} == {c} + {delta_c})')
        Formulas.add(f'And({rX} == {r} + {delta_r}, {cX} + {delta_c} == {c})')
        Formulas.add(f'And({rX} + {delta_r} == {r}, {cX} == {c} + {delta_c})')
        Formulas.add(f'And({rX} + {delta_r} == {r}, {cX} + {delta_c} == {c})')
    return 'Or(' + ', '.join(Formulas) + ')'

def all_different():
    Result = set()
    for i in range(62+1):
        for j in range(i+1, 63+1):
            Result.add(f'Or({row(i)} != {row(j)}, {col(i)} != {col(j)})')
    return Result

def all_constraints():
    Constraints = all_different()
    Constraints.add(f'{row(0)} == 0')
    Constraints.add(f'{col(0)} == 0')
    Constraints.add(f'{row(64)} == 0')
    Constraints.add(f'{col(64)} == 0')
    for i in range(63+1):
        Constraints.add(is_knight_move(i))
    for i in range(64+1):
        Constraints.add(f'{row(i)} >= 0')
        Constraints.add(f'{col(i)} >= 0')
    return Constraints

```

Figure 3.27: The Knight's Tour: Computing the constraints.


```

def solve(Constraints, Variables):
    Environment = {}
    exec('import z3', Environment)
    for v in Variables:
        exec(f'{v} = z3.BitVec(f"{v}", 4)', Environment)
    s = z3.Solver()
    for c in Constraints:
        s.add(eval(c, Environment))
    result = str(s.check())
    if result == 'sat':
        m = s.model()
        S = { v: m[eval(v, Environment)] for v in Variables }
        return S
    elif result == 'unsat':
        print('The problem is not solvable.')
    else:
        print('Z3 cannot determine whether the problem is solvable.')

```

Figure 3.28: The function `solve`.

The purpose of the function `solve` is to find a solution of the given CSP. If successful, it returns a dictionary that maps every variable name to the corresponding value of the solution that has been found.

1. In line 1 we define the dictionary `Environment` which will serve as the local environment for the functions `exec` and `eval` below.
2. We import everything from the package `z3` into this environment in line 3.
3. Then we declare that the strings from the set `Variable` represent `Z3` bit-vector variables of length 4.
4. We create a solver object in line 6 and add the constraints to this solver in the following two lines.
5. The function `check` tries to build a model satisfying the constraints, while the function `model` extracts this model if it exists.
6. Finally, in line 12 we create a dictionary that maps all of our variables to the corresponding values that are found in the model. Note that we have to turn the variable names, that are stored as strings in the set `Variables`, into objects that represent the corresponding `Z3` variables using the function `eval`.

This dictionary is then returned.

Exercise 10: Table 3.1 on page 81 shows a `sudoku` that I have taken from the `Zeit Online` magazine. Solve this Sudoku using `Z3`. I have written a frame for you to use that can be found at

<https://github.com/karlstroetmann/Artificial-Intelligence/blob/master/Python/Chapter-02/Sudoku-Z3-Frame.ipynb>

◇

	3	9						7
			7			4	9	2
				6	5		8	3
			6		3	2	7	
				4		8		
5	6							
		5	2		9			1
	2	1					4	
7						5		

Table 3.1: A super hard sudoku from the magazine “Zeit Online”.

3.7.3 Literature

In this chapter we could only give a glimpse of the theory of constraint satisfaction problems. For further details on the theory of CSPs, consult the book [Constraint Processing](#) by Rina Dechter [[Dec03](#)].

Chapter 4

Playing Games

One major breakthrough for the field of artificial intelligence happened in 1997 when the chess-playing computer **Deep Blue** was able to beat the World Chess Champion **Garry Kasparov** by $3\frac{1}{2}-2\frac{1}{2}$. While **Deep Blue** was based on special hardware, according to the **computer chess rating list** of the 2nd of January 2025, the best version of the chess program **Stockfish** runs on ordinary desktop computers and has an **Elo rating** of 3642. To compare, according to the **Fide** list of January 2025, the best human player (and former World Chess Champion) **Magnus Carlsen** has an Elo rating of 2831. If two players differ by more than 400 in their ELO ranking, the lower ranked player does not stand a chance to win or even draw against the higher ranked player. Hence, Magnus Carlsen wouldn't stand a chance to win or draw a game against Stockfish. In 2017, at the **Future of Go Summit**, the computer program **AlphaGo** was able to beat **Ke Jie**, who was at that time considered to be the best human **Go** player in the world. Besides Go and chess, there are many other games where today the performance of a computer exceeds the performance of human players. To name just one more example, in 2019 the program **Pluribus** was able to **beat** fifteen professional poker players in six-player no-limit **Texas Hold'em poker** resoundingly.¹

This chapter is structured as follows:

- (a) We define the notion of deterministic two player zero sum games in the next section.
- (b) To illustrate this definition we describe the game **tic-tac-toe** in this framework.
- (c) The **minimax algorithm** is a simple algorithm to play games and is described next.
- (d) **Alpha-beta pruning** is an improvement of the minimax algorithm.
- (e) Finally, we consider the case of those games that, due to memory limitations, can not be solved with the pure version of alpha-beta pruning. For these games we discuss **depth-limited adversarial search**.

4.1 Basic Definitions

In order to investigate how a computer can play a game we define a **game** $\mathcal{G}$ as a quintuple

$$\mathcal{G} = \langle \text{States}, s_0, \text{Players}, \text{nextStates}, \text{utility} \rangle$$

where the components are interpreted as follows:

1. **States** is the set of all possible **states** of the game.

We will only consider games where the set **States** is finite.

¹Well-informed circles report that all 15 professional players had to go home stark naked.

2. $s_0 \in \text{States}$ is the **start state**.
3. **Players** is the list of the **players** of the game. The first element in **Players** is the player to start the game and after that the players take turns. As we only consider **two person** games, we assume that **Players** is a list of length two.
4. **nextStates** is a function that takes a state $s \in \text{States}$ and a player $p \in \text{Players}$ and returns the set of states that can be reached if the player p has to make a move in the state s . Hence, the signature of **nextStates** is given as follows:

$$\text{nextStates} : \text{States} \times \text{Players} \rightarrow 2^{\text{States}}.$$

5. **utility** is a function that takes a state s as its argument. If the game is finished, it returns the **value** that the game has for the first player. Otherways, it returns the undefined value Ω . In general, the **value** of a game is a real number, but in all of our examples, this value will be an element from the set $\{-1, 0, +1\}$. If $\text{utility}(s) = -1$, then the first player has lost the game, if $\text{utility}(s) = 1$, then the first player has won the game, and if $\text{utility}(s) = 0$, then the game is drawn. Hence the signature of **utility** is

$$\text{utility} : \text{States} \rightarrow \{-1, 0, +1\} \cup \{\Omega\}.$$

If $\mathcal{G} = \langle \text{States}, s_0, \text{Players}, \text{nextStates}, \text{utility} \rangle$ is a game, we define an auxiliary function **finished** that takes a state s and decides whether the games is finished. Therefore, the signature of **finished** is

$$\text{finished} : \text{States} \rightarrow \mathbb{B}.$$

Here, $\mathbb{B}$ is the set of Boolean values, i.e. we have $\mathbb{B} := \{\text{true}, \text{false}\}$. The definition of **finished** is as follows:

$$\text{finished}(s) := (\text{utility}(s) \neq \Omega).$$

Using the function **finished**, we define the set **TerminalStates** as the set of those states such that the game is finished, i.e. we define

$$\text{TerminalStates} := \{s \in \text{States} \mid \text{finished}(s)\}.$$

We will only consider so called **two person zero sum games**. This means that the list **Players** has exactly two elements. If we call these players A and B, i.e. if we have

$$\text{Players} = [A, B],$$

then the game is called a **zero sum game** if A has won the game if and only if B has lost the game and vice versa. Games like **Go**, **chess**, and **Checkers** are two person zero sum games. We proceed to discuss a simple example.

4.2 Tic-Tac-Toe

The game **tic-tac-toe** is played on a square board of size 3×3 . On every turn, the first player puts an “X” on one of the free squares of the board when it is her turn, while the second player puts an “O” onto a free square when it is his turn. If the first player manages to place three Xs in a row, column, or diagonal, she has won the game. Similarly, if the second player manages to put three Os in a row, column, or diagonal, he is the winner. Otherwise, the game is drawn. In this section we present two different implementations of **tic-tac-toe**:

1. We begin with a naive implementation of tic-tac-toe that is easy to understand but has a high memory footprint.

2. After that, we present an implementation that is based on [bitboards](#) and has only a fraction of the memory requirements of the naive implementation.

4.2.1 A Naive Implementation of Tic-Tac-Toe

```

gPlayers = [ "X", "O" ]
gStart   = tuple( tuple(" " for col in range(3)) for row in range(3))
def to_list(State): return [list(row) for row in State]
def to_tuple(State): return tuple(tuple(row) for row in State)

def empty(State):
    return [ (row, col) for row in range(3)
              for col in range(3)
              if State[row][col] == ' ' ]

def next_states(State, player):
    Empty = empty(State)
    Result = []
    for row, col in Empty:
        NextState = to_list(State)
        NextState[row][col] = player
        Result.append( to_tuple(NextState) )
    return Result

gAllLines = [ [ (row, col) for col in range(3) ] for row in range(3) ] \
              + [ [ (row, col) for row in range(3) ] for col in range(3) ] \
              + [ [ (idx, idx) for idx in range(3) ] ] \
              + [ [ (idx, 2-idx) for idx in range(3) ] ]

def utility(State):
    for Pairs in gAllLines:
        Marks = { State[row][col] for row, col in Pairs }
        if len(Marks) == 1 and Marks != { ' ' }:
            return 1 if Marks == { 'X' } else -1
    for row in range(3):
        for col in range(3):
            if State[row][col] == ' ': # the board is not filled
                return None
    return 0

```

Figure 4.1: A *Python* implementation of tic-tac-toe.

Figure 4.1 on page 84 shows a *Python* implementation of tic-tac-toe.

1. The variable `gPlayers` stores the list of players. Traditionally, we use the characters “X” and “O” to name the players.

2. The variable `gStart` stores the start state, which is an empty board. States are represented as tuples of tuples. If S is a state and $r, c \in \{0, 1, 2\}$, then $S[r][c]$ is the mark in row r and column c . To represent states we have to use immutable data types, i.e. tuples instead of lists, as we need to store states in sets later. The entries in the inner tuples are the characters “X”, “O”, and the blank character “ ”. As the state `gStart` is the empty board, it is represented as a tuple of three tuples containing three blanks each:

```
( (' ', ' ', ' '),
  (' ', ' ', ' '),
  (' ', ' ', ' ')
).
```

3. As we need to manipulate States, we need a function that converts them into lists of lists. This function is called `to_list`.
4. We also need to convert the lists of lists back into tuples of tuples. This is achieved by the function `to_tuple`.
5. Given a state S the function `empty( $S$ )` returns the list of pairs `(row, col)` such that $S[\text{row}][\text{col}]$ is a blank character. These pairs are the coordinates of the fields on the board S that are not yet occupied by either an “X” or an “O”.
6. The function `next_states` takes a `State` and a `player` and computes the list of states that can be reached from `State` if `player` is to move next. To this end, it first computes the set of `empty` positions, i.e. those positions that have not yet been marked by either player. Every position is represented as a pair of the form `(row, col)` where `row` specifies the row and `col` specifies the column of the position. The position `(row, col)` is `empty` in `State` iff

$$\text{State}[\text{row}][\text{col}] = " ".$$

The computation of the empty position has been sourced out to the function `empty`. The function `nextStates` then iterates over these empty positions. For every empty position `(row, col)` it creates a new state `NextState` that results from the current `State` by putting the mark of `player` in this position. The resulting states are collected in the list `Result` and returned.

Note that we had to turn the `State` into a list of lists in order to manipulate it. The manipulated `State` is then cast back into a tuple of tuples.

7. The function `utility` takes a `State` as its argument. If the game is finished in the given `State`, it returns the value that this `State` has for the player “X”. If the outcome of the game is not yet decided, the value `None` is returned instead.

In order to achieve its goal, the procedure first computes the set of all sets of coordinate pairs that either specify a horizontal, vertical, or diagonal line on a 3×3 tic-tac-toe board. Concretely, the variable `gAllLines` has the following value:

```
[ [(0, 0), (0, 1), (0, 2)], [(1, 0), (1, 1), (1, 2)], [(2, 0), (2, 1), (2, 2)],
  [(0, 0), (1, 0), (2, 0)], [(0, 1), (1, 1), (2, 1)], [(0, 2), (1, 2), (2, 2)],
  [(0, 0), (1, 1), (2, 2)], [(2, 0), (1, 1), (0, 2)]
]
```

The first line in this expression gives the sets of pairs defining the rows, the second line defines the columns, and the last line yields the two diagonals. Given a state `State` and a set `Pairs`, the set

```
Marks = { State[row][col] : (row, col) in Pairs }
```

is the set of all marks in the line specified by `Pairs`. For example, if

```
Pairs = { (1, 1), (2, 2), (3, 3) },
```

then `Marks` is the set of marks on the falling diagonal. The game is decided if all entries in the set `Marks` have the value "X" or the value "O". In this case, the set `Marks` has exactly one element which is different from the blank. If this element is "X", then the game is **won** by "X", otherwise the element must be "O" and hence the game has a value of -1 for "X".

If there are any empty squares on the board, but the game has not yet been decided, then the function returns `None`. Finally, if there are no more empty squares left, the game is a **draw**.

The implementation shown so far has one important drawback: Every state needs 256 bytes in memory. This can be checked using the *Python* function `sys.getsizeof`. Therefore, we show a leaner implementation next.

4.2.2 A Bitboard-Based Implementation of Tic-Tac-Toe

If we have to reduce the memory requirements of the states, then we can store the states as integers. The first nine bits of these integers store the position of the Xs, while the next nine bits store the positions of the Os. This kind of representation where a state is coded as a series of bit in an integer is known as a **bitboard**. This is much more efficient than storing states as tuples of tuples of characters. Figure 4.2 on page 87 shows an implementation of tic-tac-toe that is based on a **bitboard**. We proceed to discuss the details of this implementation.

1. When we use bitboards to implement tic-tac-toe it is more convenient to store the players as numbers. The first player X is encoded as the number 0, while the second player O is encoded as the number 1.
2. In the state `gStart`, no mark has been placed on the board. Hence all bits are unset and therefore this state is represented by the number 0.
3. The function `set_bits` takes a list of natural numbers as its argument `Bits`. These numbers specify the bits that should be set. It returns an integer where all bits specified in the argument `Bits` are set to 1 and all other bits are set to 0.
4. The function `set_bit` takes a natural number `n` as its argument. It returns a number where the n^{th} bit is set to 1 and all other bits are set to 0.
5. Given a `state` that is represented as a number, the function `empty(state)` returns the set of indexes of those cells such that neither player X nor player O has placed a mark in the cell.
Note that there are 9 cells on the board. Each of these cells can hold either an 'X' or an 'O'. If the i^{th} cell is marked with a 'X', then the i^{th} bit of `state` is set. If instead the i^{th} cell is marked with an 'O', then the $(9 + i)^{\text{th}}$ bit of `state` is set. If the i^{th} cell is not yet marked, then both the i^{th} bit and the $(9 + i)^{\text{th}}$ bit are 0.
6. Given a `state` and the `player` who is next to move, the function `next_states` computes the set of states that can be reached from `state`. Note that player X is encoded as the number 0, while player O is encoded as the number 1.
7. The global variable `gAllLines` is a list of eight bit masks. These masks can be used to test whether there are three identical marks in a row, column, or diagonal.

```

gPlayers = [0, 1]
gStart = 0

def set_bits(Bits):
    result = 0
    for b in Bits:
        result |= 1 << b
    return result

def next_states(S: State, player: int) -> list[int]:
    Empty = { n for n in range(9)
               if S & ((1 << n) | (1 << (9 + n))) == 0 }
    return [ (S | (1 << (player * 9 + n))) for n in Empty ]

gAllLines = [ set_bits([0,1,2]), set_bits([3,4,5]), set_bits([6,7,8]),
               set_bits([0,3,6]), set_bits([1,4,7]), set_bits([2,5,8]),
               set_bits([0,4,8]), set_bits([2,4,6]) ]

def utility(state):
    for mask in gAllLines:
        if state & mask == mask:
            return 1 # player 'X' has won
        if (state >> 9) & mask == mask:
            return -1 # player 'O' has won
    # 511 == 2**9 - 1 = 0b1_1111_1111
    if (state & 511) | (state >> 9) != 511: # the board is not yet filled
        return None
    return 0 # it's a draw

```

Figure 4.2: Tic-Tac-Toe implemented by a bitboard.

8. The function `utility` takes two arguments:

- (a) `state` is an integer representing the board.
- (b) `player` specifies a player. Here player `X` is encoded as the number 0, while player `O` is encoded as the number 1.

The function returns 1 if `player` has won the game, `-1` if the game is lost for `player`, 0 if it's a draw, and `None` if the game has not yet been decided.

4.3 The Minimax Algorithm

Having defined the notion of a game, our next task is to come up with an algorithm that can play a game. The algorithm that is easiest to implement is the `minimax algorithm`. This algorithm is based on the notion of the `value` of a state. Conceptually, the notion of the `value` of a state is an extension of the notion of the `utility` of a state. While the utility is only defined for terminal states, the value is

defined for all states. Formally, we define a function

$$\text{maxValue} : \text{States} \rightarrow \{-1, 0, +1\}$$

that takes a state $s \in \text{States}$ and returns the value that the state s has for the first player, who tries to maximize the value of the state, provided that both the player p and his opponent play **optimally**. The easiest way to define this function is via recursion. As the **maxValue** function is an extension of the **utility** function, the base case is as follows:

$$\text{finished}(s) \rightarrow \text{maxValue}(s) = \text{utility}(s). \quad (1)$$

If the game is not yet finished, we define

$$\neg \text{finished}(s) \rightarrow \text{maxValue}(s) = \max(\{\text{minValue}(n) \mid n \in \text{nextStates}(s, \text{gPlayers}[0])\}). \quad (2)$$

The reason is that, if the game is not finished yet, the maximizing player **gPlayers**[0] has to evaluate all possible moves. From these, the player will choose the move that maximizes the value of the game for herself. In order to do so, the player computes the set **nextStates**($s, \text{gPlayers}[0]$) of all states that can be reached from the state s in any one move of the player **gPlayers**[0]. Now if n is a state that results from player **gPlayers**[0] making some move, then in state n it is the turn of the other player **gPlayers**[1] to make a move. However, this player is the minimizing player who tries to achieve the state with the minimal value. Hence, in order to evaluate the state n , we have to call the function **minValue** recursively as **minValue**(n). The function **minValue** has the same signature as **maxValue** and is defined by the following recursive equations

1. $\text{finished}(s) \rightarrow \text{minValue}(s) = \text{utility}(s).$
2. $\neg \text{finished}(s) \rightarrow \text{minValue}(s) = \min(\{\text{maxValue}(n) \mid n \in \text{nextStates}(s, \text{gPlayers}[1])\}).$

In the future we will sometimes speak of the **value** function. This name is used as a synonym for the function **maxValue**.

Figure 4.3 on page 89 shows an implementation of the functions **maxValue** and **minValue**. It also shows the function **best_move**. This function takes a **State** such that **X** is to move in this state. It returns a pair (v, s) where s is a state that is optimal for the player **X** and such that s can be reached in one step from **State**. Furthermore, v is the value of this state.

- (a) To this end, it first computes the set **NS** of all states that can be reached from the given **State** in one step if **X** is to move next.
- (b) **bestValue** is the best value that **X** can achieve in the given **State**.
- (c) **BestMoves** is the set of states that **X** can move to and that are optimal for her.
- (d) The function returns randomly one of those states **ns** $\in$ **NS** such that the value of **ns** is optimal, i.e. is equal to **bestValue**. We use randomization here since we want to have more interesting games. If we would always choose the first state that achieves the best value, then our program would always make the same move in a given state. Hence, playing the program would get boring much sooner.

```

def maxValue(State):
    if finished(State):
        return utility(State)
    return max([ minValue(ns) for ns in next_states(State, gPlayers[0]) ])

def minValue(State):
    if finished(State):
        return utility(State)
    return min([ maxValue(ns) for ns in next_states(State, gPlayers[1]) ])

def best_move(State):
    NS      = next_states(State, gPlayers[0])
    bestVal  = maxValue(State)
    BestMoves = [s for s in NS if minValue(s) == bestVal]
    BestState = random.choice(BestMoves)
    return bestVal, BestState

```

Figure 4.3: The Minimax algorithm.

4.3.1 Memoization

Let us consider how many states have to be explored in the case of tic-tac-toe by the minimax algorithm described previously. We have 9 possible moves for player X in the start state, then the player O can respond with 8 moves, then there are 7 moves for player O and so on until in the end player X has only 1 move left. If we disregard the fact that some games are decided after fewer than 9 moves, the functions `maxValue` and `minValue` need to consider a total of

$$9 \cdot 8 \cdot 7 \cdot \dots \cdot 2 \cdot 1 = 9! = 362\,880$$

different moves. However, if we count the number of possibilities of putting 5 Os and 4 Xs on a 3×3 board, we see that there are only

$$\binom{9}{5} = \frac{9!}{5! \cdot 4!} = \frac{9 \cdot 8 \cdot 7 \cdot 6}{1 \cdot 2 \cdot 3 \cdot 4} = 9 \cdot 2 \cdot 7 = 126$$

possibilities, because we only have to count the number of ways there are to put 5 Os on 9 different positions and that number is the same as the number of subsets of five elements from a set of 9 elements. Therefore, if we disregard the fact that some games are decided after fewer than nine moves, there are a factor of $5! \cdot 4! = 2880$ less terminal states than there are possible sequences of moves!

As we have to evaluate not just terminal states but all states, the saving is actually a bit smaller than 2880. The next exercise explores this in more detail.

We can use [memoization](#) to exploit the fact that the number of states is much smaller than the number of possible game sequences. [Figure 4.4](#) on page 90 shows how this can be implemented.

```
gCache = {}

def memoize(f):
    global gCache

    def f_memoized(*args):
        if args in gCache:
            return gCache[args]
        result = f(*args)
        gCache[args] = result
        return result

    return f_memoized

maxValue = memoize(maxValue)
minValue = memoize(minValue)
```

Figure 4.4: Memoization.

1. `gCache` is a dictionary that is initially empty. This dictionary is used as a memory cache by the function `memoize`.
2. The function `memoize` is a second order function that takes a function f as its argument. It creates a `memoized` version of the function f : This memoized version of f , which is called `f_memoized`, first tries to retrieve the value of f from the dictionary `gCache`. If this is successful, the cached value is returned. Otherwise, the function f is called to compute the result. This result is then stored in the dictionary `gCache` before it is returned, as the result of the function `f_memoized`.

In turn, the function `memoize` returns the function `f_memoized`, which is the memoized version of f .

3. In order to use memoization for the minimax algorithm, all that needs to be done is to memoize both the functions `maxValue` and `minValue`. These functions can share the same dictionary `gCache` because `maxValue` is only called for states where **X** has to make the next move, while `minValue` is only called for states where **O** has to make the next move. If this wouldn't be the case, the name of the function would have to be stored in `gCache` also.

```

def play_game(canvas):
    State = gStart
    while True:
        val, State = best_move(State);
        draw(State, canvas, f'For me, the game has the value {val}.')
        if finished(State):
            final_msg(State)
            return
    IPython.display.clear_output(wait=True)
    State = get_move(State)
    draw(State, canvas, '')
    if finished(State):
        IPython.display.clear_output(wait=True)
        final_msg(State)
        return

```

Figure 4.5: The function `play_game`.

Figure 4.5 on page 91 presents the implementation of the function `play_game` that is used to play a game.

1. Initially, `State` is the `startState`.
2. As long as the game is not finished, the procedure keeps running.
3. We assume that the computer goes first.
4. The function `best_move` is used to compute the move of the computer. This resulting state is then displayed.
5. After that, it is checked whether the game is finished.
6. If the game is not yet finished, the user is asked to make his move via the function `get_move`. The state resulting from this move is then returned and displayed.
7. Next, we have to check whether the game is finished after the move of the user has been executed.

In order to better understand the reason for using memoization in the implementation of the functions `maxValue` and `minValue` we introduce the following notion.

Definition 6 (Game Tree) Assume that

$$\mathcal{G} = \langle \text{States}, s_0, \text{Players}, \text{nextStates}, \text{utility} \rangle$$

is a game. Then a **play of length n** is a list of states of the form $[s_0, s_1, \dots, s_n]$ such that

$$s_0 = \text{Start} \quad \text{and} \quad \forall i \in \{0, \dots, n-1\} : s_{i+1} \in \text{nextStates}(s_i, p_i),$$

where the players p_i are defined as follows:

$$p_i := \begin{cases} \text{Players}[0] & \text{if } i \% 2 = 0; \\ \text{Players}[1] & \text{if } i \% 2 = 1. \end{cases}$$

Therefore, p_i is the first element of the list `Players` if i is even and p_i is the second element of this list if i is odd. The **game tree** of the game $\mathcal{G}$ is the set of all possible plays. $\diamond$

The following exercise shows why memoization is so important.

Exercise 11: In **simplified tic-tac-toe** the game only ends when there are no more empty squares left. The player **X** wins if she has more rows, columns, or diagonals of three Xs than the player **O** has rows, columns, or diagonals of three Os. Similarly, the player **O** wins if he has more rows, columns, or diagonals of three Os than the player **X** has rows, columns, or diagonals of three Xs. Otherwise, the game is a draw.

- (a) Derive a formula to compute the size of the game tree of simplified tic-tac-toe.
- (b) Write a short program to evaluate the formula derived in part (a) of this exercise.
- (c) Derive a formula that gives the number of all states of simplified tic-tac-toe.

Notice that this question does not ask for the number of all terminal states but rather asks for all states.

- (d) Evaluate the formula derived in part (c) of this exercise.

Hint: You don't have to do the calculation in your head. $\diamond$

4.4 Alpha-Beta Pruning

In this section we discuss **α - β -pruning**. This is a search technique that can prune large numbers of the search space and thereby increase the efficiency of a game playing program. Figure 4.6 gives a first idea of what α - β -pruning is about. The figure shows the game tree of a game that is finished after four moves, i.e. both players are able to make two moves, that is both players can take it in turns to make two moves. After the second player has made her second move, the game is decided. Contrary to our previous definition, the values of the game are not just elements from the set $\{-1, 0, +1\}$ but instead are natural numbers. The first player, called **Max** has the goal of achieving a big number, while the second player **Min** has the goal to achieve a small number.

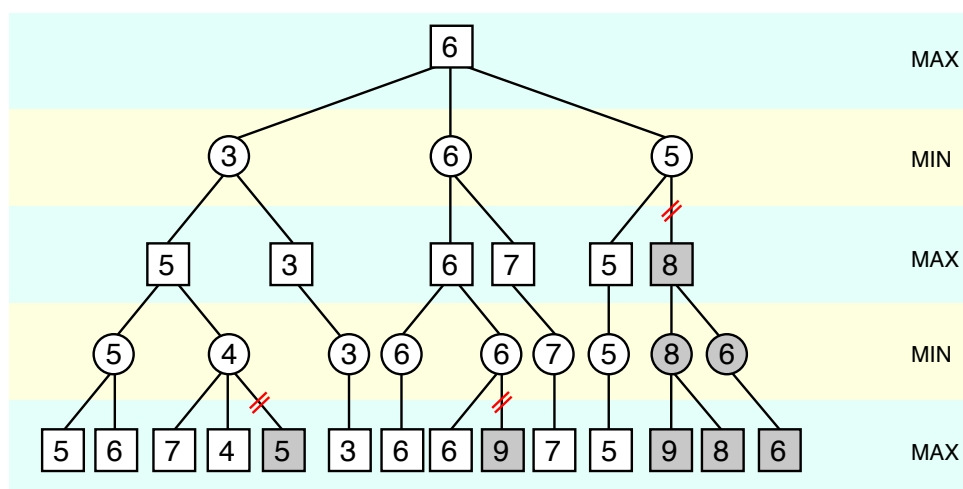


Figure 4.6: Example game tree showing α - β -Pruning. (Original: [Wikipedia](#).)

If we look at the game tree in Figure 4.6 on page 92 we notice that some numbers are greyed out. The reason is that these numbers can not influence the result of the game. If we would replace these numbers with any arbitrary numbers, the overall result of the game would not change if both players play optimally.

1. Let us first take a look at the greyed out node that is marked with a 5 in the bottom-most line. This is the result value of a move from the player **Min**. The parent of this node is marked with a 4. In this node it is the turn of the player **Min**, who will choose the smallest possible value. If the 5 would be replaced by a 0, then **Min** would choose this zero. This would change the node of the parent node, which currently is 4, to be 0. However, this would not change the result of the game because in the parent of the node that is currently marked with a 4 it is the turn of **Max** and **Max** will choose the move leading to his left child because that already guarantees him the value 5.

Similarly, if the value of the greyed out node was 10 instead of 5, the player **Min** would not choose this node because for her the node marked with 4 is a better choice. Hence the value of the node currently marked with a 5 is of no consequence for the overall value of the game and therefore there is no need to evaluate this node at all.

2. The situation is similar for the first node marked with a greyed out 9 in the bottom-most line, because in the grandparent of this node it is the turn of the player **Max** and **Max** is already guaranteed the value of 6 at this node when he chooses the move leading to its left child.
3. At the right part of the tree we have a whole subtree that is greyed out. The root of this subtree is marked with the number 8. Let us understand the reason this subtree does not influence the value of the overall tree.
 - (a) The left sibling of the root node of the greyed out tree is marked with a 5. At the parent of this node, it is the turn of the player **Min**. As **Min** can already choose the node labelled with a 5, we know that the parent node has a value that is at most five.
 - (b) However, the grandparent of this node is the root of the complete tree. In this node, it is the turn of the player **Max**. **Max** can achieve the value 6 by choosing his second child. Therefore the value of this node at least 6.
 - (c) As we have already seen that the value of the rightmost child of the root node can be at most 5, **Max** will never choose this child, but will choose his second child instead.

Therefore, the subtree whose root is marked with a greyed out 8 does not influence the outcome of the game.

To implement α - β -pruning, the basic idea is to provide two additional arguments to the functions `maxValue` and `minValue`. Traditionally, these arguments are called α and β . In order to be able to distinguish between the old functions `maxValue` and `minValue` and its improved version, we call the improved versions `alphaBetaMax` and `alphaBetaMin`. The idea is that these functions are related by the following requirements:

1. As long as `maxValue(s)` is between α and β , the function `alphaBetaMax` computes the same result as the function `maxValue`, i.e. we have

$$\alpha \leq \text{maxValue}(s) \leq \beta \rightarrow \text{alphaBetaMax}(s, \alpha, \beta) = \text{maxValue}(s).$$

2. If `maxValue(s) <  $\alpha$` , we require that the value returned by `alphaBetaMax` is less than or equal to α , i.e. we have

$$\text{maxValue}(s) < \alpha \rightarrow \text{alphaBetaMax}(s, \alpha, \beta) \leq \alpha.$$

3. Similarly, if $\text{maxValue}(s) > \beta$, we require that the value returned by `alphaBetaMax` is bigger than or equal to β , i.e. we have

$$\beta < \text{maxValue}(s) \rightarrow \beta \leq \text{alphaBetaMax}(s, \alpha, \beta).$$

Similar to the way that the function `maxValue` is approximated by the function `alphaBetaMax`, the function `minValue` is approximated by the function `alphaBetaMin`. We have:

1. $\alpha \leq \text{minValue}(s) \leq \beta \rightarrow \text{alphaBetaMin}(s, \alpha, \beta) = \text{minValue}(s).$
2. $\text{minValue}(s) < \alpha \rightarrow \text{alphaBetaMin}(s, \alpha, \beta) \leq \alpha.$
3. $\beta < \text{minValue}(s) \rightarrow \beta \leq \text{alphaBetaMin}(s, \alpha, \beta).$

Although `alphaBetaMax(s)` and `alphaBetaMin(s)` are only [approximations](#) of `maxValue(s)` and `minValue(s)`, it turns out that these approximations are all that is needed. Once the function `alphaBetaMax` is implemented, the function `maxValue` can then be computed as

$$\text{maxValue}(s) := \text{alphaBetaMax}(s, -1, +1).$$

The reason is that we already know that $-1 \leq \text{maxValue}(s) \leq +1$ and hence the first case of the specification of `alphaBetaMax` guarantees that the equation

$$\text{maxValue}(s) = \text{alphaBetaMax}(s, -1, +1)$$

holds. Similarly, the function `minValue` can be computed as

$$\text{minValue}(s) := \text{alphaBetaMin}(s, -1, +1).$$

Figure 4.7 on page 95 shows an implementation of the functions `alphaBetaMax` and `alphaBetaMin` that satisfies the specification given above. Since `alphaBetaMax` and `alphaBetaMin` are implemented as mutually recursive functions, the fact that the implementations of `alphaBetaMax` and `alphaBetaMin` satisfy the specifications given above can be established by computational induction. We proceed perform this proof. The first proof of the correctness of α - β -pruning has been given by Donald E. Knuth and Ronald W. Moore [\[KM75\]](#).

We proceed to discuss the implementation of the function `alphaBetaMax`, which is shown in Figure 4.7 on page 95.

1. If `State` is a terminal state, the function returns the `utility` of the given `State`.
2. We iterate over all successor states `ns` $\in$ `next_states(State, gPlayers[0])`.
3. We have to recursively evaluate the states `ns` with respect to the minimizing player `gPlayers[1]`. Hence we call the function `alphaBetaMin` when evaluating the state `ns`.
4. As the specification of `alphaBetaMax` asks us to compute the value of `State` only in those cases where it is less than or equal to `beta`, once we find a successor state `s` that has a `value` that is at least as big as `beta` we can [stop any further evaluation](#) of the successor states and return `value`.

This shortcut results in significant savings of computation time!

5. Once we have found a successor state that has a `value` greater than `alpha`, we can increase `alpha` to `value`. The reason is, that once we know we can achieve `value` we are no longer interested in any smaller values. This is the reason for assigning the maximum of `value` and `alpha` to `alpha`.

After this assignment, `alpha` will be at least as big as `value` and according to the specification of `alphaBetaMax` we can therefore return `alpha`.

```

1 def alphaBetaMax(State, alpha, beta):
2     if finished(State):
3         return utility(State)
4     for ns in next_states(State, gPlayers[0]):
5         value = alphaBetaMin(ns, alpha, beta)
6         if value >= beta:
7             return value
8         alpha = max(alpha, value)
9     return alpha
10
11 def alphaBetaMin(State, alpha, beta):
12     if finished(State):
13         return utility(State)
14     for ns in next_states(State, gPlayers[1]):
15         value = alphaBetaMax(ns, alpha, beta)
16         if value <= alpha:
17             return value
18         beta = min(beta, value)
19     return beta

```

Figure 4.7: α - β -Pruning.

Claim: The functions `alphaBetaMax` and `alphaBetaMin` both satisfy the specification given previously.

Proof: The proof proceeds by computational induction. However, we will only show that the function `alphaBetaMax` satisfies its specification, as the proof of the correctness of the function `alphaBetaMin` is completely analogous to the proof of the correctness of `alphaBetaMax`. Furthermore, to simplify the proof we define $A := gPlayers[0]$ and $B := gPlayers[1]$.

B.C: $S \in \text{TerminalStates}$, that is $\text{finished}(S) = \text{True}$.

In this case we have $\text{alphaBetaMax}(S, \alpha, \beta) = \text{utility}(S)$ and since in this case we also have $\text{maxValue}(S) = \text{utility}(S)$ this implies

$$\text{alphaBetaMax}(s, \alpha, \beta) = \text{maxValue}(S). \quad (*)$$

We have to check the three cases of our specification one by one.

(a) $\alpha \leq \text{maxValue}(S) \leq \beta$.

We have to show that $\text{alphaBetaMax}(s, \alpha, \beta) = \text{maxValue}(S)$ holds. This follows immediately from (*).

(b) $\text{maxValue}(S) < \alpha$. Because of (*) this implies

$$\text{alphaBetaMax}(s, \alpha, \beta) < \alpha$$

and therefore we also have $\text{alphaBetaMax}(S) \leq \alpha$.

(c) $\beta < \text{maxValue}(S)$. Because of (*) this implies

$$\beta < \text{alphaBetaMax}(s, \alpha, \beta)$$

and therefore we also have $\beta \leq \text{alphaBetaMax}(S, \alpha, \beta)$.

This finishes the proof of the base case.

I.S.: This time $S \notin \text{TerminalStates}$. As we have to check all three cases of our specification, we have to carry out a case distinction.

(a) $\alpha \leq \text{maxValue}(S) \leq \beta$.

We have to show that in this case $\text{alphaBetaMax}(S) = \text{maxValue}(S)$. There are two subcases.

i. If $\text{maxValue}(S) = \beta$ there has to be a node $\text{ns} \in \text{nextStates}(S, A)$ such that

$$\text{minValue}(\text{ns}) = \beta.$$

When the `for`-loop hits this node, the condition of the `if`-statement in line 6 is true and the value of this node, which is β , is returned. Therefore we have

$$\text{alphaBetaMax}(S, \alpha, \beta) = \beta = \text{maxValue}(S).$$

ii. If $\text{maxValue}(S) < \beta$, then we know that for all states $\text{ns} \in \text{nextStates}(S, B)$ we must have

$$\text{minValue}(\text{ns}) < \beta.$$

From the specification and the induction hypothesis it follows that

$$\text{alphaBetaMin}(\text{ns}, \alpha, \beta) < \beta.$$

Furthermore, since $\alpha \leq \text{maxValue}(S)$ there must exist a state $\text{ns} \in \text{nextStates}(S, A)$ such that

$$\alpha \leq \text{minValue}(\text{ns})$$

W.l.o.g. assume that ns is the node with the maximal value, i.e. we have

$$\text{maxValue}(S) = \text{minValue}(\text{ns}).$$

As we already know that $\text{minValue}(\text{ns}) < \beta$, the induction hypothesis tells us that

$$\text{alphaBetaMin}(\text{ns}, \alpha, \beta) = \text{minValue}(\text{ns})$$

Hence the value computed in the `for`-loop is equal to $\text{alphaBetaMin}(\text{ns}, \alpha, \beta)$ and therefore we have

$$\text{alphaBetaMax}(S, \alpha, \beta) = \text{alphaBetaMin}(\text{ns}, \alpha, \beta) = \text{minValue}(\text{ns}) = \text{maxValue}(S).$$

1. $\text{maxValue}(S) < \alpha$

In this case we only have to show that $\text{alphaBetaMax}(S, \alpha, \beta) \leq \alpha$. Since $\text{maxValue}(S) < \alpha$ it must be the case that

$$\text{minValue}(\text{ns}) < \alpha \quad \text{for all } \text{ns} \in \text{nextStates}(S).$$

By induction hypothesis this implies

$$\text{alphaBetaMin}(\text{ns}, \alpha, \beta) \leq \alpha \quad \text{for all } \text{ns} \in \text{nextStates}(S).$$

Since the `for`-loop computes the maximum of these values it follows that

$$\text{alphaBetaMax}(S, \alpha, \beta) \leq \alpha.$$

2. $\beta < \text{maxValue}(S)$

In this case we have to show that $\beta \leq \text{alphaBetaMax}(s, \alpha, \beta)$. The assumption $\beta < \text{maxValue}(S)$ implies that there is node $\text{ns} \in \text{nextStates}(S, A)$ such that

$$\beta < \text{minValue}(\text{ns}).$$

By induction hypothesis we can conclude that

$$\beta \leq \text{alphaBetaMin}(\text{ns}, \alpha, \beta).$$

But then this value (or another one greater or equal than β) is returned by the `if`-statement in line 6 and hence we have

$$\beta \leq \text{alphaBetaMax}(S, \alpha, \beta).$$

As the function `alphaBetaMax` satisfies its specification in all three cases, the proof is complete. $\square$

Remark: There is a nice simulator for alpha-beta-pruning available at the following web address: <https://pascscha.ch/info2/abTreePractice/>.

Exercise 12: The game **Nim** works as follows:

(a) There are four rows of matches:

1. the first row contains 1 match,
2. the second row contains 3 matches,
3. the third row contains 5 matches, and
4. the fourth row contains 7 matches.

(b) The player whose turn it is first selects a line.

Then she removes any number of matches from this line.

(c) The player that removes the last match has won the game.

Implement this game by adapting the notebook

[Artificial-Intelligence/blob/master/Python/Chapter-03/Nim-Frame.ipynb](#).

Then, test the game using the notebook

[Artificial-Intelligence/blob/master/Python/Chapter-03/5-Alpha-Beta-Pruning.ipynb](#). $\diamond$

4.4.1 Alpha-Beta Pruning with Memoization

Adding memoization to the functions `maxValue` and `minValue` is non-trivial. If memoization is added in a naive way, then the cache might have many entries for the same state that differ only in their values for the parameters α and β . Although this is not a problem for trivial games like Tic-Tac-Toe, it becomes a problem once we try to implement more complex games like **Connect Four**. The reason is that for those games we are no longer able to compute the complete game tree. Instead, we need to approximate the value of a state with the help of a heuristic. Then, α and β will no longer be confined to the values from the set $\{-1, 0, 1\}$ but will rather take on a continuous set of values from the interval $[-1, +1]$. Hence there will be many function calls of the form

$$\text{maxValue}(s, \alpha, \beta)$$

where the state s is the same but α and β are different. If we would try to store every combination of s , α , and β we would waste a lot of memory and, furthermore, we would have only a small number of cache hits. Therefore, we will now present a more effective way to cache the functions `maxValue` and `minValue`. The method we describe here is an adaption of the method published by Marsland and Campbell [MC82]. To this end we will define a function `evaluate` that is called as follows:

$$\text{evaluate}(s, f, \alpha, \beta)$$

where the parameters are interpreted as follows:

- (a) s is a state that is to be evaluated.
- (b) f is either the function `alphaBetaMax` or the function `alphaBetaMin`. If in state s the first player has to move, then $f = \text{alphaBetaMax}$, otherwise we have $f = \text{alphaBetaMin}$.
- (c) We interpret the parameters α and β in the same way as we did when we used them with the functions `alphaBetaMax` and `alphaBetaMin`.

The function `evaluate` encapsulates calls to the functions `alphaBetaMax` and `alphaBetaMin`. Given a state s , a function f , and the values of α and β , it first checks whether the value of $f(s, \alpha, \beta)$ has already been computed and is stored in the cache. If this is the case, the value is returned. Otherwise, the function f is called.

The function `evaluate` makes use of a global variable `gCache`. This variable is used as a cache to store the results of the function `evaluate`. This cache is implemented as a dictionary. The keys of this dictionary are the just the states, not triples of the form $\langle s, \alpha, \beta \rangle$. The values stored in the cache are pairs of the form (flag, v) , where v is a value computed by the function `evaluate`(s, f, α, β), while `flag` specifies whether v is exact, a lower bound, or an upper bound. We have

$$\text{flag} \in \{',', ', ' \geq'\}.$$

The cache satisfies the following specification:

1. $\text{gCache}[s] = (',', v) \rightarrow f(s, \alpha, \beta) = v$
If the flag is equal to `'='`, then the value stored in `gCache[s]` is the **exact** value computed for the given state s by the function f .
2. $\text{gCache}[s] = (',\leq', v) \rightarrow f(s, \alpha, \beta) \leq v$
If the flag is equal to `'≤'`, then the value stored in `gCache[s]` is an **upper bound** for the value returned from $f(s, \alpha, \beta)$.
3. $\text{gCache}[s] = (',\geq', v) \rightarrow f(s, \alpha, \beta) \geq v$
If the flag is equal to `'≥'`, then the value stored in `gCache[State]` is a **lower bound** for $f(s, \alpha, \beta)$.

If `gCache[s]` is defined, then the computation of `evaluate`(s, f, α, β) proceeds according to the following case distinction:

1. If the stored value v is exact, we can return this value:
$$\text{gCache}[s] = (',', v) \rightarrow \text{evaluate}(s, f, \alpha, \beta) = v.$$
2. If the stored value v is an upper bound, then there are two cases.
 - (a) If this upper bound v is less or equal than α , then we know that the true value of the state s is less or equal than α and hence we can also return the value v :
$$\text{gCache}[s] = (',\leq', v) \wedge v \leq \alpha \rightarrow \text{evaluate}(s, f, \alpha, \beta) = v.$$
 - (b) Otherwise we can sharpen the upper bound β by setting β to be the minimum of β and v :
$$\text{beta} = \min(\text{beta}, v).$$

If this leads to a reduction of β , then size of the interval $[\alpha, \beta]$ is reduced and hence Alpha-Beta pruning will be able to remove more nodes from the game tree, making the search more efficient.

3. If the stored value v is a lower bound, there are again two cases.

- (a) If this lower bound is greater or equal than β , then we know that the true value is bigger or equal than β and hence we can return the value v :

$$\text{gCache}[s] = (' \geq ', v) \wedge \beta \leq v \rightarrow \text{evaluate}(s, f, \alpha, \beta) = v.$$

- (b) Otherwise, we can sharpen the lower bound α by setting α to be the maximum of α and v :

$$\text{alpha} = \max(\text{alpha}, v)$$

If this leads to an increase of α , then size of the interval $[\alpha, \beta]$ is reduced and hence Alpha-Beta pruning will be able to remove more nodes from the game tree, making the search more efficient.

Finally, we call the function f on the given state with lower bound α and upper bound β and store the value that is computed in the cache via the function `store_cache`. This function has to store both the value and the appropriate flag.

```
def evaluate(State, f, alpha=-1, beta=1):
    global gCache
    if State in gCache:
        flag, v = gCache[State]
        if flag == '=':
            return v
        if flag == '<=':
            if v <= alpha:
                return v
            else:
                beta = min(beta, v)
        if flag == '>=':
            if beta <= v:
                return v
            else:
                alpha = max(alpha, v)
    v = f(State, alpha, beta)
    store_cache(State, alpha, beta, v)
    return v

def store_cache(State, alpha, beta, v):
    global gCache
    if v <= alpha:
        gCache[State] = ('<=', v)
    elif v < beta: # alpha < v
        gCache[State] = ('=', v)
    else: # beta <= v
        gCache[State] = ('>=', v)
```

Figure 4.8: Implementation of the function `evaluate`.

```

1 def maxValue(State, alpha, beta):
2     if finished(State):
3         return utility(State)
4     for ns in next_states(State, gPlayers[0]):
5         value = evaluate(ns, minValue, alpha, beta)
6         if value >= beta:
7             return value
8         alpha = max(alpha, value)
9     return alpha
10
11 def minValue(State, alpha, beta):
12     if finished(State):
13         return utility(State)
14     for ns in next_states(State, gPlayers[1]):
15         value = evaluate(ns, maxValue, alpha, beta)
16         if value <= alpha:
17             return value
18         beta = min(beta, value)
19     return beta

```

Figure 4.9: Cached implementation of the functions `alphaBetaMax` and `alphaBetaMin`.

4.5 Progressive Deepening

In practice, most games are far too complex to be evaluated completely, i.e. the size of the set `States` is so big that even the fastest computer does not stand a chance to explore this set completely. For example, it is believed² that in chess there are about $4.48 \cdot 10^{44}$ different states that could occur in a game. Hence, it is impossible to explore all possible states in chess. Instead, we have to limit the exploration in a way that is similar to the way professional players evaluate their games: Usually, a player considers all variations of a game for, say, the next three moves. After a given number of moves, the value of a position is estimated using an [evaluation heuristic](#). This function [approximates](#) the true value of a given state via a heuristic.

```

def pd_evaluate(State, limit, f=maxValue):
    for l in range(limit+1):
        value = evaluate(State, l, f)
        if value in [-1, 1]:
            return value
    return value

```

Figure 4.10: Progressive Deepening

In order to implement this idea, we add a parameter `limit` to the procedures `alphaBetaMax` and `alphaBetaMin` that were shown in the previous section. On every recursive invocation of the functions

²For reference, compare the Wikipedia article on the so-called [Shannon number](#). The Shannon number estimates that there are at least 10^{120} different plays in chess. However, the number of states is estimated to be about $(4.48 \pm 0.37) \cdot 10^{44}$.

`alphaBetaMax` and `alphaBetaMin`, the parameter `limit` is decreased. Once the limit reaches 0, instead of invoking the function `alphaBetaMax` or `alphaBetaMin` again, we try to estimate the value of the given `State` using an [evaluation heuristic](#). This leads to the code shown in Figure 4.11 on page 103.

When we compare this Figure with Figure 4.7 on page 95, the only difference is in line 4 where we test whether the `limit` is 0. In this case, instead of trying to recursively evaluate the states reachable from `State`, we evaluate the `State` with a `heuristic` function that tries to guess the approximate value of a given state. Notice that in the calls of the function `evaluate` we have to take care to decrease the parameter `limit`. The function `evaluate` is responsible for administering the cache as previously.

There is one further difference between the functions `maxValue` and `minValue` shown in Figure 4.11 and those versions of these functions that were shown previously: In Figure 4.11 the `NextStates` are stored in a priority queue such that the move that is considered to be the best has the highest priority. This way, the best moves are tried first and as a result alpha-beta-pruning is able to prune larger parts of the search space. In order to guess which move is best we use the cached values of the corresponding states. This is the real reason for using progressive deepening: When we evaluate the states with a depth limit of l , we can use the values of the states that has been stored previously when those states were evaluated with a depth limit of $l - 2$. At this point the reader might be surprised: Wouldn't the value computed for a depth limit of $l - 1$ be more accurate than the value for a depth limit of $l - 2$? The answer is no. This can be both verified experimentally and explained theoretically. To understand why the value for the depth of $l - 2$ is better than the value for $l - 1$, let us think about the game of chess. Assume first that we have a depth limit of 1, i.e. we look only one half move into the future. This would result in very aggressive play, i.e. the computer would always try to capture a piece if possible. For example, if the only capture possible would be the queen capturing a pawn, the computer would take this pawn, even if it would loose the queen in the following move because a look ahead depth of 1 is just not sufficient. With a depth limit of 2 the computer would play more defensive. However, when the depth is incremented to 3, the computer would play more aggressive again. Although it would then not sacrifice a queen to capture a pawn, it still would prepare to capture a pawn with the queen in its second move. For this reason, it is usually best to have a depth limit that is even, because if the depth limit is odd, the computer would play too aggressive as it would not be able to see the way in which his last move is answered by its opponent. Therefore, the values stored for a depth limit of $l - 2$ are actually better than those for a depth limit of $l - 1$.

For a game like tic-tac-toe it is difficult to come up with a decent heuristic. A very crude approach would be to define:

```
heuristic := [State, player] |-> 0;
```

This heuristic would simply estimate the value of all states to be 0. As this heuristic is only called after it has been tested that the game has not yet been decided, this approach is not utterly unreasonable. For a more complex game like chess, the heuristic could instead be a [weighted count](#) of all pieces. Concretely, the algorithm for estimating the value of a state would work as follows:

1. Initially, the variable `sum` is set to 0:

```
sum := 0;
```

2. We would count the number of white rooks `Rookwhite` and black rooks `Rookblack`, subtract these numbers from each other and multiply the difference by 5. The resulting number would be added to `sum`:

```
sum += (Rookwhite - Rookblack) · 5;
```

3. We would count the number of white bishops `Bishopwhite` and black bishops `Bishopblack`, subtract these numbers from each other and multiply the difference by 3. The resulting number would

be added to `sum`:

```
sum += (Bishopwhite - Bishopblack) · 3;
```

4. In a similar way we would count knights, queens, and pawns. Approximately, the weights of knights are 3, a queen is worth 9 and a pawn is worth 1.

The resulting `sum` can then be used as an approximation of the value of a state. More details about the weights of the pieces can be found in the Wikipedia article “[chess piece relative value](#)”.

I have tested the program described so far with the game [Connect Four](#). You can play this game online at

<https://connect4.gamesolver.org/en/>

The implementation can be found at:

[Artificial-Intelligence/blob/master/Python/Chapter-03/Connect-Four.ipynb](#)

The heuristic that I have implemented uses [triples](#). A [triple](#) is defined as a set of three marks of either Xs or Os in a row that is followed by a blank space. The blank space could also be between the marks. Now if there is a state s that has a triples of Xs and b triples of Os and the game is not finished, then define

$$\text{value}(s, X, \text{limit}, \alpha, \beta) = \frac{a - b}{10} \quad \text{if } \text{limit} = 0.$$

In a similar way, pairs can be defined as a set of two marks of the same player. These are valued with a factor of $\frac{1}{100}$. Using this heuristic, the resulting game engine is already quite strong when looking 10 moves ahead.

```

def maxValue(State, limit, alpha=-1, beta=1):
    if finished(State):
        return utility(State)
    if limit == 0:
        return heuristic(State)
    value = alpha
    NextStates = next_states(State, gPlayers[0])
    Moves = [] # empty priority queue
    for ns in NextStates:
        val = value_cache(ns, limit-2)
        if val == None:
            val = -1 # unknown values are assumed to be worse than known values
            # heaps are sorted ascendingly, hence the minus
            heapq.heappush(Moves, (-val, ns))
    while Moves != []:
        _, ns = heapq.heappop(Moves)
        value = max(value, evaluate(ns, limit-1, minValue, value, beta))
        if value >= beta:
            return value
    return value

def minValue(State, limit, alpha=-1, beta=1):
    if finished(State):
        return utility(State)
    if limit == 0:
        return heuristic(State)
    value = beta
    NextStates = next_states(State, gPlayers[1])
    Moves = [] # empty priority queue
    for ns in NextStates:
        val = value_cache(ns, limit-2)
        if val == None:
            val = 1
            heapq.heappush(Moves, (val, ns))
    while Moves != []:
        _, ns = heapq.heappop(Moves)
        value = min(value, evaluate(ns, limit-1, maxValue, alpha, value))
        if value <= alpha:
            return value
    return value

def value_cache(State, limit):
    flag, value = gCache.get((State, limit), ('?', None))
    return value

```

Figure 4.11: Depth-limited α - β -pruning.

Chapter 5

Equational Theorem Proving

Mathematics, particularly the field of mathematical theorem proving, is intrinsically linked to the notion of intelligence. [Automated theorem proving](#) represents a significant branch of artificial intelligence dedicated to the application of AI techniques within mathematics. The domain of [automated theorem proving](#) is extensive enough to warrant a book of its own. Due to time constraints, this chapter will focus exclusively on [equational theorem proving](#). In *equational theorem proving*, we start with a collection of axioms, which are expressed as equations, and seek to determine which additional equations can be inferred from these axioms. As an illustration, consider a [group](#) $\mathcal{G}$, defined as a quadruple:

$$\mathcal{G} = \langle G, e, \circ, i \rangle$$

subject to the following conditions:

(a) G is a set, whose elements are referred to as [group elements](#).

(b) $e \in G$, signifying that e is an element of G .

The element e is the [neutral element](#) of $\mathcal{G}$.

(c) $\circ : G \times G \rightarrow G$, indicating that $\circ$ is a binary operation on G that maps pairs of group elements to a group element. This operation is termed the [multiplication](#) of the group $\mathcal{G}$.

(d) $i : G \rightarrow G$, denoting that i is a unary operation on G mapping each group element to another group element. For any element $x \in G$, $i(x)$ is known as the [inverse](#) of x .

(e) The set G , along with the operations defined, must satisfy the [group axioms](#):

- $e \circ x = x$ for all $x \in G$,
- $i(x) \circ x = e$ for all $x \in G$,
- $(x \circ y) \circ z = x \circ (y \circ z)$ for all $x, y, z \in G$.

In [abstract algebra](#), it is shown that these axioms imply the equation

$$x \circ i(x) = e,$$

i.e. the left inverse of any group element x is also a right inverse of x . A possible proof runs as follows:

$$\begin{aligned}
 x \circ i(x) &= e \circ (x \circ i(x)) && \text{because } e \text{ is left-neutral} \\
 &= (i(x \circ i(x)) \circ (x \circ i(x))) \circ (x \circ i(x)) && \text{because } i(x \circ i(x)) \circ (x \circ i(x)) = e \\
 &= i(x \circ i(x)) \circ ((x \circ i(x)) \circ (x \circ i(x))) && \text{associativity} \\
 &= i(x \circ i(x)) \circ \left(x \circ (i(x) \circ (x \circ i(x))) \right) && \text{associativity} \\
 &= i(x \circ i(x)) \circ \left(x \circ ((i(x) \circ x) \circ i(x)) \right) && \text{associativity} \\
 &= i(x \circ i(x)) \circ (x \circ (e \circ i(x))) && \text{because } i(x) \circ x = e \\
 &= i(x \circ i(x)) \circ (x \circ i(x)) && \text{because } e \circ i(x) = i(x) \\
 &= e && \text{because } i(z) \circ z = e \text{ where } z = x \circ i(x).
 \end{aligned}$$

The formulation of proofs for such equations is far from straightforward. However, a systematic method exists for addressing these and related equational challenges. In this chapter, we introduce an algorithm capable of autonomously generating equational proofs similar to those discussed earlier. This algorithm is known as the **Knuth-Bendix completion algorithm**, a significant discovery by Donald E. Knuth and Peter B. Bendix [KB70].

The structure of this chapter is organized as follows:

1. Initially, we will formally define **equational proofs** and **term rewriting**, setting the foundation for subsequent discussions.
2. Subsequently, we explore abstract properties of relations, introducing essential concepts such as **confluence** and providing proofs for the **Church-Rosser theorem** and **Newman's lemma**.
3. The third section delves into term orderings, including the introduction of the **Knuth-Bendix ordering**, which plays an important role in the algorithm.
4. The final section presents the **Knuth-Bendix completion algorithm**.

5.1 Equational Proofs

This section defines the notion of an **equational proof** precisely and discusses how equational proofs can be carried out via **term rewriting**. In order to do this, we have to define a number of more elementary notions like **functions symbols**, **variables**, **terms**, and **substitutions**. We begin with the notion of a signature.

Definition 7 (Signature) A **signature** is a triple of the form

$$\Sigma = \langle \mathcal{V}, \mathcal{F}, \text{arity} \rangle,$$

where we have the following:

1. $\mathcal{V}$ is the set of **variables**.
2. $\mathcal{F}$ is the set of **function symbols**.
3. arity is a function such that

$$\text{arity} : \mathcal{F} \rightarrow \mathbb{N}.$$

If we have $\text{arity}(f) = n$, then f is said to be an **n -ary function symbol**.

4. We have $\mathcal{V} \cap \mathcal{F} = \{\}$, i.e. variables are different from function symbols. $\diamond$

Remark: Compared to the definition given in the lecture on logic, the new definition does not include a set $\mathcal{P}$ of predicate symbols.

Example: The signature of [group theory](#) Σ_G can be defined as follows:

(a) $\mathcal{V} := \{w, x, y, z\}$,

(b) $\mathcal{F} := \{e, i, \circ\}$,

(c) $\text{arity} := \{e \mapsto 0, i \mapsto 1, \circ \mapsto 2\}$,

i.e. e is a constant symbol, i is a unary function symbol, and $\circ$ is a binary function symbol.

(d) $\Sigma = \langle \mathcal{V}, \mathcal{F}, \text{arity} \rangle$. $\diamond$

Having defined the notion of a signature we proceed to define terms.

Definition 8 (Term, $\mathcal{T}_\Sigma$) If $\Sigma = \langle \mathcal{V}, \mathcal{F}, \text{arity} \rangle$ is a signature, the set of [\$\Sigma\$ -terms](#) $\mathcal{T}_\Sigma$ is defined inductively:

1. For every variable $x \in \mathcal{V}$ we have $x \in \mathcal{T}_\Sigma$.
2. If $f \in \mathcal{F}$ and $\text{arity}(f) = 0$, then $f \in \mathcal{T}_\Sigma$.
3. If $f \in \mathcal{F}$ and $n := \text{arity}(f) > 0$ and, furthermore, $t_1, \dots, t_n \in \mathcal{T}_\Sigma$, then we have

$$f(t_1, \dots, t_n) \in \mathcal{T}_\Sigma. \quad \diamond$$

Example: Given the signature Σ_G defined above, we have the following:

1. $x \in \mathcal{T}_{\Sigma_G}$,
because every variable is a Σ_G -term.
2. $e \in \mathcal{T}_{\Sigma_G}$.
3. $\circ(e, x) \in \mathcal{T}_{\Sigma_G}$.
4. $\circ(\circ(x, y), z) \in \mathcal{T}_{\Sigma_G}$.

Remark: Later on we will often use an [infix notation](#) for binary function symbols. In general, if f is a binary function symbol, then the term $f(t_1, t_2)$ is written as $t_1 f t_2$. If this notation would result in an ambiguity because either t_1 or t_2 is also written in infix notation, then we use parenthesis to resolve the ambiguity. For example, we will write

$$(x \circ y) \circ z \quad \text{instead of} \quad \circ(\circ(x, y), z) \in \mathcal{T}_{\Sigma_G}.$$

Note that we cannot write the term $\circ(\circ(x, y), z)$ as $x \circ y \circ z$ because that notation is ambiguous, since it can be interpreted as either $(x \circ y) \circ z$ or $x \circ (y \circ z)$. $\diamond$

Definition 9 (Σ -Equation) Assume a signature Σ is given. A [\$\Sigma\$ -equation](#) is a pair $\langle s, t \rangle$ such that both s and t are Σ -terms. The Σ -equation $\langle s, t \rangle$ is written as

$$s \approx t. \quad \diamond$$

Remark: We use the notation $s \approx t$ instead of the notation $s = t$ in order to distinguish between the notion of a Σ -equation and the notion of equality of terms. So when s and t are Σ -terms and we write $s = t$ we do mean that s and t are literally the same terms, while writing $s \approx t$ means that we are interested in the logical consequences that would follow from the assumption that the interpretation of s and t are the same in certain Σ -algebras. The notion of a Σ -algebra is defined next. $\diamond$

Definition 10 (Σ -Algebra) Assume a signature $\Sigma = \langle \mathcal{V}, \mathcal{F}, \text{arity} \rangle$ is given. A Σ -algebra¹ is a pair of the form $\mathfrak{A} = \langle A, \mathcal{J} \rangle$ where:

1. A is a nonempty set that is called the **universe** of the Σ -algebra $\mathfrak{A}$.
2. $\mathcal{J}$ is the **interpretation** of the function symbols. Technically, $\mathcal{J}$ is a function that is defined on the set $\mathcal{F}$ of all function symbols. For every function symbol $f \in \mathcal{F}$ we have that

$$\mathcal{J}(f) : A^{\text{arity}(f)} \rightarrow A,$$

i.e. $\mathcal{J}(f)$ is a function from A^n to A where n is the arity of the function symbol f .

If $\mathfrak{A} = \langle A, \mathcal{J} \rangle$ is a Σ -algebra, then the function $\mathcal{J}(f)$ is usually written more concisely as $f^{\mathfrak{A}}$.

The set of all Σ -algebras is written as $\text{Alg}(\Sigma)$. $\diamond$

Example: In this example we construct a Σ_G -algebra where Σ_G is the signature of group theory defined earlier. We define $G := \{0, 1\}$ and define the interpretations $\mathcal{J}(f)$ for $f \in \{e, i, \circ\}$ as follows:

1. $\mathcal{J}(e) := 0$.
2. $\mathcal{J}(i) := \{0 \mapsto 0, 1 \mapsto 1\}$.
3. $\mathcal{J}(\circ) := \{\langle 0, 0 \rangle \mapsto 0, \langle 0, 1 \rangle \mapsto 1, \langle 1, 0 \rangle \mapsto 1, \langle 1, 1 \rangle \mapsto 0\}$.

Then $\mathfrak{G} = \langle G, \mathcal{J} \rangle$ is a Σ_G -algebra. $\diamond$

Remark: Alternatively, we could have given the interpretation of the multiplication symbol $\circ$ as

$$\mathcal{J}(\circ)(x, y) := (x + y) \% 2.$$

Definition 11 (Variable Assignment)

If $\Sigma = \langle \mathcal{V}, \mathcal{F}, \text{arity} \rangle$ is a signature and $\mathfrak{A} = \langle A, \mathcal{J} \rangle$ is a Σ -algebra, then a **variable assignment** is a function of the form

$$I : \mathcal{V} \rightarrow A$$

that is the variable assignment I maps every variable $v \in \mathcal{V}$ to a value in the set A .

Definition 12 (Evaluation, Valid Equation)

If $\Sigma = \langle \mathcal{V}, \mathcal{F}, \text{arity} \rangle$ is a signature, $\mathfrak{A} = \langle A, \mathcal{J} \rangle$ is a Σ -algebra, and I is a **variable assignment**, then we can **evaluate** Σ -terms in $\mathfrak{A}$ as follows:

1. $\text{eval}(x, I) := I(x)$ for all $x \in \mathcal{V}$.
2. $\text{eval}(c, I) := c^{\mathfrak{A}}$ for every constant symbol $c \in \mathcal{F}$.

¹The notion of a Σ -algebra is a notion that is used both in logic and in **universal algebra**. In universal algebra, a Σ -algebra is also known as an **algebraic structure**. This notion is not related to and should not be confused with the notion of a **σ -algebra**, which is a notion used in the field of **measure theory**. Note that the notion used in measure theory is always written with a lower case σ , while the notion used in logic is written with a capital Σ .

$$3. \text{eval}(f(t_1, \dots, t_n), I) := f^{\mathfrak{A}}(\text{eval}(t_1, I), \dots, \text{eval}(t_n, I)).$$

A Σ -equation $s \approx t$ is **valid** in the Σ -algebra $\mathfrak{A}$ iff we have

$$\text{eval}(s, I) = \text{eval}(t, I) \text{ for all variable assignments } I : \mathcal{V} \rightarrow A.$$

This is written as

$$\mathfrak{A} \models s \approx t$$

and we say that $\mathfrak{A}$ **satisfies** the equation $s \approx t$. ◇

Example: Continuing the previous example we have the following:

$$1. \mathfrak{G} \models e \circ x \approx x,$$

$$2. \mathfrak{G} \models i(x) \circ x \approx e,$$

$$3. \mathfrak{G} \models (x \circ y) \circ z \approx x \circ (y \circ z). \quad \diamond$$

Definition 13 (E -Variety) Assume that Σ is a signature and E is a set of Σ -equations. The collection of all Σ -algebras that satisfy every equation from E is called the **E -variety**.

$$\text{Variety}(E) := \{\mathfrak{A} \in \text{Alg}(\Sigma) \mid \forall (s \approx t) \in E : \mathfrak{A} \models s \approx t\}.$$

To put it differently, the Σ -structure $\mathfrak{A}$ is a member of $\text{Variety}(E)$ iff

$$\mathfrak{A} \models s \approx t \quad \text{for every equation } s \approx t \text{ in } E. \quad \diamond$$

Example: Define $E := \{e \circ x = x, i(x) \circ x = e, (x \circ y) \circ z = x \circ (y \circ z)\}$. This set of equations defines the variety of **groups**. You can check that the Σ_G -algebra $\mathfrak{G}$ is a member of this variety and hence it is a group. ◇

Given a set of Σ -equations E it is natural to ask which other equations are **logical consequences** of the equations in E . This notion is defined below.

Definition 14 (Logical Consequence) Assume a signature Σ and a set E of Σ -equations to be given. Then the equation $s \approx t$ is a **logical consequence** of E iff we have

$$\mathfrak{A} \models s \approx t \quad \text{for every } \mathfrak{A} \in \text{Variety}(E).$$

If $s \approx t$ is a logical consequence of the set of equations E , then this is written as

$$E \models s \approx t.$$

Therefore we have $E \models s \approx t$ if and only if every Σ -algebra that satisfies all equations from E also satisfies the equation $s \approx t$. ◇

Example: In the introduction of this chapter we have already seen that if we define

$$E := \{e \circ x = x, i(x) \circ x = e, (x \circ y) \circ z = x \circ (y \circ z)\},$$

then we have

$$E \models x \circ i(x) \approx e. \quad \diamond$$

The notion $E \models s \approx t$ is a **semantic notion**. We cannot hope to implement this notion directly because if a set of equations E and a possible logical consequence $s \approx t$ is given, there are, in general, infinitely many Σ -algebras that have to be checked. Fortunately, the notion $E \models s \approx t$ has a **syntactical analog** $E \vdash s \approx t$ (read: E proves $s \approx t$) that can be implemented and that is at least **semi-decidable**,

i.e. we can create a program that given a set of equations E and an equation $s \approx t$ will return **True** if $E \vdash s \approx t$ holds, and will either return **False** or run forever if $E \vdash s \approx t$ does not hold. Even more fortunately, **Gödel's completeness theorem** implies that the syntactical notion coincides with the semantic notion, i.e. we have

$$E \models s \approx t \quad \text{if and only if} \quad E \vdash s \approx t.$$

5.1.1 A Calculus for Equality

In this subsection we assume a signature Σ and a set of Σ -equations E to be given. Our goal is to define the provability notion $E \vdash s \approx t$, which is read as E proves $s \approx t$. However, in order to do this we first need to define the notion of a **substitution**.

Definition 15 (Σ -Substitution) Assume that a signature $\Sigma = \langle \mathcal{V}, \mathcal{F}, \text{arity} \rangle$ is given. A Σ -substitution σ is a map of the form

$$\sigma : \mathcal{V} \rightarrow \mathcal{T}_\Sigma$$

such that the set $\text{dom}(\sigma) := \{x \in \mathcal{V} \mid \sigma(x) \neq x\}$ is finite. If we have $\text{dom}(\sigma) = \{x_1, \dots, x_n\}$ and $t_i = \sigma(x_i)$ for all $i = 1, \dots, n$, then we use the following notation:

$$\sigma = \{x_1 \mapsto t_1, \dots, x_n \mapsto t_n\}.$$

The set of all Σ -Substitutions is denoted as $\text{Subst}(\Sigma)$. ◇

A substitution $\sigma = \{x_1 \mapsto t_1, \dots, x_n \mapsto t_n\}$ can be **applied** to a term t by replacing the variables x_i with the terms t_i . We will use the postfix notation $t\sigma$ to denote the **application** of the substitution σ to the term t . Formally, the notation $t\sigma$ is defined by induction on t :

1. $x\sigma := \sigma(x)$ for all $x \in \mathcal{V}$.
2. $c\sigma = c$ for every constant $c \in \mathcal{F}$.
3. $f(t_1, \dots, t_n)\sigma := f(t_1\sigma, \dots, t_n\sigma)$.

Definition 16 ($E \vdash s \approx t$)

Given a signature Σ and a set of Σ -equations E , the notion $E \vdash s \approx t$ is defined inductively.

1. $E \vdash s \approx t$ for every Σ -equations $(s \approx t) \in E$. (Axioms)
2. $E \vdash s \approx s$ for every Σ -term s . (Reflexivity)
3. If $E \vdash s \approx t$, then $E \vdash t \approx s$. (Symmetry)
4. If $E \vdash r \approx s$ and $E \vdash s \approx t$, then $E \vdash r \approx t$. (Transitivity)
5. If $\text{arity}(f) = n$ and $E \vdash s_i \approx t_i$ for all $i \in \{1, \dots, n\}$,
then $E \vdash f(s_1, \dots, s_n) \approx f(t_1, \dots, t_n)$. (Congruence)
6. If $E \vdash s \approx t$ and σ is a Σ -substitution, then $E \vdash s\sigma \approx t\sigma$. (Stability)

We read $E \vdash s \approx t$ as E proves $s \approx t$. ◇

The definition of $E \vdash s \approx t$ given above is due to **Gottfried Wilhelm Leibniz**.

5.1.2 Term Rewriting

It turns out that, although it is possible to implement the notion $E \vdash s \approx t$ directly, it is much more efficient to refine this notion a little bit. To this end we need to introduce the notion of a **position** u in a term t , the notion of the **subterm** of a given term t at a given position, and the notion of **replacing** a subterm at a given position by another term. We define these notions next.

Definition 17 (Positions of a Term t , $\mathcal{Pos}(t)$)

Given a term t the set of all **positions** of t is written as $\mathcal{Pos}(t)$ and is defined by induction on t :

1. $\mathcal{Pos}(x) := \{[]\}$ for every variable x .
2. $\mathcal{Pos}(c) := \{[]\}$ for every constant c .
3. $\mathcal{Pos}(f(t_1, \dots, t_n)) := \{[]\} \cup \bigcup_{i=1}^n \{[i] + u \mid u \in \mathcal{Pos}(t_i)\}$ for every term $f(t_1, \dots, t_n)$. ◇

Definition 18 (Subterm at a given Position, t/u)

Given a term t and a position $u \in \mathcal{Pos}(t)$, the **subterm of t at position u** is written as t/u . This expression is defined by induction on u .

1. $t/[] := t$,
2. $f(t_1, \dots, t_n)/([i] + u) := t_i/u$. ◇

Definition 19 (Subterm Replacement, $t[u \mapsto s]$)

Given a term t , a position $u \in \mathcal{Pos}(t)$, and a term s , the expression $t[u \mapsto s]$ denotes the term that results from t when the subterm t/u is replaced by the term s . This expression is defined by induction on u .

1. $t/[] \mapsto s := s$,
2. $f(t_1, \dots, t_n)/([i] + u \mapsto s) := f(t_1, \dots, t_{i-1}, t_i[u \mapsto s], t_{i+1}, \dots, t_n)$. ◇

Example: Define $t := (x \circ y) \circ z$. Then we have

$$\mathcal{Pos}((x \circ y) \circ z) = \{[], [1], [1, 1], [1, 2], [2]\}.$$

Furthermore, we have the following:

1. $((x \circ y) \circ z)/[] = (x \circ y) \circ z$,
2. $((x \circ y) \circ z)/[1] = x \circ y$,
3. $((x \circ y) \circ z)/[1, 1] = x$,
4. $((x \circ y) \circ z)/[1, 2] = y$,
5. $((x \circ y) \circ z)/[2] = z$.

We also have

$$((x \circ y) \circ z)/[1] \mapsto y \circ x = (y \circ x) \circ z. \quad \diamond$$

Definition 20 ($\leftrightarrow_E$) Given a set of Σ -equations E and two Σ -terms s and t we define that

$$s \leftrightarrow_E t$$

holds if and only if the following conditions are satisfied:

(a) There exists an equation $l \approx r$ such that either $(l \approx r) \in E$ or $(r \approx l) \in E$.

(b) There is a position $u \in \text{Pos}(s)$ and a substitution σ such that $s/u = l\sigma$.

(c) $t = s[u \mapsto r\sigma]$. ◇

To put this in words: We have $s \leftrightarrow_E t$ iff there is an equation $l \approx r$ such that either the equation $l \approx r$ or the equation $r \approx l$ is an element of the set of equations E and, furthermore, s contains the subterm $l\sigma$ and t results from s by replacing the subterm $l\sigma$ with the subterm $r\sigma$.

Example: If we have $E = \{i(x) \circ x \approx e\}$ then

$$(i(a) \circ a) \circ b \leftrightarrow_E e \circ b$$

because the right hand side $e \circ b$ results from the left hand side $(i(a) \circ a) \circ b$ by replacing the subterm $i(a) \circ a$ that occurs at position [1] by the term e . This is possible because the equation $i(x) \circ x \approx e$ tells us that $i(a) \circ a$ is equal to e . ◇

Next, we define the relation $\leftrightarrow_E^*$ as the [reflexive and transitive closure](#) of the relation $\leftrightarrow_E$.

Definition 21 ($\leftrightarrow_E^*$) For Σ -terms s and t the notion $s \leftrightarrow_E^* t$ is defined inductively as follows:

1. We have $s \leftrightarrow_E^* s$ for every Σ -term s .
2. If $s \leftrightarrow_E t$, then $s \leftrightarrow_E^* t$.
3. If u is a Σ -term such that both $s \leftrightarrow_E u$ and $u \leftrightarrow_E^* t$ holds, then we have $s \leftrightarrow_E^* t$. ◇

Given this definition it is now possible to show the following theorem.

Theorem 22 If E is a set of equations and $s \approx t$ is an equation, then we have

$$E \vdash s \approx t \quad \text{if and only if} \quad s \leftrightarrow_E^* t.$$

For each of the two directions that has to be proven, the proof can be done by a straightforward, but tedious induction.

5.1.3 Proofs via Term Rewriting

The implementation of the relation $\leftrightarrow_E$ remains inefficient due to the dual utility of each equation within E . Specifically, for an equation $l \approx r$ contained in E , it can be applied in two directions: we can replace a subterm matching $l\sigma$ with $r\sigma$ utilizing the equation from left to right, or conversely, substitute a subterm resembling $r\sigma$ with $l\sigma$ by applying the equation from right to left. The seminal insight of Donald E. Knuth and Peter B. Bendix [KB70] was to recognize that if the equations could be ordered such that the left-hand side is consistently more complex than the right-hand side, then it would be feasible to apply these equations in a singular direction by incorporating certain supplementary equations into E throughout this procedure. This approach necessitates the subsequent definition.

Definition 23 (Rewrite Order)

A binary relation $\prec \subseteq \mathcal{T}_\Sigma \times \mathcal{T}_\Sigma$ is a [rewrite order](#) if and only if we have the following:

1. $\prec$ is a [strict partial order](#) on $\mathcal{T}_\Sigma$, i.e. we have
 - (a) $\neg(s \prec s)$ for all $s \in \mathcal{T}_\Sigma$, i.e. $\prec$ is [irreflexive](#).
 - (b) $r \prec s \wedge s \prec t \Rightarrow r \prec t$ for all $r, s, t \in \mathcal{T}_\Sigma$, i.e. $\prec$ is [transitive](#).

2. $\prec$ is **stable under substitutions**, i.e. we have

$$r \prec l \Rightarrow r\sigma \prec l\sigma \quad \text{for every substitution } \sigma.$$

3. $\prec$ is a **congruence**, i.e. we have

$$r \prec l \Rightarrow s[u \mapsto r] \prec s[u \mapsto l] \quad \text{for every } \Sigma\text{-term } s \text{ and every } u \in \mathcal{Pos}(s).$$

4. The relation $\prec$ is **well-founded**, i.e. there is no infinite sequence of the form $(s_n)_{n \in \mathbb{N}}$ such that we have

$$s_{n+1} \prec s_n \quad \text{for all } n \in \mathbb{N}.$$

If E is a set of equations, then a binary relation $\prec \subseteq \mathcal{T}_\Sigma \times \mathcal{T}_\Sigma$ is a **rewrite order w.r.t. E** if, in addition to being a rewrite order, it satisfies

$$r \prec l \quad \text{for every equation } (l \approx r) \in E.$$

This means that all equations in E are ordered such that the right hand side is smaller than the left hand side w.r.t. $\prec$. $\diamond$

Later we will introduce the Knuth-Bendix order as an example of a rewrite order.

In the following let us assume that a binary relation $\prec$ on terms is given and let us, furthermore, assume that this relation is a rewrite order with respect to a some set of equations R . We will call these equations **rewrite rules**. We proceed to define the relation $\rightarrow_R$ on the set of Σ -terms $\mathcal{T}_\Sigma$.

Definition 24 (Rewrite Relation $\rightarrow_R$)

Given a set of rewrite rules R and two Σ -terms s and t we define that

$$s \rightarrow_R t \quad (\text{read: } s \text{ rewrites to } t)$$

if and only if there exists a rewrite rule $(l \approx r) \in R$ such that the following conditions are satisfied:

- (a) There is a position $u \in \mathcal{Pos}(s)$ and a substitution σ such that $s/u = l\sigma$.
- (b) $t = s[u \mapsto r\sigma]$. $\diamond$

To put it differently, we have $s \rightarrow_R t$ if there is a rewrite rule $l \approx r$ in R and the left hand side l of this rewrite rule matches a subterm s/u of s via a substitution σ . If replacing this subterm by $r\sigma$ results in t , then s rewrites to t .

Similar to the definition of $\leftrightarrow_E^*$ we next define $\rightarrow_R^*$ as the reflexive and transitive closure of $\rightarrow_R$.

Definition 25 ($\rightarrow_R^*$) For Σ -terms s and t the notion $s \rightarrow_R^* t$ is defined inductively as follows:

- 1. We have $s \rightarrow_R^* s$ for every Σ -term s .
- 2. If $s \rightarrow_R t$, then $s \rightarrow_R^* t$.
- 3. If u is a Σ -term such that both $s \rightarrow_R u$ and $u \rightarrow_R^* t$ holds, then we have $s \rightarrow_R^* t$. $\diamond$

Definition 26 (Normal Form)

A Σ -term s is in **normal form** w.r.t. a rewrite relation $\rightarrow_R$ iff there is no Σ -term t such that $s \rightarrow_R t$, i.e. the term s cannot be simplified anymore by rewriting. $\diamond$

The basic idea of a **rewrite proof** of an equation $s \approx t$ is now the following:

1. We rewrite s using the rewrite rules from R into a term $\widehat{s}$ that is in normal form:

$$s \rightarrow_R s_1 \rightarrow_R s_2 \rightarrow_R \cdots \rightarrow_R s_m = \widehat{s}$$

2. Similarly, we rewrite t using the rewrite rules from R into a term $\widehat{t}$ that is in normal form:

$$t \rightarrow_R t_1 \rightarrow_R t_2 \rightarrow_R \cdots \rightarrow_R t_n = \widehat{t}.$$

3. If the relation $s \rightarrow t$ is **confluent** (this notion is defined in the next section), then we have

$$s \leftrightarrow_E t \quad \Leftrightarrow \quad \widehat{s} = \widehat{t}.$$

By this method, the question of whether $s \leftrightarrow_E^* t$ holds can be simplified to the computation of normal forms, which is often achievable with high efficiency. The subsequent sections of this chapter are structured as follows:

- (a) The forthcoming section explores the concept of **confluence** and establishes a theorem instrumental in demonstrating the confluence of a relation.
- (b) Subsequently, we investigate rewrite orderings. Specifically, we examine the **Knuth-Bendix ordering**, the designated rewrite ordering that facilitates the construction of several confluent term rewriting systems.
- (c) Thereafter, we introduce the **Knuth-Bendix completion algorithm**, a methodology capable of augmenting a set of equations to ensure the rewrite relation $\rightarrow_R$ is made confluent.
- (d) Finally, an implementation of the Knuth-Bendix completion algorithm is given.
- (e) Lastly, we apply this algorithm to the axioms of group theory and a number of theories that are related to group theory.

5.2 Confluence

In this section we assume that a binary relation $\rightarrow$ is given on a set M , that is we have $\rightarrow \subseteq M \times M$. Instead of writing $(a, b) \in \rightarrow$ we use infix notation and write $a \rightarrow b$. Furthermore, we assume that $\rightarrow$ is **well-founded**, i.e. there is no infinite sequence $(x_n)_{n \in \mathbb{N}}$ such that

$$s_n \rightarrow s_{n+1} \quad \text{holds for all } n \in \mathbb{N}.$$

We denote the **equivalence relation** generated by $\rightarrow$ as $\leftrightarrow^*$ and the reflexive and transitive closure of $\rightarrow$ is written as $\rightarrow^*$.

Definition 27 (Confluence) The relation $\rightarrow \subseteq M \times M$ is **confluent** iff the following holds:

$$\forall a, b, c \in M : \left(a \rightarrow^* b \wedge a \rightarrow^* c \quad \Rightarrow \quad \exists d \in M : (b \rightarrow^* d \wedge c \rightarrow^* d) \right) \quad \diamond$$

Figure 5.1 shows a diagram picturing the notion of confluence. Here, the relation $\rightarrow^*$ is denoted by a snakelike arrow.

The next theorem shows that confluence is all we need to reduce the relation $\leftrightarrow^*$ to the relation $\rightarrow^*$.

Theorem 28 (Church-Rosser) If the relation $\rightarrow \subseteq M \times M$ is confluent, then we have

$$\forall a, b \in M : \left(a \leftrightarrow^* b \Leftrightarrow \exists c \in M : (a \rightarrow^* c \wedge b \rightarrow^* c) \right).$$

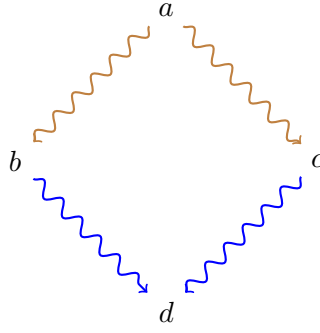


Figure 5.1: Confluence

Proof: If $a \leftrightarrow^* b$ then there is a finite sequence $(s_k)_{k \in \{0, \dots, n\}}$ such that

$$a = s_0 \leftrightarrow s_1 \leftrightarrow \dots \leftrightarrow s_{n-1} \leftrightarrow s_n = b.$$

We prove by induction on n that there is an element $c \in M$ such that both $a \rightarrow^* c$ and $b \rightarrow^* c$ holds.

Base Case: $n = 0$.

Then we have $a = b$ and we can define $c := a$.

Induction Step: $n \mapsto n + 1$

We have $a = s_0 \leftrightarrow s_1 \leftrightarrow \dots \leftrightarrow s_n \leftrightarrow s_{n+1} = b$. By induction hypotheses we know that there exists a $d \in M$ such that

$$a \rightarrow^* d \quad \text{and} \quad s_n \rightarrow^* d$$

hold. Furthermore, we either have

$$s_n \rightarrow b \quad \text{or} \quad b \rightarrow s_n.$$

We discuss these cases one by one.

1. Case: $s_n \rightarrow b$.

Since we also have $s_n \rightarrow^* d$, the confluence of the relation $\rightarrow$ shows that there is an element $c \in M$ such

$$b \rightarrow^* c \quad \text{and} \quad d \rightarrow^* c$$

holds. From $a \rightarrow^* d$ and $d \rightarrow^* c$ we have that $a \rightarrow^* c$. Since we already know that $b \rightarrow^* c$, the proof is complete in this case.

2. Case: $b \rightarrow s_n$.

Since we have $b \rightarrow s_n$ and $s_n \rightarrow^* d$, we can conclude $b \rightarrow^* d$. Since we also have $a \rightarrow^* d$, the proof is complete if we define $c := d$. $\square$

In general, it is hard to prove that a relation $\rightarrow$ is confluent. Things get easier if the relation $\rightarrow$ is well-founded, since then there is a weaker notion than confluence that is already sufficient to guarantee confluence.

Definition 29 (Local Confluence)

The relation $\rightarrow \subseteq M \times M$ is **locally confluent** iff the following holds:

$$\forall a, b, c \in M : \left(a \rightarrow b \wedge a \rightarrow c \Rightarrow \exists d \in M : (b \rightarrow^* d \wedge c \rightarrow^* d) \right) \quad \diamond$$

Figure 5.2 shows a diagram picturing the notion of local confluence. Here, the relation $\rightarrow^*$ is denoted by a snakelike arrow.

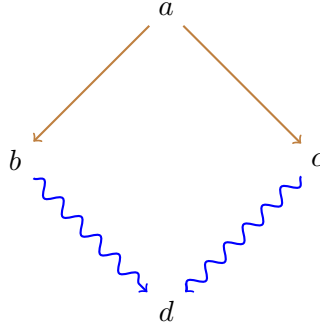


Figure 5.2: Local Confluence

Lemma 30 Assume that the binary relation $\prec \subseteq M \times M$ is a well-founded partial ordering and that $A \subseteq M$ is a non-empty set. Then A must have a minimal element, i.e. there is an element $m \in A$ such that there is no element $n \in A$ that satisfies $n \prec m$.

Proof: The proof is indirect. Assume that the set A has no minimal element. We will define an infinite sequence $(a_n)_{n \in \mathbb{N}}$ by induction on n . This sequence will satisfy the following conditions:

- (a) $a_n \in A$ for all $n \in \mathbb{N}$,
- (b) $a_{n+1} \prec a_n$ for all $n \in \mathbb{N}$.

However, the second property would contradict the fact that $\prec$ is well-founded. Therefore, if we are able to construct the sequence $(a_n)_{n \in \mathbb{N}}$ with the properties stated above, then our assumption that A has no minimal element must be wrong.

B.C.: $n = 0$.

Since A is not empty, there is some element $x \in A$. We pick any such x and define $a_0 := x$.

I.S.: $n \mapsto n + 1$

Since we have assumed that A has no minimal element, a_n is not minimal. Therefore the set

$$B := \{y \in A \mid y \prec a_n\}$$

is not empty. We pick an arbitrary element y from this set and define

$$a_{n+1} := y.$$

Then we have both $a_{n+1} \in A$ and $a_{n+1} \prec a_n$ and hence we have constructed the sequence $(a_n)_{n \in \mathbb{N}}$ with the properties stated above.

Since the existence of this infinite decreasing sequence contradicts the well-foundedness of $\prec$, our proof is complete. $\square$

Theorem 31 (Transfinite Induction)

Assume the binary relation $\prec \subseteq M \times M$ is a well-founded partial ordering and $F(x)$ is some formula. If we have that

$$\forall a \in M : \left(\forall b \in M : (b \prec a \Rightarrow F(b)) \Rightarrow F(a) \right), \quad (\text{TI})$$

then we can conclude that $\forall c \in M : F(c)$ holds.

Proof: Before we start with the proof, let us put the formula given above in words. The formula

$$\forall b \in M : (b \prec a \Rightarrow F(b))$$

expresses that $F(b)$ holds for all b smaller than a . The principle of transfinite induction tells us that, if we are able to conclude $F(a)$ for an arbitrary a from the fact that $F(b)$ holds for all b that are strictly less than a , then we can conclude that $F(c)$ holds for all $c \in M$.

The proof that $F(c)$ holds for all $c \in M$ is indirect. Assume that there is an $a \in M$ such that $F(a)$ does not hold. By the last Lemma we may furthermore assume that this a is minimal w.r.t. the ordering $\prec$. But this implies that

$$\{b \in M \mid (b \prec a \wedge \neg F(b))\} = \emptyset,$$

Therefore, $F(b)$ holds for all b less than a . By the assumption (TI) we can conclude that $F(a)$ holds, contradicting the assumption that $F(a)$ does not hold. $\square$

Theorem 32 (Newman's Lemma)

If the relation $\rightarrow \subseteq M \times M$ is well-founded and locally confluent, then it is already confluent.

Proof: Given any $a \in M$, we define the following formula:

$$F(a) := \forall b, c \in M : \left(a \rightarrow^* b \wedge a \rightarrow^* c \Rightarrow \exists d \in M : (b \rightarrow^* d \wedge c \rightarrow^* d) \right)$$

We prove that $F(a)$ holds for all $a \in M$ by transfinite induction. The relation $b \prec a$ that is needed in the proof of transfinite induction is defined as $a \rightarrow^+ b$, that is we have

$$b \prec a \quad \text{iff} \quad a \rightarrow^+ b.$$

Here, $\rightarrow^+$ is the transitive closure of $\rightarrow$, i.e. we have

$$a \rightarrow^+ b$$

iff there is a finite sequence $c_0, c_1, \dots, c_n, c_{n+1}$ such that we have

$$a = c_0, \quad c_i \rightarrow c_{i+1} \text{ for all } i = 1, \dots, n, \quad \text{and} \quad c_{n+1} = b.$$

Therefore, in order to prove $F(a)$ we may assume that $F(b)$ already holds for all b such that $a \rightarrow^+ b$. So let us assume that we have

$$a \rightarrow^* b \quad \text{and} \quad a \rightarrow^* c.$$

We have to find an element $d \in M$ such that both $b \rightarrow^* d$ and $c \rightarrow^* d$ holds. Now since $a \rightarrow^* b$, either $a = b$ or there is an element b_1 such that

$$a \rightarrow b_1 \rightarrow^* b$$

holds. If $a = b$ we can define $d := c$ and because of $a \rightarrow^* c$ we would then have both

$$b \rightarrow^* d \quad \text{and} \quad c \rightarrow^* d$$

and therefore, in the case $a = b$, we are done. Similarly, since $a \rightarrow^* c$ we either have $a = c$ or there is an element c_1 such that

$$a \rightarrow c_1 \rightarrow^* c$$

holds. If $a = c$ we can define $d := b$ and because of $a \rightarrow^* b$ we would then have both

$$b \rightarrow^* d \quad \text{and} \quad c \rightarrow^* d$$

and are done again. Now the case that is left is the following:

$$a \rightarrow b_1 \rightarrow^* b \quad \text{and} \quad a \rightarrow c_1 \rightarrow^* c.$$

Since $\rightarrow$ is locally confluent and we have both $a \rightarrow b_1$ and $a \rightarrow c_1$ there exists an element d_1 such that we have

$$b_1 \rightarrow^* d_1 \quad \text{and} \quad c_1 \rightarrow^* d_1.$$

Now as b_1 is a successor of a and we have both

$$b_1 \rightarrow^* b \quad \text{and} \quad b_1 \rightarrow^* d_1,$$

our induction hypotheses tells us that there is an element d_2 such that we have both

$$b \rightarrow^* d_2 \quad \text{and} \quad d_1 \rightarrow^* d_2.$$

Now we have $c_1 \rightarrow^* d_1$ and $d_1 \rightarrow^* d_2$, which implies

$$c_1 \rightarrow^* d_2$$

As we also have $c_1 \rightarrow^* c$ we have both

$$c_1 \rightarrow^* d_2 \quad \text{and} \quad c_1 \rightarrow^* c.$$

Since c_1 is a successor of a , the induction hypotheses tells us that there is an element d such that we have both

$$d_2 \rightarrow^* d \quad \text{and} \quad c \rightarrow^* d.$$

As we have $b \rightarrow^* d_2$ and $d_2 \rightarrow^* d$ we can conclude $b \rightarrow^* d$. Hence we have

$$b \rightarrow^* d \quad \text{and} \quad c \rightarrow^* d$$

and the proof is complete. Figure 5.3 on page 118 shows how the different elements are related and conveys the idea of the proof in a concise way. $\square$

Exercise 13: Assume that M is a non-empty set and that $\prec \subseteq M \times M$ is a well-founded partial order on M . Define the relation $\preceq$ as follows:

$$x \preceq y \quad \text{iff} \quad x = y \vee x \prec y.$$

Furthermore, assume that $f : M \rightarrow M$ has the following property:

$$f(x) \preceq x \quad \text{for all } x \in M$$

Prove that f has a [fixed point](#), i.e. there is an element $z \in M$ such that $f(z) = z$. $\diamond$

5.3 The Knuth-Bendix Order

In this section we define the [Knuth-Bendix order](#) $\prec$ on the set $\mathcal{T}_\Sigma$ of Σ -terms.

Definition 33 (Knuth-Bendix Order)

Assume $\Sigma = \langle \mathcal{V}, \mathcal{F}, \text{arity} \rangle$ is a signature. A Knuth-Bendix order for Σ is a pair $\langle w, < \rangle$ such that

1. $w : \mathcal{F} \rightarrow \mathbb{N}$,

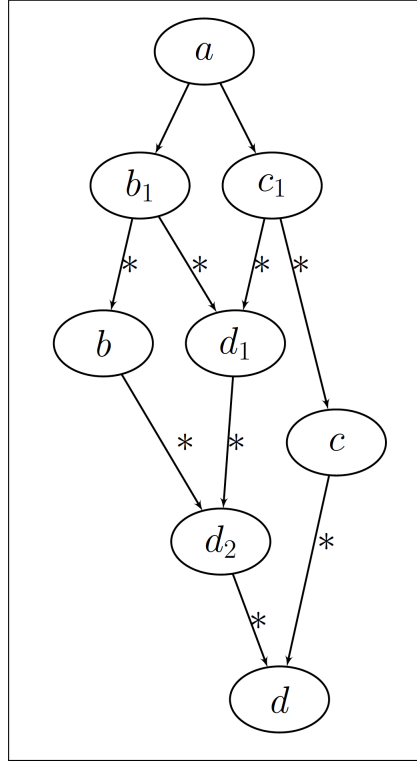


Figure 5.3: The Proof of Newman's Lemma.

i.e. w is a function assigning a natural number to every function symbol. This function w is called the **weight function**. Furthermore, we must have

- (a) There must be at most one function symbol g such that $w(g) = 0$.
- (b) If $w(g) = 0$, then g has to be unary, i.e. $\text{arity}(g) = 1$.
2. $<$ is a **strict total order** $<$ on the set of function symbols, i.e. the following conditions need to be satisfied:

- (a) The relation $<$ is **irreflexive**, that is we have $\neg(f < f)$ for all function symbols f .
- (b) The relation $<$ is **transitive**, that is we have

$$f < g \wedge g < h \Rightarrow f < h \quad \text{for all function symbols } f, g, \text{ and } h.$$

- (c) The relation $<$ is **total**, that is we have

$$f < g \vee f = g \vee g < f \quad \text{for all function symbols } f \text{ and } g.$$

3. The order $<$ on the function symbols has to be **admissible** with respect to the weight function w , i.e. the following condition needs to be satisfied:

$$w(f) = 0 \rightarrow \forall g : (g \neq f \rightarrow g < f).$$

To put this in words: If the function symbol f has a weight of 0, then all other function symbols g have to be smaller than f w.r.t. the strict order $<$. Note that this implies that there can be at most one function symbol with f such that $w(f) = 0$. This function symbol f is then the maximum w.r.t. the order $<$. $\diamond$

Example: For the signature $\Sigma_g = \langle \{w, x, y, z\}, \{e, i, o\}, \{e \mapsto 0, i \mapsto 1, o \mapsto 2\} \rangle$, we can define

- (a) $w := \{w(e) \mapsto 1, w(i) \mapsto 0, w(o) \mapsto 1\}$
- (b) The relation $<$ is defined as

$$e < o < i.$$

Then $\langle w, < \rangle$ is a Knuth-Bendix order. $\diamond$

Definition 34 (Weight of a Term) If Σ is a signature and $\langle w, < \rangle$ is a Knuth-Bendix w.r.t. Σ we define the weight of a term $t \in \mathcal{T}_\Sigma$ by induction on t .

- (a) $w(x) := 1$ for all variables x ,
- (b) $w(f(t_1, \dots, t_n)) := w(f) + \sum_{i=1}^n w(t_i)$. $\diamond$

Definition 35 (Count) If $\Sigma = \langle \mathcal{V}, \mathcal{F}, \text{arity} \rangle$ is a signature, then we define the function

$$\text{count} : \mathcal{T}_\Sigma \times \mathcal{V} \rightarrow \mathbb{N}$$

that takes a term t and a variable x and returns the number of times that x occurs in t . We define $\text{count}(t, x)$ by induction on t .

- 1. $\text{count}(x, x) := 1$ for every variable $x \in \mathcal{V}$.
- 2. $\text{count}(y, x) := 0$ if $x \neq y$ for all variables $x, y \in \mathcal{V}$.
- 3. $\text{count}(f(t_1, \dots, t_n), x) := \sum_{i=1}^n \text{count}(t_i, x)$. $\diamond$

Definition 36 ($\prec_{kb}$) Now we are ready to define the [Knuth-Bendix ordering on terms](#). Assume Σ is a signature and $\langle w, < \rangle$ is a Knuth-Bendix order. Given two terms s and t we define $s \prec_{kb} t$ iff one of the following two conditions hold:

- 1. $w(s) < w(t)$ and $\text{count}(s, x) \leq \text{count}(t, x)$ for all variables x occurring in s .
- 2. $w(s) = w(t)$, $\text{count}(s, x) \leq \text{count}(t, x)$ for all variables x occurring in s , and one of the following subconditions holds:
 - (a) $t = f^n(s)$ where $n \geq 1$ and f is the maximum w.r.t. the order $<$ on function symbols, i.e. we have $g < f$ for all function symbols $g \neq f$.
 - (b) $s = f(s_1, \dots, s_m)$, $t = g(t_1, \dots, t_n)$, and $f < g$.
 - (c) $s = f(s_1, \dots, s_m)$, $t = f(t_1, \dots, t_m)$, and $[s_1, \dots, s_m] \prec_{\text{lex}} [t_1, \dots, t_m]$.

Here, $\prec_{\text{lex}}$ denotes the [lexicographic extension](#) of the ordering $\prec_{kb}$ to lists of terms. It is defined as follows:

$$[x] + R_1 \prec_{\text{lex}} [y] + R_2 \stackrel{\text{def}}{\iff} x \prec_{kb} y \vee (x = y \wedge R_1 \prec_{\text{lex}} R_2)$$

In order to remove clutter we will write $s \prec t$ instead of $s \prec_{kb} t$. $\diamond$

Example: For the signature of group theory $\Sigma_G = \langle \{w, x, y, z\}, \{e, i, \circ\}, \{e \mapsto 0, i \mapsto 1, \circ \mapsto 2\} \rangle$ we define the weight function w as follows:

$$w(e) := 1, \quad w(\circ) := 1, \quad \text{and} \quad w(i) := 0.$$

Furthermore, we define a strict total order on the function symbols by setting

$$e < \circ \quad \text{and} \quad \circ < i.$$

Then the order $<$ is admissible with respect to the weight function because the only function symbol that has a weight of 0 is the largest function symbol with respect to the order $<$. Furthermore, we have the following:

- (a) $x \prec e \circ x$,
because $w(x) = 1$, $w(e \circ x) = 3$, and $1 < 3$.
- (b) $e \prec i(x) \circ x$,
because $w(e) = 1$, $w(i(x) \circ x) = 3$, and $1 < 3$
- (c) $x \circ (y \circ z) \prec (x \circ y) \circ z$
because $w((x \circ y) \circ z) = 5 = w(x \circ (y \circ z))$ and $x \prec x \circ y$, since $w(x) = 1$ and $w(x \circ y) = 3$.
- (d) $i(y) \circ i(x) \prec i(x \circ y)$
because we have the following:
 - (a) $w(i(y) \circ i(x)) = 3 = w(i(x \circ y))$,
 - (b) $\text{count}(i(y) \circ i(x), x) = 1 = \text{count}(i(x \circ y), x)$,
 - (c) $\text{count}(i(y) \circ i(x), y) = 1 = \text{count}(i(x \circ y), y)$, and
 - (d) $\circ < i$. ◇

Theorem 37 If $\langle w, < \rangle$ is a Knuth-Bendix order, then the Knuth-Bendix ordering $\prec_{kb}$ is a rewrite order.

Proving that the Knuth-Bendix order is a strict partial order on the set $\mathcal{T}_\Sigma$ that is stable and a congruence can be done via induction on the structure of the terms. This part of the proof is tedious, but straightforward. The hard part of the proof is to show that the Knuth-Bendix order is well-founded. A proof is given in the book by Franz Baader and Tobias Nipkow [BN98].

5.4 Unification

This section introduces the notion of a **most general unifier** of two terms. To begin, we define the composition of two Σ -substitutions.

Definition 38 (Composition of Substitutions) Assume that

$$\sigma = \{x_1 \mapsto s_1, \dots, x_m \mapsto s_m\} \quad \text{and} \quad \tau = \{y_1 \mapsto t_1, \dots, y_n \mapsto t_n\}$$

are two substitutions such that $\text{dom}(\sigma) \cap \text{dom}(\tau) = \{\}$. We define the **composition** $\sigma\tau$ of σ and τ as

$$\sigma\tau := \{x_1 \mapsto s_1\tau, \dots, x_m \mapsto s_m\tau, y_1 \mapsto t_1, \dots, y_n \mapsto t_n\} \quad \diamond$$

Example: If we define

$$\sigma := \{x_1 \mapsto c, x_2 \mapsto f(x_3)\} \quad \text{and} \quad \tau := \{x_3 \mapsto h(c, c), x_4 \mapsto d\},$$

then we have

$$\sigma\tau = \{x_1 \mapsto c, x_2 \mapsto f(h(c, c)), x_3 \mapsto h(c, c), x_4 \mapsto d\}. \quad \diamond$$

Proposition 39 If t is a term and σ and τ are substitutions such that $\text{dom}(\sigma) \cap \text{dom}(\tau) = \{\}$ holds, then we have

$$(t\sigma)\tau = t(\sigma\tau). \quad \diamond$$

This proposition may be proven by induction on t .

Definition 40 (Syntactical Equation) A **syntactical equation** is a pair $\langle s, t \rangle$ of terms. It is written as $s \doteq t$. A **system of syntactical equations** is a set of syntactical equations. $\diamond$

Definition 41 (Unifier) A substitution σ **solves** a syntactical equation $s \doteq t$ iff we have $s\sigma = t\sigma$. If E is a system of syntactical equations and σ is a substitution that solves every syntactical equations in E , then σ is a **unifier** of E . $\diamond$

If $E = \{s_1 \doteq t_1, \dots, s_n \doteq t_n\}$ is a system of syntactical equations and σ is a substitution, then we define

$$E\sigma := \{s_1\sigma \doteq t_1\sigma, \dots, s_n\sigma \doteq t_n\sigma\}.$$

Example: Let us consider the syntactical equation

$$p(x_1, f(x_4)) \doteq p(x_2, x_3)$$

and define the substitution

$$\sigma := \{x_1 \mapsto x_2, x_3 \mapsto f(x_4)\}.$$

Then σ solves the given syntactical equation because we have

$$\begin{aligned} p(x_1, f(x_4))\sigma &= p(x_2, f(x_4)) \quad \text{and} \\ p(x_2, x_3)\sigma &= p(x_2, f(x_4)). \end{aligned} \quad \diamond$$

Next we develop an algorithm for solving a system of syntactical equations. The algorithm we present was published by Martelli and Montanari [MM82]. To begin, we first consider the cases where a syntactical equation $s \doteq t$ is **unsolvable**. There are two cases: A syntactical equation of the form

$$f(s_1, \dots, s_m) \doteq g(t_1, \dots, t_n)$$

is certainly unsolvable if f and g are different function symbols. The reason is that for any substitution σ we have that

$$f(s_1, \dots, s_m)\sigma = f(s_1\sigma, \dots, s_m\sigma) \quad \text{and} \quad g(t_1, \dots, t_n)\sigma = g(t_1\sigma, \dots, t_n\sigma).$$

If $f \neq g$, then the terms $f(s_1, \dots, s_m)\sigma$ and $g(t_1, \dots, t_n)\sigma$ start with different function symbols and hence they can't be identical.

The other case where a syntactical equation is unsolvable, is a syntactical equation of the following form:

$$x \doteq f(t_1, \dots, t_n) \quad \text{where } x \in \text{var}(f(t_1, \dots, t_n)).$$

This syntactical equation is unsolvable because the term $f(t_1, \dots, t_n)\sigma$ will always contain at least one more occurrence of the function symbol f than the term $x\sigma$.

Now we are able to present an algorithm for solving a system of syntactical equations, provided the system is solvable. The algorithm will also discover if a system of syntactical equations is unsolvable. The algorithm works on pairs of the form $\langle F, \tau \rangle$ where F is a system of syntactical equations and τ is a substitution. The algorithm starts with the pair $\langle E, \{\} \rangle$. Here E is the system of syntactical equations that is to be solved and $\{\}$ represents the empty substitution. The system works by simplifying the pairs $\langle F, \tau \rangle$ using certain reduction rules that are presented below. These reduction rules are applied until we either discover that the system of syntactical equations is unsolvable or else we reduce the pairs until we finally arrive at a pair of the form $\langle \{\}, \mu \rangle$. In this case μ is a unifier of the system of syntactical equations E . The reduction rules are as follows:

1. If $y \in \mathcal{V}$ is a variable that does **not** occur in the term t , then we can perform the following reduction:

$$\langle E \cup \{y \doteq t\}, \sigma \rangle \rightsquigarrow \langle E\{y \mapsto t\}, \sigma\{y \mapsto t\} \rangle \quad \text{if } y \in \mathcal{V} \text{ and } y \notin \text{var}(t)$$

This reduction rule can be understood as follows: If the system of syntactical equations that is to be solved contains a syntactical equation of the form $y \doteq t$, where the variable y does not occur in the term t , then the syntactical equation $y \doteq t$ can be removed if we apply the substitution $\{y \mapsto t\}$ to both components of the pair

$$\langle E \cup \{y \doteq t\}, \sigma \rangle.$$

2. If the variable y occurs in the term t , i.e. if $y \in \text{Var}(t)$ and, furthermore, $t \neq y$, then the system of syntactical equations $E \cup \{y \doteq t\}$ has no solution. We write this as

$$\langle E \cup \{y \doteq t\}, \sigma \rangle \rightsquigarrow \Omega \quad \text{if } y \in \text{var}(t) \text{ and } y \neq t.$$

3. If $y \in \mathcal{V}$ and $t \notin \mathcal{V}$, then we have:

$$\langle E \cup \{t \doteq y\}, \sigma \rangle \rightsquigarrow \langle E \cup \{y \doteq t\}, \sigma \rangle \quad \text{if } y \in \mathcal{V} \text{ and } t \notin \mathcal{V}.$$

After we apply this rule, we can apply either the first or the second reduction rule thereafter.

4. Trivial syntactical equations can be deleted:

$$\langle E \cup \{x \doteq x\}, \sigma \rangle \rightsquigarrow \langle E, \sigma \rangle \quad \text{if } x \in \mathcal{V}.$$

5. If f is an n -ary function symbol we have

$$\langle E \cup \{f(s_1, \dots, s_n) \doteq f(t_1, \dots, t_n)\}, \sigma \rangle \rightsquigarrow \langle E \cup \{s_1 \doteq t_1, \dots, s_n \doteq t_n\}, \sigma \rangle.$$

This rule is the reason that we have to work with a system of syntactical equations, because even if we start with a single syntactical equation the rule given above can increase the number of syntactical equations.

A special case of this rule is the following:

$$\langle E \cup \{c \doteq c\}, \sigma \rangle \rightsquigarrow \langle E, \sigma \rangle.$$

Here c is a nullary function symbol.

6. The system of syntactical equations $E \cup \{f(s_1, \dots, s_m) \doteq g(t_1, \dots, t_n)\}$ has no solution if the function symbols f and g are different. Hence we have

$$\langle E \cup \{f(s_1, \dots, s_m) \doteq g(t_1, \dots, t_n)\}, \sigma \rangle \rightsquigarrow \Omega \quad \text{provided } f \neq g.$$

If a system of syntactical equations E is given and we start with the pair $\langle E, \{\} \rangle$, then we can apply the rules given above until one of the following two cases happens:

1. We use the second or the sixth of the reduction rules given above. In this case the system of syntactical equations E is unsolvable.
2. The pair $\langle E, \{\} \rangle$ is reduced into a pair of the form $\langle \{\}, \mu \rangle$. Then μ is a **unifier** of E . In this case we write $\mu = \text{mgu}(E)$. If $E = \{s \doteq t\}$, we write $\mu = \text{mgu}(s, t)$. The abbreviation **mgu** is short for “**most general unifier**”.

Example: We show how to solve the syntactical equation

$$p(x_1, f(x_4)) \doteq p(x_2, x_3).$$

We have the following reductions:

$$\begin{aligned} & \langle \{p(x_1, f(x_4)) \doteq p(x_2, x_3)\}, \{\} \rangle \\ \rightsquigarrow & \langle \{x_1 \doteq x_2, f(x_4) \doteq x_3\}, \{\} \rangle \\ \rightsquigarrow & \langle \{f(x_4) \doteq x_3\}, \{x_1 \mapsto x_2\} \rangle \\ \rightsquigarrow & \langle \{x_3 \doteq f(x_4)\}, \{x_1 \mapsto x_2\} \rangle \\ \rightsquigarrow & \langle \{\}, \{x_1 \mapsto x_2, x_3 \mapsto f(x_4)\} \rangle \end{aligned}$$

Hence the method is successful and we have that the substitution

$$\{x_1 \mapsto x_2, x_3 \mapsto f(x_4)\}$$

is a solution of the syntactical equation given above. $\diamond$

Example: Next we try to solve the following system of syntactical equations:

$$E = \{p(h(x_1, c)) \doteq p(x_2), q(x_2, d) \doteq q(h(d, c), x_4)\}$$

We have the following reductions:

$$\begin{aligned} & \langle \{p(h(x_1, c)) \doteq p(x_2), q(x_2, d) \doteq q(h(d, c), x_4)\}, \{\} \rangle \\ \rightsquigarrow & \langle \{p(h(x_1, c)) \doteq p(x_2), x_2 \doteq h(d, c), d \doteq x_4\}, \{\} \rangle \\ \rightsquigarrow & \langle \{p(h(x_1, c)) \doteq p(x_2), x_2 \doteq h(d, c), x_4 \doteq d\}, \{\} \rangle \\ \rightsquigarrow & \langle \{p(h(x_1, c)) \doteq p(x_2), x_2 \doteq h(d, c)\}, \{x_4 \mapsto d\} \rangle \\ \rightsquigarrow & \langle \{p(h(x_1, c)) \doteq p(h(d, c))\}, \{x_4 \mapsto d, x_2 \mapsto h(d, c)\} \rangle \\ \rightsquigarrow & \langle \{h(x_1, c) \doteq h(d, c)\}, \{x_4 \mapsto d, x_2 \mapsto h(d, c)\} \rangle \\ \rightsquigarrow & \langle \{x_1 \doteq d, c \doteq c\}, \{x_4 \mapsto d, x_2 \mapsto h(d, c)\} \rangle \\ \rightsquigarrow & \langle \{x_1 \doteq d\}, \{x_4 \mapsto d, x_2 \mapsto h(d, c)\} \rangle \\ \rightsquigarrow & \langle \{\}, \{x_4 \mapsto d, x_2 \mapsto h(d, c), x_1 \mapsto d\} \rangle \end{aligned}$$

Hence the substitution $\{x_4 \mapsto d, x_2 \mapsto h(d, c), x_1 \mapsto d\}$ is a solution of the system of syntactical equations given above. $\diamond$

5.5 The Knuth-Bendix Algorithm

Assume we have been given a set R of rewrite rules such that

$$r \prec l \quad \text{holds for all } l \approx r \text{ in } R$$

such that the relation $\prec$ is a rewrite order. Given two terms s and t , the Church-Rosser Theorem tells us, that we can decide the question whether $s \leftrightarrow_R^* t$ holds by rewriting s and t into normal forms, provided the relation $\rightarrow_R$ is confluent. By Newman's Lemma we know that local confluence is sufficient. Donald E. Knuth and Peter B. Bendix [KB70] have discovered a way to decide whether the term rewriting relation $\rightarrow_R$ is locally confluent. To understand their idea, we introduce the notion of a **critical pair**.

Definition 42 (Critical Pair)

Assume we have been given the equations $l_1 \approx r_1$ and $l_2 \approx r_2$. These equations **generate** a critical pair if and only if the following conditions hold:

- (a) There exists a position $u \in \mathcal{Pos}(l_1)$ such that l_1/u is not a variable.
- (b) The subterm l_1/u of l_1 is unifiable with l_2 . For the following, assume that μ is a most general unifier of l_1/u and l_2 , i.e. we have

$$\mu = \text{mgu}(l_1/u, l_2).$$

- (c) The term s results from rewriting the term $l_1\mu$ by rewriting the subterm $l_1\mu/u$ to the new subterm $r_2\mu$ using the rewrite rule $l_2 \approx r_2$:

$$s = l_1\mu[u \mapsto r_2\mu].$$

- (d) The term t results from rewriting the term $l_1\mu$ into the term $r_1\mu$ using the rule $l_1 \approx r_1$, i.e. we have

$$t = r_1\mu.$$

Then the pair $\langle s, t \rangle$, which is

$$\langle l_1\mu[u \mapsto r_2\mu], r_1\mu \rangle$$

is a **critical pair** of $l_1 \approx r_1$ and $l_2 \approx r_2$. ◇

Example: The following example assumes the signature Σ_G from group theory as given. We start with the two equations $(x \circ y) \circ z \approx x \circ (y \circ z)$ and $i(w) \circ w \approx e$. Then $u = [1]$ is a position in the term $(x \circ y) \circ z$ and we have

$$((x \circ y) \circ z)/[1] = x \circ y, \text{ which is not a variable.}$$

The term $x \circ y$ can be unified with the term $i(w) \circ w$ and we have

$$\mu := \text{mgu}(x \circ y, i(w) \circ w) = \{x \mapsto i(w), y \mapsto w\}.$$

Therefore we have

$$((x \circ y) \circ z)\mu = (i(w) \circ w) \circ z$$

and the right hand side of this equations can be rewritten by the equation $i(w) \circ w \approx e$ into the term $e \circ z$, i.e. we have

$$(i(w) \circ w) \circ z \rightarrow_{\{i(w) \circ w \approx e\}} e \circ z.$$

Furthermore, the same term $(i(w) \circ w) \circ z$ can be rewritten by the equation $(x \circ y) \circ z \approx x \circ (y \circ z)$

into the term $i(w) \circ (w \circ z)$:

$$(i(w) \circ w) \circ z \rightarrow_{\{(x \circ y) \circ z \approx x \circ (y \circ z)\}} i(w) \circ (w \circ z).$$

Therefore, the pair

$$\langle e \circ z, i(w) \circ (w \circ z) \rangle$$

is a critical pair of the two equations $(x \circ y) \circ z \approx x \circ (y \circ z)$ and $i(w) \circ w \approx e$. $\diamond$

Remark: If $\langle s, t \rangle$ is a critical pair from two equations in a set R , then the equation $s \approx t$ is a logical consequence of the equations from R , i.e. we have

$$R \models s \approx t. \quad \diamond$$

Definition 43 (Confluent Critical Pair)

A critical pair $\langle s_1, s_2 \rangle$ is **confluent** w.r.t. a rewrite relation R iff there is a term t such that we have both

$$s_1 \rightarrow_R^* t \quad \text{and} \quad s_2 \rightarrow_R^* t.$$

Theorem 44 (Knuth-Bendix) If R is a set of rewrite equations such that all critical pairs between equations from R are confluent, then the rewrite relation $\rightarrow_R^*$ is confluent and hence the question, whether $R \models s \approx t$ can be decided by rewriting both s and t into normal forms $\hat{s}$ and $\hat{t}$:

$$s \rightarrow_R^* \hat{s} \quad \text{and} \quad t \rightarrow_R^* \hat{t}$$

Then we have

$$R \models s \approx t \quad \text{if and only if} \quad \hat{s} = \hat{t}.$$

To make the above theorem work, if we start with a set E of equations, we first have to order them into a set of rewrite rules R . In general, this will not be sufficient because there will be critical pairs that are not confluent. However, if we can orient these newly derived critical pairs into new rewrite rules, we might be able to extend the set R to a new set of rewrite $\hat{R}$ such that all critical pairs from equations from $\hat{R}$ are confluent.

Knuth-Bendix Algorithm: Given a set of equations E the Knuth-Bendix algorithm proceeds as follows:

1. We define a suitable Knuth-Bendix order $\langle w, < \rangle$ for the function symbols occurring in E such that every equation $(s \approx t) \in E$ can be ordered as either $s < t$ or $t < s$. If this is not possible, the algorithm fails.
2. Otherwise, call R the set of oriented rewrite rules that result from orienting the equations in E into rewrite rules.
3. Compute all critical pairs that can be built from equations in R .
 - (a) If all critical pairs are confluent, then the rewrite relation $\rightarrow_R^*$ is confluent and the algorithm is successful.
 - (b) If we have found a critical pair $\langle s, t \rangle$ that is not confluent, we use the rewrite rules to simplify s and t into terms $\hat{s}$ and $\hat{t}$ that are in normal form with respect to $\rightarrow_r$.
 - (c) Then we try to orient the equation $\hat{s} \approx \hat{t}$ into a rewrite rule $l \approx r$ such that $r < l$.
 - (d) If this is impossible, the algorithm fails.
 - (e) Otherwise, we add the equation $l \approx r$ to R :

$$R := R \cup \{l \approx r\}.$$

- (f) The newly added equation could generate additional critical pairs. Hence we must go back to the beginning of step 3. $\diamond$

The algorithm shown above can have three different outcomes:

1. It can fail because it has generated an equation that can not be oriented into a rewrite rule.
2. It can stop with a set of rewrite rules R such that $\rightarrow_R$ is confluent.
3. It can run forever because an infinite set of critical pairs is generated.

My GitHub repository contains the Jupyter notebook

[1-Knuth-Bendix-Algorithm-KBO.ipynb](#)

which contains an implementation of the Knuth-Bendix algorithm. It also contains a number of equational theories E where the Knuth-Bendix algorithm is successful.

Example: We test the Knuth-Bendix algorithm with the axioms of group theory. We extend the signature of group theory to contain all lowercase letters

$$a, b, c, d, e, f, g, h, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z$$

with the exception of i as variables. Then we have to denote the neutral element as 1, since e is now a variable. The multiplication is still denoted as $\circ$, and $i(x)$ denotes the inverse of x . Then the axioms are:

- (a) $1 \circ x \approx x$,
- (b) $i(x) \circ x \approx 1$, and
- (c) $(x \circ y) \circ z \approx x \circ (y \circ z)$.

Using the Knuth-Bendix ordering described previously, we can turn these equations into rewrite rules as follows:

- $1 \circ x \rightarrow x$,
- $i(x) \circ x \rightarrow 1$, and
- $(x \circ y) \circ z \rightarrow x \circ (y \circ z)$.

By taking the rewrite rule $(x \circ y) \circ z \rightarrow x \circ (y \circ z)$ and superposing the rewrite rule $i(a) \circ a \rightarrow 1$ at position [1], utilizing the most general unifier $[x \mapsto i(a), y \mapsto a]$, we deduce the following relationships:

$$\begin{aligned} (i(a) \circ a) \circ z &\rightarrow i(a) \circ (a \circ z) \quad \text{and} \\ (i(a) \circ a) \circ z &\rightarrow 1 \circ z. \end{aligned}$$

As $1 \circ z$ can be rewritten to z and $z \prec i(a) \circ (a \circ z)$ in the Knuth-Bendix ordering, we have found the new rewrite rule

$$i(a) \circ (a \circ z) \rightarrow z.$$

By taking the rewrite rule $i(a) \circ (a \circ z) \rightarrow z$ and superposing the rewrite rule $i(x) \circ x \rightarrow 1$ at position [2] using the most general unifier

$$\text{mgu}(i(x) \circ x, a \circ z) = [a \mapsto i(x), z \mapsto x]$$

we conclude

$$i(i(x)) \circ (i(x) \circ x) \rightarrow x \quad \text{and} \quad i(i(x)) \circ (i(x) \circ x) \rightarrow i(i(x)) \circ 1.$$

Hence we have found the new rewrite rule

$$i(i(x)) \circ 1 \rightarrow x.$$

In the next step, we build a critical pair between a rule $l \rightarrow r$ and the same rule $l \rightarrow r$. However, note that we have to rename the variables in the second instance of the rule to prevent accidental name clashes. We take the rewrite rule $i(x) \circ (x \circ y) \rightarrow y$ and superposing the (renamed) rewrite rule $i(a) \circ (a \circ b) \rightarrow b$ at position [2] using the most general unifier

$$\text{mgu}(x \circ y, i(a) \circ (a \circ b)) = [x \mapsto i(a), y \mapsto a \circ b].$$

This gives

$$i(i(a)) \circ (i(a) \circ (a \circ b)) \rightarrow a \circ b \quad \text{and} \quad i(i(a)) \circ (i(a) \circ (a \circ b)) \rightarrow i(i(a)) \circ b.$$

Since $a \circ b \prec i(i(a)) \circ b$ we have found the new rewrite rule

$$i(i(a)) \circ b \rightarrow a \circ b.$$

By taking the rewrite rule $i(i(a)) \circ b \rightarrow a \circ b$ and superposing the rewrite rule $i(i(x)) \circ 1 \rightarrow x$ at position [] using the most general unifier

$$\text{mgu}(i(i(a)) \circ b, i(i(x)) \circ 1) = [x \mapsto a, b \mapsto 1]$$

we conclude

$$i(i(a)) \circ 1 \rightarrow a \circ 1 \quad \text{and} \quad i(i(a)) \circ 1 \rightarrow a$$

Hence we have found the new rewrite rule

$$a \circ 1 \rightarrow a.$$

At this point, the rewrite rule $i(i(x)) \circ 1 \rightarrow x$ is simplified into the rule

$$i(i(x)) \rightarrow x.$$

By taking the rewrite rule $i(x) \circ (x \circ y) \rightarrow y$ and superposing the rewrite rule $i(i(a)) \rightarrow a$ at position [1] using the most general unifier

$$\text{mgu}(i(x), i(i(a))) = [x \mapsto i(a)],$$

we conclude

$$i(i(a)) \circ (i(a) \circ y) \rightarrow y \quad \text{and} \quad i(i(a)) \circ (i(a) \circ y) \rightarrow a \circ (i(a) \circ y).$$

Hence we have found the new rule

$$a \circ (i(a) \circ y) \rightarrow y.$$

By taking the rewrite rule $i(x) \circ x \rightarrow 1$ and superposing the rewrite rule $i(i(a)) \rightarrow a$ at position [1] using the most general unifier

$$\text{mgu}(i(x), i(i(a))) = [x \mapsto i(a)]$$

we conclude

$$i(i(a)) \circ i(a) \rightarrow 1 \quad \text{and} \quad i(i(a)) \circ i(a) \rightarrow a \circ i(a).$$

Hence we have found the new rule

$$a \circ i(a) \rightarrow 1.$$

By taking the rewrite rule $a \circ i(a) \rightarrow 1$ and superposing the rewrite rule $1 \circ x \rightarrow x$ at position [] using the most general unifier

$$\text{mgu}(a \circ i(a), 1 \circ x) = [a \mapsto 1, x \mapsto i(1)],$$

we conclude

$$1 \circ i(1) \rightarrow 1 \quad \text{and} \quad 1 \circ i(1) \rightarrow i(1).$$

Hence we have shown

$$i(1) \rightarrow 1.$$

By taking the rewrite rule $a \circ i(a) \rightarrow 1$ and superposing the rewrite rule $(x \circ y) \circ z \rightarrow x \circ (y \circ z)$ at position $[]$ using the most general unifier

$$\text{mgu}(a \circ i(a), (x \circ y) \circ z) = [a \mapsto x \circ y, z \mapsto i(x \circ y)],$$

we conclude

$$(x \circ y) \circ i(x \circ y) \rightarrow 1 \quad \text{and} \quad (x \circ y) \circ i(x \circ y) \rightarrow x \circ (y \circ i(x \circ y)).$$

Hence we have found the new rule

$$x \circ (y \circ i(x \circ y)) \rightarrow 1.$$

By taking the rewrite rule $a \circ (i(a) \circ b) \rightarrow b$ and superposing the rewrite rule $x \circ (y \circ i(x \circ y)) \rightarrow 1$ at position $[2]$ using the most general unifier

$$\text{mgu}(i(a) \circ b, x \circ (y \circ i(x \circ y))) = [x \mapsto i(a), b \mapsto y \circ i(i(a) \circ y)],$$

we conclude

$$a \circ (i(a) \circ (y \circ i(i(a) \circ y))) \rightarrow y \circ i(i(a) \circ y) \quad \text{and} \quad a \circ (i(a) \circ (y \circ i(i(a) \circ y))) \rightarrow a \circ 1.$$

As we already know that $a \circ 1 \rightarrow a$ we have found the new rule

$$y \circ i(i(a) \circ y) \rightarrow a.$$

By taking the rewrite rule $y \circ i(i(a) \circ y) \rightarrow a$ and superposing the rewrite rule $i(i(x)) \rightarrow x$ at position $[2, 1, 1]$ using the most general unifier

$$\text{mgu}(i(a), i(i(x))) = [a \mapsto i(x)],$$

we conclude

$$y \circ i(i(i(x)) \circ y) \rightarrow i(x) \quad \text{and} \quad y \circ i(i(i(x)) \circ y) \rightarrow y \circ i(x \circ y).$$

Hence we have found the new rule

$$y \circ i(x \circ y) \rightarrow i(x).$$

By taking the rewrite rule $i(a) \circ (a \circ b) \rightarrow b$ and superposing the rewrite rule $y \circ i(x \circ y) \rightarrow i(x)$ at the position $[2]$ using the most general unifier

$$\text{mgu}(a \circ b, y \circ i(x \circ y)) = [y \mapsto a, b \mapsto i(x \circ a)],$$

we conclude

$$i(a) \circ (a \circ i(x \circ a)) \rightarrow i(x \circ a) \quad \text{and} \quad i(a) \circ (a \circ i(x \circ a)) \rightarrow i(a) \circ i(x).$$

Since $i(a) \circ i(y) \prec i(x \circ a)$, we have found the new rule

$$i(x \circ a) \rightarrow i(a) \circ i(x).$$

This last rule makes the rules

$$y \circ i(x \circ y) \rightarrow i(x), \quad y \circ i(i(a) \circ y) \rightarrow a, \quad \text{and} \quad x \circ (y \circ i(x \circ y)) \rightarrow 1$$

redundant as all of these rules can be simplified to an identity using the rule $i(x \circ a) \rightarrow i(a) \circ i(x)$. Therefore, we have found the following set of rewrite rules.

1. $1 \circ x \rightarrow x$,
2. $i(x) \circ x \rightarrow 1$,
3. $(x \circ y) \circ z \rightarrow x \circ (y \circ z)$,
4. $i(a) \circ (a \circ z) \rightarrow z$.
5. $a \circ 1 \rightarrow a$,
6. $i(1) \rightarrow 1$,
7. $i(i(x)) \rightarrow x$,
8. $a \circ i(a) \rightarrow 1$,
9. $a \circ (i(a) \circ y) \rightarrow y$,
10. $i(x \circ a) \rightarrow i(a) \circ i(x)$.

It can be shown that all critical pairs resulting from these rules can be simplified to identities. Hence this set is a confluent set of rewrite rules for group theory. Therefore, the validity of any equation $s \approx t$ in group theory can be checked by rewriting s and t into normal forms using the rewrite rules given above. Then the equation $s \approx t$ is valid in group theory if and only if the normal forms of s and t are identical. $\diamond$

Exercise 14: A [quasi-group](#) is a structure

$$\mathcal{G} = \langle G, \circ, /, \backslash \rangle$$

such that

1. G is a non-empty set,
2. $\circ : G \times G \rightarrow G$,
3. $/ : G \times G \rightarrow G$,
4. $\backslash : G \times G \rightarrow G$.
5. Furthermore, the following axioms have to be satisfied:
 - (a) $x \circ (x \backslash y) = y$,
 - (b) $(x / y) \circ y = x$,
 - (c) $x \backslash (x \circ y) = y$,
 - (d) $(x \circ y) / y = x$.

Compute the set of all non-trivial critical pairs from these equations.

Hint: The two non-trivial critical pairs arise from trying to simplify the left hand side of equation (d) with equation (a) and from simplifying the left hand side of equation (c) with (b). $\diamond$

5.6 Literature

The book [Term Rewriting and All That](#) by Franz Baader and Tobias Nipkow [BN98] gives a much more detailed account of equational theorem proving via term rewriting.

Chapter 6

Linear Regression

A great deal of the current success of artificial intelligence is due to recent advances in **machine learning**. In order to get a first taste of what machine learning is about, we introduce **linear regression** in this chapter, since linear regression is one of the most basic algorithms in machine learning. It is also the foundation for more advanced forms of machine learning like **logistic regression** and **neural networks**. Furthermore, linear regression is surprisingly versatile and powerful. Finally, many of the fundamental problems of machine learning can already be illustrated with linear regression. Therefore it is only natural that we begin our study of machine learning with the study of linear regression.

6.1 Simple Linear Regression

Assume we want to know how the **engine displacement** of a car engine relates to its **fuel consumption**. One approach to understand this relation would be to derive a **theoretical model** that is able to predict the fuel consumption from the engine displacement by using the appropriate laws of physics and chemistry. However, due to our lack of understanding of the underlying theory, this is not an option for us. Instead, we follow a **statistical approach** and collect data from a large number of cars. For these cars, we compare their engine displacement with the corresponding fuel consumption. This way, we will collect a set of m **observations** of the form

$$\langle x_1, y_1 \rangle, \dots, \langle x_m, y_m \rangle$$

where x_i is the engine displacement of the engine in the i -th car, while y_i is the fuel consumption of the i -th car. We call x the **independent variable**, while y is the **dependent variable**. We define the vectors $\mathbf{x}$ and $\mathbf{y}$ as follows:

$$\mathbf{x} := \langle x_1, \dots, x_m \rangle^\top \quad \text{and} \quad \mathbf{y} := \langle y_1, \dots, y_m \rangle^\top.$$

Here, the postfix operator $^\top$ is interpreted as the **transposition operator**. The reason is that $\mathbf{x}$ and $\mathbf{y}$ are column vectors, but writing them as column vectors would take too much space. By using the transposition operator we are able to write these vectors in a single line. In these lecture notes, I will write vectors using bold face. On the blackboard I will write $\vec{x}$ instead of $\mathbf{x}$.

In linear regression, we use a **linear model** and assume that the dependent variable y_i is related to the independent variable x_i via an equation of the form

$$y_i = \vartheta_1 \cdot x_i + \vartheta_0.$$

We do not assume that this equation will hold precisely. This is because, in addition to engine displacement, numerous other factors affect fuel consumption. For instance, the **weight** of a car and its **aerodynamics** undoubtedly play significant roles in determining fuel consumption. We want to

calculate those values ϑ_0 and ϑ_1 such that the [mean squared error](#), which is defined as

$$\text{MSE}(\vartheta_0, \vartheta_1) := \frac{1}{m-1} \cdot \sum_{i=1}^m (\vartheta_1 \cdot x_i + \vartheta_0 - y_i)^2, \quad (6.1)$$

is minimized. It can be shown (and you will do so in an exercise) that the solution to this minimization problem is given as follows:

$$\vartheta_1 = r_{x,y} \cdot \frac{s_y}{s_x} \quad \text{and} \quad \vartheta_0 = \bar{y} - \vartheta_1 \cdot \bar{x}. \quad (6.2)$$

This solution utilizes the values $r_{x,y}$, s_x , and s_y . To define these values, we initially establish the [sample mean values](#) $\bar{x}$ and $\bar{y}$ for the vectors $\mathbf{x}$ and $\mathbf{y}$ respectively. Specifically, we calculate

$$\bar{x} = \frac{1}{m} \cdot \sum_{i=1}^m x_i \quad \text{and} \quad \bar{y} = \frac{1}{m} \cdot \sum_{i=1}^m y_i.$$

Furthermore, s_x and s_y are the [corrected sample standard deviations](#) of $\mathbf{x}$ and $\mathbf{y}$, i.e. we have

$$s_x = \sqrt{\frac{1}{m-1} \cdot \sum_{i=1}^m (x_i - \bar{x})^2} \quad \text{and} \quad s_y = \sqrt{\frac{1}{m-1} \cdot \sum_{i=1}^m (y_i - \bar{y})^2}.$$

In the rest of these lecture notes, s_x and s_y will be referred to as [sample standard deviation](#), i.e. we drop the attribute *corrected*. Next, $\text{Cov}[\mathbf{x}, \mathbf{y}]$ is the [sample covariance](#) and is defined as

$$\text{Cov}[\mathbf{x}, \mathbf{y}] = \frac{1}{(m-1)} \cdot \sum_{i=1}^m (x_i - \bar{x}) \cdot (y_i - \bar{y}).$$

Finally, $r_{x,y}$ is the [sample correlation coefficient](#) that is defined as

$$r_{x,y} = \frac{1}{(m-1) \cdot s_x \cdot s_y} \cdot \sum_{i=1}^m (x_i - \bar{x}) \cdot (y_i - \bar{y}) = \frac{\text{Cov}[\mathbf{x}, \mathbf{y}]}{s_x \cdot s_y}.$$

The number $r_{x,y}$ is also known as the [Pearson correlation coefficient](#) or [Pearson's r](#). It is named after [Karl Pearson](#) (1857 – 1936). Note that the formula for the parameter ϑ_1 can be simplified to

$$\vartheta_1 = \frac{\sum_{i=1}^m (x_i - \bar{x}) \cdot (y_i - \bar{y})}{\sum_{i=1}^m (x_i - \bar{x})^2} \quad (6.3)$$

This latter formula should be used to calculate ϑ_1 . However, the previous formula is also useful because it shows that the correlation coefficient is identical to the coefficient ϑ_1 , provided the variables $\mathbf{x}$ and $\mathbf{y}$ have been [normalized](#) so that their standard deviation is 1.

Exercise 15: Prove Equation 6.2 and Equation 6.3.

Hint: The expression $\text{MSE}(\vartheta_0, \vartheta_1)$ is a quadratic function with respect to the parameters ϑ_0 and ϑ_1 . Therefore, it has exactly one global minimum. Take the partial derivatives of $\text{MSE}(\vartheta_0, \vartheta_1)$ with respect to ϑ_0 and ϑ_1 . If the expression $\text{MSE}(\vartheta_0, \vartheta_1)$ is minimal, then these partial derivatives have to be equal to 0. $\diamond$

6.1.1 Assessing the Quality of Linear Regression

Assuming we are provided with a set of m observations in the form $\langle x_1, y_1 \rangle, \dots, \langle x_m, y_m \rangle$, and that we have calculated the parameters ϑ_0 and ϑ_1 as per Equation 6.2 and Equation 6.3, these formulas

will yield values for ϑ_0 and ϑ_1 , assuming the x_i values are not all the same. These values define a linear model for $\mathbf{y}$ as a function of $\mathbf{x}$. Yet, to assess the effectiveness of this linear model, we require a metric that quantifies its accuracy. To evaluate the quality of the linear model represented by

$$y = \vartheta_0 + \vartheta_1 \cdot x,$$

we can calculate the mean squared error using Equation 6.1. However, the mean squared error alone may not provide a complete picture due to its absolute nature and the potential inherent noise within $\mathbf{y}$. To contextualize this noise relative to the mean squared error, we measure the noise within $\mathbf{y}$ through the [sample variance](#), calculated as follows:

$$\text{Var}(y) := \frac{1}{m-1} \cdot \sum_{i=1}^m (y_i - \bar{y})^2. \quad (6.4)$$

If we compare this formula to the formula for the mean squared error

$$\text{MSE}(\vartheta_0, \vartheta_1) := \frac{1}{m-1} \cdot \sum_{i=1}^m (\vartheta_1 \cdot x_i + \vartheta_0 - y_i)^2 = \frac{1}{m-1} \cdot \sum_{i=1}^m (y_i - \vartheta_1 \cdot x_i - \vartheta_0)^2,$$

we see that the sample variance of $\mathbf{y}$ is an upper bound for the mean squared error since we have

$$\text{Var}(\mathbf{y}) = \text{MSE}(\bar{\mathbf{y}}, 0),$$

i.e. the sample variance is the value that we would get for the mean squared error if we set ϑ_0 to the average value of $\mathbf{y}$ and ϑ_1 to zero. Since ϑ_0 and ϑ_1 are chosen to minimize the mean squared error, we have

$$\text{MSE}(\vartheta_0, \vartheta_1) \leq \text{MSE}(\bar{\mathbf{y}}, 0) = \text{Var}(\mathbf{y}).$$

The mean squared error is an absolute value and, therefore, difficult to interpret. The fraction

$$\frac{\text{MSE}(\vartheta_0, \vartheta_1)}{\text{Var}(\mathbf{y})}$$

is called the [proportion of the unexplained variance](#) because it is the variance that is still left if we use our linear model to predict the values of $\mathbf{y}$ given the values of $\mathbf{x}$. The [proportion of the explained variance](#) which is also known as the [R² statistic](#) is defined as

$$\text{R}^2 := \frac{\text{Var}(\mathbf{y}) - \text{MSE}(\vartheta_0, \vartheta_1)}{\text{Var}(\mathbf{y})} = 1 - \frac{\text{MSE}(\vartheta_0, \vartheta_1)}{\text{Var}(\mathbf{y})}. \quad (6.5)$$

The statistic R^2 measures the quality of our model: If it is small, then our model does not explain the variation of the value of $\mathbf{y}$ when the value of $\mathbf{x}$ changes. On the other hand, if it is near to 100%, then our model does a good job in explaining the variation of $\mathbf{y}$ when $\mathbf{x}$ changes.

Since the formulas for $\text{Var}(\mathbf{y})$ and $\text{MSE}(\vartheta_0, \vartheta_1)$ have the same denominator $m-1$, this denominator can be cancelled when R^2 is computed. To this end we define the [total sum of squares TSS](#) as

$$\text{TSS} := \sum_{i=1}^m (y_i - \bar{y})^2 = (m-1) \cdot \text{Var}(\mathbf{y})$$

and the [residual sum of squares RSS](#) as

$$\text{RSS} := \sum_{i=1}^m (\vartheta_1 \cdot x_i + \vartheta_0 - y_i)^2 = (m-1) \cdot \text{MSE}(\vartheta_0, \vartheta_1).$$

Then the formula for the R^2 statistic can be written as

$$\text{R}^2 = 1 - \frac{\text{RSS}}{\text{TSS}}.$$

This is the formula that we will use when we implement simple linear regression.

It should be noted that R^2 is the square of Pearson’s r . The notation is a bit inconsistent since Pearson’s r is written in lower case, while R^2 is written in upper case. However, since this is the notation used in most books on statistics, we will use it too. The number R^2 is also known as the **coefficient of determination**. It tells us to what extent the value of the variable y is **determined** by the value of x .

6.1.2 Putting the Theory to the Test

In order to get a better feeling for linear regression, we want to test it to investigate the factors that determine the fuel consumption of cars. Figure 6.1 on page 133 shows the head of the data file “Auto.csv” which I have adapted from the file

<https://www.statlearning.com/s/Auto.csv>.

Figure 6.1 on page 133 shows the column headers and the first ten data entries contained in this file. Altogether, this file contains data of 392 different car models.

	mpg,	cyl,	displacement,	hp,	weight,	acc,	year,	name
1	18.0,	8,	307.0,	130.0,	3504.0,	12.0,	70,	chevrolet chevelle malibu
2	15.0,	8,	350.0,	165.0,	3693.0,	11.5,	70,	buick skylark 320
3	18.0,	8,	318.0,	150.0,	3436.0,	11.0,	70,	plymouth satellite
4	16.0,	8,	304.0,	150.0,	3433.0,	12.0,	70,	amc rebel sst
5	17.0,	8,	302.0,	140.0,	3449.0,	10.5,	70,	ford torino
6	15.0,	8,	429.0,	198.0,	4341.0,	10.0,	70,	ford galaxie 500
7	14.0,	8,	454.0,	220.0,	4354.0,	9.0,	70,	chevrolet impala
8	14.0,	8,	440.0,	215.0,	4312.0,	8.5,	70,	plymouth fury iii
9	14.0,	8,	455.0,	225.0,	4425.0,	10.0,	70,	pontiac catalina
10	15.0,	8,	390.0,	190.0,	3850.0,	8.5,	70,	amc ambassador dpl

Figure 6.1: The head of the file `cars.csv`.

The file “cars.csv” is part of the data set accompanying the excellent book **Introduction to Statistical Learning** by Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani [JWHT14]. The file `cars.csv` contains the fuel consumption of a number of different cars that were in widespread use during the seventies and early eighties of the last century. The first column of this data set gives the **miles per gallon** of a car, i.e. the number of miles a car can drive with one gallon of gas. Note that this number is in **reciprocal** relation to the fuel consumption: If a car A can drive **twice** as many miles per gallon than another car B, then the fuel consumption of A is **half** of the fuel consumption of B. Furthermore, besides the miles per gallon, for every car the following other parameters are listed:

1. `cyl` is the number of cylinders,
2. `displacement` is the engine displacement in **cubic inches**, (100 cubic inch is 1.638 706 4 litres)
3. `hp` is the engine power given in units of **horsepower**,
4. `weight` is the weight in **pounds** (1 pound is the same as 0.453 592 37 kg),
5. `acc` is the acceleration given as the time in seconds needed to accelerate from 0 miles per hour to 60 miles per hour,

6. `year` is the year in which the model was introduced, and
7. `name` is the name of the model.

Our aim is to determine what part of the fuel consumption of a car is explained by its engine displacement. To this end, I have written the function `simple_linear_regression` shown in Figure 6.2 on page 134.

```
def simple_linear_regression(X, Y):
    """
    This function implements linear regression.

    * X:      explaining variable, numpy array
    * Y:      dependent variable, numpy array

    Output: The R2 value of the linear regression.
    """
    m      = len(X)
    xMean  = np.mean(X);
    yMean  = np.mean(Y);
    theta1 = np.sum( (X - xMean) * (Y - yMean) ) / np.sum((X - xMean) ** 2)
    theta0 = yMean - theta1 * xMean;
    TSS    = np.sum((Y - yMean) ** 2)
    RSS    = np.sum((theta1 * X + theta0 - Y) ** 2)
    R2     = 1 - RSS / TSS;
    return R2
```

Figure 6.2: Simple Linear Regression

The function `simple_linear_regression` takes two arguments:

- (a) `X` is a NumPy array containing the independent variable.
- (b) `Y` is a NumPy array containing the dependent variable.

The implementation of the function `simple_linear_regression` works as follows:

1. `m` is the number of data that are present in the array `X`.
2. `xMean` is the mean value $\bar{x}$ of the independent variable `x`.
3. `yMean` is the mean value $\bar{y}$ of the dependent variable `y`.
4. The coefficient `theta1` is computed according to Equation 6.3, which is repeated here for convenience:

$$\vartheta_1 = \frac{\sum_{i=1}^m (x_i - \bar{x}) \cdot (y_i - \bar{y})}{\sum_{i=1}^m (x_i - \bar{x})^2}.$$

Note that the expression `(X - xMean)` computes an array of the same shape as `X` by subtracting `xMean` from every entry of `X`. Next, the expression `(X - xMean) * (Y - yMean)` computes the

elementwise product of the arrays $X - \bar{x}$ and $Y - \bar{y}$. The expression $(X - \bar{x})^2$ computes the elementwise squares of the array $X - \bar{x}$. Finally, the function `sum` computes the sum of all the elements of an array.

5. The coefficient `theta0` is computed according to Equation 6.2, which reads

$$\vartheta_0 = \bar{y} - \vartheta_1 \cdot \bar{x}.$$

6. TSS is the **total sum of squares** and is computed using the formula

$$\text{TSS} = \sum_{i=1}^m (y_i - \bar{y})^2.$$

7. RSS is the **residual sum of squares** and is computed as

$$\text{RSS} := \sum_{i=1}^m (\vartheta_1 \cdot x_i + \vartheta_0 - y_i)^2.$$

8. R^2 is the R^2 statistic and measures the **proportion of the explained variance**. It is computed using the formula

$$R^2 = \frac{\text{TSS} - \text{RSS}}{\text{TSS}}.$$

```
import csv
import numpy as np

with open('cars.csv') as input_file:
    reader = csv.reader(input_file, delimiter=',')
    line_count = 0
    kpl = []
    displacement = []
    for row in reader:
        if line_count != 0:
            kpl.append(float(row[0]) * 0.00425144)
            displacement.append(float(row[2]) * 0.0163871)
            line_count += 1
m = len(displacement)
X = np.array(displacement)
Y = np.array([1 / kpl[i] for i in range(m)])
R2 = simple_linear_regression(X, Y)
print(f'The explained variance is {R2}%')
```

Figure 6.3: Calling the function `simple_linear_regression`.

In order to use the function we can use the code that is shown in Figure 6.3 on page 135.

1. We import the module `csv` in order to be able to read the Csv file “cars.csv” conveniently.
2. We import the module `numpy` in order to use NumPy arrays.
3. We open the file “cars.csv”.

dependent variable	explained variance
displacement	0.75
number of cylinders	0.70
horsepower	0.73
weight	0.78
acceleration	0.21
year of build	0.31

Table 6.1: Explained variance for various dependent variables.

4. This file is processed as a Csv file where different columns are separated by the character “,”.
5. `kpl` is a list of the numbers that appear in the first column of the Csv file. The numbers in the Csv file are interpreted as the [miles per gallon](#) of a car. These numbers are converted into metric units, i.e. how many kilometers a car can run on a litre.
6. `displacement` is a list the numbers appearing in the third column of the CSV file. These numbers are interpreted as the *engine displacement* in cubic inches. These numbers are converted to litres.
7. The first line of the Csv file contains a header. This header is skipped. In order to do so we use the variable `line_count`.
8. `m` is the number of data pairs that have been read.
9. The independent variable `X` is given by the engine displacement. In order to be able to use NumPy features later we convert this list into a NumPy array.
10. The dependent variable `Y` is given by the inverse of the variable `kph`.
11. Finally, the coefficient of determination R^2 is computed. Here we use the function `simple_linear_regression` that is shown in Figure [6.2](#) on page [134](#).

In the same way as we have computed the coefficient of determination that measures how the fuel consumption is influenced by the engine displacement we can also compute the coefficient of determination for other variables like the number of cylinders or the weight of the car. The resulting values are shown in Table [6.1](#). It seems that, given the data in the file “`cars.csv`”, the best indicator for the fuel consumption is the `weight` of a car. The `displacement`, the power `hp` of an engine, and the number of cylinders `cyl` are also good predictors. But notice that the `weight` is the real cause of fuel consumption: If a car is heavy, it will also need a more powerful engine. Hence the variable `hp` is correlated with the variable `weight` and will therefore also provide a reasonable explanation of the fuel consumption, although the high engine power is not the most important cause of the fuel consumption.

Exercise 16: In this exercise we are going to investigate the concept of a [random walk](#). To understand the idea of a [random walk](#), imagine a person standing at the origin $(0, 0)$ of a two dimensional plane. Every second the robot chooses a random direction from the set

`{north, east, south, west}`

and moves one unit into this direction. For example, if the robot chooses the direction `north` in the first step, it will move from $(0, 0)$ to the position $(0, 1)$. Then, it could choose the direction `west` to arrive at the position $(-1, 1)$. The link

https://upload.wikimedia.org/wikipedia/commons/c/cb/Random_walk_25000.svg

shows a random walk with 25,000 steps. Our goal is to find a formula for the average distance of the robot after its i^{th} step. Our assumption is that the average distance after i steps satisfies the following formula:

$$\text{Distance}[i] = \alpha \cdot i^\beta.$$

In order to find α and β we will simulate a large number of random walks and compute the average distance of the robot in all of these walks. Then we use an appropriate variation of linear regression to find the coefficients α and β formula. Complete the notebook

[Artificial-Intelligence/blob/master/Python/Chapter-05/Random-Walk-Frame.ipynb](#).

to solve this exercise. ◇

6.2 General Linear Regression

In practice, it is uncommon for an observed variable y to depend solely on a single variable x . Taking the example of a car's fuel consumption further, it is generally anticipated that the fuel consumption depends not only on the car's engine displacement but also on various other parameters. For instance, it is plausible to assume that the car's mass has a significant influence on its fuel consumption. To model such complex relationships, we introduce the theory of **general linear regression**.

In the context of a **general regression problem**, we are provided with a list of m pairs in the form $\langle \mathbf{x}^{(i)}, y^{(i)} \rangle$, where $\mathbf{x}^{(i)} \in \mathbb{R}^p$ and $y^{(i)} \in \mathbb{R}$ for all $i \in \{1, \dots, m\}$. The term p denotes the number of **features**, and the pairs are referred to as the **training examples**. Our objective is to compute a linear function

$$F : \mathbb{R}^p \rightarrow \mathbb{R}$$

such that $F(\mathbf{x}^{(i)})$ approximates $y^{(i)}$ as accurately as possible for all $i \in \{1, \dots, m\}$; that is, we aim to achieve

$$F(\mathbf{x}^{(i)}) \approx y^{(i)} \quad \text{for all } i \in \{1, \dots, m\}.$$

In order to make the notation $F(\mathbf{x}^{(i)}) \approx y^{(i)}$ more precise, we define the **mean squared error**

$$\text{MSE} := \frac{1}{m-1} \cdot \sum_{i=1}^m \left(F(\mathbf{x}^{(i)}) - y^{(i)} \right)^2. \quad (6.6)$$

Then, given the list of training examples $[\langle \mathbf{x}^{(1)}, y^{(1)} \rangle, \dots, \langle \mathbf{x}^{(m)}, y^{(m)} \rangle]$, our goal is to minimize the MSE. In order to proceed, we assume that F is given as

$$F(\mathbf{x}) = \sum_{j=1}^p w_j \cdot x_j + b = \mathbf{x}^\top \cdot \mathbf{w} + b \quad \text{where } \mathbf{w} \in \mathbb{R}^p \text{ and } b \in \mathbb{R}.$$

Here, the expression $\mathbf{x}^\top \cdot \mathbf{w}$ denotes the matrix product of the vector $\mathbf{x}^\top$, which is viewed as a 1-by- p matrix, and the vector $\mathbf{w}$, where $\mathbf{w}$ is viewed as a p -by-1 matrix. Alternatively, this expression could be interpreted as the dot product of the vector $\mathbf{x}$ and the vector $\mathbf{w}$. At this point you might wonder why it is useful to introduce matrix notation here. The reason is that this notation shortens the formula and, furthermore, is more efficient to implement since most programming languages used in machine learning have special library support for matrix operations. Provided the computer is equipped with a graphics card, some programming languages are even able to delegate matrix operations to the graphics card. This results in a considerable speed-up.

The definition of F given above is the model used in **linear regression**. Here, $\mathbf{w}$ is called the **weight vector** and b is called the **bias**. It turns out that the notation can be simplified if we extend the p -dimensional feature vector $\mathbf{x}$ to a $(p+1)$ -dimensional vector $\mathbf{x}'$ such that

$$x'_j := x_j \quad \text{for all } j \in \{1, \dots, p\} \quad \text{and} \quad x'_{p+1} := 1.$$

To put it in words, the feature vector $\mathbf{x}'$ results from the feature vector $\mathbf{x}$ by appending the number 1 as a constant feature to every data point:

$$\mathbf{x}' = \langle x_1, \dots, x_p, 1 \rangle^\top \quad \text{where } \langle x_1, \dots, x_p \rangle = \mathbf{x}^\top.$$

Furthermore, we define

$$\mathbf{w}' := \langle w_1, \dots, w_p, b \rangle^\top \quad \text{where } \langle w_1, \dots, w_p \rangle = \mathbf{w}^\top.$$

Then we have

$$F(\mathbf{x}) = \mathbf{x}^\top \cdot \mathbf{w} + b = \mathbf{x}'^\top \cdot \mathbf{w}'.$$

Hence, the bias has been incorporated into the weight vector at the cost of appending the number 1 at the end of input vector $\mathbf{x}$. This is also known as the **bias trick** or **homogenization**. As we want to use this simplification, from now on we assume that the input vectors $\mathbf{x}^{(i)}$ have all been extended so that their last component is always 1. Then we can just write $\mathbf{x}$ and $\mathbf{w}$ instead of $\mathbf{x}'$ and $\mathbf{w}'$. Using this simplification, we define the function F as

$$F(\mathbf{x}) := \mathbf{x}^\top \cdot \mathbf{w}.$$

Now equation (6.6) can be rewritten as follows:

$$\text{MSE}(\mathbf{w}) = \frac{1}{m-1} \cdot \sum_{i=1}^m \left((\mathbf{x}^{(i)})^\top \cdot \mathbf{w} - y^{(i)} \right)^2. \quad (6.7)$$

Our aim is to rewrite the sum appearing in this equation as a scalar product of a vector with itself. To this end, we first define the vector $\mathbf{y}$ as follows:

$$\mathbf{y} := \langle y^{(1)}, \dots, y^{(m)} \rangle^\top.$$

Note that $\mathbf{y} \in \mathbb{R}^m$ since it has a component for all of the m training examples. Next, we define the **design matrix** X as follows:

$$X := \begin{pmatrix} (\mathbf{x}^{(1)})^\top \\ \vdots \\ (\mathbf{x}^{(m)})^\top \end{pmatrix}$$

In the literature, X is also called the **feature matrix**. If X is defined in this way, the row vectors of the matrix X are the transpositions of the vectors $\mathbf{x}^{(i)}$. Then we have the following:

$$X \cdot \mathbf{w} - \mathbf{y} = \begin{pmatrix} (\mathbf{x}^{(1)})^\top \\ \vdots \\ (\mathbf{x}^{(m)})^\top \end{pmatrix} \cdot \mathbf{w} - \mathbf{y} = \begin{pmatrix} (\mathbf{x}^{(1)})^\top \cdot \mathbf{w} - y^{(1)} \\ \vdots \\ (\mathbf{x}^{(m)})^\top \cdot \mathbf{w} - y^{(m)} \end{pmatrix}$$

Taking the square of the vector $X \cdot \mathbf{w} - \mathbf{y}$ we discover that we can rewrite equation (6.7) as follows:

$$\text{MSE}(\mathbf{w}) = \frac{1}{m-1} \cdot (X \cdot \mathbf{w} - \mathbf{y})^\top \cdot (X \cdot \mathbf{w} - \mathbf{y}). \quad (6.8)$$

6.2.1 Some Useful Gradients

In the last section, we have computed the mean squared error $\text{MSE}(\mathbf{w})$ using equation (6.8). Our goal is to minimize the $\text{MSE}(\mathbf{w})$ by choosing the weight vector $\mathbf{w}$ appropriately. A necessary condition for $\text{MSE}(\mathbf{w})$ to be minimal is

$$\nabla \text{MSE}(\mathbf{w}) = \mathbf{0},$$

i.e. the **gradient** of $\text{MSE}(\mathbf{w})$ with respect to $\mathbf{w}$ needs to be zero. In order to prepare for the computation of $\nabla \text{MSE}(\mathbf{w})$, we first compute the gradient of two simpler functions.

Computing the Gradient of $f(\mathbf{x}) = \mathbf{x}^\top \cdot C \cdot \mathbf{x}$

Suppose the function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is defined as

$$f(\mathbf{x}) := \mathbf{x}^\top \cdot C \cdot \mathbf{x} \quad \text{where } C \in \mathbb{R}^{n \times n}.$$

If we write the matrix C as $C = (c_{i,j})_{\substack{i=1,\dots,n \\ j=1,\dots,n}}$ and the vector $\mathbf{x}$ as $\mathbf{x} = \langle x_1, \dots, x_n \rangle^\top$, then $f(\mathbf{x})$ can be computed as follows:

$$f(\mathbf{x}) = \sum_{i=1}^n x_i \cdot \sum_{j=1}^n c_{i,j} \cdot x_j = \sum_{i=1}^n \sum_{j=1}^n x_i \cdot c_{i,j} \cdot x_j.$$

We compute the partial derivative of f with respect to x_k and use the **product rule** together with the definition of the **Kronecker delta** $\delta_{i,j}$, which is defined as 1 if $i = j$ and as 0 otherwise:

$$\delta_{i,j} := \begin{cases} 1 & \text{if } i = j; \\ 0 & \text{otherwise.} \end{cases}$$

Then the partial derivative of f with respect to x_k , which is written as $\frac{\partial f}{\partial x_k}$, is computed as follows:

$$\begin{aligned} \frac{\partial f}{\partial x_k} &= \sum_{i=1}^n \sum_{j=1}^n \left(\frac{\partial x_i}{\partial x_k} \cdot c_{i,j} \cdot x_j + x_i \cdot c_{i,j} \cdot \frac{\partial x_j}{\partial x_k} \right) \\ &= \sum_{i=1}^n \sum_{j=1}^n \left(\delta_{i,k} \cdot c_{i,j} \cdot x_j + x_i \cdot c_{i,j} \cdot \delta_{j,k} \right) \\ &= \sum_{j=1}^n c_{k,j} \cdot x_j + \sum_{i=1}^n x_i \cdot c_{i,k} \\ &= (C \cdot \mathbf{x})_k + (C^\top \cdot \mathbf{x})_k \end{aligned}$$

Hence we have shown that

$$\nabla f(\mathbf{x}) = (C + C^\top) \cdot \mathbf{x}.$$

If the matrix C is **symmetric**, i.e. if $C = C^\top$, this simplifies to

$$\nabla f(\mathbf{x}) = 2 \cdot C \cdot \mathbf{x}.$$

Next, if the function $g : \mathbb{R}^n \rightarrow \mathbb{R}$ is defined as

$$g(\mathbf{x}) := \mathbf{b}^\top \cdot A \cdot \mathbf{x}, \quad \text{where } \mathbf{b} \in \mathbb{R}^m \text{ and } A \in \mathbb{R}^{m \times n},$$

then a similar, but slightly easier, calculation shows that

$$\nabla g(\mathbf{x}) = A^\top \cdot \mathbf{b}.$$

Exercise 17: Prove this equation.

6.2.2 Deriving the Normal Equation

Next, we will derive the so called **normal equation** for linear regression. To this end, we first expand the product in equation (6.8):

$$\begin{aligned}
 \text{MSE}(\mathbf{w}) &= \frac{1}{m-1} \cdot (\mathbf{X} \cdot \mathbf{w} - \mathbf{y})^\top \cdot (\mathbf{X} \cdot \mathbf{w} - \mathbf{y}) \\
 &= \frac{1}{m-1} \cdot (\mathbf{w}^\top \cdot \mathbf{X}^\top - \mathbf{y}^\top) \cdot (\mathbf{X} \cdot \mathbf{w} - \mathbf{y}) && \text{since } (A \cdot B)^\top = B^\top \cdot A^\top \\
 &= \frac{1}{m-1} \cdot (\mathbf{w}^\top \cdot \mathbf{X}^\top \cdot \mathbf{X} \cdot \mathbf{w} - \mathbf{y}^\top \cdot \mathbf{X} \cdot \mathbf{w} - \mathbf{w}^\top \cdot \mathbf{X}^\top \cdot \mathbf{y} + \mathbf{y}^\top \cdot \mathbf{y}) \\
 &= \frac{1}{m-1} \cdot (\mathbf{w}^\top \cdot \mathbf{X}^\top \cdot \mathbf{X} \cdot \mathbf{w} - 2 \cdot \mathbf{y}^\top \cdot \mathbf{X} \cdot \mathbf{w} + \mathbf{y}^\top \cdot \mathbf{y}) && \text{since } \mathbf{w}^\top \cdot \mathbf{X}^\top \cdot \mathbf{y} = \mathbf{y}^\top \cdot \mathbf{X} \cdot \mathbf{w}
 \end{aligned}$$

The fact that

$$\mathbf{w}^\top \cdot \mathbf{X}^\top \cdot \mathbf{y} = \mathbf{y}^\top \cdot \mathbf{X} \cdot \mathbf{w}$$

might not be immediately obvious. It follows from two facts:

1. For two matrices A and B such that the matrix product $A \cdot B$ is defined we have

$$(A \cdot B)^\top = B^\top \cdot A^\top.$$

2. The matrix product $\mathbf{w}^\top \cdot \mathbf{X}^\top \cdot \mathbf{y}$ is a real number. The transpose $r^\top$ of a real number r is the number itself, i.e. $r^\top = r$ for all $r \in \mathbb{R}$. Therefore, we have

$$\mathbf{w}^\top \cdot \mathbf{X}^\top \cdot \mathbf{y} = (\mathbf{w}^\top \cdot \mathbf{X}^\top \cdot \mathbf{y})^\top = \mathbf{y}^\top \cdot \mathbf{X} \cdot \mathbf{w}.$$

Hence we have shown that

$$\text{MSE}(\mathbf{w}) = \frac{1}{m-1} \cdot (\mathbf{w}^\top \cdot (\mathbf{X}^\top \cdot \mathbf{X}) \cdot \mathbf{w} - 2 \cdot \mathbf{y}^\top \cdot \mathbf{X} \cdot \mathbf{w} + \mathbf{y}^\top \cdot \mathbf{y}) \quad (6.9)$$

holds. The matrix $\mathbf{X}^\top \cdot \mathbf{X}$ used in the first term is symmetric because

$$(\mathbf{X}^\top \cdot \mathbf{X})^\top = \mathbf{X}^\top \cdot (\mathbf{X}^\top)^\top = \mathbf{X}^\top \cdot \mathbf{X}.$$

Using the results from the previous section, i.e.

$$\nabla(\mathbf{x} \mapsto \mathbf{x}^\top \cdot \mathbf{C} \cdot \mathbf{x}) = 2 \cdot \mathbf{C} \cdot \mathbf{x} \quad \text{provided that } \mathbf{C}^\top = \mathbf{C} \quad \text{and}$$

$$\nabla(\mathbf{x} \mapsto \mathbf{b}^\top \cdot \mathbf{A} \cdot \mathbf{x}) = \mathbf{A}^\top \cdot \mathbf{b},$$

we can now compute the gradient of $\text{MSE}(\mathbf{w})$ with respect to $\mathbf{w}$. The gradient of the term

$$\mathbf{w}^\top \cdot (\mathbf{X}^\top \cdot \mathbf{X}) \cdot \mathbf{w}$$

with respect to $\mathbf{w}$ is $2 \cdot (\mathbf{X}^\top \cdot \mathbf{X}) \cdot \mathbf{w}$, while the gradient of the term

$$\mathbf{y}^\top \cdot \mathbf{X} \cdot \mathbf{w}$$

is $\mathbf{X}^\top \cdot \mathbf{y}$. Since the gradient of $\mathbf{y}^\top \cdot \mathbf{y}$ with respect to $\mathbf{w}$ vanishes, the gradient of $\text{MSE}(\mathbf{w})$ is given as

$$\nabla \text{MSE}(\mathbf{w}) = \frac{2}{m-1} \cdot (\mathbf{X}^\top \cdot \mathbf{X} \cdot \mathbf{w} - \mathbf{X}^\top \cdot \mathbf{y}).$$

If the squared error $\text{MSE}(\mathbf{w})$ has a minimum for the weights $\mathbf{w}$, then we must have

$$\nabla \text{MSE}(\mathbf{w}) = \mathbf{0}.$$

This leads to the equation

$$\frac{2}{m-1} \cdot (X^\top \cdot X \cdot \mathbf{w} - X^\top \cdot \mathbf{y}) = \mathbf{0}.$$

This equation can be rewritten as

$$(X^\top \cdot X) \cdot \mathbf{w} = X^\top \cdot \mathbf{y} \quad (6.10)$$

and is known as the [normal equation](#).

Remark: Although the matrix $X^\top \cdot X$ will often be invertible, for numerical reasons it is not advisable to rewrite the normal equation as

$$\mathbf{w} = (X^\top \cdot X)^{-1} \cdot X^\top \cdot \mathbf{y}.$$

Instead, when solving the normal equation we will use the *Python* function `numpy.linalg.solve(A, b)`, which takes a matrix $A \in \mathbb{R}^{n \times n}$ and a vector $\mathbf{b} \in \mathbb{R}^n$ and solves the equation

$$A \cdot \mathbf{x} = \mathbf{b}. \quad \diamond$$

6.2.3 Implementation

Figure 6.4 on page 142 shows an implementation of general linear regression. The function

```
linear_regression(fileName, target, explaining, f)
```

takes four arguments:

1. `fileName` is a string that is interpreted as the name of a Csv file containing the data.
2. `target` is an integer that specifies the column that contains the dependent variable.
3. `explaining` is a list of integers. These integers specify the columns of the Csv file that contain the independent variables. These are also called the [explaining variables](#).
4. `f` is a function that takes one floating point argument and outputs one floating point function. This function is used to modify the dependent variable.

Later, when we call the function `linear_regression` to investigate the fuel consumption, we will use the function

$$x \mapsto \frac{1}{x}$$

to transform the variable *miles per gallon* into a variable expressing the fuel consumption. The reason is that there is reciprocal relation between the number of miles that a car drives on one gallon of gasoline and the fuel consumption: If you drive only a few miles with one gallon of gas, then your fuel consumption is high.

The function `linear_regression` works as follows:

1. It reads the specified Csv file line by line and stores the data in the variables `goal` and `Causes`. This is done by creating a `csv` reader in line 6. This reader returns the entries in the specified input file line by line in the for loop in line 10.
 - (a) `goal` is a list containing the data of the dependent variable that was specified by `target`. This list is initialized in line 8. It is filled with data in line 12. Note that the values stored in `goal` are transformed by the function `f`.

```

import csv
import numpy as np

def linear_regression(fileName, target, explaining, f):
    with open(fileName) as input_file:
        reader = csv.reader(input_file, delimiter=',')
        line_count = 0
        goal = []
        Causes = []
        for row in reader:
            if line_count != 0:
                goal.append(f(float(row[target])))
                Causes.append([float(row[i]) for i in explaining] + [1.0])
            line_count += 1
    m = len(goal)
    X = np.array(Causes)
    y = np.array(goal)
    w = np.linalg.solve(X.T @ X, X.T @ y)
    RSS = np.sum((X @ w - y) ** 2)
    yMean = np.sum(y) / m
    TSS = sum((y - yMean) ** 2)
    R2 = 1 - RSS / TSS
    return R2

def main():
    explaining = [1, 2, 3, 4, 5, 6]
    R2 = linear_regression("cars.csv", 0, explaining, lambda x: 1/x)
    print(f'portion of explained variance : {R2}')

```

Figure 6.4: General linear regression.

(b) **Causes** is a list of lists containing the data of the explaining variables. Every row in the Csv file corresponds to one list in the list **Causes**. Note also that we append the number 1.0 to each of these lists. This corresponds to adding a constant feature to our data and it enables us to use the normal equations as we have derived them.

2. **m** is the number of data pairs and is computed in line 15.
3. **Causes** is transformed into the NumPy matrix **X** in line 16.
4. **goal** is transformed into the NumPy array **y** in line 17.
5. The normal equation $(X^T \cdot X) \cdot \mathbf{w} = X^T \cdot \mathbf{y}$ is formulated and solved using the function `np.linalg.solve` in line 18.

Note that **X.T** is the transpose of the matrix **X**. The operator `@` computes the matrix product. Hence the expression `X.T @ X` is interpreted as $X^T \cdot X$. Similarly, the expression `X.T @ y` is interpreted as $X^T \cdot \mathbf{y}$.

6. The expression $(X @ w - y)$ is the difference between the predictions of the linear model and the observed values y . By squaring it and the summing over all entries of the resulting vector we compute is the residual sum of squares RSS in line 19.
7. `yMean` is the mean value of the variable y .
8. TSS is the total sum of squares.
9. R2 is the proportion of the explained variance.

When we run the program shown in Figure 6.4 on page 142 with the data stored in `cars.csv`, which had been discussed previously, then the proportion of explained variance is 88%. Considering that our data does not take the aerodynamics of the cars into take account, this seems like a reasonable result. A Jupyter notebook containing a similar program is available at

<https://github.com/karlstroetmann/Artificial-Intelligence/blob/master/Python/Chapter-05/6-Linear-Regression.ipynb>.

6.3 Polynomial Regression

Sometimes the model of linear regression is not flexible enough to capture the underlying patterns in data. One effective way to extend its capabilities without changing the fundamental algorithm is to add [higher order features](#). In general, these are products or powers of the given features. For example, assume we have a single feature x and a dependent variable y . If the relationship between x and y is non-linear, we can extend the feature matrix by adding powers of x , such as x^2, x^3 , etc. A model of the form:

$$y = w_0 + w_1 \cdot x + w_2 \cdot x^2$$

is still considered a [linear model](#) in the context of machine learning because it is linear in terms of the parameters w_0, w_1, w_2 , even though it describes a non-linear curve (a parabola) in terms of the feature x . Therefore, we can still use the [normal equation](#) to solve for the optimal weights.

6.3.1 Case Study: German Civil Servant Salaries

To illustrate this concept, we will examine a real-world dataset: the salaries of German Federal Civil Servants. In Germany, state employees are paid according to a table known as the [Bundes-Besoldungs-Ordnung](#). The pay grades range from A3 to A16. The monthly salaries are shown in Figure 6.5 on page 144. The columns correspond to the length of service (*Dienstalter*). A plot of the data for a given pay grade with the highest length of service are shown in Figure 6.6 on page 144.

Typically, the salary gaps widen as one moves up the hierarchy—the jump from A15 to A16 is significantly larger than the jump from A3 to A4. This suggests that a simple straight line (linear model) might be insufficient to predict the salary based on the grade.

Besoldungstabelle Beamte Bund 2025 - PROGNOSE								
€	1	2	3	4	5	6	7	8
A 3	2788.20	2846.21	2904.25	2950.96	2997.67	3044.39	3091.11	3137.81
A 4	2842.01	2911.35	2980.70	3035.89	3091.11	3146.32	3201.51	3252.49
A 5	2861.79	2948.13	3017.48	3085.45	3153.42	3222.78	3290.69	3357.24
A 6	2918.40	3018.93	3120.82	3198.68	3279.38	3357.24	3443.56	3518.59
A 7	3052.89	3142.09	3259.59	3379.86	3497.35	3616.27	3705.46	3794.62
A 8	3217.09	3324.69	3476.12	3629.03	3781.88	3888.04	3995.62	4101.79
A 9	3454.89	3561.06	3728.11	3897.95	4064.96	4178.50	4296.61	4411.80
A 10	3682.78	3828.58	4039.52	4251.38	4467.19	4617.38	4767.53	4917.77
A 11	4178.50	4401.57	4623.20	4846.28	4999.37	5152.47	5305.57	5458.71
A 12	4464.29	4728.20	4993.56	5257.45	5441.18	5621.98	5804.24	5989.42
A 13	5197.69	5445.55	5691.96	5939.83	6110.42	6282.50	6453.06	6620.73
A 14	5339.11	5658.42	5979.20	6298.51	6518.66	6740.33	6960.46	7182.11
A 15	6477.85	6766.56	6986.72	7206.91	7427.06	7645.77	7864.49	8081.71
A 16	7123.78	7459.16	7712.84	7966.56	8218.79	8473.97	8727.66	8978.48
Besoldungstabelle mit Monatswerten								

Figure 6.5: Salary table for civil servants in Germany.

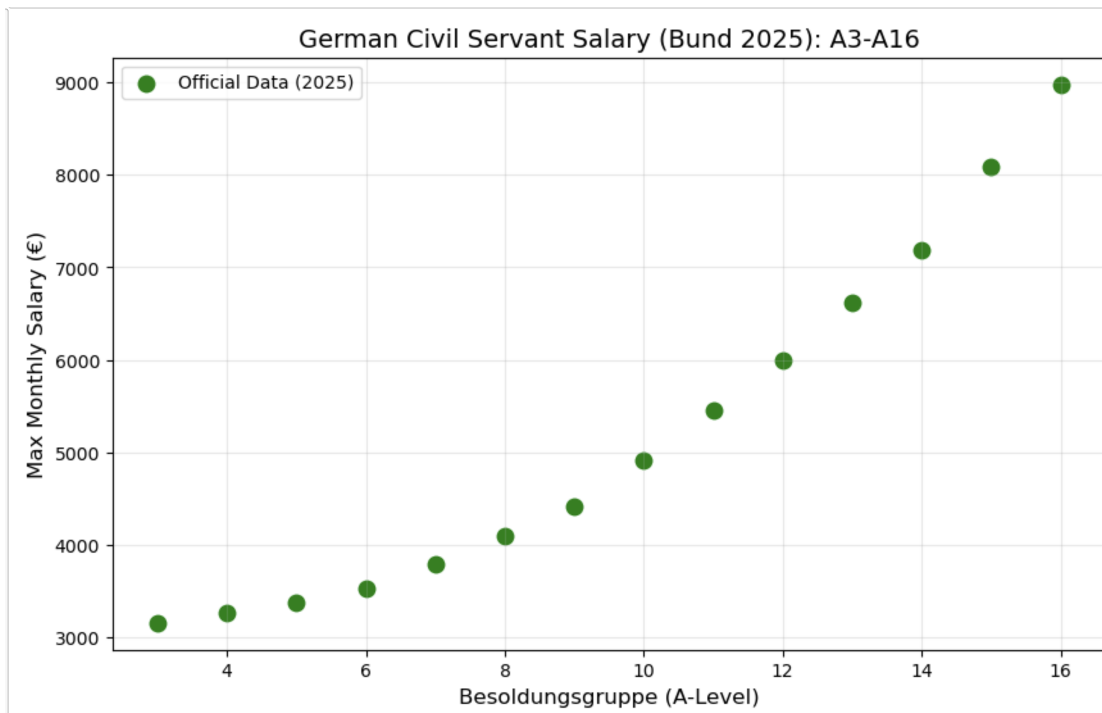


Figure 6.6: Salaries for civil servants w.r.t. to pay grade.

Data Preparation

First, we define our dataset manually using the official values valid from April 2025. The variable `grades_list` contains the pay grades (3 to 16), and `salary_list` contains the corresponding maximum monthly salaries in Euro.

```

1 # Grade (A3 to A16)
2 grades_list = list(range(3, 16+1))
3
4 # Maximum Monthly Salary in Euro
5 salary_list = [
6     3156.42, 3267.76, 3369.46, 3526.11, 3794.62,
7     4101.79, 4411.80, 4917.77, 5458.71, 5989.42,
8     6620.73, 7182.11, 8081.71, 8978.48
9 ]

```

Figure 6.7: Data entry for German Civil Servant Salaries.

The Linear Model (Degree 1)

We first attempt to model this data using simple linear regression. We construct a feature matrix X_{linear} where each row consists of the grade x and the constant 1 (for the bias term).

$$\mathbf{x}_{\text{linear}} = [x, 1]$$

We solve for the weights using the standard linear algebra library in NumPy.

```

1 import numpy as np
2
3 # Construct X as a list of lists: [[x, 1], ...]
4 X_linear = [[x, 1.0] for x in grades_list]
5
6 # Convert to NumPy arrays
7 X_linear = np.array(X_linear)
8 Y        = np.array(salary_list)
9
10 # Solve: (X^T * X) * w = X^T * y
11 w_linear = np.linalg.solve(X_linear.T @ X_linear, X_linear.T @ Y)
12
13 print(f"Linear Weights: Slope={w_linear[0]:.2f}, Intercept={w_linear[1]:.2f}")

```

Figure 6.8: Implementing Simple Linear Regression.

The resulting R^2 score for this model is approximately 0.93. While this appears high, a visual inspection of Figure 6.10 on page 147 reveals that the model significantly underestimates the salaries of top-level civil servants. The linear model cannot capture the accelerating growth (curvature) of the salary structure.

The Polynomial Model (Degree 2)

To improve the model, we introduce a **quadratic term**. We hypothesize that the salary relates to the pay grade via a polynomial of degree 2:

$$y = w_2 \cdot x^2 + w_1 \cdot x + w_0$$

To implement this, we do not need a new algorithm. We simply modify how we construct our feature matrix by providing **higher order features**. We construct a new matrix X_{poly} where each row contains the square of the grade, the grade itself, and the bias constant:

$$\mathbf{x}_{poly} = [x^2, x, 1]$$

This time, the resulting R^2 score is better than 0.9993%.

```

1 # Construct X as a list of lists: [[x^2, x, 1], ...]
2 X_poly = [[x**2, x, 1.0] for x in grades_list]
3
4 # Convert to NumPy array
5 X_poly = np.array(X_poly)
6
7 # Solve using Normal Equation
8 w_poly = np.linalg.solve(X_poly.T @ X_poly, X_poly.T @ Y)
9
10 print(f"Poly Weights: Quad={w_poly[0]:.2f}, Lin={w_poly[1]:.2f}, Bias={w_poly[2]:.2f}")

```

Figure 6.9: Implementing Polynomial Regression (Degree 2).

Exercise 18: The file “trees.csv”, which is available at

[Artificial-Intelligence/blob/master/Python/Chapter-05/trees.csv](https://github.com/karlstroetmann/Artificial-Intelligence/blob/master/Python/Chapter-05/trees.csv),

contains data about 31 lovely cherry trees from the Allegheny National Forest in Pennsylvania that have fallen prey to a **chainsaw massacre**. I have taken this data from

<http://www.statsci.org/data/general/cherry.txt>.

1. The first column of this CSV file contains the diameter of these trees at a height of 54 inches above the ground.
2. The second column lists the heights of these trees in feet.
3. The third column list the volume of wood that has been harvested from these trees. This volume is given in cubic inches.

Try to derive a model that estimates the volume of the trees from the diameter and the height. ◇

Exercise 19: The file “nba.csv”, which is available at

<https://github.com/karlstroetmann/Artificial-Intelligence/blob/master/Python/Chapter-05/nba.csv>,

contains various data about professional basket ball players.

1. The first column gives the name of the player.

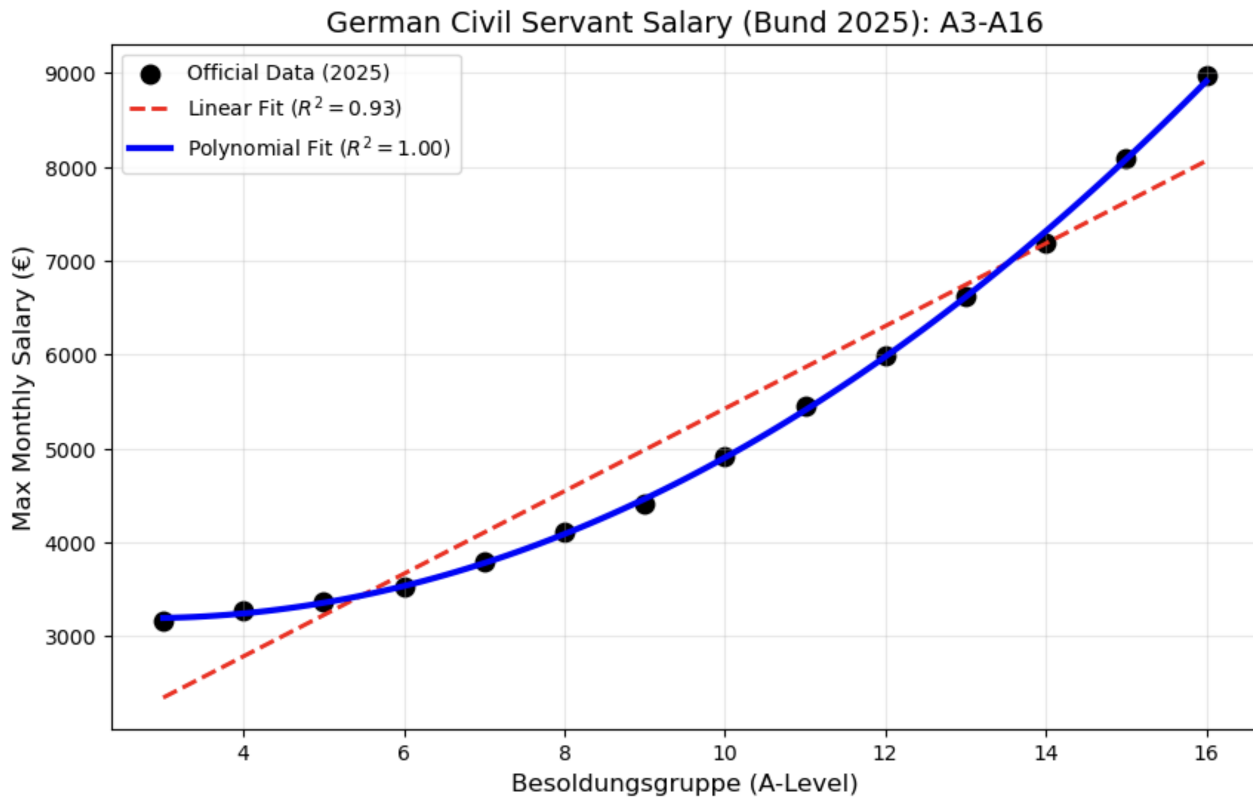


Figure 6.10: Attempting to match salaries with a linear and a quadratic model.

2. The second column specifies the position of the player.
3. The third column lists the height of the player.
4. The fourth column contains the weight of the player.
5. The fifth column shows the age of each player.

To what extent can you predict the weight of a player given his height and his age?

◇

6.4 Overfitting and Underfitting in Linear Regression

In machine learning, our primary goal is not merely to do well on the data we have seen (the training data), but to generalize to new data we have not seen (the test data). When building regression models, we often encounter two primary failure modes:

1. **Underfitting (High Bias):** This occurs when the model is too simple to capture the underlying structure of the data. For example, trying to fit a straight line to a parabolic curve. The model performs poorly on both the training set and the test set.
2. **Overfitting (High Variance):** This occurs when the model is too complex relative to the amount of training data. The model begins to "memorize" the random noise and fluctuations in the training set rather than learning the true signal. As a result, it performs exceptionally well on the training data but fails to generalize, leading to poor performance on the test set.

To illustrate these concepts, we analyze a real-world dataset using the Python library `scikit-learn`. The corresponding Jupyter Notebook is `9-Overfitting-SK.ipynb`.

We utilize the **Hitters** dataset, which contains statistics for Major League Baseball players (e.g., hits, runs, years in the league) and their salaries. Our objective is to predict a player’s salary based on these statistics. To demonstrate overfitting, we will simulate a *low data* scenario where the number of features is high relative to the number of training examples. We will incrementally add features to our model and observe the divergence between [training performance](#) and [test performance](#).

6.4.1 Data Preprocessing

First, we load the data using `pandas`. We perform necessary cleaning steps, such as removing columns that are not useful (like row names), dropping rows with missing salary information, and converting categorical variables (like League or Division) into numerical values using One-Hot Encoding.

```
import pandas as pd

df = pd.read_csv("Hitters.csv")
df = df.drop(columns=["rownames"])
df = df.dropna(subset=['Salary'])
df = pd.get_dummies(df, drop_first=True)
```

Figure 6.11: Data loading and preprocessing.

We discuss the important lines of the code fragment in [Figure 6.11](#).

1. Line 4 creates a [data frame](#) containing the data from the file `Hitters.csv`.
2. Line 5 removes the column with the title “`rownames`” as this column contains the names of the players.
3. Line 6 removes those columns where the attribute “`Salary`” is missing.
4. Line 7 is the most important line. Some of the columns have non-numerical entries. For example, there is a column with the attribute “`League`”. There are two leagues in American baseball:
 - (a) The [American League](#) is often referred to as the “Junior Circuit”.
 - (b) The [National League](#) was historically the first baseball league and is therefore referred to as the “Senior Circuit”.

In the `hitters` dataset, the values are represented as the values “A” and “N”. Furthermore, there are three divisions in American baseball:

- (a) The “`East Divison`”,
- (b) the “`Central Divison`”,
- (c) the “`West Divison`” divison.

In the `hitters` dataset, the values are represented as the values “E”, “C”, and “W”.

The purpose of the method `pd.get_dummies` is to transform these attributes into numerical values. This is done using [one hot encoding](#). We explain how this procedure works by example

of the attribute “Divison”. First, for each of the three values of this attribute `pd.get_dummies` adds a column to the dataframe `df` that encodes, whether the corresponding attribute is set. Therefore the new dataframe will have three new columns with the attributes

`Divison_E`, `Divison_C`, and `Divison_W`.

If the value of the attribute `Divison` was “C” for a player, then the values of these new attributes are set to the binary values

0, 1, and 0

respectively, because the player is not in the East Divison, hence `Divison_E` = 0, he is in the Central Divison, hence `Divison_C` = 1, and he is not in the West Divison, hence `Divison_W` = 0.

Observe that the column “`Divison_E`” is redundant. Once we know whether a player is in the West Divison or in the Central Divison, we know he can’t be in the East Divison. Similarly, if a player is neither in the West Divison nor in the Central Divison, he must be in the East Divison. Therefore, the column “`Divison_E`” is automatically dropped by the method `pd.get_dummies`, because we have added the argument “`drop_first=True`” to the call of the method `df.get_dummies`.

Actually, in the data set there is no player from the Central Divison. Therefore, there is only one divison and hence the resulting data frame only contains the attribute `Divison_W`. I have included the discussion above only to illustrate the concept of *one hot encoding*.

The same transformation is performed for the attribute “`League`”. Since there are only two values for this attribute, the attribute “`League`” is replaced by the attribute “`League_N`” which is 1 if the player is playing in the National League and 0 if he is playing in the American League.

6.4.2 Feature Selection by Importance

To see the effect of model complexity clearly, we do not add features randomly. Instead, we first determine which features are most predictive. We calculate the correlation of every feature with the target variable (`Salary`) and sort them by their absolute value.

```
1 # Calculate correlation of all columns with 'Salary'
2 correlations = df.corr()['Salary'].abs().sort_values(ascending=False)
3
4 # Drop 'Salary' itself from the list to get feature list
5 sorted_features = correlations.drop('Salary').index.tolist()
```

Figure 6.12: Sorting features by correlation importance.

Splitting the Data

We split our data into a training set and a test set using the function `train_test_split` from `scikit-learn`.

- **Training Set:** Used to calculate the model weights. We intentionally restrict this to only **50 samples** to make overfitting more likely.
- **Test Set:** Used strictly for evaluation.

```

1 from sklearn.model_selection import train_test_split
2
3 X = df[sorted_features]
4 y = df['Salary']
5
6 # Split: 50 samples for training, rest for testing
7 X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=50, random_state=42)

```

Figure 6.13: Creating a small training set to induce overfitting.

6.4.3 The Experiment

We now perform an iterative experiment. We start by training a model with only the single most important feature. Then, we train a model with the top 2 features, then top 3, and so on, until we use all available features. For each iteration, we use the `LinearRegression` class from `scikit-learn`. We use the `.fit()` method to train the model and the `.score()` method to calculate the R^2 (coefficient of determination).

```

1 from sklearn.linear_model import LinearRegression
2
3 train_scores = []
4 test_scores = []
5 num_features = []
6
7 # Loop from k=1 to all features
8 for k in range(1, len(sorted_features) + 1):
9     top_k_features = sorted_features[:k]
10
11     # Subset the data
12     X_train_k = X_train[top_k_features]
13     X_test_k = X_test[top_k_features]
14
15     # Initialize and Train
16     model = LinearRegression()
17     model.fit(X_train_k, y_train)
18
19     # Record Scores
20     train_scores.append(model.score(X_train_k, y_train))
21     test_scores.append(model.score(X_test_k, y_test))
22     num_features.append(k)

```

Figure 6.14: Iteratively training models with increasing complexity.

6.4.4 Results and Interpretation

When we plot the training and test scores against the number of features, we observe a classic overfitting pattern:

1. **Training Score (Blue Line):** This score generally increases as we add more features. With enough features, the model has enough degrees of freedom to fit the specific 50 data points in the training set almost perfectly.
2. **Test Score (Red Line):** Initially, the score increases as we add relevant information (features strongly correlated with salary). However, after a certain point (the "Sweet Spot"), adding more features causes the test score to drop.

Figure 6.15 on page 151 shows the results. The drop in the test score indicates that the model has started to overfit. The "less important" features added later in the loop (which have low correlation with salary) are essentially noise. The model uses these features to "explain" the random quirks of the small training set, which harms its ability to predict salaries for players in the test set.

This experiment demonstrates that **more features are not always better**. A simpler model (with fewer, higher-quality features) often generalizes better than a complex model.

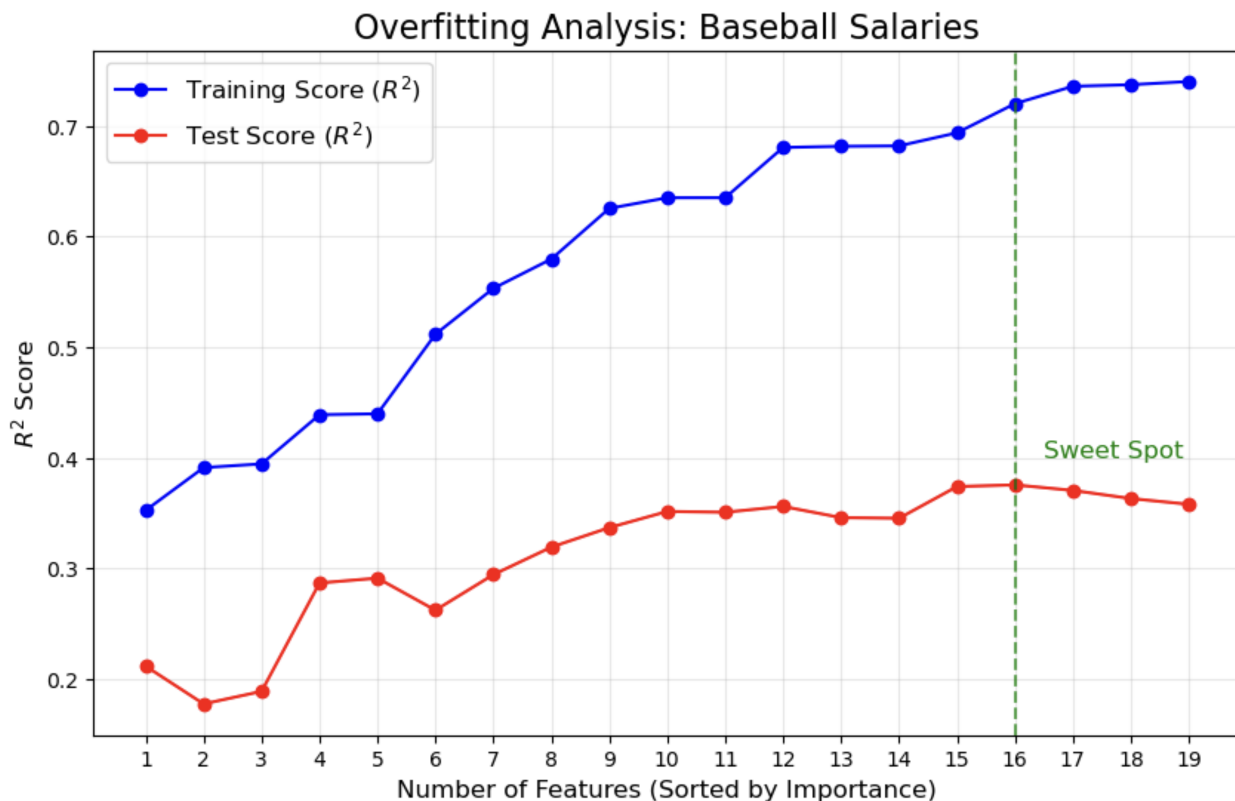


Figure 6.15: The effect of overfitting.

Remark: There is an interesting book with the title

Moneyball: The Art of Winning an Unfair Game

by Michael Lewis [Lew03]. The book follows Billy Beane, the General Manager of the Oakland Athletics baseball team. Faced with a tight budget and unable to compete with wealthy teams like the New York Yankees, Beane adopted a radical new approach to scouting and analyzing players. This book has been turned into the film *Moneyball*.

Beane and his assistant, Paul DePodesta, relied on *sabermetrics*—specialized baseball statistics—rather than traditional scouting methods (like a player’s appearance or raw athleticism). The

term [sabermetrics](#) was coined by Bill James [[Jam86](#)], deriving its name from the acronym for the Society for American Baseball Research (SABR). James defined the field as “the search for objective knowledge about baseball”, utilizing empirical statistical analysis to measure player performance more accurately than traditional methods.

They discovered that the market undervalued skills like the ability to get on base (On-Base Percentage or OBP) and overvalued traditional stats like Stolen Bases or Batting Average. This allowed the Oakland Athletics to buy players who were statistically effective but ignored by other teams because they were older, injured, or looked “funny” when they played. Despite having one of the lowest payrolls in Major League Baseball, the Oakland Athletics became one of the most successful teams, famously achieving a 20-game winning streak in 2002.

Chapter 7

Classification

One of the earliest applications of artificial intelligence is **classification**. A good example of classification is **spam detection**. A system for spam detection classifies an email as either spam or not spam, where “not spam” is often abbreviated as “ham”. To do so, it first computes various **features** of the email and then uses these features to determine whether the email is likely to be spam. For example, a possible feature would be the number of occurrences of the word “**pharmacy**” in the text of the email.

Another famous example of classification is **character recognition**. In character recognition, the input is an image depicting a character. This image is usually coded as a vector of gray values. The task is then to recognize the letter shown. While spam detection is a **binary** classification problem, character recognition is a **multi-class** classification problem, since there are 26 different characters.

7.1 Introduction

Formally, the **classification problem** in machine learning can be stated as follows: We are given a set of objects $S := \{o_1, \dots, o_n\}$ and a set of classes $C := \{c_1, \dots, c_k\}$. Furthermore, there exists a function

$$\text{classify} : S \rightarrow C$$

that assigns a class $\text{classify}(o)$ to every object $o \in S$. The set S is called the **sample space**. In the example of spam detection, the sample space S is the set of all emails that we might receive, i.e. S is the set of all strings, while the set of classes C is given as

$$C = \{\text{spam}, \text{ham}\}.$$

Our goal is to compute the function **classify**. In order to do this, we use an approach known as **supervised learning**: We take a subset $S_{\text{train}} \subseteq S$ of emails where we already know whether the emails are spam or not. This set S_{train} is called the **training set**. Next, we define a set of D **features** for every $o \in S$. These features have to be **computable**, i.e. we must have a function

$$\text{feature} : S \times \{1, \dots, D\} \rightarrow \mathbb{R}$$

such that $\text{feature}(o, j)$ computes the j -th feature and we have to be able to implement this function with reasonable efficiency. In general, the values of the features are real values. However, there are cases where these values are just Boolean values. If

$$\text{feature}(o, j) \in \mathbb{B} \quad \text{for all } o \in S,$$

then the j -th feature is called a **binary feature**. If we encode **False** as -1 and **True** as $+1$, then the set of Boolean values $\mathbb{B}$ can be considered a subset of $\mathbb{R}$ and hence Boolean features can be considered as real numbers. For example, in the case of spam detection, the first feature could be the occurrence of the string “**pharmacy**”. In this case, we would have

$$\text{feature}(o, 1) := \begin{cases} +1 & \text{if } \text{pharmacy} \in o, \\ -1 & \text{if } \text{pharmacy} \notin o, \end{cases}$$

i.e. the first feature would be to check whether the email o contains the string “**pharmacy**”. If we want to be more precise, we can instead define the first feature as

$$\text{feature}(o, 1) := \text{count}(\text{"pharmacy"}, o),$$

i.e. we would count the number of occurrences of the string “**pharmacy**” in our email o . As the value of

$$\text{count}(\text{"pharmacy"}, o)$$

is always a natural number, in this case the first feature would be a **discrete** feature. However, we can be even more precise: Instead of just counting the number of occurrences of “**pharmacy**” we can compute its **frequency**. After all, there is a difference whether the string “**pharmacy**” occurs once in an email containing but a hundred characters or whether it occurs once in an email with a length of several thousand characters. To this end, we would then define the first feature as

$$\text{feature}(o, 1) := \frac{\text{count}(\text{"pharmacy"}, o)}{\text{len}(o)},$$

where $\text{len}(o)$ defines the number of characters in the string o . In this case, the first feature would be a **continuous** feature and as this is the most general case, unless stated otherwise, we deal with the continuous case.

Having defined the features, we next need a **model** of the function **classify** that tries to approximate the function **classify** via the features. This model is given by a function

$$\text{model} : \mathbb{R}^d \rightarrow C$$

such that

$$\text{model}(\text{feature}(o, 1), \dots, \text{feature}(o, D)) \approx \text{classify}(o).$$

Using the function **model**, we can then approximate the function **classify** using a function **guess** that is defined as

$$\text{guess}(o) := \text{model}(\text{feature}(o, 1), \dots, \text{feature}(o, D))$$

Most of the time, the function **guess** will only **approximate** the function **classify**, i.e. we will have

$$\text{guess}(o) = \text{classify}(o)$$

for most objects of $o \in S$ but not for all of them. The **accuracy** of our model is defined as the fraction of those objects that are classified correctly, i.e.

$$\text{accuracy} := \frac{\text{card}(\{o \in S \mid \text{guess}(o) = \text{classify}(o)\})}{\text{card}(S)}.$$

To keep matters simple, we will assume that the sample space is finite.

The function **model** is usually determined by a set of **parameters** or **weights** $\mathbf{w}$. In this case, we have

$$\text{model}(\mathbf{x}) = \text{model}(\mathbf{x}, \mathbf{w})$$

where $\mathbf{x}$ is the feature vector, while $\mathbf{w}$ is the weight vector. Later, when we introduce **logistic regression**, we will assume that the number of weights is one more than the number of features. Then, the weights specify the relative importance of the different features. Furthermore, there will be a weight that is interpreted as a **bias term**.

When it comes to the choice of model, it is important to understand that, at least in practical

applications, all models are wrong. Nevertheless, some models are useful. There are two reasons for this:

1. We do not fully understand the function `classify` that we want to approximate by the function `model`.
2. In the most general setting, the function `classify` is so complex, that even if we could compute it exactly, the resulting model would be much too complicated.

The situation is similar in physics: Let us assume that we intend to model the fall of an object. A model that is a hundred percent accurate would have to include the following forces:

- (a) gravitational acceleration,
- (b) air friction,
- (c) tidal forces, i.e. the effects that the rotation of the earth has on moving objects,
- (d) celestial forces, i.e. the gravitational acceleration caused by celestial objects like the moon or the sun.
- (e) In the case of an iron object we have to be aware of the magnetic forces caused by the **geomagnetic field**.
- (f) To be fully accurate, we might have to include corrections from relativistic physics and even quantum physics.
- (g) As physics is not a closed subject, there might be other forces at work which we still do not know of.

Hence, a correct model would be so complicated that it would be unmanageable and therefore useless.

Let us summarize our introductory discussion of machine learning in general and classification in particular. A set S of objects and a set C of classes are given. Our goal is to approximate a function

$$\text{classify} : S \rightarrow C$$

using certain **features** of our objects. The function `classify` is then approximated using a function `model` as follows:

$$\text{model}(\text{feature}(o, 1), \dots, \text{feature}(o, D), \mathbf{w}) \approx \text{classify}(o).$$

The model depends on a vector of parameters $\mathbf{w}$. In order to **learn** these parameters, we are given a **training set** S_{train} that is a subset of S . As we are dealing with **supervised learning**, the function `classify` is known for all objects $o \in S_{\text{train}}$. Our goal is to determine the parameters $\mathbf{w}$ such that the number of mistakes we make on the training set is minimized.

7.1.1 Notation

We conclude this introductory section by fixing some notation. Let us assume that the objects $o \in S_{\text{train}}$ are numbered from 1 to n , while the features are numbered from 1 to d . Then we define

1. $\mathbf{x}_i := \langle \text{feature}(o_i, 1), \dots, \text{feature}(o_i, d) \rangle^\top$ for all $i \in \{1, \dots, n\}$.
i.e. $\mathbf{x}_i$ is a D -dimensional column vector that collects the features of the i -th training object.

2. $x_{i,j} := \text{feature}(o_i, j)$ for all $i \in \{1, \dots, n\}$ and $j \in \{1, \dots, d\}$.

i.e. $x_{i,j}$ is the j -th feature of the i -th object. The numbers $x_{i,j}$ are combined into the **feature matrix** X , i.e. we have

$$X = (x_{i,j})_{\substack{i \in \{1, \dots, n\} \\ j \in \{1, \dots, d\}}}$$

The matrix X is similar to the **feature matrix** that was introduced in the previous chapter.

3. $y_i := \text{classify}(o_i)$ for all $i \in \{1, \dots, n\}$

i.e. y_i is the class of the i -th object. These numbers are collected into the n -dimensional column vector $\mathbf{y}$.

7.1.2 Applications of Classification

Besides spam detection, there are many other classification problems that can be solved using machine learning. To give just one more example, imagine a doctor that receives a patient and examines her symptoms. In this case, the symptoms can be seen as the features of the patient. For example, these features could be

- (a) body temperature,
- (b) blood pressure,
- (c) heart rate,
- (d) body weight,
- (e) breathing difficulties,
- (f) age,

to name but a few of the possible features. Based on these symptoms, the doctor would then decide on an illness, i.e. the set of classes for the classification problem would be

$$\{\text{commonCold}, \text{pneumonia}, \text{asthma}, \text{flu}, \text{Covid-19}, \dots, \text{unknown}\}.$$

Hence, the task of disease diagnosis is a classification problem. This was one of the earliest problems that was tackled by artificial intelligence. As of today, **computer-aided diagnosis** and **clinical decision support systems** have been used for more than 40 years in many hospitals. Today, there are a number of diseases that can be diagnosed more accurately by a computer than by a specialist. One such example is the **diagnosis of heart disease**. Other applications of classification are the following:

- (a) image recognition,
- (b) speech recognition,
- (c) credit card fraud detection,
- (d) credit approval.

7.2 Digression: The Method of Gradient Ascent

In machine learning, it is often the case that we have to find either the **maximum** or the **minimum** of a function

$$f : \mathbb{R}^n \rightarrow \mathbb{R}.$$

For example, when we discuss **logistic regression** in the next section, we will have to find the maximum of the **likelihood** function. To proceed, let us introduce the **arg max** function. The idea is that

$$\hat{\mathbf{x}} = \arg \max_{\mathbf{x} \in \mathbb{R}^n} f$$

is that value of $\mathbf{x} \in \mathbb{R}^n$ that maximizes $f(\mathbf{x})$. Formally, we have

$$\forall \mathbf{x} \in \mathbb{R}^n : f(\mathbf{x}) \leq f\left(\arg \max_{\mathbf{x} \in \mathbb{R}^n} f\right).$$

Of course, the expression $\arg \max_{\mathbf{x} \in \mathbb{R}^n} f$ is only defined when the maximum of f is unique. If the function f is differentiable, we know that a necessary condition for a vector $\hat{\mathbf{x}} \in \mathbb{R}^n$ to satisfy

$$\hat{\mathbf{x}} = \arg \max_{\mathbf{x} \in \mathbb{R}^n} f \quad \text{is that we must have} \quad \nabla f(\hat{\mathbf{x}}) = \mathbf{0},$$

i.e. the **gradient** of f , which we will write as ∇f , vanishes at the maximum $\hat{\mathbf{x}}$.

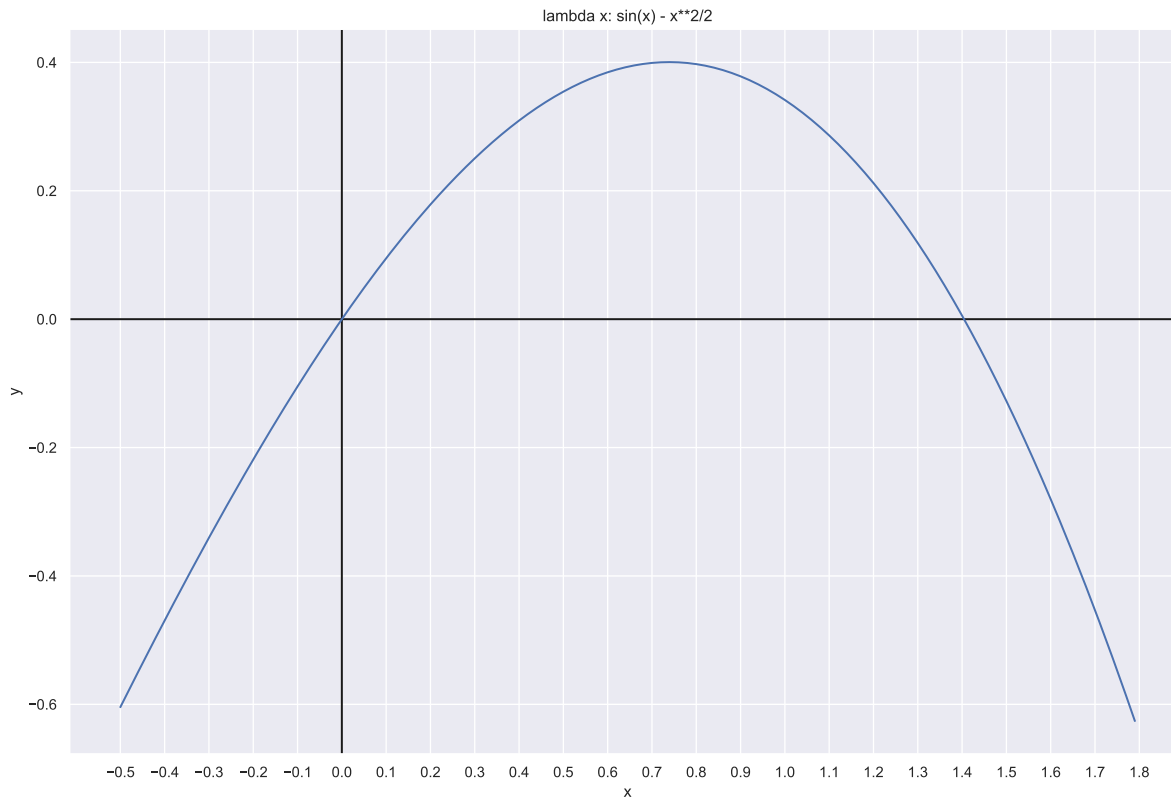


Figure 7.1: The function $x \mapsto \sin(x) - \frac{1}{2} \cdot x^2$.

Remember that the gradient of the function f is defined as the column vector

$$\nabla f := \left\langle \frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n} \right\rangle^\top.$$

Unfortunately, in many cases the equation

$$\nabla f(\hat{\mathbf{x}}) = \mathbf{0}$$

cannot be solved in **closed terms**. This is already true in the one-dimensional case, i.e. if $n = 1$. For example, consider the function $f : \mathbb{R} \rightarrow \mathbb{R}$ that is defined as

$$f(x) := \sin(x) - \frac{1}{2} \cdot x^2.$$

This function is shown in Figure 7.1 on page 157. From the graph of the function it is obvious that this function has a maximum somewhere between 0.6 and 0.8. In order to compute this maximum, we can compute the derivative of f . This derivative is given as

$$f'(x) = \cos(x) - x$$

As it happens, the equation $\cos(x) - x = 0$ does not seem to have a solution in **closed form**. Hence, we can only approximate the solution numerically via a sequence of numbers $(x_n)_{n \in \mathbb{N}}$ such that the limit $\lim_{n \rightarrow \infty} x_n$ exists and is a solution of the equation $\cos(x) - x = 0$, i.e. we want to have

$$\cos\left(\lim_{n \rightarrow \infty} x_n\right) = \lim_{n \rightarrow \infty} x_n.$$

The method of **gradient ascent** is a numerical method that can be used to find the maximum of a function

$$f : \mathbb{R}^n \rightarrow \mathbb{R}$$

numerically. The basic idea is to take a vector $\mathbf{x}_0 \in \mathbb{R}^n$ as the start value and define a sequence of vectors $(\mathbf{x}_n)_{n \in \mathbb{N}}$ such that we have

$$f(\mathbf{x}_{n+1}) \geq f(\mathbf{x}_n) \quad \text{for all } n \in \mathbb{N}.$$

Hopefully, this sequence will converge against $\hat{\mathbf{x}} = \arg \max_{\mathbf{x} \in \mathbb{R}^n} f$. If we do not really know where to start our search, we define $\mathbf{x}_0 := \mathbf{0}$. In order to compute $\mathbf{x}_{n+1}$ given $\mathbf{x}_n$, the idea is to move from $\mathbf{x}_n$ in that direction where we have the biggest change in the values of f . This direction happens to be the gradient of f at $\mathbf{x}_n$. Therefore, the definition of $\mathbf{x}_{n+1}$ is given as follows:

$$\mathbf{x}_{n+1} := \mathbf{x}_n + \alpha \cdot \nabla f(\mathbf{x}_n) \quad \text{for all } n \in \mathbb{N}_0.$$

Here, α is called the **step size** and is also known as the **learning rate**. It determines by how much we move in the direction of the gradient. In practise, it is best to adapt the step size dynamically during the iterations. The Python function shown in Figure 7.2 on page 160 demonstrates how this is done. The function `findMaximum` takes four arguments:

1. `f` is the function that is to be maximized. It is assumed that `f` takes a vector $\mathbf{x} \in \mathbb{R}^n$ as its input and that it returns a real number. Note that n might be 1. In that case the input to `f` is a real number.
2. `gradF` is the gradient of `f`. It takes a vector $\mathbf{x} \in \mathbb{R}^n$ as its input and returns the vector $\nabla f(\mathbf{x})$.
3. `start` is a vector from $\mathbb{R}^n$ that is used as the value of $\mathbf{x}_0$.
4. `eps` is the precision that we want to obtain when locating the maximum. We will have to say more on how `eps` is related to the precision later. As we are using double precision floating point arithmetic, it won't make sense to use a value for `eps` that is smaller than 10^{-15} .

Next, let us discuss the implementation of gradient ascent.

1. `x` is initialized with the parameter `start`. Hence, `start` is really the same as $\mathbf{x}_0$.
2. `fx` is the value $f(\mathbf{x})$.
3. `alpha` is the [learning rate](#). We initialize `alpha` as 1.0. The learning rate will be adapted dynamically.
4. The body of the `while` loop starting in line 6 executes one iteration of gradient ascent.
5. In each iteration, we store the values of $\mathbf{x}_n$ and $f(\mathbf{x}_n)$ in the variables `xOld` and `fOld`. This is needed since we need to ensure that the values of $f(\mathbf{x}_n)$ are increasing. If this value of $f(\mathbf{x}_{n+1})$ is not bigger than $f(\mathbf{x}_n)$ we revert to the old values.
6. Next, we compute $\mathbf{x}_{n+1}$ in line 8 using the formula

$$\mathbf{x}_{n+1} := \mathbf{x}_n + \alpha \cdot \nabla f(\mathbf{x}_n).$$

7. The corresponding value $f(\mathbf{x}_{n+1})$ is computed in line 9.
8. If we are unlucky, $f(\mathbf{x}_{n+1})$ is smaller than $f(\mathbf{x}_n)$ instead of bigger. This might happen if the learning rate α is too large. Hence, in this case we decrease the value of α , discard both $\mathbf{x}_{n+1}$ and $f(\mathbf{x}_{n+1})$ and start over again via the `continue` statement in line 13.
9. Otherwise, if $f(\mathbf{x}_{n+1})$ is indeed bigger than $f(\mathbf{x}_n)$, the vector $\mathbf{x}_{n+1}$ is a better approximation of the maximum than the vector $\mathbf{x}_n$. In this case, in order to increase the speed of the convergence of our algorithm we will then increase the learning rate α by 20%.
10. The idea of our implementation is to stop the iteration when the relative difference of $f(\mathbf{x}_{n+1})$ and $f(\mathbf{x}_n)$ is less than ε or, to be more precise, if

$$f(\mathbf{x}_{n+1}) < f(\mathbf{x}_n) \cdot (1 + \varepsilon).$$

As the sequence $(f(\mathbf{x}_n))_{n \in \mathbb{N}}$ will be monotonically increasing, i.e. we have

$$f(\mathbf{x}_{n+1}) \geq f(\mathbf{x}_n) \quad \text{for all } n \in \mathbb{N},$$

the condition given above is sufficient. Now, if the increment of $f(\mathbf{x}_{n+1})$ is less than $f(\mathbf{x}_n) \cdot (1 + \varepsilon)$ we assume that we have reached the maximum with the required precision. In this case we return both the value of `x` and the corresponding function value $f(\mathbf{x})$.

```

def findMaximum(f, gradF, start, eps):
    x      = start
    fx     = f(x)
    alpha  = 1.0
    while True:
        xOld, fOld = x, fx
        x += alpha * gradF(x)
        fx = f(x)
        if fx <= fOld:
            alpha *= 0.5
            x, fx = xOld, fOld
            continue
        else:
            alpha *= 1.2
        if abs(fx - fOld) <= abs(fx) * eps:
            return x, fx

```

Figure 7.2: The gradient ascent algorithm.

The implementation of gradient ascent given above is not the most sophisticated variant of this algorithm. Furthermore, there are algorithms that are more powerful than gradient ascent. The first of these methods is the [conjugate gradient method](#). A refinement of this method is the [BFGS-algorithm](#) that has been invented by Broyden, Fletcher, Goldfarb, and Shanno. Unfortunately, we do not have the time to discuss these algorithms. However, our implementation of gradient ascent is sufficient for our applications and as this is not a course on numerical analysis but rather on artificial intelligence we will not delve deeper into this topic but, instead, we refer readers interested in more efficient algorithms to the literature [\[Sny05\]](#). If you ever need to find the maximum of a function numerically, you should try to use a predefined library routine that implements a state of the art algorithm. For example, in [Python](#) the method [minimize](#) from the package `scipy.optimize` offers various algorithms for minimization.

7.3 Logistic Regression

If we have a model such that

$$\text{model}(\mathbf{x}, \mathbf{w}) \approx \text{classify}(\mathbf{x})$$

holds, then we want to choose the weight vector $\mathbf{w}$ in a way such that the accuracy

$$\text{accuracy}(\mathbf{w}) := \frac{\text{card}(\{\mathbf{o} \in S \mid \text{model}(\text{feature}(\mathbf{o}), \mathbf{w}) = \text{classify}(\mathbf{o})\})}{\text{card}(S)}$$

is maximized. However, there is a snag: The accuracy is not a smooth function of the weight vector $\mathbf{w}$. It can't be a smooth function because the number of errors of our model is a natural number and not a real number that could change smoothly when the weight vector $\mathbf{w}$ is changed. Hence, the accuracy is not differentiable as a function of the weight vector. The way to proceed is to work with [probabilities](#) instead. Instead of assigning a class to an object $\mathbf{o}$ we rather assign a [probability](#) p to the object $\mathbf{o}$ that measures how probable it is that object $\mathbf{o}$ has a given class c . Then we try to maximize this probability. In [logistic regression](#) we use a linear model that is combined with the [sigmoid function](#).

Before we can discuss the details of logistic regression we need to define this function and state some of its properties.

7.3.1 The Sigmoid Function

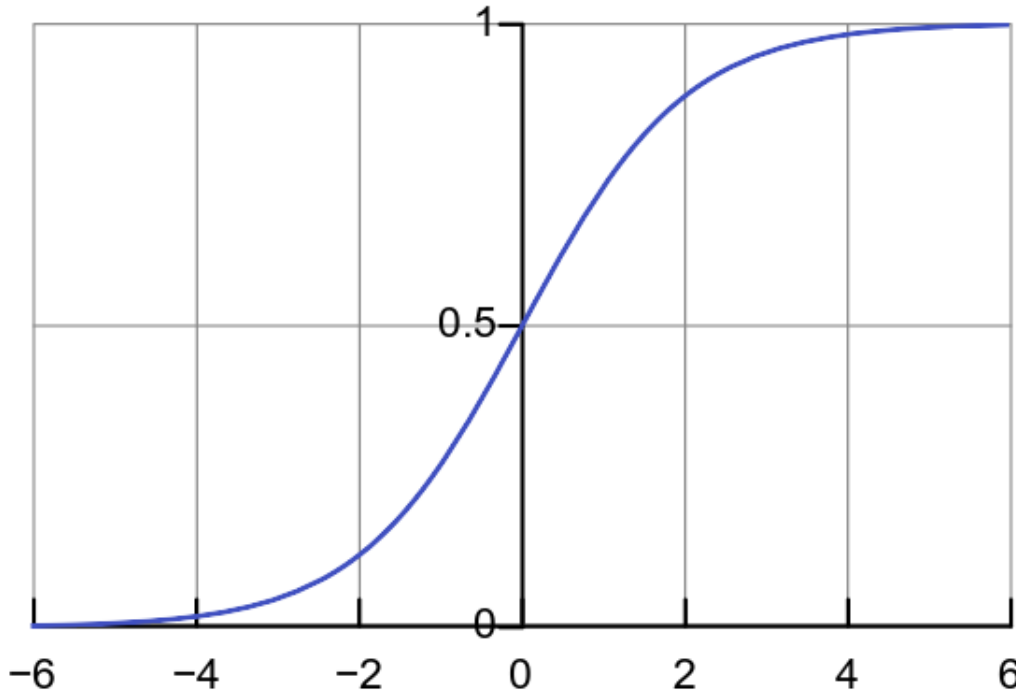


Figure 7.3: The sigmoid function.

Definition 45 (Sigmoid Function) The **sigmoid function** $S : \mathbb{R} \rightarrow [0, 1]$ is defined as

$$S(t) = \frac{1}{1 + \exp(-t)}.$$

Figure 7.3 on page 161 shows the sigmoid function. The sigmoid function is also known as the **logistic function**. $\diamond$

Let us note some immediate consequences of the definition of the sigmoid function. As we have

$$\lim_{x \rightarrow -\infty} \exp(-x) = \infty, \quad \lim_{x \rightarrow +\infty} \exp(-x) = 0, \quad \text{and} \quad \lim_{x \rightarrow \infty} \frac{1}{x} = 0,$$

the sigmoid function has the following properties:

$$\lim_{t \rightarrow -\infty} S(t) = 0 \quad \text{and} \quad \lim_{t \rightarrow +\infty} S(t) = 1.$$

As the sigmoid function is monotonically increasing, this shows that indeed

$$0 \leq S(t) \leq 1 \quad \text{for all } t \in \mathbb{R}.$$

Therefore, the value of the sigmoid function can be interpreted as a **probability**. Another important property of the sigmoid function is its symmetry. Figure 7.3 shows that if the sigmoid function is shifted down by $\frac{1}{2}$, the resulting function is **centrally symmetric**, i.e. we have

$$S(-t) - \frac{1}{2} = -\left(S(t) - \frac{1}{2}\right).$$

Adding $\frac{1}{2}$ on both sides of this equation shows that this is equivalent to the equation

$$S(-t) = 1 - S(t),$$

The proof of this fact runs as follows:

$$\begin{aligned} 1 - S(t) &= 1 - \frac{1}{1 + \exp(-t)} && \text{(by definition of } S(t)) \\ &= \frac{1 + \exp(-t) - 1}{1 + \exp(-t)} && \text{(common denominator)} \\ &= \frac{\exp(-t)}{1 + \exp(-t)} \\ &= \frac{1}{\exp(t) + 1} && \text{(expand fraction by } \exp(t)) \\ &= \frac{1}{1 + \exp(+t)} \\ &= S(-t) && \text{(by definition of } S(-t)). \quad \square \end{aligned}$$

The exponential function can be expressed via the sigmoid function. Let us start with the definition of the sigmoid function.

$$S(t) = \frac{1}{1 + \exp(-t)}$$

Multiplying this equation with the denominator yields

$$S(t) \cdot (1 + \exp(-t)) = 1.$$

Dividing both sides by $S(t)$ gives:

$$\begin{aligned} 1 + \exp(-t) &= \frac{1}{S(t)} \\ \Leftrightarrow \exp(-t) &= \frac{1}{S(t)} - 1 \\ \Leftrightarrow \exp(-t) &= \frac{1 - S(t)}{S(t)} \end{aligned}$$

We highlight this formula, as we need it later

$$\exp(-t) = \frac{1 - S(t)}{S(t)}. \tag{7.1}$$

If we take the reciprocal of both sides of this equation, we have

$$\exp(t) = \frac{S(t)}{1 - S(t)}.$$

Applying the natural logarithm on both sides of this equation yields

$$t = \ln \left(\frac{S(t)}{1 - S(t)} \right).$$

This shows that the inverse of the sigmoid function is given as

$$S^{-1}(y) = \ln\left(\frac{y}{1-y}\right).$$

This function is known as the **logit function**. Next, let us compute the derivative of $S(t)$, i.e. $S'(t) = \frac{dS}{dt}$. We have

$$\begin{aligned} S'(t) &= -\frac{-\exp(-t)}{(1 + \exp(-t))^2} \\ &= \exp(-t) \cdot S(t)^2 \\ &= \frac{1 - S(t)}{S(t)} \cdot S(t)^2 \quad (\text{by Equation 7.1}) \\ &= (1 - S(t)) \cdot S(t) \end{aligned}$$

We have shown

$$S'(t) = (1 - S(t)) \cdot S(t). \tag{7.2}$$

We will later need the derivative of the natural logarithm of the logistic function. We define

$$L(t) := \ln(S(t)).$$

Then we have

$$\begin{aligned} L'(t) &= \frac{S'(t)}{S(t)} \quad (\text{by the chain rule}) \\ &= \frac{(1 - S(t)) \cdot S(t)}{S(t)} \\ &= 1 - S(t) \\ &= S(-t) \quad (\text{by symmetry}) \end{aligned}$$

As this is our most important result, we highlight it:

$$L'(t) = S(-t) \quad \text{where} \quad L(t) := \ln(S(t)).$$

7.3.2 The Model of Logistic Regression

In logistic regression we deal with **binary classification**, i.e. we assume that we just need to decide whether a given object is a member of a given class or not. We use the following model to compute the **probability** that an object o with features $\mathbf{x}$ will be of the given class:

$$P(y = +1 \mid \mathbf{x}, \mathbf{w}) = S(\mathbf{x} \cdot \mathbf{w}).$$

Note that $P(y = +1 \mid \mathbf{x}, \mathbf{w})$ is the **conditional probability** that o has the given class, given its features $\mathbf{x}$ and the weights $\mathbf{w}$. The expression $\mathbf{x} \cdot \mathbf{w}$ denotes the **dot product** of the vectors $\mathbf{x}$ and $\mathbf{w}$, i.e. we have

$$\mathbf{x} \cdot \mathbf{w} = \sum_{i=1}^d x_i \cdot w_i,$$

where D is the number of features. To simplify the notation, it is assumed that $\mathbf{x}$ contains a **constant feature** that always takes the value of 1. If you see this model for the first time, you might think that it is not very general and that it can only be used in very special circumstances. However, the features x_i can be functions of arbitrary complexity and hence this model is much more general than it appears initially.

We assume that y can only take the values $+1$ or -1 , e.g. in the example of spam detection $y = 1$ if the email is spam and $y = -1$ otherwise. Since complementary probabilities add up to 1, we have

$$P(y = -1 \mid \mathbf{x}, \mathbf{w}) = 1 - P(y = +1 \mid \mathbf{x}, \mathbf{w}) = 1 - S(\mathbf{x} \cdot \mathbf{w}) = S(-\mathbf{x} \cdot \mathbf{w}).$$

Hence, we can combine the equations for $P(y = -1 \mid \mathbf{x}, \mathbf{w})$ and $P(y = +1 \mid \mathbf{x}, \mathbf{w})$ into a single equation

$$P(y \mid \mathbf{x}, \mathbf{w}) = S(y \cdot (\mathbf{x} \cdot \mathbf{w})).$$

Given N objects $o_1, \dots, o_n$ with feature vectors $\mathbf{x}_1, \dots, \mathbf{x}_n$ and classes $y_1, \dots, y_n$, we want to determine the weight vector $\mathbf{w}$ such that the **likelihood** $\ell(\mathbf{X}, \mathbf{y})$ of all of our observations is maximized, where $\mathbf{X}$ is the **feature matrix** that is defined as

$$\mathbf{X} := \begin{pmatrix} (\mathbf{x}^{(1)})^\top \\ \vdots \\ (\mathbf{x}^{(n)})^\top \end{pmatrix}.$$

This approach is called the **maximum likelihood estimation** of the weights. As we assume that the probabilities of different observations are independent, the individual probabilities have to be multiplied to compute the overall likelihood $\ell(\mathbf{X}, \mathbf{y}, \mathbf{w})$ of a given training set:

$$\ell(\mathbf{X}, \mathbf{y}, \mathbf{w}) = \prod_{i=1}^n P(y_i \mid \mathbf{x}_i, \mathbf{w}).$$

Since it is easier to work with sums than with products, instead of maximizing the function $\ell(\mathbf{X}, \mathbf{y}, \mathbf{w})$ we instead maximize the logarithm of $\ell(\mathbf{X}, \mathbf{y}, \mathbf{w})$. This logarithm is called the **log-likelihood** and is defined as

$$\ell\ell(\mathbf{X}, \mathbf{y}, \mathbf{w}) := \ln(\ell(\mathbf{X}, \mathbf{y}, \mathbf{w})).$$

As the natural logarithm is a **monotonically increasing** function, the functions $\ell(\mathbf{X}, \mathbf{y}, \mathbf{w})$ and $\ell\ell(\mathbf{X}, \mathbf{y}, \mathbf{w})$ take their maximum at the same value of $\mathbf{w}$. As we have

$$\ln(a \cdot b) = \ln(a) + \ln(b),$$

the natural logarithm of the likelihood is

$$\ell\ell(\mathbf{X}, \mathbf{y}, \mathbf{w}) = \sum_{i=1}^n \ln(S(y_i \cdot (\mathbf{x}_i \cdot \mathbf{w}))) = \sum_{i=1}^n L(y_i \cdot (\mathbf{x}_i \cdot \mathbf{w})).$$

Our goal is to choose the weights $\mathbf{w}$ such that the likelihood is maximized. Since this is the same as maximizing the log-likelihood, we need to determine the gradient of the log-likelihood with respect to the weights w_j , i.e. we need to compute the partial derivatives

$$\frac{\partial}{\partial w_j} \ell\ell(\mathbf{X}, \mathbf{y}, \mathbf{w}) \quad \text{for all } j \in \{1, \dots, d\}.$$

In order to compute the partial derivative of $\ell\ell(\mathbf{X}, \mathbf{y}, \mathbf{w})$ with respect to the coefficients $\mathbf{w}$ we need to compute the partial derivative of the dot product $\mathbf{x}_i \cdot \mathbf{w}$ with respect to the weights w_j . We define

$$h(\mathbf{w}) := \mathbf{x}_i \cdot \mathbf{w} = \sum_{k=1}^d x_{i,k} \cdot w_k.$$

Then we have

$$\frac{\partial}{\partial w_j} h(\mathbf{w}) = \frac{\partial}{\partial w_j} \sum_{k=1}^d x_{i,k} \cdot w_k = \sum_{k=1}^d x_{i,k} \cdot \frac{\partial}{\partial w_j} w_k = \sum_{k=1}^d x_{i,k} \cdot \delta_{j,k} = x_{i,j}.$$

Now we are ready to compute the partial derivative of $\ell(\mathbf{X}, \mathbf{y}, \mathbf{w})$ with respect to $\mathbf{w}$:

$$\begin{aligned} & \frac{\partial}{\partial w_j} \ell(\mathbf{X}, \mathbf{y}, \mathbf{w}) \\ &= \frac{\partial}{\partial w_j} \sum_{i=1}^n L(y_i \cdot (\mathbf{x}_i \cdot \mathbf{w})) \\ &= \sum_{i=1}^n y_i \cdot x_{i,j} \cdot S(-y_i \cdot (\mathbf{x}_i \cdot \mathbf{w})), \quad \text{since} \quad \frac{dL(x)}{dx} = S(-x). \end{aligned}$$

Hence, the partial derivative of the log-likelihood function is given as follows:

$$\frac{\partial}{\partial w_j} \ell(\mathbf{X}, \mathbf{y}, \mathbf{w}) = \sum_{i=1}^n y_i \cdot x_{i,j} \cdot S(-y_i \cdot \mathbf{x}_i \cdot \mathbf{w})$$

Next, we have to find the value of $\mathbf{w}$ such that

$$\sum_{i=1}^n y_i \cdot x_{i,j} \cdot S(-y_i \cdot \mathbf{x}_i \cdot \mathbf{w}) = 0 \quad \text{for all } j \in \{1, \dots, d\}.$$

These are d equations for the d variables $w_1, \dots, w_d$. Due to the occurrence of the sigmoid function, these equations are nonlinear. Unfortunately, these equations do not have a solution in closed terms. Nevertheless, our computation of the gradient of the log-likelihood was not in vain: We will use the method of [gradient ascent](#) to find the value of $\mathbf{w}$ that maximizes the log-likelihood. This method has been outlined in the previous section.

7.3.3 Implementing Logistic Regression

In this section we will give a simple implementation of logistic regression. We will use our implementation of logistic regression to predict whether a student will pass or fail a given exam. Figure 7.4 shows a [Csv](#) file I have borrowed from the Wikipedia page on [logistic regression](#) that contains the data we are going to explore. Concretely, this file stores the hours a student has learned for a particular exam and the fact whether the student has passed the exam or has failed. A passed exam is encoded as the number 1, while a failed exam is encoded as 0. The first column of the file stores these numbers. The second column stores the number of hours that the student has learned in order to pass the exam.

The program shown in Figure 7.5 on page 167 implements logistic regression. As there are a number of subtle points that might easily be overlooked otherwise, we proceed to discuss this program line by line.

	Pass,	Hours
1		
2	0,	0.50
3	0,	0.75
4	0,	1.00
5	0,	1.25
6	0,	1.50
7	0,	1.75
8	1,	1.75
9	0,	2.00
10	1,	2.25
11	0,	2.50
12	1,	2.75
13	0,	3.00
14	1,	3.25
15	0,	3.50
16	1,	4.00
17	1,	4.25
18	1,	4.50
19	1,	4.75
20	1,	5.00
21	1,	5.50

Figure 7.4: Results of an exam.

```

import numpy as np

# compute  $\frac{1}{1 + \exp(-t)}$ 
def sigmoid(t):
    return 1.0 / (1.0 + np.exp(-t))

# compute  $\ln\left(\frac{1}{1 + \exp(-t)}\right)$  and avoid overflow
def logSigmoid(t):
    if t > -100:
        return -np.log(1.0 + np.exp(-t))
    else:
        return t

def ll(X, y, w):
    """
    given the matrix X and the observations y,
    return the log likelihood for the weight vector w
    """
    return np.sum([logSigmoid(y[i] * (X[i] @ w)) for i in range(len(X))])

def gradLL(X, y, w):
    """
    Compute the gradient of the log-likelihood with respect to w
    """
    Gradient = []
    for j in range(len(X[1])):
        L = [y[i]*X[i][j]*sigmoid(-y[i] * (X[i] @ w)) for i in range(len(X))]
        Gradient.append(sum(L))
    return np.array(Gradient)

```

Figure 7.5: An implementation of logistic regression.

1. First, we `import` the module `numpy`. This module provides us with the functions `log` and `exp` for computing the logarithm and the exponential of a number or a vector. Furthermore, we need this module because the gradient of the log-likelihood is a vector and for efficiency reasons this vector should be stored as a NumPy array.

2. Line 3 implements the sigmoid function

$$S(x) = \frac{1}{1 + \exp(-x)}.$$

Since we are using NumPy to compute the exponential function, the parameter t that is used in our implementation can also be a vector.

3. Line 7 starts the implementation of the natural logarithm of the sigmoid function, i.e. we implement

$$L(x) = \ln(S(x)) = \ln\left(\frac{1}{1 + \exp(-x)}\right) = -\ln(1 + \exp(-x)).$$

The implementation is more complicated than you might expect. The reason has to do with [numerical overflow](#). Consider values of x that are smaller than, say, -1000 . The problem is that the expression `exp(1000)` evaluates to the value `inf`, which represents the mathematical concept of infinity, denoted as ∞ . But then $1 + \exp(1000)$ is also `inf` and finally `log(1 + exp(1000))` is still `inf`. However, in reality we have

$$\ln(1 + \exp(1000)) \approx 1000$$

because $\exp(1000)$ is so big that adding 1 to it does not make much of a difference. In fact, if we use *Python*'s 64-bit double arithmetic, then even adding 1 to $\exp(100)$ does not make any difference at all as w.r.t. machine arithmetic we have

$$1.0 \oplus 2.6881171418161356 \cdot 10^{43} = 2.6881171418161356 \cdot 10^{43}.$$

Here, $\oplus$ denotes machine addition.

The argument that $\ln(1 + \exp(1000)) \approx 1000$ works as follows:

$$\begin{aligned} \ln(1 + \exp(x)) &= \ln(\exp(x) \cdot (1 + \exp(-x))) \\ &= \ln(\exp(x)) + \ln(1 + \exp(-x)) \\ &= x + \ln(1 + \exp(-x)) \\ &\approx x + \ln(1) + \exp(-x) && \text{Taylor expansion of } \ln(1 + x) \\ &= x + 0 + \exp(-x) \\ &\approx x && \text{since } \exp(-x) \approx 0 \text{ for large } x \end{aligned}$$

This is the reason that `logSigmoid` returns `x` if the value of `x` is less than -100 .

4. The function `ll(X, y, w)` defined in line 14 computes the log-likelihood of the parameter `w` given the available data `X` and `y`. We have

$$\ell\ell(\mathbf{X}, \mathbf{y}, \mathbf{w}) = \sum_{i=1}^n L(y_i \cdot (\mathbf{x}_i \cdot \mathbf{w})).$$

Here L denotes the natural logarithm of the sigmoid of the argument. It is assumed that $\mathbf{X}$ is the feature matrix. Every observation corresponds to a row in this matrix, i.e. the vector $\mathbf{x}_i$ is the feature vector containing the features of the i -th observation. $\mathbf{y}$ is a vector describing

the outcomes, i.e. the elements of this vector are either $+1$ or -1 . Finally, $\mathbf{w}$ is the vector of coefficients.

5. The function `gradLL( $\mathbf{x}, \mathbf{y}, \mathbf{w}$ )` in line 21 computes the gradient of the log-likelihood according to the formula

$$\frac{\partial}{\partial w_j} \ell(\mathbf{X}, \mathbf{y}, \mathbf{w}) = \sum_{i=1}^n y_i \cdot x_{i,j} \cdot S(-y_i \cdot \mathbf{x}_i \cdot \mathbf{w}).$$

The different components of this gradient are combined into a vector. The arguments are the same as the arguments to the log-likelihood.

6. Finally, the function `logisticRegressionFile` that is shown in Figure 7.6 takes one argument. This argument is the name of the Csv file containing the data that are to be analysed. The task of this function is to read the Csv file, convert the data in the feature matrix $\mathbf{X}$ and the vector $\mathbf{y}$, and then to use the method of gradient ascent to find the weight vector $\mathbf{w}$ that maximize the likelihood. In detail this function works as follows.

- (a) The `with` statement that extends from line 34 to line 43 reads the data in the file that is specified by the parameter `name`.
- (b) After the data has been read, the list `Pass` contains a list of floating point numbers that are either 0 or 1 specifying whether the student has passed the exam, while the list `Hours` contains the numbers of hours that the students have spent studying.
- (c) These data are converted into the NumPy arrays `x` and `y` in line 44 and 45.
- (d) `n` is the number of data points we have, i.e. it is the number of students.
- (e) We reshape the vector `x` into a matrix `X` in line 47. As there is only a single feature, namely the hours a student has studied, all rows of this matrix have a length of 1.
- (f) Next, we prepend a column of ones to this matrix. This is done in line 48. This frees us from dealing explicitly with a bias term in our model.
- (g) In logistic regression we assume that the entries of the vector $\mathbf{y}$ are either $+1$ or -1 . As the data provided in our input file contains 1 and 0, we need to apply a function that maps 1 to $+1$ and 0 to -1 . The function

$$y \mapsto 2 \cdot y - 1$$

fits this job description and is applied to transform the vector $\mathbf{y}$ appropriately in line 49.

- (h) Now we are ready to run gradient ascent. As the start vector we use a vector containing only zeros. This vector is defined in line 50. The precision we use is 10^{-8} . We want to maximize the log-likelihood of a given weight vector $\mathbf{w}$. Hence we define the function `f( $\mathbf{w}$ )` as `ll( $\mathbf{X}, \mathbf{y}, \mathbf{w}$ )` in line 52, while the gradient of this function is defined in line 53. Line 54 call the function `gradient_ascent` that computes the value of $\mathbf{w}$ that maximizes the log-likelihood.

```

import csv
import gradient_ascent

def logisticRegression(name):
    with open(name) as file:
        reader = csv.reader(file, delimiter=',')
        count = 0 # line count
        Pass = []
        Hours = []
        for row in reader:
            if count != 0: # skip header
                Pass.append(float(row[0]))
                Hours.append(float(row[1]))
            count += 1
    y = np.array(Pass)
    x = np.array(Hours)
    n = len(y)
    X = np.reshape(x, (n,1))
    X = np.append(np.ones((n, 1)), X, axis=-1)
    y = 2 * y - 1
    start = np.zeros((2,))
    eps = 10 ** -8
    f = lambda w: ll(X, y, w)
    gradF = lambda w: gradLL(X, y, w)
    w, _, _ = gradient_ascent.findMaximum(f, gradF, start, eps)
    return w

```

Figure 7.6: The function `logisticRegression`.

If we run the function `logisticRegressionFile` using the data shown in Figure 7.4 the resulting values of the weight vector `w` are

```
[-4.0746468959343405, 1.5033787070592017]
```

This shows that the probability $P(h)$ that a student who has studied for h hours will pass the exam is given approximately as follows:

$$P(h) \approx \frac{1}{1 + \exp(4.1 - 1.5 \cdot h)}$$

Figure 7.7 shows a plot of this probability $P(x)$.

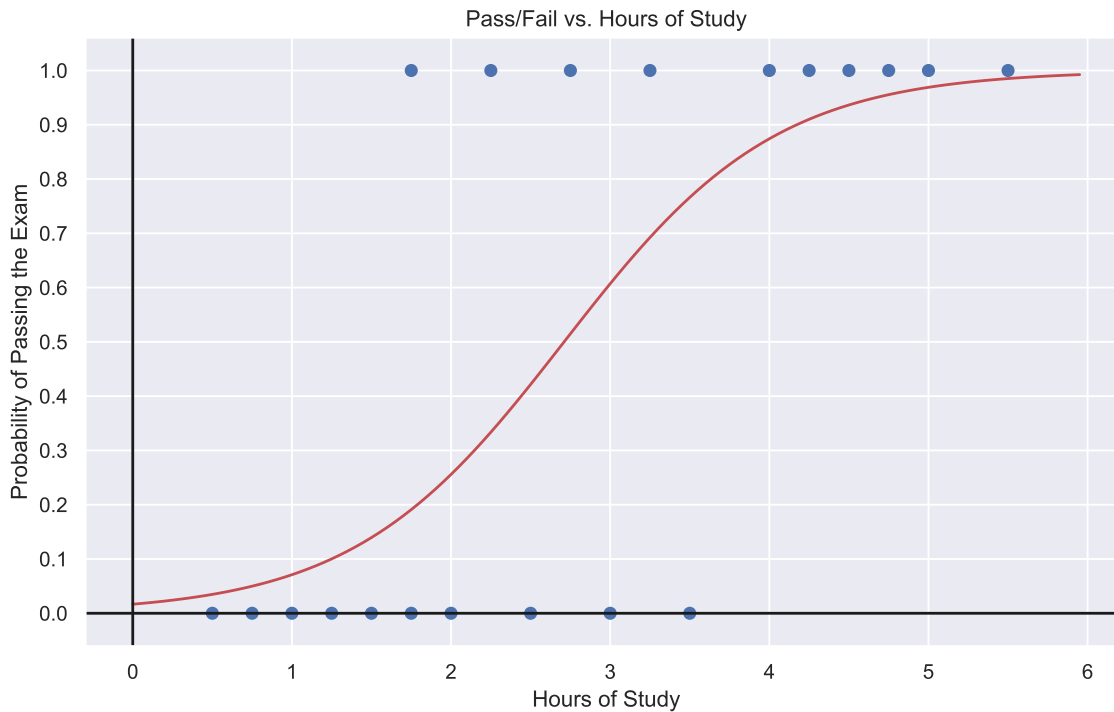


Figure 7.7: Probability of passing an exam versus hours of studying.

7.3.4 Logistic Regression with SciKit-Learn

In this section we discuss how linear regression is done in the SciKit-Learn environment. We will improve on the previous example and study the data that is shown in Figure 7.8 on page 172. This Csv file contains data about a fictional exam. The first column indicates whether the student has passed or failed the exam. A passed exam is encoded as the integer 1, while a failed exam is encoded as the integer 0. The second column contains the number of hours that the student has studied for the exam. The third column contains the *intelligence quotient*, abbreviated as IQ. To better understand the data, we first plot it. This plot is shown in Figure 7.9 on page 173. The horizontal axis is used for the hours of study, while the vertical axis shows the IQ of the student. Students who have passed the exam are shown as blue dots, while those students who have failed their exam are shown as red dots.

When we inspect the diagram shown in Figure 7.9 we see that there are two outliers: There is one student who failed although he has an IQ of 125 and he did study for 3.5 hours. Maybe he was still drunk when he had to write the exam. There is also a student with an IQ of 104 who did pass while only studying for 2 hours. He just might have gotten lucky. We expect logistic regression to classify all other students correctly.

1	Pass,Hours,IQ
2	0,0.50,110
3	0,0.75,95
4	0,1.00,118
5	0,1.25,97
6	0,1.50,100
7	0,1.75,110
8	0,1.75,115
9	1,2.00,104
10	1,2.25,120
11	0,2.50,98
12	1,2.75,118
13	0,3.00,88
14	1,3.25,108
15	0,3.50,125
16	1,4.00,109
17	1,4.25,110
18	1,4.50,112
19	1,4.75,97
20	1,5.00,102
21	1,5.50,109

Figure 7.8: Results of an exam given hours of study and IQ.

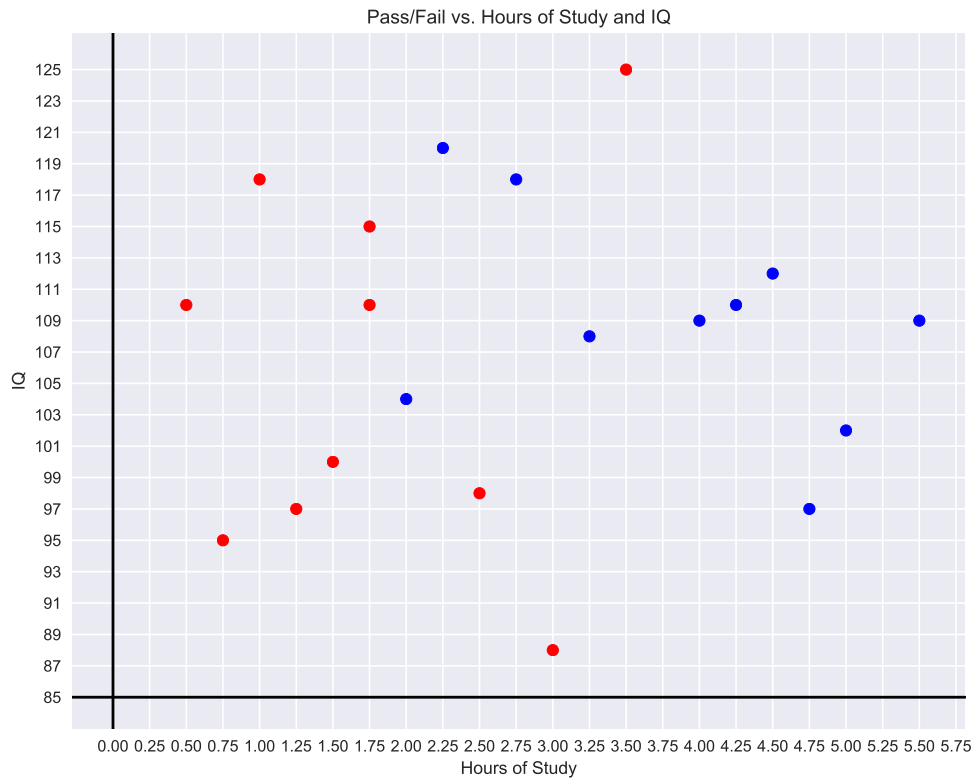


Figure 7.9: Probability of passing an exam versus hours of studying.

```

1  import numpy                as np
2  import pandas               as pd
3  import sklearn.linear_model as lm
4
5  ExamDF = pd.read_csv('exam-iq.csv')
6  X      = np.array(ExamDF[['Hours', 'IQ']])
7  Y      = np.array(ExamDF['Pass'], dtype=float)
8  model  = lm.LogisticRegression(C=10.000, tol=1e-6, solver='newton-cg')
9  M      = model.fit(X, Y)
10   $\vartheta_0$     = M.intercept_[0]
11   $\vartheta_1, \vartheta_2$  = M.coef_[0]
12  errors = np.sum(np.abs(Y - model.predict(X)))
13  print((len(Y) - errors) / len(Y))

```

Figure 7.10: Logistic Regression using SciKit-Learn

Figure 7.10 on page 173 shows a *Python* script that creates a logistic regression classifier with the help of the SciKit-Learn package.

1. In the first three lines we import the necessary modules. The support for logistic regression is located in the module `sklearn.linear_model`.
2. In line 5, the data from the file “`exam-iq.csv`” is read as a Pandas `DataFrame`.
3. Line 6 creates the feature matrix **X** by extracting the two independent variables “Hours” and “IQ”.
4. Line 7 extracts the dependent variable “Pass”. Since this variable is stored as an integer in the CSV file, we convert it into a floating point number. This is necessary because the method `fit` that we use later expects the dependent variable to be encoded as a floating point number.
5. Line 8 creates an object of class `LogisticRegression`. This object is initialized with a number of parameters:
 - (a) `C` specifies the amount of [regularization](#). We will discuss the concept of regularization later when we discuss [polynomial logistic regression in the next section](#). In this example we do not need any regularization. Setting `C` to a high value like 10 000 prevents regularization.
 - (b) `tol` is the tolerance that specifies when the iterative algorithm to maximize the log-likelihood should stop.
 - (c) `solver` specifies the numerical method that is used to find the maximum of the log-likelihood. By choosing “`newton-cg`” we specify that the [conjugate gradient](#) method should be used. This method is more sophisticated than gradient descent, but as this is not a course on numerical optimization we do not have the time to discuss it.
6. All the real work is happening in line 9, because there we use the method `fit` to create the logistic regression model.
7. The next two lines are needed to extract the coefficients ϑ_0 , ϑ_1 , and ϑ_2 that specify the logistic model. According to the model we have learned, the probability $P(h)$ that a student, who has learned for h hours and has an IQ of q , will pass the exam, is given as

$$P(Y = 1|h, q) = S(\vartheta_0 + \vartheta_1 \cdot h + \vartheta_2 \cdot q)$$

In general, the model predicts that she will pass the exam if

$$S(\vartheta_0 + \vartheta_1 \cdot h + \vartheta_2 \cdot q) \geq \frac{1}{2}$$

and that is the case if and only if

$$\vartheta_0 + \vartheta_1 \cdot h + \vartheta_2 \cdot q \geq 0.$$

This can be rewritten as follows:

$$q \geq -\frac{\vartheta_0 + \vartheta_1 \cdot h}{\vartheta_2}.$$

The [decision boundary](#) are those values of (h, q) such that $P(h, q) = \frac{1}{2}$. This set of values satisfies the linear equation

$$q = -\frac{\vartheta_0 + \vartheta_1 \cdot h}{\vartheta_2}.$$

We have plotted this decision boundary as a green line in Figure 7.11 on page 175. Everybody who is located to the right of this line is predicted to pass the exam, while everybody who is located to the left is predicted to fail.

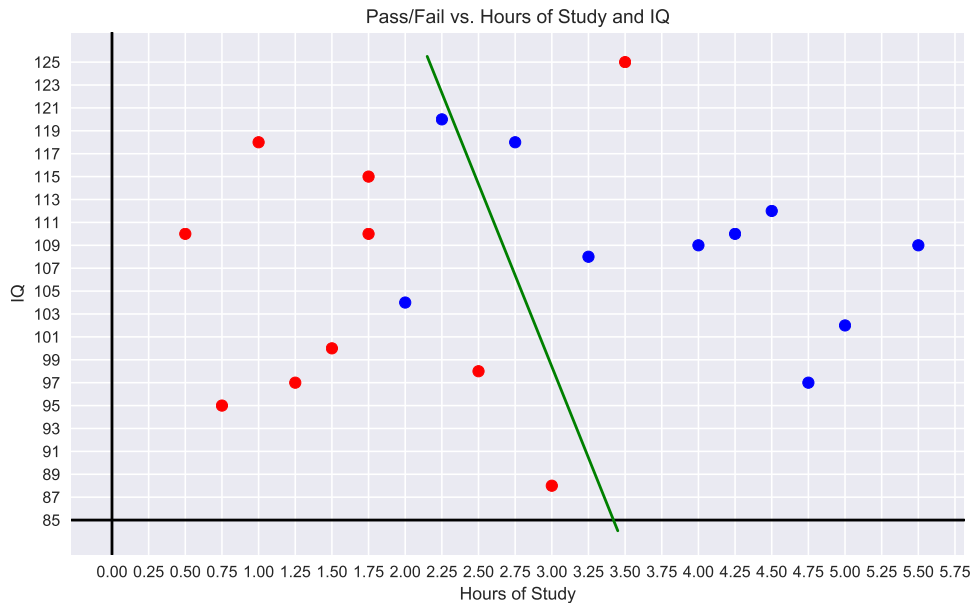


Figure 7.11: Probability of passing an exam versus hours of studying.

8. Line 12 computes the number of data for which the predictions of the model are wrong. The method `predict(X)` takes a design matrix X . Each row of X is assumed to be a feature vector. It creates a prediction vector. The i -th entry of this vector is 1 if the model predicts a pass for the i -th student. Otherwise this entry is 0.
9. Line 13 prints the accuracy. As Figure 7.11 shows, 3 examples have been miss-predicted. Two of these examples were bound to be miss-predicted, but the fact that the student with an IQ of 120 who has studied for 2.25 hours has also been miss-predicted is disappointing. The reason is that logistic regression does not maximize the accuracy but rather maximizes the log-likelihood. Later, we will discuss so called [support vector machines](#). Often, a support vector machine is able to achieve a higher accuracy than logistic regression. However, support vector machines are also more complicated than logistic regression.

The jupyter notebook containing the computation discussed previously can be found at my [github repository](#):

[Artificial-Intelligence/Python/Chapter-06/04-Logistic-Regression-with-SciKit-Learn.ipynb](#)

Exercise 20: The file `iris.csv` contains the sizes of both the [sepals](#) and the [petals](#) of three different specimen of the iris flower. The data is described in more detail [here](#). Use logistic regression to predict whether a given plant is of the species [iris setosa](#) (Deutsch: Borsten-Schwertlilie), [iris virginica](#)¹, or [iris versicolor](#) (Deutsch: verschiedenfarbige Schwertlilie). As logistic regression is only able to distinguish between two different classes, you have to build three different classifiers:

- The first classifier is able to distinguish [iris setosa](#) from other irises.
- The second classifier is able to distinguish [iris virginica](#) from other irises.

¹This plant is native to North America and hence has no German name.

- The third classifier is able to distinguish *iris versicolor* from other irises.

Your task is to implement these classifiers and to evaluate their accuracy. You should divide the data randomly into a *training dataset*, which is used for computing the coefficients of logistic regression and a *test dataset*, which you should only use to predict the accuracy of your model. To this end, the function

```
sklearn.model_selection.train_test_split
```

might be useful. Once you have created these classifiers, proceed to implement a classifier that inputs a feature vector and that outputs the class of the iris flower as a string. If you do this correctly, you can achieve an accuracy that exceeds 95%.

You should also plot the data and the decision boundary. Since we now have four features, you need to restrict yourself to use only two of the features. The most important features are *petal length* and *petal width*.

◇

7.4 Polynomial Logistic Regression

Sometimes logistic regression does not work because the data is not *linearly separable*. For example, Figure 7.12 on page 177 shows a classification problem where we have two features x and y and, obviously, it is not possible to separate the blue dots from the red dots by a vertical line.

If we try to separate the data in Figure 7.12 by logistic regression, we get the result shown in Figure 7.13 on page 178. The data points above the green line are classified as red, while the data points below the green line are classified as blue. The accuracy achieved is about 61%, so more than 38% of the data have been miss-classified.

We can arrive at a better model if we extend our data with *second order polynomial features*, i.e. we will not only consider the features x and y but also use x^2 , y^2 , and $x \cdot y$ as features. Then the resulting *decision boundary* will be a *conic section*, i.e. it might be an *ellipse*, a *parabola*, or a *hyperbola*. Figure 7.14 on page 179 shows a *Python* script that reads the fake data shown in Figure 7.12, adds second order polynomial features to this data and then performs linear regression.

1. The first three lines import the modules needed for this task.
2. As we want to split our data into a *training set* and a *test set*, we import the function `train_test_split` from `sklearn.model_selection`.
3. Line 6 reads the data from the file “`fake-data.csv`”.
4. Line 7 and 8 extract the independent variables “`x`” and “`y`” and the dependent variable “`class`” and stores them in the *design matrix* `X` and the vector `Y`, respectively.
5. Line 10 splits the data into a *training set* and a *test set*. We allocate 20% of our data to the test set, while the remaining 80% are used as training data. In order to be able to reproduce our results, we set `random_state` to a fixed value. This forces the random number generator to always produce the same split of the data.
6. Line 12 defines the function `logistic_regression`. This function takes 5 arguments.
 - (a) `X_train` and `Y_train` are the training data.
 - (b) `X_test` and `Y_test` are the test data.

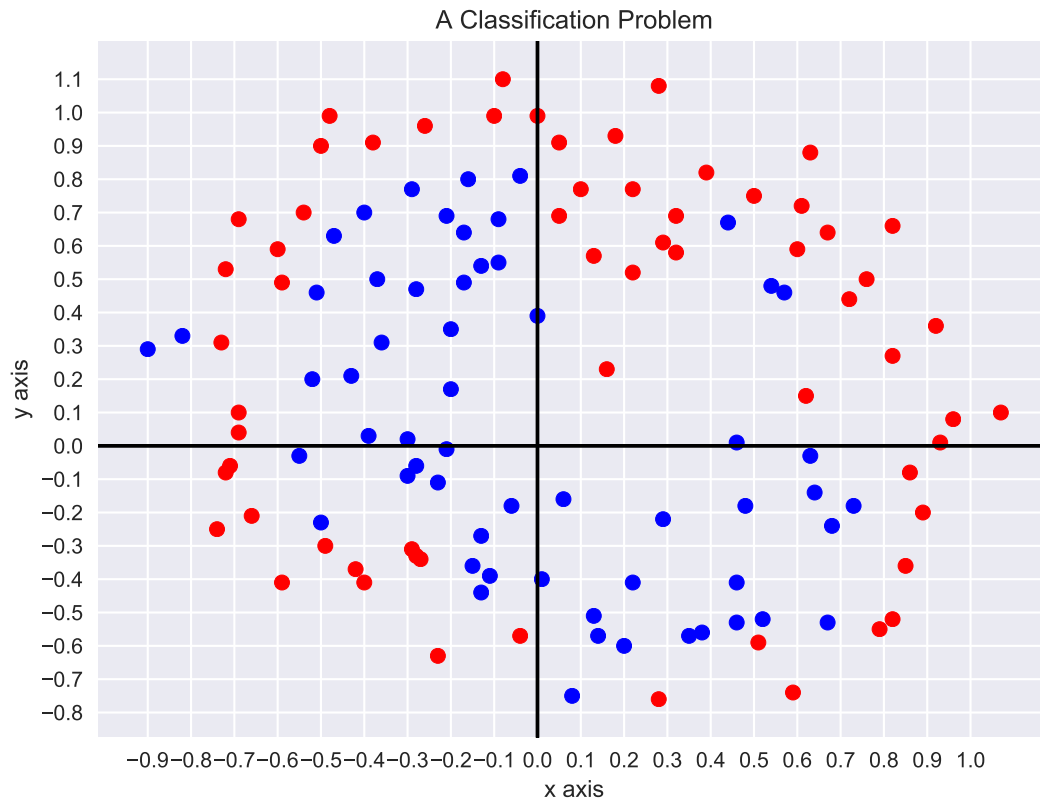


Figure 7.12: Some fake data.

- (c) `reg` is a [regularization](#) parameter. By default this parameter is set to a high value. Setting this parameter to a high value ensures that there is no regularization. The default is to use next to no regularization.

The function returns a triple of the form $(M, \text{score}, \text{accuracy})$ where

- (a) M is the model that has been found.
 - (b) `score` is the fraction of data points from the training set that have been classified correctly.
 - (c) `accuracy` is the fraction of data points from the test set that have been classified correctly.
7. The function `extend(X)` takes a design matrix X as its argument. In order to keep the implementation of this function simple, we assume that X has just two features, i.e. the matrix X has shape $(n, 2)$ where n is the number of rows of X .
- (a) We extract the two features of X in line 22 and 23.
 - (b) In line 24 the new feature matrix is created by stacking the original features `fx` and `fy` together with the new features `fx2`, `fy2`, `fx · fy`,
8. Line 26 extends both the training set and the test set with second order features.

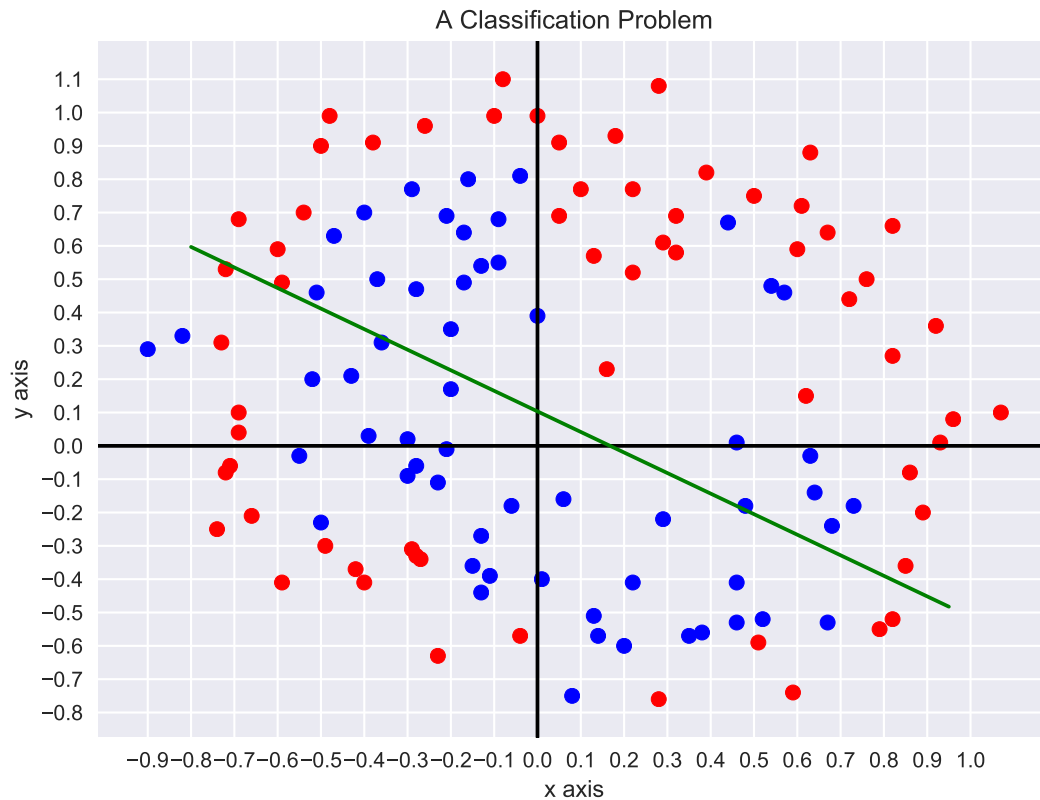


Figure 7.13: Fake data with linear decision boundary.

- Line 27 performs logistic regression using the new features. The use of second order feature improves the accuracy on the training set to 84%, while the accuracy on the test set improves to 76%. Figure 7.15 on page 180 shows the resulting decision boundary.

As adding second order features has increased the accuracy considerably, we proceed to add higher order features. Figure 7.16 on page 180 shows how we can add arbitrary polynomial features of higher degree. This script is a continuation of the script shown in Figure 7.14.

- Line 28 imports the function `PolynomialFeatures` from `sklearn.preprocessing`. This function can be used to add all polynomial features up to a given order. We do not need a bias term here as it is automatically added by the function `LogisticRegression`.
- Line 30 creates an object that can be used to extend a design matrix with polynomial features up to degree 4
- Line 31 and 32 extend the training set and the test set with all polynomial features up to degree 4.
- When we perform logistic regression with these new features, we achieve an accuracy of 88.4% on the training set. However, the accuracy on the test set does not improve. Figure 7.17 on page 181 shows the data with the resulting decision boundary.

```

1  import numpy as np
2  import pandas as pd
3  import sklearn.linear_model as lm
4  from sklearn.model_selection import train_test_split
5
6  DF = pd.read_csv('fake-data.csv')
7  X = np.array(DF[['x', 'y']])
8  Y = np.array(DF['class'])
9
10 X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=1)
11
12 def logistic_regression(X_train, Y_train, X_test, Y_test, reg=10000):
13     model = lm.LogisticRegression(C=reg, tol=1e-6, solver='newton-cg')
14     M = model.fit(X_train, Y_train)
15     score = M.score(X_train, Y_train)
16     yPredict = M.predict(X_test)
17     accuracy = np.sum(yPredict == Y_test) / len(Y_test)
18     return M, score, accuracy
19
20 def extend(X):
21     n = len(X)
22     fx = np.reshape(X[:,0], (n, 1))
23     fy = np.reshape(X[:,1], (n, 1))
24     return np.hstack([fx, fy, fx*fx, fy*fy, fx*fy])
25
26 X_train_quadratic, X_test_quadratic = extend(X_train), extend(X_test)
27 logistic_regression(X_train_quadratic, Y_train, X_test_quadratic, Y_test)

```

Figure 7.14: A script for second order logistic regression.

Nothing stops us from cranking the order of the polynomial features to be added higher. Figure 7.18 on page 182 shows what happens when we include all polynomial features up to a degree of 14. In this case, we 100% accuracy on the training set. However, the accuracy on the test set is only 80%. Clearly, we are overfitting the data.

In order to combat overfitting we need to [regularize](#), i.e. we need to penalize high values of the parameters. If we lower the regularization parameter down to 100, then the accuracy on the training set drops down to 89.6%, but the accuracy on the test set increases to 88%. The resulting decision boundary is shown in Figure 7.19 on page 183. Clearly, this decision boundary looks less complicated than the boundary shown in Figure 7.18. Contrary to the previous figures, this figure shows all the data. The previous figures had only shown the training data.

Exercise 21:

- Assume that a design matrix X has two features x_1 and x_2 . Given a natural number n , you want to extend this design matrix by adding all features of the form $x_1^{k_1} \cdot x_2^{k_2}$ such that $k_1 + k_2 \leq n$, i.e. you want to add all features up to a degree of n . How many features will the extended design matrix have?
- Next, assume that the design matrix X has three features x_1 , x_2 , and x_3 . This time, you want to

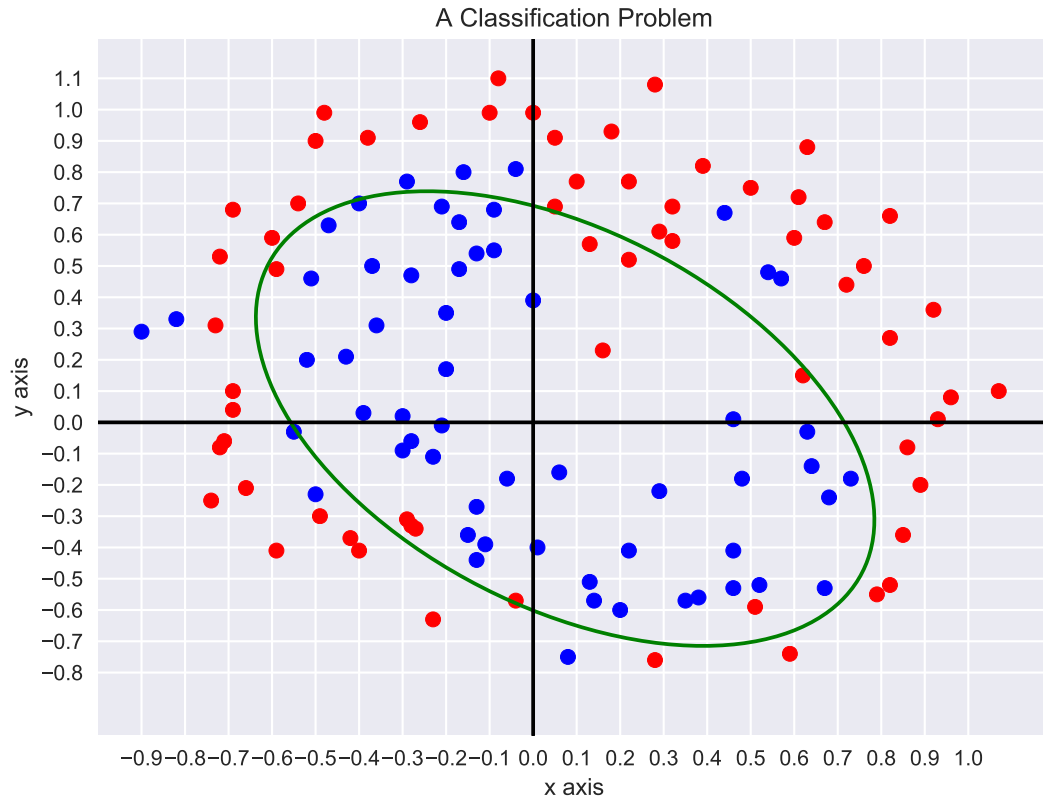


Figure 7.15: Elliptical decision boundary for fake data.

```

1  from sklearn.preprocessing import PolynomialFeatures
2
3  quartic = PolynomialFeatures(4, include_bias=False)
4  X_train_quartic = quartic.fit_transform(X_train)
5  X_test_quartic  = quartic.fit_transform(X_test)
6
7  logistic_regression(X_train_quartic, Y_train, X_test_quartic, Y_test)

```

Figure 7.16: Polynomial Logistic Regression.

extend the design matrix by adding all features of the form $x_1^{k_1} \cdot x_2^{k_2} \cdot x_3^{k_3}$ where $k_1 + k_2 + k_3 \leq n$. How many features will the extended design matrix have in this case?

- (c) In the general case, the design matrix has d features $x_1, \dots, x_d$. Assume you want to add all polynomial terms up to order n as new features, i.e. you want to add all features of the form

$$x_1^{k_1} \cdot x_2^{k_2} \cdot \dots \cdot x_d^{k_d} \quad \text{such that } k_1 + k_2 + \dots + k_d \leq n.$$

How many features will the extended design matrix have in the general case?

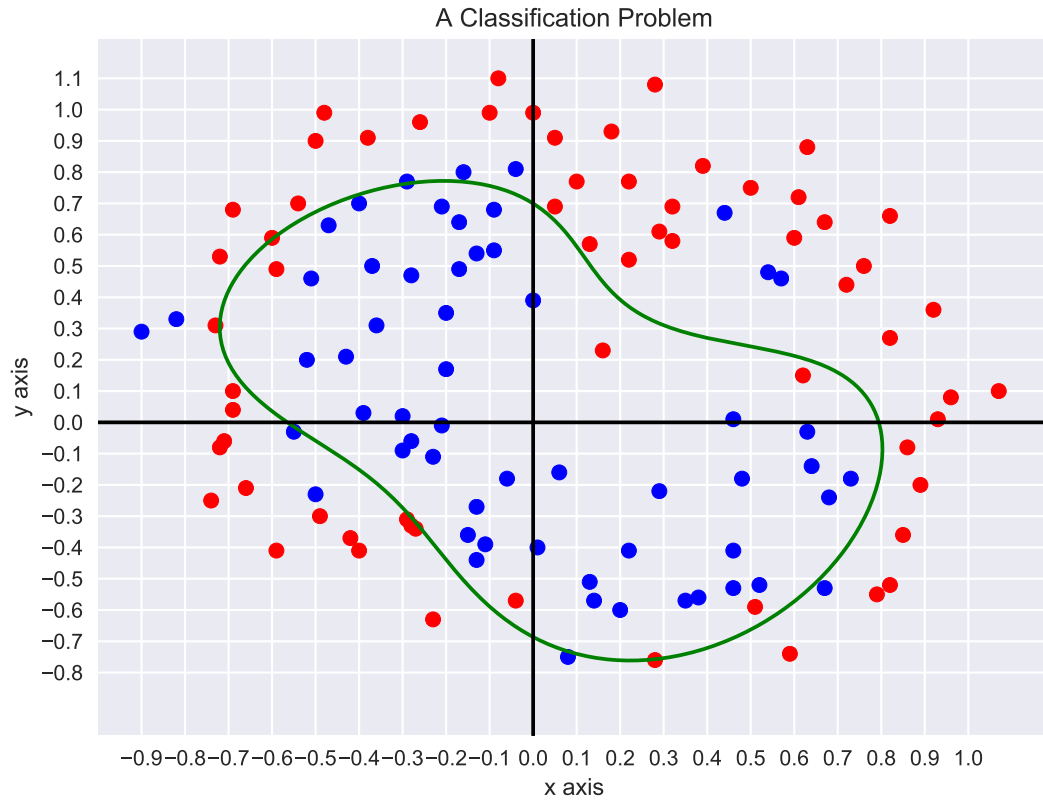


Figure 7.17: Fake data with a decision boundary of fourth order.

Hint: You might find it helpful to revisit your old lecture notes on statistics.

◇

7.5 Naive Bayes Classifiers

In this section we discuss **naive Bayes classifiers**. Naive Bayes classifiers are an alternative method for classification which is appropriate in cases where the features are not numerical but rather are categorical. It starts with **Bayes' theorem**: Assume we have some evidence E about an object o and want to know whether o belongs to some class C . Bayes' theorem tell us that the **conditional probability** $P(C|E)$, i.e. the probability that o has class C given that we have observed the evidence E , is related to the conditional probability $P(E|C)$, which is the probability that we observe the evidence E given that o has class C , in the following way:

$$P(C|E) = \frac{P(E|C) \cdot P(C)}{P(E)}.$$

For example, the evidence E could be the fact that the email contains the string "viagra" and the class C could be the class **spam**.

This theorem is useful because often the conditional probability $P(E|C)$ that we observe some evidence E in an object o of class C is readily available, but the conditional probability $P(C|E)$ that an object has class C if we have observed the evidence E is unknown. For example, we know from experience how many spam mails contain the word "viagra" and hence can estimate the conditional

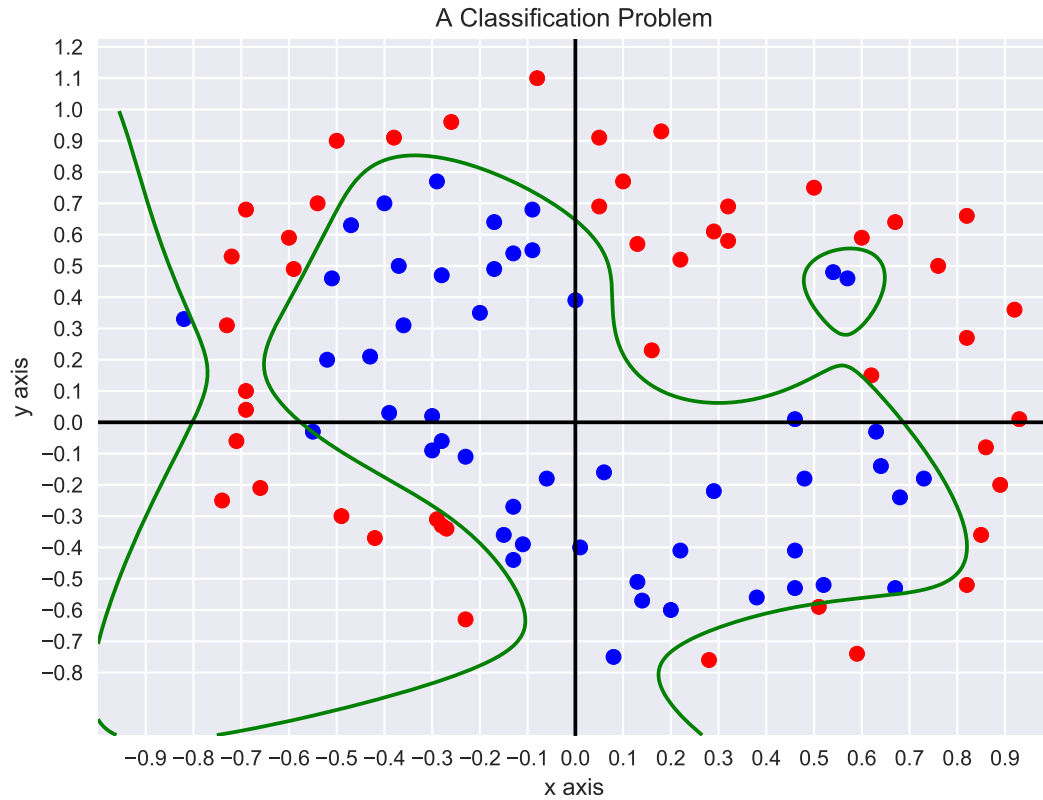


Figure 7.18: Fake data with a decision boundary of order 14.

probability

$$P(\text{"viagra"} \in m \mid \text{spam})$$

for a given mail m , but we do not know the conditional probability that a given mail m is **spam** if it contains the word "viagra", i.e. we do not know

$$P(\text{spam} \mid \text{"viagra"} \in m).$$

However, the later probability $P(\text{spam} \mid \text{"viagra"} \in m)$ is really what we are interested in. Bayes' theorem show us how to compute $P(\text{spam} \mid \text{"viagra"} \in m)$ from $P(\text{"viagra"} \in m \mid \text{spam})$. In the context of machine learning, the evidence E is often not just a single feature but is given as a list of features $f_1, \dots, f_m$ that we are able to observe or compute. In this case we have to rewrite Bayes' theorem as follows:

$$P(C \mid f_1 \wedge \dots \wedge f_m) = \frac{P(f_1 \wedge \dots \wedge f_m \mid C) \cdot P(C)}{P(f_1 \wedge \dots \wedge f_m)}.$$

In order to apply this form of Bayes' theorem to the problem of classification, we have to rewrite the expression

$$P(f_1 \wedge \dots \wedge f_m \mid C).$$

In order to be able to do this, we need some theory which will be developed next. The conditional probability $P(A \mid B)$ that an event A happens when it is already known that B has happened is defined

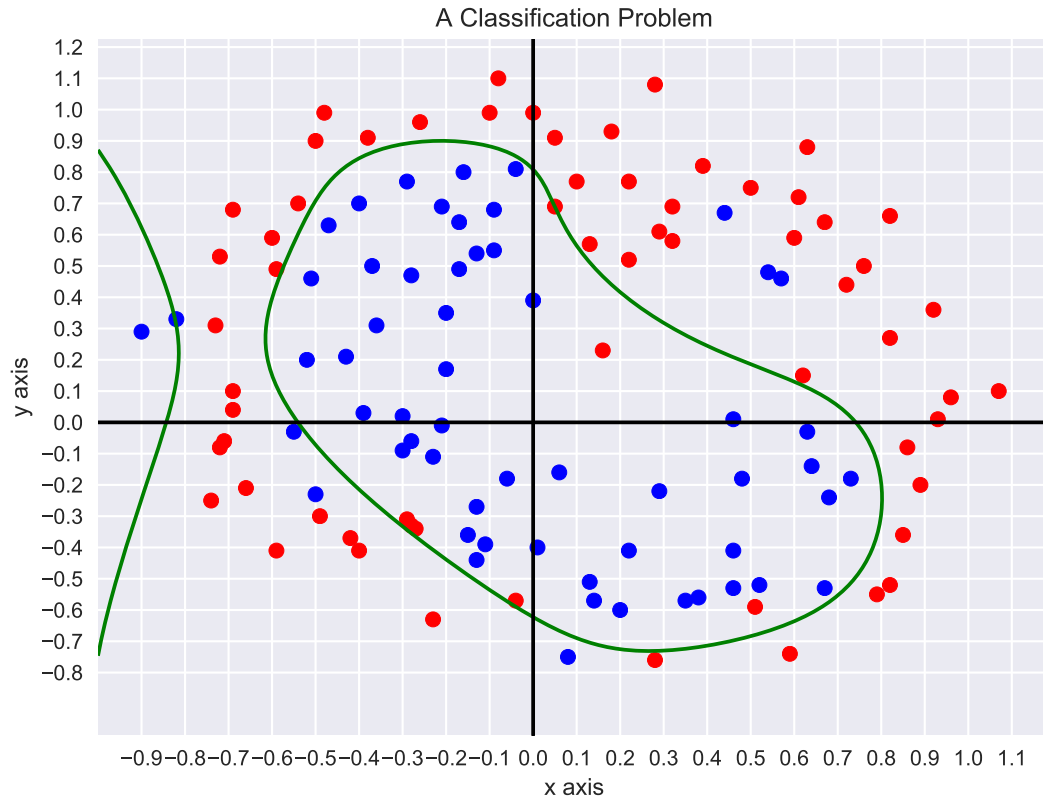


Figure 7.19: Fake data with a decision boundary of order 14, regularized.

as

$$P(A|B) = \frac{P(A \wedge B)}{P(B)}.$$

This equation can be rewritten as

$$P(A \wedge B) = P(A|B) \cdot P(B).$$

Because conditional probabilities are probabilities and hence obey all the laws for probabilities, this equation is also true for conditional probabilities:

$$P(A \wedge B | C) = P(A | B \wedge C) \cdot P(B | C).$$

This equation can be generalized to the so called **chain rule of probability**:

$$\begin{aligned} & P(A_1 \wedge \dots \wedge A_m | C) \\ &= P(A_1 \wedge \dots \wedge A_{m-1} | A_m \wedge C) \cdot P(A_m | C) \\ &= P(A_1 \wedge \dots \wedge A_{m-2} | A_{m-1} \wedge A_m \wedge C) \cdot P(A_{m-1} | A_m \wedge C) \cdot P(A_m | C) \\ &= \dots \\ &= P(A_1 | A_2 \wedge \dots \wedge A_m \wedge C) \cdot \dots \cdot P(A_{m-1} | A_m \wedge C) \cdot P(A_m | C) \\ &= \prod_{i=1}^m P(A_i | A_{i+1} \wedge \dots \wedge A_m \wedge C) \end{aligned}$$

Two events A and B are defined to be **conditionally independent** given an event C if and only if we have

$$P(A \mid C) = P(A \mid B \wedge C).$$

To put this equation differently, once we know that C holds, when it comes to the estimating the probability of A , then it does not matter whether B holds or not. Now the important assumption that a **naive Bayes classifier** makes is that in order to estimate the class C of an object o that has features $f_1, \dots, f_m$ it is assumed that the features $f_1, \dots, f_m$ are conditionally independent once the class is known. In most applications of naive Bayes classifiers this assumption is not true because there might be some weak correlation between the features. That explains why naive Bayes classifiers are called **naive**. Still, in practise the conditional independence of the features given the class is often approximately true and therefore these classifiers are useful. If we make the assumption of conditional independence, then the probability $P(C \mid f_1 \wedge \dots \wedge f_m)$ of an object o with features $f_1, \dots, f_m$ to be of class C is given as

$$\begin{aligned} P(C \mid f_1 \wedge \dots \wedge f_m) &= \frac{P(f_1 \wedge \dots \wedge f_m \mid C)}{P(f_1 \wedge \dots \wedge f_m)} \cdot P(C) \\ &= \frac{\prod_{i=1}^m P(f_i \mid f_{i+1} \wedge \dots \wedge f_m \wedge C)}{P(f_1 \wedge \dots \wedge f_m)} \cdot P(C) \\ &= \frac{\prod_{i=1}^m P(f_i \mid C)}{P(f_1 \wedge \dots \wedge f_m)} \cdot P(C) \end{aligned}$$

In the last line of the previous chain of equations we have used the fact that the features $f_1, \dots, f_m$ are conditionally independent given C . Now a naive Bayes classifier works as follows: Assume we have a set of n classes $\mathcal{C} = \{C_1, \dots, C_n\}$ from which we have to choose the class of an object o given the features $f_1, \dots, f_m$. We assume that o has class C_k if and only if the probability $P(C_k \mid f_1 \wedge \dots \wedge f_m)$ is maximal with respect to all classes of $\mathcal{C}$. In order to be able to specify this in a more formal way, we define the **arg max** function: Given a set S and a function $g : S \rightarrow \mathbb{R}$ that has exactly one maximum, we define

$$\arg \max_{x \in S} g(x) := \text{arb}(\{x \in S \mid \forall y \in S : g(y) \leq g(x)\}),$$

where the function $\text{arb}(M)$ returns an arbitrary element from the set M . Since we assume that g has exactly one maximum on the set S , the expression $\arg \max_{x \in S} g(x)$ is well defined. To put the definition of $\arg \max_{x \in S} g(x)$ differently, the idea is that $\arg \max_{x \in S} g(x)$ computes the value of $x \in S$ that maximizes g . Given the features $f_1, \dots, f_m$, the naive Bayes classifier computes the most probable class as follows:

$$\text{NaiveBayes}(f_1, \dots, f_m) := \arg \max_{C \in \mathcal{C}} \frac{\prod_{i=1}^m P(f_i \mid C)}{P(f_1 \wedge \dots \wedge f_m)} \cdot P(C)$$

It is important to observe that the denominator $P(f_1 \wedge \dots \wedge f_m)$ does not depend on the class C . As we only need to determine the class with the maximal probability, not the exact probability of the class, we can simplify the definition by dropping this denominator. Therefore the definition of the naive Bayes classifier can be rewritten as follows:

$$\text{NaiveBayes}(f_1, \dots, f_m) := \arg \max_{C \in \mathcal{C}} \left(\prod_{i=1}^m P(f_i \mid C) \right) \cdot P(C)$$

This equation can be implemented once we have a training set T of objects with known classes: The

probability $P(C)$ is the probability that an object o has class C if nothing else is known about this object. $P(C)$ is estimated as the proportion of objects in T that are of class C :

$$P(C) \approx \frac{\text{card}(\{t \in T \mid \text{class}(t) = C\})}{\text{card}(T)}.$$

This expression is called the **prior probability** of C or sometimes just the **prior** of C . In this equation, given an object $t \in T$ the function $\text{class}(t)$ determines the class of the object t , while $\text{card}(M)$ returns the number of elements of the set M .

Next, given a feature f and a class C , we have to determine the **conditional probability** that an object of class C exhibits the feature f_i , i.e. we have to determine $P(f_i \mid C)$. This probability can be estimated as the proportion of those objects of class C in the training set T that possess the feature f :

$$P(f \mid C) \approx \frac{\text{card}(\{t \in T \mid \text{class}(t) = C \wedge \text{has}(t, f)\})}{\text{card}(\{t \in T \mid \text{class}(t) = C\})}$$

Here, for an object t and a feature f the expression $\text{has}(t, f)$ is true if and only if t has the feature f .

7.5.1 Example: Spam Detection

Spam detection is an important application of classification. We will see in this subsection that naive Bayes classifiers work well for spam detection. The directory

<https://github.com/karlstroetmann/Artificial-Intelligence/tree/master/Python/Chapter-06/EmailData>

contains 960 emails that are partitioned into four subdirectories:

1. **spam-train** contains 350 spam emails for training,
2. **ham-train** contains 350 non-spam emails for training,
3. **spam-test** contains 130 spam emails for testing,
4. **ham-test** contains 130 non-spam emails for testing.

This data has been collected by Ion Androutsopoulos. Figure 7.20 on page 186 shows the first part of the notebook that implements a naive Bayes classifier.

The code performs the following steps:

1. It imports the necessary modules. **os** is used to read directories, **re** for regular expressions, **math** for the logarithmic function, and **Counter** for counting word occurrences.
2. It defines the directories containing the data and computes the prior probabilities **spam_prior** and **ham_prior**.
3. The function **get_words** reads a file, converts the text to lower case, and extracts all words using a regular expression. It returns a set of words, ensuring each word is counted only once per email.
4. **read_all_files** reads all emails and creates a global counter **Word_Counter** of all words found.
5. Finally, **Common_Words** is defined as the set of words that appear in at least 10 emails. This helps in feature selection by ignoring rare words.

Figure 7.21 on page 187 shows the second part of the implementation.

```

1  import os
2  import re
3  import math
4  from collections import Counter
5
6  spam_dir_train = 'EmailData/spam-train/'
7  ham__dir_train = 'EmailData/ham-train/'
8  spam_dir_test  = 'EmailData/spam-test/'
9  ham__dir_test  = 'EmailData/ham-test/'
10 Directories    = [spam_dir_train, ham__dir_train, spam_dir_test, ham__dir_test]
11
12 no_spam        = len(os.listdir(spam_dir_train))
13 no_ham         = len(os.listdir(ham__dir_train))
14 spam_prior     = no_spam / (no_spam + no_ham)
15 ham__prior     = no_ham  / (no_spam + no_ham)
16
17 def get_words(fn: str) -> set[str]:
18     with open(fn, 'r') as file:
19         text: str = file.read().lower()
20         return set(re.findall(r"[\w']+", text))
21
22 def read_all_files(Directories: list[str]) -> Counter:
23     Words: Counter = Counter()
24     for directory in Directories:
25         for file_name in os.listdir(directory):
26             Words.update(get_words(directory + file_name))
27     return Words
28
29 Word_Counter = read_all_files(Directories)
30
31 Common_Words = { w for w in Word_Counter if Word_Counter[w] >= 10 }

```

Figure 7.20: A Naive Bayes Classifier for Spam Detection: Part I

1. `get_common_words` extracts only the common words from a given file.
2. `count_common_words` counts the occurrences of common words in all files within a directory.
3. `Spam_Counter` and `Ham_Counter` store the counts of each common word in spam and ham emails, respectively.
4. We then compute the conditional probabilities $P(w|\text{spam})$ and $P(w|\text{ham})$ for each common word. Laplace smoothing is applied by adding 1 to the counts and the denominators to avoid zero probabilities.

The final part of the classifier implements the classification logic and evaluates the model. In order to compute whether a mail is spam or ham we have to compute

$$\arg \max_{C \in \{\text{Spam}, \text{Ham}\}} \left(\prod_{i=1}^m P(f_i | C) \right) \cdot P(C)$$

```

32 def get_common_words(fn: str) -> set[str]:
33     return get_words(fn) & Common_Words
34
35 def count_common_words(directory: str) -> Counter:
36     Words: Counter = Counter()
37     for file_name in os.listdir(directory):
38         Words.update(get_common_words(directory + file_name))
39     return Words
40
41 Spam_Counter = count_common_words(spam_dir_train)
42 Ham__Counter = count_common_words(ham__dir_train)
43
44 Spam_Probability = {}
45 Ham__Probability = {}
46 for w in Common_Words:
47     Spam_Probability[w] = (Spam_Counter[w] + 1) / (no_spam + 1)
48     Ham__Probability[w] = (Ham__Counter[w] + 1) / (no_ham + 1)

```

Figure 7.21: A Naive Bayes Classifier for Spam Detection: Part II

Therefore, we have to check, whether

$$\left(\prod_{i=1}^m P(f_i \mid \text{Spam}) \right) \cdot P(\text{Spam}) > \left(\prod_{i=1}^m P(f_i \mid \text{Ham}) \right) \cdot P(\text{Ham})$$

holds. When implementing the formula

$$\arg \max_{C \in \mathcal{C}} \left(\prod_{i=1}^m P(f_i \mid C) \right) \cdot P(C)$$

we have to be careful, because a naive implementation will evaluate the product

$$\prod_{i=1}^m P(f_i \mid C)$$

as the number 0 due to numerical underflow. The trick to compute this product is to remember that

$$\ln(a \cdot b) = \ln(a) + \ln(b)$$

and to transform the product into a sum of logarithms. As the logarithm is a monotone function, we have

$$\begin{aligned}
 & \left(\prod_{i=1}^m P(f_i \mid \text{Spam}) \right) \cdot P(\text{Spam}) > \left(\prod_{i=1}^m P(f_i \mid \text{Ham}) \right) \cdot P(\text{Ham}) \\
 \Leftrightarrow & \sum_{i=1}^m \ln(P(f_i \mid \text{Spam})) + \ln(P(\text{Spam})) > \sum_{i=1}^m \ln(P(f_i \mid \text{Ham})) + \ln(P(\text{Ham}))
 \end{aligned}$$

The function `log_probabilities(fn)` takes a filename `fn` as its first argument and returns the pair

$$\left(\sum_{i=1}^m \ln(P(f_i \mid \text{Ham})) + \ln(P(\text{Ham})), \sum_{i=1}^m \ln(P(f_i \mid \text{Spam})) + \ln(P(\text{Spam})) \right)$$

as its result. It should be noted that these numbers are not really the logarithms of probabilities. The reason is that the formula for the probability of a class C is

$$\frac{\prod_{i=1}^m P(f_i | C)}{P(f_1 \wedge \dots \wedge f_m)} \cdot P(C)$$

and we are not computing the denominator $P(f_1 \wedge \dots \wedge f_m)$. However, this denominator is the same for spam and ham and hence we don't need it when comparing the respective probabilities. Therefore, what the function `log_probabilities` is really computing are pairs of [relative logarithmic probabilities](#).

```

49 def log_probabilities(fn: str) -> tuple[float, float]:
50     log_p_spam: float = math.log(spam_prior)
51     log_p_ham: float = math.log(ham_prior)
52     words: set[str] = get_common_words(fn)
53     for w in Common_Words:
54         if w in words:
55             log_p_spam += math.log(Spam_Probability[w])
56             log_p_ham += math.log(Ham_Probability[w])
57     return (log_p_ham, log_p_spam)

```

Figure 7.22: A Naive Bayes Classifier for Spam Detection: Part III

The function `precision_recall` takes two directories as arguments: `spam_dir` is supposed to contain spam emails, while `ham_dir` contains ham emails. It computes the [precision](#) and the [recall](#) of our spam classifier with respect to these test data. Since it would be quite bad when we misclassify a valid email as spam but we can tolerate an occasional spam email that gets through our filter, we add a third parameter ϑ . We will only classify an email as spam if

$$\sum_{i=1}^m \ln(P(f_i | \text{Spam})) + \ln(P(\text{Spam})) > \vartheta \cdot \left(\sum_{i=1}^m \ln(P(f_i | \text{Ham})) + \ln(P(\text{Ham})) \right)$$

The third parameter ϑ is the logarithmic threshold. Later, we will use a logarithmic threshold of 0.8. To make things concrete, assume that

- The relative logarithmic probability p_1 of a mail being ham is $p_1 = -250$, while
- the relative logarithmic probability p_2 of this mail being spam is $p_2 = -200$.

As we have

$$p_1 = -250 < -200 = p_2$$

we would classify this mail as spam if we would just compare the relative logarithmic probabilities. However, if we use the logarithmic threshold of 0.8 we have

$$\vartheta \cdot p_1 = 0.8 \cdot (-250) = -200 = p_2$$

and hence the inequality

$$\vartheta \cdot p_1 < p_2$$

would not be valid. Therefore, we would conservatively classify the email as ham.

```

58 def precision_recall(spam_dir: str,
59                     ham_dir: str,
60                     θ: float) -> tuple[float, float, float]:
61     TN: int = 0 # true negatives
62     FP: int = 0 # false positives
63     for email in os.listdir(spam_dir):
64         log_p_ham, log_p_spam = log_probabilities(spam_dir + email)
65         if log_p_spam > θ * log_p_ham:
66             TN += 1
67         else:
68             FP += 1
69     FN: int = 0 # false negatives
70     TP: int = 0 # true positives
71     for email in os.listdir(ham_dir):
72         log_p_ham, log_p_spam = log_probabilities(ham_dir + email)
73         if log_p_spam > θ * log_p_ham:
74             FN += 1
75         else:
76             TP += 1
77     precision: float = TP / (TP + FP)
78     recall: float = TP / (TP + FN)
79     accuracy: float = (TN + TP) / (TN + TP + FN + FP)
80     return precision, recall, accuracy

```

Figure 7.23: A Naive Bayes Classifier for Spam Detection: Part III

7.5.2 Naive Bayes Classifier with Numerical Features

We can build a naive Bayes classifier even if some of our features are numerical. Assume we have a feature f that is a numerical attribute. The tricky part is to come up with a way to compute the conditional probability

$$P(f = x \mid C)$$

which is the conditional probability that the feature f has the value x if the object to classify has class C . The idea is to assume that for every class C the values of the feature f have a Gaussian distribution. Then we have

$$P(f = x \mid C) = \frac{1}{\sqrt{2 \cdot \pi} \cdot \sigma_{f,C}} \cdot \exp\left(-\frac{(x - \mu_{f,C})^2}{2 \cdot \sigma_{f,C}^2}\right),$$

where $\mu_{f,C}$ is the [mean value](#) of the feature f for those objects that have class C , while $\sigma_{f,C}^2$ is the [variance](#) of the feature f for objects of class C .

Exercise 22: We have already investigated the file [iris.csv](#) in a previous exercise. This time, your task is to implement a [naive Bayes classifier](#) that is able to classify iris flowers. You can achieve an accuracy that exceeds 95%. ◇

7.6 Support Vector Machines

Support Vector Machines (abbreviated as SVMs) had been invented in 1963 by Vladimir Naumovich Vapnik and Alexey Yakovlevich Chervonenkis. However, they only got widespread acceptance in 1995 when Cortes and Vapnik published a paper explaining the [kernel trick](#) [CV95]. This section will introduce support vector machines. In order to motivate SVMs, we first explain why logistic regression sometimes behaves suboptimally. After that, we explain the mathematical theory of support vector machines. Finally, we show how we can use the support vector machines provided by SciKit Learn.

7.6.1 Non-Optimality of Logistic Regression

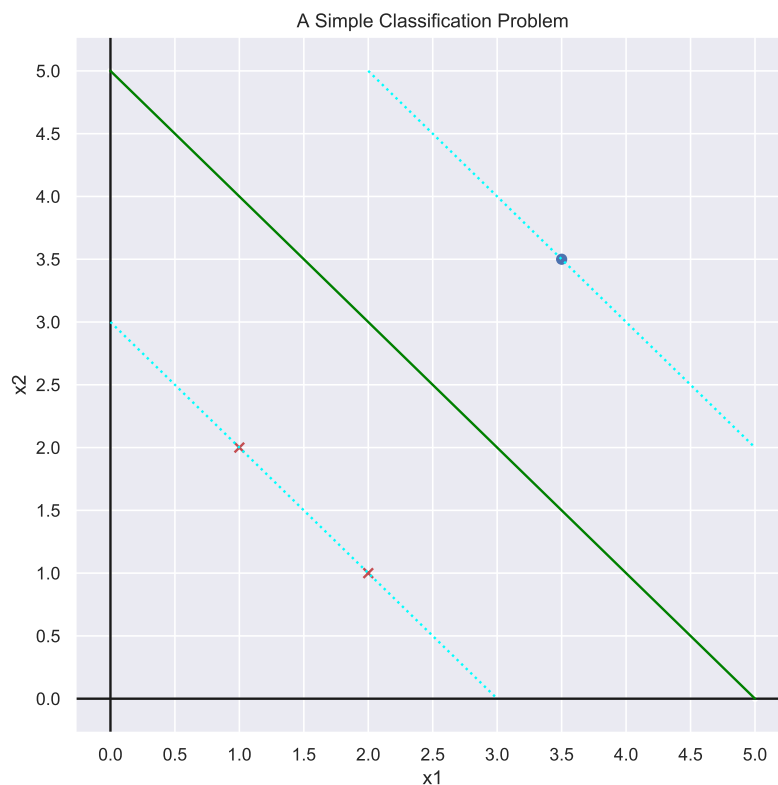


Figure 7.24: Three points to separate.

Figure 7.24 on page 190 shows three points that belong to two different classes. The blue dot at position (3.5, 3.5) belongs to class 1, while the two red crosses at position (1, 2) and (2, 1) belong to the class -1 . When we build a classifier to separate these two classes, we would ideally like the decision boundary to be the green line that passes through the points (0, 5) and (5, 0), since this line has the biggest distance from both classes. Figure 7.25 on page 191 shows how logistic regression deals with this problem.

A close inspection of Figure 7.25 shows that the decision boundary calculated by logistic regression is nearer to the blue dot than it is to the red crosses. If we add a large number of blue points right next to the first blue points, the decision boundary found by logistic regression moves away from the

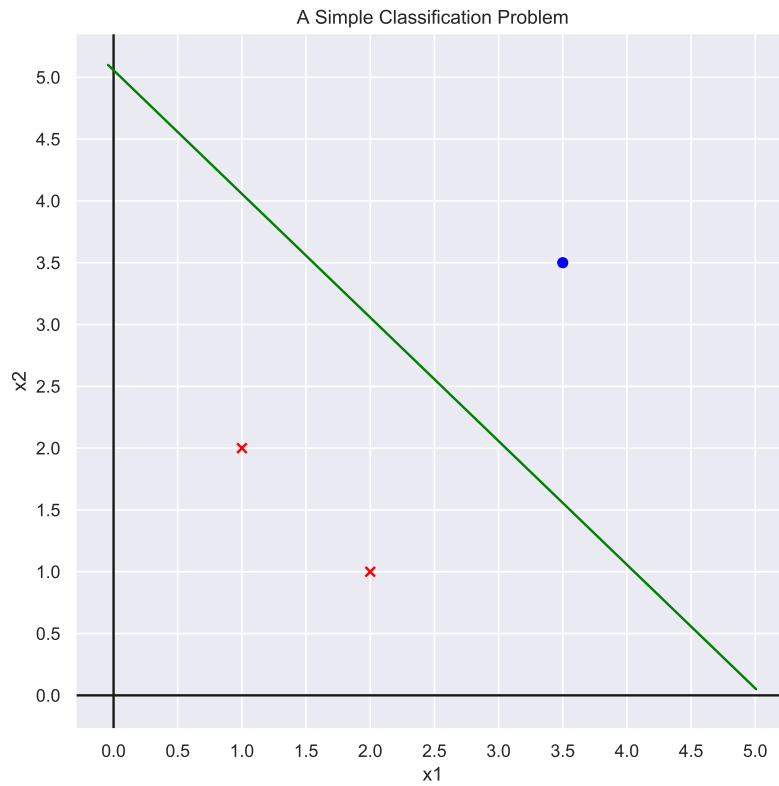


Figure 7.25: Three points separated by logistic regression.

blue points, as shown in Figure 7.26 on page 192, then the decision boundary moves away from the first blue point that marks the margin of the two classes. These new blue points do not add real information, since they are further away from the red crosses than the first blue point. Hence it is counter-intuitive that the addition of these points changes the decision boundary.

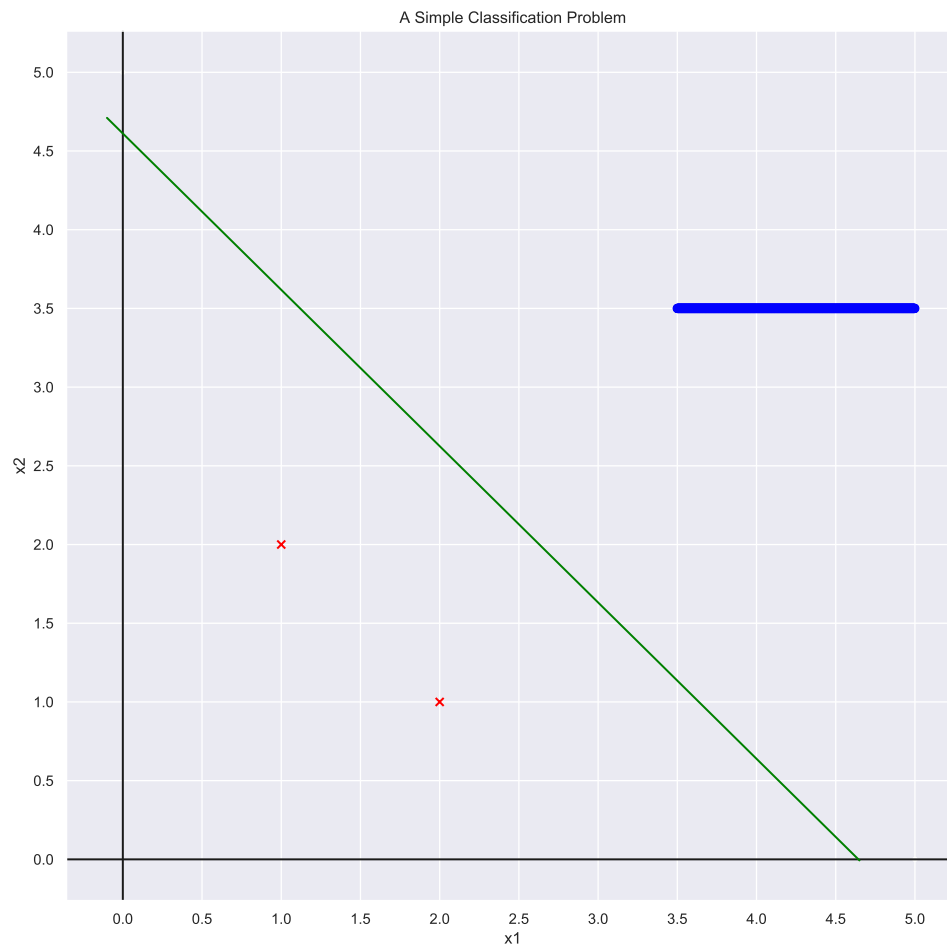


Figure 7.26: Points separated by logistic regression.

7.7 Digression: The Method of Lagrange Multipliers

In machine learning, we often need to find the maximum or minimum of a function $f(\mathbf{x})$ where the variable $\mathbf{x}$ is not allowed to vary freely but is constrained to a specific set of values. This is known as a [constrained optimization problem](#).

The [method of Lagrange multipliers](#) is a strategy for finding the local maxima and minima of a function subject to equality constraints.

7.7.1 Geometric Intuition

To understand the method, let us look at a concrete example in 2D space. We want to maximize the function:

$$f(x, y) = x \cdot y$$

subject to the constraint that the point (x, y) must lie on a circle of radius $\sqrt{2}$:

$$g(x, y) = x^2 + y^2 - 2 = 0.$$

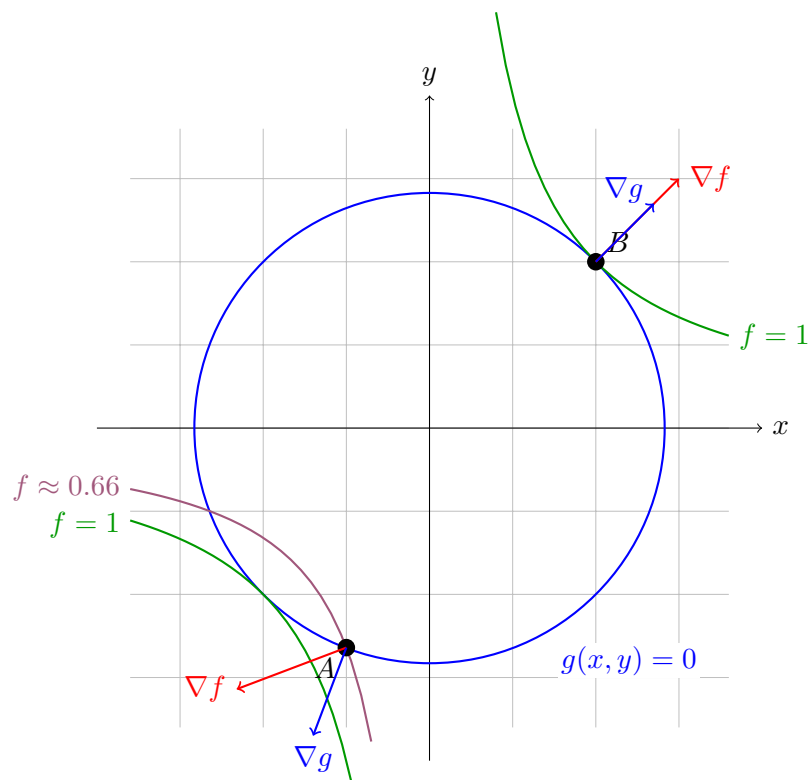


Figure 7.27: Maximizing $f(x, y) = xy$ subject to $x^2 + y^2 = 2$. At point A (3rd quadrant), the level curve cuts through the constraint; the gradients are not parallel. At point B (1st quadrant), the level curve is tangent to the constraint; the gradients are parallel.

Figure 7.27 on page 193 illustrates this problem.

1. The [blue circle](#) represents our constraint $g(x, y) = 0$. We have to stay on this line.
2. The [green lines](#) (hyperbolas) represent the [level curves](#) of $f(x, y)$. Every point on a green curve has the same value for $x \cdot y$. For the green curve in the first quadrant $f(x, y) = 1$, while on the green curve in the third quadrant this value is approximately 0.66.

Consider the point A in the third quadrant. At this point, the value of our function is approximately $f = 0.66$. Notice that the circle [crosses](#) the dashed level curve.

- The red arrow (∇f) points in the direction where f increases most rapidly.
- The blue arrow (∇g) points in the direction perpendicular to the circle (the constraint).

At point A , these vectors point in different directions. This geometry implies that if we move along the blue circle, we can still find a path that increases the value of f . Therefore, the point A cannot be the maximum of the function f .

Now consider point B at $(1, 1)$. Here, the value of the function is $f = 1.0$. Notice that the circle is [tangent](#) to the green level curve. At this optimal point, the direction of greatest increase for f is exactly parallel to the normal vector of the circle. We cannot move along the constraint without decreasing f .

Mathematically, this condition of parallel gradients is written as:

$$\nabla f(x, y) = \lambda \cdot \nabla g(x, y).$$

Here, λ is a real number called the [Lagrange multiplier](#). We can rewrite this as finding the stationary points of a new function, the [Lagrangian](#):

$$\mathcal{L}(x, y, \lambda) := f(x, y) - \lambda \cdot g(x, y).$$

7.7.2 Solving the Example

Let us solve the specific example from Figure [7.27](#) mathematically.

- We want to maximize the function $f(x, y) = x \cdot y$
- subject to the constraint: $g(x, y) = x^2 + y^2 - 2 = 0$.

The [Lagrangian](#) is:

$$\mathcal{L}(x, y, \lambda) = x \cdot y - \lambda \cdot (x^2 + y^2 - 2).$$

We compute the partial derivatives of the Lagrangian $\mathcal{L}$ and set them to zero:

1. $\frac{\partial \mathcal{L}}{\partial x} = y - 2 \cdot \lambda \cdot x = 0 \quad \Rightarrow \quad y = 2 \cdot \lambda \cdot x$
2. $\frac{\partial \mathcal{L}}{\partial y} = x - 2 \cdot \lambda \cdot y = 0 \quad \Rightarrow \quad x = 2 \cdot \lambda \cdot y$
3. $\frac{\partial \mathcal{L}}{\partial \lambda} = -(x^2 + y^2 - 2) = 0 \quad \Rightarrow \quad x^2 + y^2 = 2$

Substituting the first equation ($y = 2 \cdot \lambda \cdot x$) into the second equation gives:

$$x = 2 \cdot \lambda \cdot (2 \cdot \lambda \cdot x) = 4 \cdot \lambda^2 \cdot x.$$

This implies $x \cdot (1 - 4 \cdot \lambda^2) = 0$. Since $x = 0$ would imply $y = 0$ (which violates the constraint), we must have:

$$1 - 4 \cdot \lambda^2 = 0 \quad \Rightarrow \quad \lambda^2 = \frac{1}{4} \quad \Rightarrow \quad \lambda = \pm \frac{1}{2}.$$

If $\lambda = 1/2$, then $y = x$. Substituting into the constraint $x^2 + y^2 = 2$:

$$2 \cdot x^2 = 2 \quad \Rightarrow \quad x = \pm 1.$$

This gives us the maxima at $(1, 1)$ and $(-1, -1)$, where $f(x, y) = 1$. This matches point B in our figure.

If $\lambda = -1/2$, then $y = -x$. Substituting into the constraint $x^2 + y^2 = 2$ yields:

$$2 \cdot x^2 = 2 \quad \Rightarrow \quad x = \pm 1.$$

This gives us the minima at $(1, -1)$ and $(-1, 1)$, where $f(x, y) = -1$.

Exercise 23: Assume that $\mathbf{w} \in \mathbb{R}^n$ is a vector of Euclidean length 1, that is the equation $\mathbf{w} \cdot \mathbf{w} = 1$ holds. Furthermore, assume $b \in \mathbb{R}$. Let us define the hyperplane P as the set

$$P := \{ \mathbf{x} \in \mathbb{R}^n \mid \mathbf{w} \cdot \mathbf{x} + b = 0 \}.$$

What is the distance between the hyperplane P and the origin, i.e. the point $\mathbf{0}$?

Hint: Solve this problem using the method of Lagrangian multipliers. For any vector $\mathbf{x} \in \mathbb{R}$, the Euclidean length of $\mathbf{x}$ is defined as $\sqrt{\mathbf{x} \cdot \mathbf{x}}$. If we want to find the distance of the plane to the origin, we have to find a vector $\mathbf{x} \in P$ such that the length of $\mathbf{x}$ is minimal. The constraint is that the vector $\mathbf{x}$ has to be a member of the hyperplane and therefore we must have $\mathbf{w} \cdot \mathbf{x} + b = 0$.

Furthermore, we can simplify the calculation if we minimize the function $\mathbf{x} \cdot \mathbf{x}$ instead of the function $\sqrt{\mathbf{x} \cdot \mathbf{x}}$. This is possible because the function $x \mapsto \sqrt{x}$ is a monotonous function. $\diamond$

Exercise 24: You are a landscape architect designing a rectangular fish pond for a client's garden. To maximize the happiness of the fish, your goal is to maximize the volume of the pond. However, you are limited by two specific material inventories left over from a previous job:

1. You have exactly 20 meters of expensive stone coping that must be used to create the border around the rim of the pond. Hence the perimeter of the pond must be 20 meters.
2. You have exactly 48 square meters of high-grade rubber liner to cover the bottom and the vertical sides of the pond. $\diamond$

Hint: Somewhat surprisingly, the optimal pond is not square!

7.7.3 The Mathematical Theory of Support Vector Machines

The main idea of support vector machines is to have a decision boundary that is [as simple as possible](#) and that [separates the different classes as much as possible](#). In two dimensions, the simplest decision boundary is a line. In n dimensions, the simplest decision boundary is an $(n - 1)$ -dimensional hyperplane. A hyperplane separates two different classes as much as possible if the distance to both classes is maximized.

A hyperplane can be defined by a vector $\mathbf{w}$ that is perpendicular to the hyperplane together with a bias b : A vector $\mathbf{x}$ is an element of the hyperplane if and only if

$$\mathbf{w} \cdot \mathbf{x} + b = 0.$$

In order for the decision boundary to separate the positive examples from the negative examples, we add the following two conditions. If $\mathbf{x}^{(i)}$ is a positive example, then we don't just want that

$$\mathbf{w} \cdot \mathbf{x}^{(i)} + b \geq 0.$$

Instead, we demand that

$$\mathbf{w} \cdot \mathbf{x}^{(i)} + b \geq 1 \tag{7.3}$$

holds. Similarly, if $\mathbf{x}^{(i)}$ is a negative example, we want to have that

$$\mathbf{w} \cdot \mathbf{x}^{(i)} + b \leq -1 \tag{7.4}$$

holds. Let us define the class of a positive example to be +1 and the class of a negative example to be -1. Let y_i denote the class of example $\mathbf{x}^{(i)}$. Let us multiply equation 7.3 by $y_i = 1$. Unsurprisingly, we get

$$y_i \cdot (\mathbf{w} \cdot \mathbf{x}^{(i)} + b) \geq 1. \quad (7.5)$$

Similarly, let us multiply equation 7.3 by $y_i = -1$. This time, things get more interesting as the direction of the inequality is flipped:

$$y_i \cdot (\mathbf{w} \cdot \mathbf{x}^{(i)} + b) \geq 1. \quad (7.6)$$

Notice that the equations 7.5 and 7.6 are the same! Hence inequation 7.5 holds for both positive and negative examples. We rewrite the last inequation as

$$y_i \cdot (\mathbf{w} \cdot \mathbf{x}^{(i)} + b) - 1 \geq 0. \quad (7.7)$$

Those vectors $\mathbf{x}^{(i)}$ that satisfy the equation

$$y_i \cdot (\mathbf{w} \cdot \mathbf{x}^{(i)} + b) - 1 = 0$$

are at the **margins** of their respective classes and are called **support vectors**. These vectors have the smallest distance to the hyperplane defined by $\mathbf{w}$ and b . Let us compute the width of the separation of the positive class from the negative class if the decision boundary is given by the equation $\mathbf{w} \cdot \mathbf{x} + b = 0$. To this end, assume that $\mathbf{x}_+$ is a **positive support vector**, i.e. a support vector belonging to the positive class. Then we have

$$\mathbf{w} \cdot \mathbf{x}_+ + b = 1, \quad (7.8)$$

while $\mathbf{x}_-$ is a **negative support vector**, i.e. a support vector belonging to the negative class and therefore satisfies

$$\mathbf{w} \cdot \mathbf{x}_- + b = -1. \quad (7.9)$$

Since the vector $\mathbf{w}$ is perpendicular to the hyperplane that defines the decision boundary, the **width** between the positive and the negative example is given by the projection $\mathbf{x}_+ - \mathbf{x}_-$ on the normalized vector $\mathbf{w}$:

$$\text{width} = (\mathbf{x}_+ - \mathbf{x}_-) \cdot \frac{\mathbf{w}}{\|\mathbf{w}\|} = (\mathbf{x}_+ \cdot \mathbf{w} - \mathbf{x}_- \cdot \mathbf{w}) \cdot \frac{1}{\|\mathbf{w}\|} \quad (7.10)$$

If we subtract equation 7.9 from equation 7.8, the constant b cancels and we are left with

$$\mathbf{x}_+ \cdot \mathbf{w} - \mathbf{x}_- \cdot \mathbf{w} = 2.$$

Substituting this equation into equation 7.10 yields the equation

$$\text{width} = \frac{2}{\|\mathbf{w}\|}.$$

Hence in order to maximize the width of the separation of the two classes from the decision boundary we have to minimize the size of the vector $\mathbf{w}$ subject to the constraints given in equation 7.7. Now minimizing $\mathbf{w}$ is the same as minimizing

$$\frac{1}{2} \cdot \|\mathbf{w}\|^2 = \frac{1}{2} \cdot \sum_{k=1}^d w_k^2,$$

where d is the number of features. Determining a minimum of a function that is subject to a set of constraints requires us to use **Lagrange multipliers**. Assuming our training set has the form $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n)}\}$,

we define the **Lagrangian** $\mathcal{L}(\mathbf{w}, b, \alpha_1, \dots, \alpha_n)$ as follows:

$$\mathcal{L}(\mathbf{w}, b, \alpha_1, \dots, \alpha_n) := \frac{1}{2} \cdot \|\mathbf{w}\|^2 - \sum_{i=1}^n \alpha_i \cdot (y_i \cdot (\mathbf{w} \cdot \mathbf{x}_i + b) - 1).$$

The sum in this Lagrangian sums over all training examples although not all training examples have to satisfy the constraint

$$y_i \cdot (\mathbf{w} \cdot \mathbf{x}_i + b) - 1 = 0.$$

This is not a problem because for those $i \in \{1, \dots, n\}$ where we only have the inequality

$$y_i \cdot (\mathbf{w} \cdot \mathbf{x}_i + b) - 1 \geq 0$$

we can just assume that the corresponding Lagrange multiplier α_i is equal to 0. A necessary condition for the values of $\mathbf{w}$, b and α_i that minimize $\mathcal{L}$ is that the partial derivatives of $\mathcal{L}$ with respect to w_k , b and α_i are all 0. Let us first compute the partial derivative of $\mathcal{L}$ with respect to w_k :

$$\frac{\partial \mathcal{L}}{\partial w_k} = w_k - \sum_{i=1}^n \alpha_i \cdot y_i \cdot x_k^{(i)} = 0$$

Therefore, we must have that

$$w_k = \sum_{i=1}^n \alpha_i \cdot y_i \cdot x_k^{(i)}$$

and this implies

$$\mathbf{w} = \sum_{i=1}^n \alpha_i \cdot y_i \cdot \mathbf{x}^{(i)} \quad (7.11)$$

Therefore the vector $\mathbf{w}$ is a linear combination of the **support vectors**, where a vector $\mathbf{x}^{(i)}$ is a **support vector** iff $\alpha_i \neq 0$. Hence a support vector $\mathbf{x}^{(i)}$ must satisfy the equation

$$y_i \cdot (\mathbf{w} \cdot \mathbf{x}^{(i)} + b) - 1 = 0.$$

Next, let us compute the partial derivative of $\mathcal{L}$ with respect to b . We have

$$\frac{\partial \mathcal{L}}{\partial b} = - \sum_{i=1}^n \alpha_i \cdot y_i = 0$$

which implies that

$$\sum_{i=1}^n \alpha_i \cdot y_i = 0 \quad (7.12)$$

Let us rewrite the Lagrangian by substituting $\mathbf{w}$ with the right hand side of equation 7.11:

$$\begin{aligned} \mathcal{L} &= \frac{1}{2} \cdot \left(\sum_{i=1}^n \alpha_i \cdot y_i \cdot \mathbf{x}^{(i)} \right) \cdot \left(\sum_{j=1}^n \alpha_j \cdot y_j \cdot \mathbf{x}^{(j)} \right) - \sum_{i=1}^n \alpha_i \cdot \left(y_i \cdot \left(\mathbf{x}^{(i)} \cdot \left(\sum_{j=1}^n \alpha_j \cdot y_j \cdot \mathbf{x}^{(j)} \right) + b \right) - 1 \right) \\ &= \frac{1}{2} \cdot \sum_{i=1}^n \sum_{j=1}^n \alpha_i \cdot \alpha_j \cdot y_i \cdot y_j \cdot (\mathbf{x}^{(i)} \cdot \mathbf{x}^{(j)}) - \sum_{i=1}^n \sum_{j=1}^n \alpha_i \cdot \alpha_j \cdot y_i \cdot y_j \cdot (\mathbf{x}^{(i)} \cdot \mathbf{x}^{(j)}) - b \cdot \underbrace{\sum_{i=1}^n \alpha_i \cdot y_i}_{=0} + \sum_{i=1}^n \alpha_i \\ &= \sum_{i=1}^n \alpha_i - \frac{1}{2} \cdot \sum_{i=1}^n \sum_{j=1}^n \alpha_i \cdot \alpha_j \cdot y_i \cdot y_j \cdot (\mathbf{x}^{(i)} \cdot \mathbf{x}^{(j)}) \end{aligned}$$

Now, the **crucial observation** is the following: The Lagrangian $\mathcal{L}$ only depends on the dot products $\mathbf{x}^{(i)} \cdot \mathbf{x}^{(j)}$. Why is this a big deal? Often, a set of data point is not linearly separable in the given space. However, it might be possible to transform the feature vectors $\mathbf{x} \in \mathbb{R}^d$ into some higher dimensional space $\mathbb{R}^h$ where $h > d$ and in this higher dimensional space the two classes are separable. Concretely, it is sometimes possible to define a transformation function

$$\Phi : \mathbb{R}^d \rightarrow \mathbb{R}^h$$

such that the set of transformed data points $\{\langle \Phi(\mathbf{x}^{(1)}), y_1 \rangle, \dots, \langle \Phi(\mathbf{x}^{(n)}), y_n \rangle\}$ is linearly separable. The question then is to find such a transformation Φ . Here is the punch line: As the Lagrangian does only depend on the dot products $\Phi(x^{(i)}) \cdot \Phi(x^{(j)})$, it is sufficient to define the transformed dot products

$$\Phi(\mathbf{x}) \cdot \Phi(\mathbf{y}).$$

This is done with the help of so called **kernel functions**: We define the dot product $\Phi(\mathbf{x}) \cdot \Phi(\mathbf{y})$ as

$$\Phi(\mathbf{x}) \cdot \Phi(\mathbf{y}) := k(\mathbf{x}, \mathbf{y})$$

where k is called a **kernel function**. There are two kernel functions that are quite popular:

1. **Polynomial kernels** have the form

$$k(\mathbf{x}, \mathbf{y}) = (\mathbf{x} \cdot \mathbf{y} + c)^n,$$

where n is a natural number called the **degree** of the kernel. The number c is a hyperparameter that is often set to 1.

2. **Gaussian kernels** have the form

$$k(\mathbf{x}, \mathbf{y}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{y}\|^2}{2 \cdot \sigma^2}\right).$$

Here, σ is a hyperparameter.

Using a kernel function to simulate a parameter transformation is known as the **kernel trick**. Experience has shown that the kernel functions given above often enable us to transform a data set into a space where the data set is linearly separable. Figure 7.28 on page 199 shows a set of point that is separable using a support vector machine with a Gaussian kernel.

If you want to know more about support vector machines, the free book **Support Vector Machines Succinctly** by Alexandre Kowalczyk [Kow17] is a good place to start.

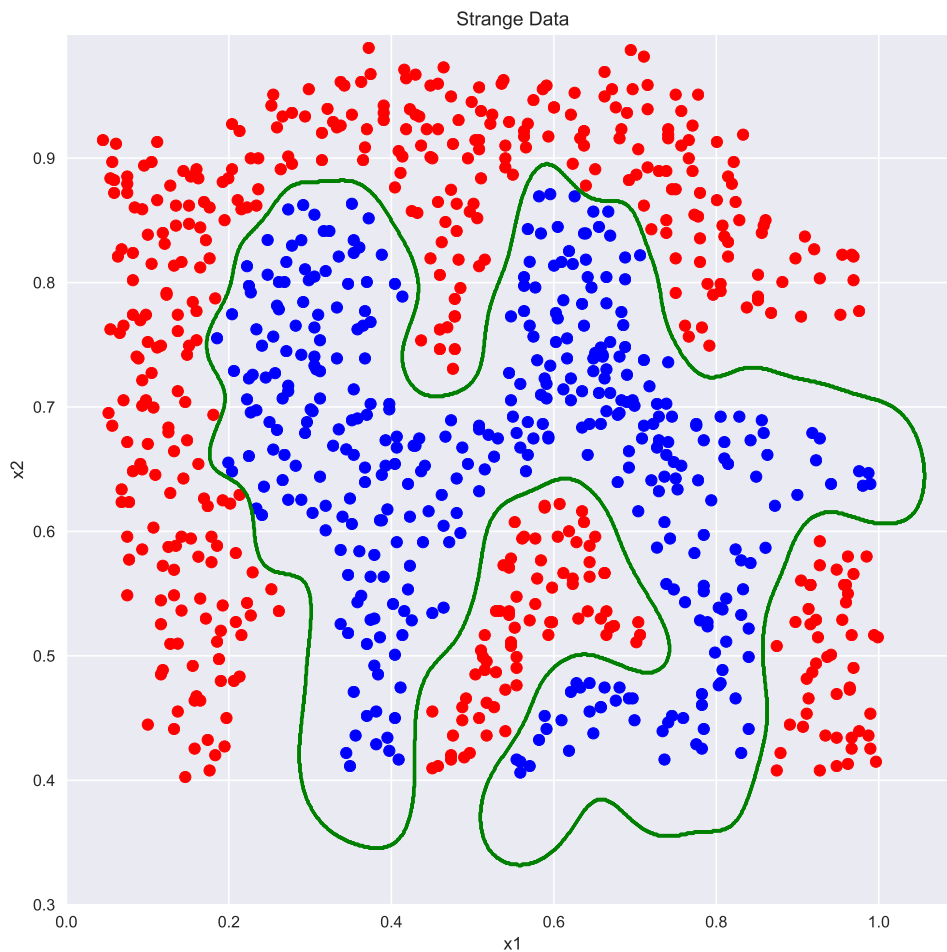


Figure 7.28: Points separated by a support vector machine.

7.8 Support Vector Machines on Medical Data

In this section, we apply our implementation of Support Vector Machines to a real-world medical dataset: the [Breast Cancer Wisconsin \(Diagnostic\) Dataset](#). This problem involves classifying breast mass samples as either [Malignant](#) (harmful) or [Benign](#) (non-harmful) based on features computed from digitized images of a fine needle aspirate (FNA) of a breast mass.

7.8.1 Data Preprocessing: Standardization

Real-world data often presents a challenge: features have vastly different scales. For example, the mean area of a cell nucleus might range from 200 to 2500, while the smoothness might range from 0.05 to 0.16. This is problematic for the [Gaussian Kernel](#) (RBF), which relies on the Euclidean distance $\|\mathbf{x} - \mathbf{y}\|^2$. If we do not normalize the data, features with large numerical values (like Area) will dominate the distance calculation, effectively rendering features with small values (like Smoothness)

irrelevant. To fix this, we apply [standardization](#) (also known as Z-score normalization) to scale the data such that each feature has a mean of 0 and a standard deviation of 1.

7.8.2 From Hard Margin to Soft Margin

The SVM theory derived in Section 7 assumed that the data is [linearly separable](#) (potentially after a kernel transformation). This is known as the [Hard Margin SVM](#). The constraints derived were:

$$y_i \cdot (\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1.$$

In medical diagnosis, data often overlaps significantly. A Hard Margin SVM would fail to find a solution because no hyperplane can perfectly separate the classes. To handle this, we introduce [slack variables](#) $\xi_i \geq 0$ which allow individual data points to violate the margin constraint:

$$y_i \cdot (\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \xi_i.$$

Our goal is still to maximize the margin (minimize $\|\mathbf{w}\|^2$), but we must now also penalize the slack variables. The new [primal objective](#) becomes:

$$\text{Minimize } \frac{1}{2} \|\mathbf{w}\|^2 + C \cdot \sum_{i=1}^n \xi_i.$$

Here, C is a [regularization parameter](#). If C is large, we penalize mistakes heavily. On the other hand, if C is small, we allow more margin violations to obtain a simpler boundary. Remarkably, when we solve the Lagrangian for this new problem, the Lagrangian remains exactly the same as derived previously:

$$\mathcal{L}(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \cdot \alpha_j \cdot y_i \cdot y_j \cdot K(\mathbf{x}_i, \mathbf{x}_j).$$

Here, $K(\mathbf{x}_i, \mathbf{x}_j)$ is the kernel function discussed earlier. However, the constraints on the Lagrange multipliers α_i change. Instead of just $\alpha_i \geq 0$, we obtain the [box constraints](#):

$$0 \leq \alpha_i \leq C.$$

This result allows us to implement a so called [soft margin SVM](#) by simply adding an upper bound to our Lagrange multipliers α_i .

Derivation of the Soft Margin Lagrangian

In this subsection, we formally derive the [Dual Optimization Problem](#) for the Soft Margin SVM. This derivation explains why the objective function remains unchanged compared to the Hard Margin case, while the constraints on α_i change to $0 \leq \alpha_i \leq C$.

The Primal Problem

Recall that for the Soft Margin SVM, we introduce [slack variables](#) $\xi_i \geq 0$ to allow misclassifications. We want to maximize the margin (minimize $\|\mathbf{w}\|^2$) while penalizing the sum of the slack variables. The [Primal Optimization Problem](#) is defined as follows:

$$\text{Minimize } \frac{1}{2} \|\mathbf{w}\|^2 + C \cdot \sum_{i=1}^n \xi_i$$

Subject to the constraints:

$$1. \ y_i \cdot (\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \xi_i \quad \text{for all } i = 1, \dots, n$$

2. $\xi_i \geq 0$ for all $i = 1, \dots, n$

Constructing the Lagrangian

To solve this constrained optimization problem, we use the method of [Lagrange multipliers](#). We need to introduce a multiplier for every constraint:

- We associate a multiplier $\alpha_i \geq 0$ with the margin constraint:

$$y_i \cdot (\mathbf{w} \cdot \mathbf{x}_i + b) - 1 + \xi_i \geq 0.$$

- We associate a multiplier $\beta_i \geq 0$ with the non-negativity constraint of the slack variable:

$$\xi_i \geq 0.$$

The [Lagrangian](#) $\mathcal{L}(\mathbf{w}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\beta})$ is constructed by subtracting the weighted sum of constraints from the objective function:

$$\mathcal{L} = \underbrace{\frac{1}{2} \cdot \|\mathbf{w}\|^2 + C}_{\text{Objective}} \cdot \sum_{i=1}^n \xi_i - \sum_{i=1}^n \alpha_i [y_i \cdot (\mathbf{w} \cdot \mathbf{x}_i + b) - 1 + \xi_i] - \sum_{i=1}^n \beta_i \cdot \xi_i$$

Minimizing with respect to Primal Variables

We must minimize $\mathcal{L}$ with respect to the primal variables $\mathbf{w}$, b , and ξ_i . To do this, we compute the partial derivatives and set them to zero.

1. Derivative with respect to $\mathbf{w}$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \mathbf{w} - \sum_{i=1}^n \alpha_i \cdot y_i \cdot \mathbf{x}_i = 0 \quad \Rightarrow \quad \mathbf{w} = \sum_{i=1}^n \alpha_i \cdot y_i \cdot \mathbf{x}_i$$

This result is identical to the hard margin case.

2. Derivative with respect to b :

$$\frac{\partial \mathcal{L}}{\partial b} = - \sum_{i=1}^n \alpha_i \cdot y_i = 0 \quad \Rightarrow \quad \sum_{i=1}^n \alpha_i \cdot y_i = 0$$

This is also identical to the hard margin case.

3. Derivative with respect to ξ_i :

$$\frac{\partial \mathcal{L}}{\partial \xi_i} = C - \alpha_i - \beta_i = 0 \quad \Rightarrow \quad \alpha_i + \beta_i = C$$

This equation is the key to the Soft Margin formulation. It relates the [regularization parameter](#) C to our Lagrange multipliers α_i and β_i .

Deriving the Box Constraints

We now show how the constraints $0 \leq \alpha_i \leq C$ for $i = 1, \dots, n$ are derived. From the definition of the Lagrangian, we know that all Lagrangian multipliers must be non-negative, i.e. we have

$$\alpha_i \geq 0 \quad \text{and} \quad \beta_i \geq 0 \quad \text{for all } i = 1, \dots, n. \quad \text{That is very wrong what you are telling here.}$$

The derivative with respect to ξ_i gave us the relation $\alpha_i + \beta_i = 0$. Therefore we have

$$\beta_i = C - \alpha_i \quad \text{for all } i = 1, \dots, n.$$

Substituting this into the inequality $\beta_i \geq 0$, we get:

$$C - \alpha_i \geq 0 \quad \Rightarrow \quad \alpha_i \leq C.$$

Combining this with $\alpha_i \geq 0$, we obtain the [box constraints](#)

$$0 \leq \alpha_i \leq C \quad \text{for all } i = 1, \dots, n.$$

Finally, we substitute the relations back into the Lagrangian to eliminate the variables $\mathbf{w} = (w_1, \dots, w_d)^\top$. Let us collect the terms involving the slack variables ξ_i :

$$\sum_{i=1}^n \xi_i \cdot (C - \alpha_i - \beta_i).$$

Since $\alpha_i + \beta_i = C$, the term inside the parenthesis is zero. Therefore all terms involving ξ_i vanish and we are left with the exact same terms as previously when we had not introduced slack variables! Therefore, the [optimization problem](#) is again

$$\text{Maximize} \quad \sum_{i=1}^n \alpha_i - \frac{1}{2} \cdot \sum_{i=1}^n \sum_{j=1}^n \alpha_i \cdot \alpha_j \cdot y_i \cdot y_j \cdot (\mathbf{x}_i \cdot \mathbf{x}_j)$$

subject to the constraints

$$\sum_{i=1}^n \alpha_i \cdot y_i = 0 \quad \text{and} \quad 0 \leq \alpha_i \leq C.$$

Of course, the constraints have changed now, as we have added the constraints $\alpha_i \leq C$.

7.8.3 Implementation

Finally, we are ready to present the implementation of the class `SVM`. This class implements soft margin support vector machines. We have split the code into several figures to make it easier to digest.

Initialization

Figure 7.29 shows the initialization of the class. The class accepts a [kernel](#) function and the regularization parameter `C`. If `C` is set to `None`, we assume a hard margin.

```
class SVM:
    def __init__(self, kernel=gaussian_kernel, C=1.0):
        self.kernel = kernel
        self.C = C # Regularization parameter
        self.alpha = None
        self.support_vectors = None
        self.support_vector_labels = None
        self.b = 0
```

Figure 7.29: SVM Initialization.

Optimization (The fit Method)

The core training logic is implemented in the `fit` method, shown in Figure 7.30. The optimization process involves the following mathematical details:

1. Lines 11–14 compute the matrix K where $K_{i,j} = K(\mathbf{x}_i, \mathbf{x}_j)$. In the literature K is called the **Gram matrix**. Pre-computing K is essential for performance, as these values are used n^2 times in every step of the optimization.
2. Lines 17–18 define the function to minimize. The mathematical goal is to **maximize** $\mathcal{L}(\alpha)$. However, standard numerical solvers like `minimize` are designed to find the **minimum**. Therefore, we minimize the **negative** of the dual objective:

$$\text{minimize } J(\alpha) = \frac{1}{2} \cdot \sum_{i,j} \alpha_i \cdot \alpha_j \cdot y_i \cdot y_j \cdot K_{ij} - \sum_i \alpha_i.$$

3. **Box Constraints:** Lines 21–22 implement the Soft Margin constraints derived earlier.
 - The equality constraint `np.dot(alpha, y)` corresponds to $\sum \alpha_i \cdot y_i = 0$.
 - The bounds `(0, self.C)` correspond to $0 \leq \alpha_i \leq C$.

```
def fit(self, X, y):
    n_samples, n_features = X.shape
    K = np.zeros((n_samples, n_samples))
    for i in range(n_samples):
        for j in range(n_samples):
            K[i, j] = self.kernel(X[i], X[j])
    def objective(alpha):
        return 0.5 * np.sum((np.outer(alpha, alpha) * \
                               np.outer(y, y)) * K) - np.sum(alpha)
    constraints = ({'type': 'eq', 'fun': lambda alpha: np.dot(alpha, y)})
    bounds = [(0, self.C) for _ in range(n_samples)]
    initial_alpha = np.zeros(n_samples)
    result = minimize(objective, initial_alpha, method='SLSQP',
                      bounds=bounds, constraints=constraints)
    self.alpha = result.x
    sv_indices = self.alpha > 1e-4
    self.support_vectors = X[sv_indices]
    self.support_vector_labels = y[sv_indices]
    self.alpha = self.alpha[sv_indices]
    self.compute_bias()
```

Figure 7.30: The training method `fit`.

Computing the Bias

Calculating the bias b in a Soft Margin SVM requires careful consideration of the **Karush-Kuhn-Tucker (KKT) conditions**. The KKT conditions for the primal problem imply the following complementarity conditions for the optimal solution:

$$\begin{aligned} \alpha_i \cdot (y_i \cdot (\mathbf{w} \cdot \mathbf{x}_i + b) - 1 + \xi_i) &= 0 \\ (C - \alpha_i) \cdot \xi_i &= 0 \end{aligned}$$

These conditions divide our support vectors into two types:

1. **Bound Support Vectors** ($\alpha_i = C$): Here $\xi_i > 0$, meaning the point violates the margin. We cannot use these to calculate b precisely because the exact value of ξ_i is unknown.
2. **Free Support Vectors** ($0 < \alpha_i < C$): Here $C - \alpha_i > 0$, which implies $\xi_i = 0$. These points lie exactly on the margin.

For any free support vector $\mathbf{x}_k$, we have:

$$y_k \cdot (\mathbf{w} \cdot \mathbf{x}_k + b) = 1.$$

Multiplying by y_k (and noting $y_k^2 = 1$) gives:

$$\mathbf{w} \cdot \mathbf{x}_k + b = y_k \quad \Rightarrow \quad b = y_k - \mathbf{w} \cdot \mathbf{x}_k.$$

Substituting the dual form $\mathbf{w} = \sum \alpha_j \cdot y_j \cdot \mathbf{x}_j$, we get the formula implemented in Figure 7.31:

$$b = y_k - \sum_{j \in SV} \alpha_j \cdot y_j \cdot K(\mathbf{x}_j, \mathbf{x}_k).$$

We compute this value for all free support vectors and take the average to ensure numerical stability.

```
def compute_bias(self):
    free_sv_indices = (self.alpha < self.C - 1e-4)
    if np.any(free_sv_indices):
        b_values = []
        free_alphas = self.alpha[free_sv_indices]
        free_vecs = self.support_vectors[free_sv_indices]
        free_labels = self.support_vector_labels[free_sv_indices]
        for i in range(len(free_alphas)):
            sum_term = 0
            for j in range(len(self.alpha)):
                sum_term += self.alpha[j] * self.support_vector_labels[j] * \
                    self.kernel(self.support_vectors[j], free_vecs[i])
            b_values.append(free_labels[i] - sum_term)
        self.b = np.mean(b_values)
    else:
        self.b = 0
```

Figure 7.31: Computing the bias b .

Prediction

Finally, Figure 7.32 shows the prediction logic. The decision boundary is determined by the sign of the hyperplane equation:

$$f(\mathbf{x}) = \text{sign} \left(\sum_{i \in SV} \alpha_i \cdot y_i \cdot K(\mathbf{x}_i, \mathbf{x}) + b \right).$$

If the result is positive, we classify the tumour as +1, i.e. benign, otherwise we classify it as -1, i.e. malignant.

```
def predict(self, X):
    y_pred = []
    for x in X:
        prediction = 0
        for i in range(len(self.alpha)):
            prediction += self.alpha[i] * self.support_vector_labels[i] * \
                self.kernel(self.support_vectors[i], x)
        y_pred.append(prediction + self.b)
    return np.sign(y_pred)
```

Figure 7.32: The method `predict`.

Chapter 8

Neural Networks

In this chapter, we discuss **artificial neural networks**. Many of the most visible breakthroughs in artificial intelligence have been achieved through the use of neural networks:

1. **DeepL** is a language translator that is based on neural networks.
2. **AlphaGo** uses neural networks together with tree search [SHM⁺16]. It has **beaten** the world champion **Ke Jie** in the game of **Go**.
3. **Image recognition** is best done via neural networks.
4. **Autonomous driving** makes heavy use of neural networks.
5. **ChatGPT** is a large language model that is based on a gigantic neural network.

The list given above is far from being complete. In this chapter, we will only discuss **feedforward neural networks**. Although recently both **convolutional neural networks** and **recurrent neural networks** have gotten a lot of attention, these type of neural networks are more difficult to understand and are therefore beyond the scope of this introduction. The rest of this chapter is strongly influenced by the online book

<http://neuralnetworksanddeeplearning.com/index.html>

that has been written by Michael Nielsen [Nie19]. This book is easy to read, carefully written, and free to access. I recommend this book to anybody who wants to dive deeper into the fascinating topic of neural networks.

We proceed to give an overview of the content of this chapter.

1. We start with the definition of a fully connected **feed forward** neural networks and discuss their **topology**.
2. We introduce **forward propagation**, which is the way a neural network computes its predictions.
3. Similarly to our treatment of logistic regression, we define a cost function that measures the quality of the predictions of a neural network on a training set. In order to minimize this cost function using gradient descent, we have to compute the **gradient** of the cost function with respect to the weights of the neural network. The algorithm which is used to compute this gradient is called **backpropagation**.
4. In order to find the minimum of the cost function efficiently, we need an improved version of **gradient descent**. This improved version is known as **stochastic gradient descent**.

5. After having covered the theory, we implement a simple neural network that is able to recognize [handwritten digits](#).
6. Finally, we discuss [automatic differentiation](#), which is the backbone of state-of-the-art machine learning platforms like [PyTorch](#) or [TensorFlow](#).

8.1 Feed Forward Neural Networks

A neural network is built from [neurons](#). Neural networks are inspired by biological [neurons](#). However, in order to understand artificial neural networks it is not necessary to know how biological neurons work and it is definitely not necessary to understand how networks of biological neurons, i.e. brains, work¹. Instead, we will use a mathematical abstraction of neurons that will serve as the foundation of the theory developed in this chapter. At the abstraction level that we are using to look at neural networks, a single neuron with m inputs is specified by a pair $\langle \mathbf{w}, b \rangle$ where the vector $\mathbf{w} \in \mathbb{R}^m$ is called the [weight vector](#) and the number $b \in \mathbb{R}$ is called the [bias](#). Conceptually, a neuron is a function that maps an input vector $\mathbf{x} \in \mathbb{R}^m$ into the set $\mathbb{R}$ of the real numbers. This function is defined as follows:

$$\mathbf{x} \mapsto f(\mathbf{x} \cdot \mathbf{w} + b).$$

Here, f is called the [activation function](#). In our applications, we will use the sigmoid function as our activation function. This function has been defined previously in Definition 45 on page 161 as follows:

$$f(t) := S(t) := \frac{1}{1 + \exp(-t)}.$$

Another useful activation function is the so called [ReLU function](#), which is defined as

$$f(t) := \text{ReLU}(t) := \max(0, t).$$

The abbreviation ReLU is short for [rectified linear unit](#). The function modelling the neuron can be written more explicitly using index notation. If

$$\mathbf{w} = \langle w_1, \dots, w_m \rangle^\top$$

is the weight vector and

$$\mathbf{x} = \langle x_1, \dots, x_m \rangle^\top$$

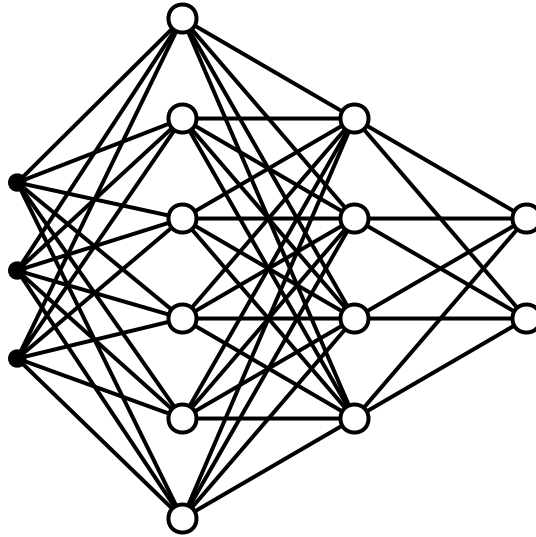
is the input vector, and $b \in \mathbb{R}$ is the [bias](#), then we have

$$\mathbf{x} \mapsto f\left(\left(\sum_{i=1}^m x_i \cdot w_i\right) + b\right), \quad \text{where } f \text{ is the activation function.}$$

If we use the sigmoid function as the activation function, then a single neuron works just like a classifier in logistic regression. The only difference is that the bias b is now explicit in our notation. In logistic regression, we had assumed that the first component x_1 of our feature vector $\mathbf{x}$ was always equal to 1. This assumption enabled us to incorporate the bias b into the weight vector $\mathbf{w}$.

A [feed forward neural network](#) is a layered network of neurons. Formally, the [topology](#) of a neural network is given by a number $L \in \mathbb{N}$ and a list $[m(1), \dots, m(L)]$ of L natural numbers. The number L is called the [number of layers](#) and for $l \in \{2, \dots, L\}$ the number $m(l)$ is the number of neurons in the l -th layer. The first layer is called the [input layer](#). The input layer does not contain neurons but instead just contains [input nodes](#). The last layer (i.e. the layer with index L) is called the [output layer](#) and the remaining layers are called [hidden layers](#). If there is more than one hidden layer, the neural network is called a [deep neural network](#). Figure 8.1 on page 208 shows a small neural network

¹Actually, when it comes to brains, although there are many speculations, surprisingly little is known for a fact.

Figure 8.1: A neural network with topology $[3, 6, 4, 2]$.

with two hidden layers. Including the input layer it has four layers and its topology is given by the list $[3, 6, 4, 2]$. A larger neural network with three hidden layers is shown in Figure 8.2 on page 209. I have written a small Jupyter notebook that can be used to draw diagrams of this kind. This notebook is available at [NN-Architecture.ipynb](#) in the `Python` subdirectory of my GitHub repository for this lecture.

If the topology of a neural network is $[m(1), \dots, m(L)]$, the **input dimension** is defined as $m(1)$. Similarly, the **output dimension** is defined as $m(L)$. The feedforward neural networks discussed in this chapter are **fully connected**: Every node in the l -th layer is connected to every node in the $(l + 1)$ -th layer via a **weight**. The weight $w_{j,k}^{(l)}$ is the weight of the connection from the k -th neuron in layer $l - 1$ to the j -th neuron in layer l . The weights in layer l are combined into the **weight matrix** $W^{(l)}$ of the layer l : This matrix is defined as

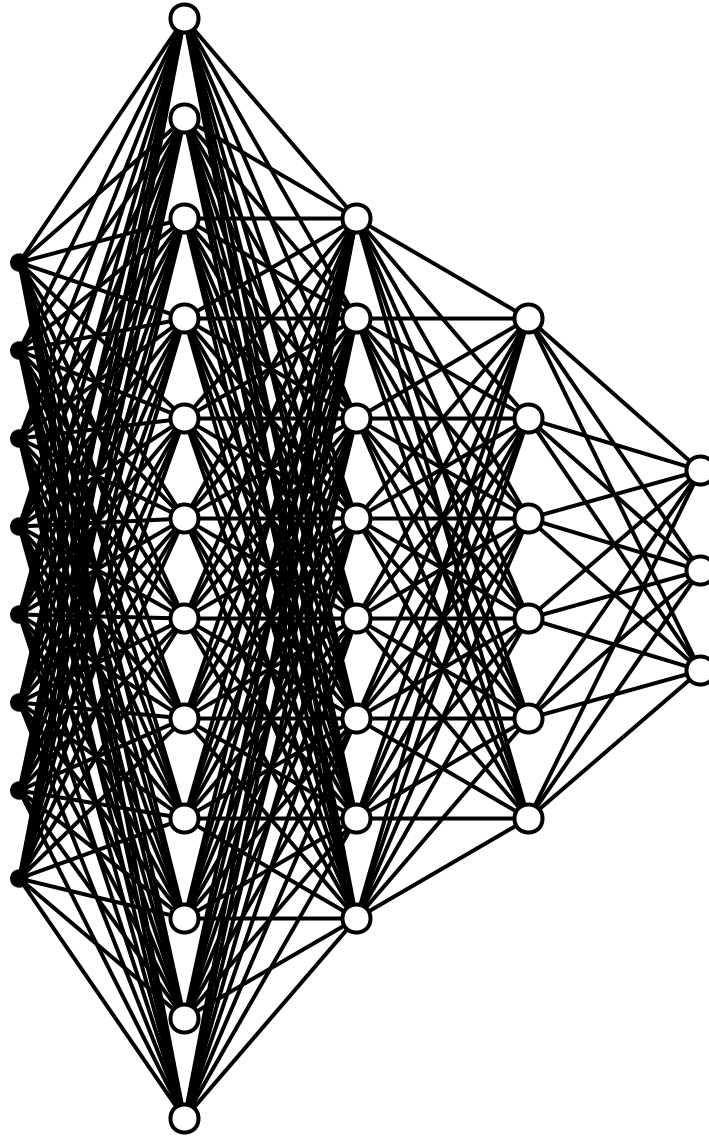
$$W^{(l)} := (w_{j,k}^{(l)}).$$

Note that $W^{(l)}$ is an $m(l) \times m(l - 1)$ matrix, i.e. we have

$$W^{(l)} \in \mathbb{R}^{m(l) \times m(l-1)}.$$

The j -th neuron in layer l has the **bias** $b_j^{(l)}$. These biases of layer l are combined into the **bias vector**

$$\mathbf{b}^{(l)} := \langle b_1^{(l)}, \dots, b_{m(l)}^{(l)} \rangle^\top.$$

Figure 8.2: A neural network with topology $[8, 12, 8, 6, 3]$.

The **activation** of the j -th neuron in layer l is denoted as $a_j^{(l)}$ and is defined recursively as follows:

1. For the input layer we have

$$a_j^{(1)}(\mathbf{x}) := x_j. \quad (\text{FF1})$$

To put it differently, the input vector $\mathbf{x}$ is the activation of the input nodes.

2. For all other layers we have

$$a_j^{(l)}(\mathbf{x}) := f \left(\left(\sum_{k=1}^{m^{(l-1)}} w_{j,k}^{(l)} \cdot a_k^{(l-1)}(\mathbf{x}) \right) + b_j^{(l)} \right) \quad \text{for all } l \in \{2, \dots, L\}. \quad (\text{FF2})$$

The **activation vector** of layer l is defined as

$$\mathbf{a}^{(l)}(\mathbf{x}) := \langle a_1^{(l)}(\mathbf{x}), \dots, a_{m^{(l)}}^{(l)}(\mathbf{x}) \rangle^\top.$$

Using vector notation, the [feed forward equations](#) (FF1) and (FF2) can be rewritten as follows:

$$\mathbf{a}^{(1)}(\mathbf{x}) := \mathbf{x}, \tag{FF1v}$$

$$\mathbf{a}^{(l)}(\mathbf{x}) := f\left(W^{(l)} \cdot \mathbf{a}^{(l-1)}(\mathbf{x}) + \mathbf{b}^{(l)}\right) \quad \text{for all } l \in \{2, \dots, L\}. \tag{FF2v}$$

The output of our neural network for an input $\mathbf{x}$ is given by the neurons in the output layer, i.e. the output vector $\mathbf{o}(\mathbf{x}) \in \mathbb{R}^{m^{(L)}}$ is defined as

$$\mathbf{o}(\mathbf{x}) := \langle a_1^{(L)}(\mathbf{x}), \dots, a_{m^{(L)}}^{(L)}(\mathbf{x}) \rangle^\top = \mathbf{a}^{(L)}(\mathbf{x}).$$

Note that the equations (FF1) and (FF2) describe how information propagates through the neural network:

1. Initially, the input vector $\mathbf{x}$ is given and stored in the input layer of the neural network:

$$\mathbf{a}^{(1)}(\mathbf{x}) := \mathbf{x}.$$

2. The first layer of neurons, which is the second layer of nodes, is activated and computes the activation vector $\mathbf{a}^{(2)}$ according to the formula

$$\mathbf{a}^{(2)}(\mathbf{x}) := f(W^{(2)} \cdot \mathbf{a}^{(1)}(\mathbf{x}) + \mathbf{b}^{(2)}) = f(W^{(2)} \cdot \mathbf{x} + \mathbf{b}^{(2)}).$$

3. The second layer of neurons, which is the third layer of nodes, is activated and computes the activation vector $\mathbf{a}^{(3)}(\mathbf{x})$ according to the formula

$$\mathbf{a}^{(3)}(\mathbf{x}) := f(W^{(3)} \cdot \mathbf{a}^{(2)}(\mathbf{x}) + \mathbf{b}^{(3)}) = f(W^{(3)} \cdot S(W^{(2)} \cdot \mathbf{x} + \mathbf{b}^{(2)}) + \mathbf{b}^{(3)})$$

4. This proceeds until the output layer is reached and the output

$$\mathbf{o}(\mathbf{x}) := \mathbf{a}^{(L)}(\mathbf{x})$$

has been computed. If we use the sigmoid function as our activation function f , every neuron of the neural network performs logistic regression.

Next, we assume that we have n [training examples](#)

$$\langle \mathbf{x}^{(i)}, \mathbf{y}^{(i)} \rangle \quad \text{for } i = 1, \dots, n$$

such that

$$\mathbf{x}^{(i)} \in \mathbb{R}^{m^{(1)}} \text{ and } \mathbf{y}^{(i)} \in \mathbb{R}^{m^{(L)}}.$$

Our goal is to choose the weight matrices $W^{(l)}$ and the bias vectors $\mathbf{b}^{(l)}$ in a way such that

$$\mathbf{o}(\mathbf{x}^{(i)}) = \mathbf{y}^{(i)} \quad \text{for all } i \in \{1, \dots, n\}.$$

Unfortunately, in general we will not be able to achieve equality for all $i \in \{1, \dots, n\}$. Therefore, our goal is to minimize the [error](#) instead. To be more precise, the [quadratic error cost function](#) is defined as

$$C(W^{(2)}, \dots, W^{(L)}, \mathbf{b}^{(2)}, \dots, \mathbf{b}^{(L)}; \mathbf{x}^{(1)}, \mathbf{y}^{(1)}, \dots, \mathbf{x}^{(n)}, \mathbf{y}^{(n)}) := \frac{1}{2 \cdot n} \cdot \sum_{i=1}^n \left(\mathbf{o}(\mathbf{x}^{(i)}) - \mathbf{y}^{(i)} \right)^2.$$

Note that this cost function is additive in the training examples $\langle \mathbf{x}^{(i)}, \mathbf{y}^{(i)} \rangle$. In order to simplify the notation we define

$$C_{\mathbf{x}, \mathbf{y}}(W^{(2)}, \dots, W^{(L)}, \mathbf{b}^{(2)}, \dots, \mathbf{b}^{(L)}) := \frac{1}{2} \cdot \left(\mathbf{a}^{(L)}(\mathbf{x}) - \mathbf{y} \right)^2,$$

i.e. $C_{\mathbf{x},\mathbf{y}}$ is the part of the cost function that is associated with a single training example $\langle \mathbf{x}, \mathbf{y} \rangle$. Then we have

$$\begin{aligned} & C(W^{(2)}, \dots, W^{(L)}, \mathbf{b}^{(2)}, \dots, \mathbf{b}^{(L)}; \mathbf{x}^{(1)}, \mathbf{y}^{(1)}, \dots, \mathbf{x}^{(n)}, \mathbf{y}^{(n)}) \\ &:= \frac{1}{n} \cdot \sum_{i=1}^n C_{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}}(W^{(2)}, \dots, W^{(L)}, \mathbf{b}^{(2)}, \dots, \mathbf{b}^{(L)}). \end{aligned}$$

As the notation

$$C_{\mathbf{x},\mathbf{y}}(W^{(2)}, \dots, W^{(L)}, \mathbf{b}^{(2)}, \dots, \mathbf{b}^{(L)})$$

is still far too heavy, we will abbreviate this term as $C_{\mathbf{x},\mathbf{y}}$ in the following discussion of the backpropagation algorithm. Similarly, we abbreviate the quadratic error cost function as C . Our goal is to choose the weight matrices $W^{(l)}$ and the bias vectors $\mathbf{b}^{(l)}$ such that the quadratic error cost function C is minimized. We will use a variation of [gradient descent](#) to find this minimum². Unfortunately, the cost function C when regarded as a function of the weights and biases has many local minima. Hence, in practical applications all we can hope for is to find a local minimum that is good enough for the goal that we want to achieve.

8.2 Backpropagation

There are three reasons for the recent success of neural networks.

1. The computing power that is available today has vastly increased in the last 20 years. For example, today the [NVIDIA RTX 5090](#) graphic card offers 419 teraflops in single precision performance. It needs a power supply that outputs 450 watt. Contrast this with [ASCI White](#), which was the most powerful supercomputer in 2000: According to the article “[History of Supercomputing](#)”, it offered a performance of 7.2 teraflops and needed 6 megawatt to operate. The cost to build ASCI White was about 110,000,000\$. To compare, the NVIDIA RTX 5090 costs 2,099\$.

The NVIDIA RTX 5090 is a consumer graphics card. There are so called [Tensor Core GPUs](#) like the NVIDIA Blackwell B200 Tensor Core GPU that can achieve more than a petaflop when training neural networks. Furthermore, these cards can be bundled in clusters. However, a card containing two NVIDIA B200 GPUs and one NVIDIA Grace CPU can cost up to 70,000\$.

2. The breakthrough in the theory of neural networks was the rediscovering of the [backpropagation algorithm](#) by David Rumelhart, Geoffrey Hinton, and Ronald Williams [[RHW86](#)] in 1986. The backpropagation algorithm had first been discovered by Arthur E. Bryson, Jr. and Yu-Chi Ho [[BH69](#)]. In recent years, there have been a number of other theoretical advances that have helped in speeding up the learning algorithms for neural networks.
3. Lastly, as neural networks have large sets of parameters, they need large sets of training examples. The recent digitization of our society has made these large data sets available.

Essentially, the [backpropagation](#) algorithm is an efficient way to compute the partial derivatives of the cost function C with respect to the weights $w_{j,k}^{(l)}$ and the biases $b_j^{(l)}$. Before we can proceed to compute these partial derivatives, we need to define some auxiliary variables.

²In logistic regression we have tried to *maximize* the log-likelihood. Here, instead we *minimize* the quadratic error cost function. Hence, instead of *gradient ascent* we use *gradient descent*.

8.2.1 Definition of some Auxiliary Variables

We start by defining the auxiliary variables $z_j^{(l)}$. The expressions $z_j^{(l)}$ are defined as the inputs of the activation function f of the j -th neuron in layer l :

$$z_j^{(l)} := \left(\sum_{k=1}^{m(l-1)} w_{j,k}^{(l)} \cdot a_k^{(l-1)} \right) + b_j^{(l)} \quad \text{for all } j \in \{1, \dots, m(l)\} \text{ and } l \in \{2, \dots, L\}.$$

Of course, the term $a_k^{(l-1)}$ really is a function of the input vector $\mathbf{x}$. However, it is better to suppress this dependence in the notation since otherwise the formulas get too cluttered. The point is that we want to compute the partial derivative of the cost function w.r.t. the weights and biases and for this purpose the input vector is a constant.

Essentially, $z_j^{(l)}$ is the input to the activation function when the activation $a_j^{(l)}$ is computed, i.e. we have

$$a_j^{(l)} = f(z_j^{(l)}).$$

Later, we will see that the partial derivatives of the cost function $C_{\mathbf{x},\mathbf{y}}$ with respect to both the weights $w_{j,k}^{(l)}$ and the biases $b_j^{(l)}$ can be computed easily if we first compute the partial derivatives of $C_{\mathbf{x},\mathbf{y}}$ with respect to $z_j^{(l)}$. Therefore we define

$$\varepsilon_j^{(l)} := \frac{\partial C_{\mathbf{x},\mathbf{y}}}{\partial z_j^{(l)}} \quad \text{for all } j \in \{1, \dots, m(l)\} \text{ and } l \in \{2, \dots, L\},$$

that is we regard $C_{\mathbf{x},\mathbf{y}}$ as a function of the $z_j^{(l)}$ and take the partial derivatives according to these variables. Note that $\varepsilon_j^{(l)}$ depends on both $\mathbf{x}$ and $\mathbf{y}$. We call $\varepsilon_j^{(l)}$ the **error in the j -th neuron in the l -th layer**. Since the notation would get too cumbersome if we would write this as $\varepsilon(\mathbf{x}, \mathbf{y})_j^{(l)}$, we regard the training example $\langle \mathbf{x}, \mathbf{y} \rangle$ as fixed for now. Next, the quantities $\varepsilon_j^{(l)}$ are combined into a vector:

$$\boldsymbol{\varepsilon}^{(l)} := \begin{pmatrix} \varepsilon_1^{(l)} \\ \vdots \\ \varepsilon_{m(l)}^{(l)} \end{pmatrix}.$$

The vector $\boldsymbol{\varepsilon}^{(l)}$ is called the **error in layer l** .

8.2.2 The Hadamard Product

Later, we will have need of the **Hadamard product** of two vectors. Assume that $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$. The **Hadamard product** of $\mathbf{x}$ and $\mathbf{y}$ is a **vector** that is defined by multiplying the vectors $\mathbf{x}$ and $\mathbf{y}$ elementwise:

$$\begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} \odot \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} := \begin{pmatrix} x_1 \cdot y_1 \\ x_2 \cdot y_2 \\ \vdots \\ x_n \cdot y_n \end{pmatrix},$$

i.e. the i -th component of the Hadamard product $\mathbf{x} \odot \mathbf{y}$ is the product of the i -th component of $\mathbf{x}$ with the i -th component of $\mathbf{y}$. Do not confuse the Hadamard product with the **dot product**! Although both multiply the vectors componentwise, the Hadamard product returns a vector, while the dot product returns a number. Later, we will use the **NumPy** package to represent vectors. In NumPy, the Hadamard product of two vectors $\mathbf{x}$ and $\mathbf{y}$ is conveniently computed by the expression $\mathbf{x} * \mathbf{y}$.

8.2.3 Backpropagation: The Equations

Now we are ready to state the [backpropagation equations](#). The first of these four equations reads as follows:

$$\varepsilon_j^{(L)} = (a_j^{(L)} - y_j) \cdot f'(z_j^{(L)}) \quad \text{for all } j \in \{1, \dots, m(L)\}, \quad (\text{BP1})$$

where $f'(x)$ denotes the derivative of the activation function. The equation (BP1) can also be written in vectorized form using the Hadamard product:

$$\boldsymbol{\varepsilon}^{(L)} = (\mathbf{a}^{(L)} - \mathbf{y}) \odot f'(\mathbf{z}^{(L)}) \quad (\text{BP1v})$$

Here, we have [vectorized](#) the application of the function f' to the vector $\mathbf{z}^{(L)}$, i.e. the expression $f'(\mathbf{z}^{(L)})$ is defined as follows:

$$f' \left(\begin{pmatrix} z_1^{(L)} \\ \vdots \\ z_{m(L)}^{(L)} \end{pmatrix} \right) := \begin{pmatrix} f'(z_1^{(L)}) \\ \vdots \\ f'(z_{m(L)}^{(L)}) \end{pmatrix}.$$

The next equation computes $\varepsilon_j^{(l)}$ for $l < L$.

$$\varepsilon_j^{(l)} = \sum_{i=1}^{m(l+1)} w_{i,j}^{(l+1)} \cdot \varepsilon_i^{(l+1)} \cdot f'(z_j^{(l)}) \quad \text{for all } j \in \{1, \dots, m(l)\} \text{ and } l \in \{2, \dots, L-1\}. \quad (\text{BP2})$$

This equation is more succinct in vectorized notation:

$$\boldsymbol{\varepsilon}^{(l)} = \left((W^{(l+1)})^\top \cdot \boldsymbol{\varepsilon}^{(l+1)} \right) \odot f'(\mathbf{z}^{(l)}) \quad \text{for all } l \in \{2, \dots, L-1\}. \quad (\text{BP2v})$$

Note that this equation computes the error in layer l for $l < L$ in terms of the error in layer $l+1$: The error $\boldsymbol{\varepsilon}^{(l+1)}$ at layer $l+1$ is [propagated backwards](#) through the neural network to produce the error $\boldsymbol{\varepsilon}^{(l)}$ at layer l . This is the reason for calling the associated algorithm [backpropagation](#).

Next, we have to compute the partial derivative of $C_{\mathbf{x}, \mathbf{y}}$ with respect to the bias of the j -th neuron in layer l , which is denoted as $b_j^{(l)}$. We have

$$\frac{\partial C_{\mathbf{x}, \mathbf{y}}}{\partial b_j^{(l)}} = \varepsilon_j^{(l)} \quad \text{for all } j \in \{1, \dots, m(l)\} \text{ and } l \in \{2, \dots, L\}. \quad (\text{BP3})$$

This equation shows the reason for defining the error terms $\varepsilon_j^{(l)}$: What we really need to compute is the partial derivative of $C_{\mathbf{x}, \mathbf{y}}$ with respect to the biases and the weights. The equation (BP3) and the equation (BP4) below show how this can be done once we have computed the error terms $\varepsilon_j^{(l)}$. In vectorized notation, the equation (BP3) takes the following form:

$$\nabla_{\mathbf{b}^{(l)}} C_{\mathbf{x}, \mathbf{y}} = \boldsymbol{\varepsilon}^{(l)} \quad \text{for all } l \in \{2, \dots, L\}. \quad (\text{BP3v})$$

Here, $\nabla_{\mathbf{b}^{(l)}} C_{\mathbf{x}, \mathbf{y}}$ denotes the gradient of $C_{\mathbf{x}, \mathbf{y}}$ with respect to the bias vector $\mathbf{b}^{(l)}$. Finally, we can compute the partial derivative of $C_{\mathbf{x}, \mathbf{y}}$ with respect to the weights $w_{j,k}^{(l)}$:

$$\frac{\partial C_{\mathbf{x}, \mathbf{y}}}{\partial w_{j,k}^{(l)}} = \varepsilon_j^{(l)} \cdot a_k^{(l-1)} \quad \text{for all } j \in \{1, \dots, m(l)\}, k \in \{1, \dots, m(l-1)\}, \text{ and } l \in \{2, \dots, L\}. \quad (\text{BP4})$$

In vectorized notation, this equation can be written as:

$$\nabla_{W^{(l)}} C_{\mathbf{x}, \mathbf{y}} = \boldsymbol{\varepsilon}^{(l)} \cdot (\mathbf{a}^{(l-1)})^\top \quad \text{for all } l \in \{2, \dots, L\}. \quad (\text{BP4v})$$

Here, the expression $\boldsymbol{\varepsilon}^{(l)} \cdot (\mathbf{a}^{(l-1)})^\top$ denotes the matrix product of the column vector $\boldsymbol{\varepsilon}^{(l)}$ that is regarded as an $m(l) \times 1$ matrix and the row vector $(\mathbf{a}^{(l-1)})^\top$ that is regarded as an $1 \times m(l-1)$ matrix.

As the backpropagation equations are at the very core of the theory of fully connected feed-forward neural networks, we highlight the vectorized form of these equations:

$$\boldsymbol{\varepsilon}^{(L)} = (\mathbf{a}^{(L)} - \mathbf{y}) \odot f'(\mathbf{z}^{(L)}) \quad (\text{BP1v})$$

$$\boldsymbol{\varepsilon}^{(l)} = \left((W^{(l+1)})^\top \cdot \boldsymbol{\varepsilon}^{(l+1)} \right) \odot f'(\mathbf{z}^{(l)}) \quad \text{for all } l \in \{2, \dots, L-1\} \quad (\text{BP2v})$$

$$\nabla_{\mathbf{b}^{(l)}} C_{\mathbf{x}, \mathbf{y}} = \boldsymbol{\varepsilon}^{(l)} \quad \text{for all } l \in \{2, \dots, L\} \quad (\text{BP3v})$$

$$\nabla_{W^{(l)}} C_{\mathbf{x}, \mathbf{y}} = \boldsymbol{\varepsilon}^{(l)} \cdot (\mathbf{a}^{(l-1)})^\top \quad \text{for all } l \in \{2, \dots, L\} \quad (\text{BP4v})$$

The equations (BP3) and (BP4) show why it was useful to introduce the vectors $\boldsymbol{\varepsilon}^{(l)}$: These vectors enable us to compute the partial derivatives of the cost function with respect to both the biases and the weights. The equations (BP1) and (BP2) show how the vectors $\boldsymbol{\varepsilon}^{(l)}$ can be computed. An implementation of backpropagation should use the vectorized versions of these equations since this is more efficient for two reasons:

1. Interpreted languages like *Python* take much more time to execute a loop than to execute a simple matrix-vector multiplication. The reason is that in a loop, in addition to executing the statement a given number of times, the statement has to be interpreted every time it is executed.
2. Languages that are optimized for machine learning often take care to delegate the execution of matrix operations to highly optimized functions that have been written in more efficient low level languages like *C* or assembler. Often, these functions are able to utilize all cores of the processor simultaneously. Furthermore, sometimes these functions can even use the graphical coprocessor which, because of parallelization, can do a matrix multiplication much faster than the floating point unit of a conventional processor.

8.2.4 A Proof of the Backpropagation Equations

Next, we are going to prove the backpropagation equations. Although the proof is a bit tedious, it should be accessible: We only need the [chain rule](#) from [multivariable calculus](#).

Let us start with the proof of equations BP1. Remember that we have defined the numbers $\varepsilon_j^{(l)}$ as

$$\varepsilon_j^{(l)} = \frac{\partial C_{\mathbf{x}, \mathbf{y}}}{\partial z_j^{(l)}},$$

while the numbers $z_j^{(l)}$ have been defined as

$$z_j^{(l)} := \left(\sum_{k=1}^{m(l-1)} w_{j,k}^{(l)} \cdot a_k^{(l-1)}(\mathbf{x}) \right) + b_j^{(l)}.$$

Since the quadratic error cost function $C_{\mathbf{x}, \mathbf{y}}$ for the training example $\langle \mathbf{x}, \mathbf{y} \rangle$ has been defined in terms of the activation $\mathbf{a}^{(L)}(\mathbf{x})$ as

$$C_{\mathbf{x}, \mathbf{y}} = \frac{1}{2} \cdot (\mathbf{a}^{(L)}(\mathbf{x}) - \mathbf{y})^2$$

and we have $\mathbf{a}^{(L)}(\mathbf{x}) = f(\mathbf{z}^{(L)})$, the **chain rule** of calculus tells us that $\varepsilon_j^{(L)}$ can be computed as shown below. Note that we suppress the fact that $\mathbf{a}^{(L)}(\mathbf{x})$ is a function of the input vector $\mathbf{x}$ in the following calculation. The reason is that we are computing the partial derivative of the cost function with respect to the weights $w_{j,k}^{(l)}$ and biases $b_j^{(l)}$ and these weights and biases do not depend on the inputs x_i . Therefore, it makes sense to suppress the input vector $\mathbf{x}$ in the notation.

$$\begin{aligned} \varepsilon_j^{(L)} &= \frac{\partial C_{\mathbf{x}, \mathbf{y}}}{\partial z_j^{(L)}} \\ &= \frac{\partial}{\partial z_j^{(L)}} \frac{1}{2} \cdot (\mathbf{a}^{(L)} - \mathbf{y})^2 \\ &= \frac{1}{2} \cdot \frac{\partial}{\partial z_j^{(L)}} \sum_{i=1}^{m(L)} (a_i^{(L)} - y_i)^2 \\ &= \frac{1}{2} \cdot \frac{\partial}{\partial z_j^{(L)}} \sum_{i=1}^{m(L)} (f(z_i^{(L)}) - y_i)^2 \\ &= \frac{1}{2} \cdot \sum_{i=1}^{m(L)} 2 \cdot (f(z_i^{(L)}) - y_i) \cdot \frac{\partial}{\partial z_j^{(L)}} f(z_i^{(L)}) \\ &= \sum_{i=1}^{m(L)} (f(z_i^{(L)}) - y_i) \cdot f'(z_i^{(L)}) \cdot \frac{\partial z_i^{(L)}}{\partial z_j^{(L)}} \\ &= \sum_{i=1}^{m(L)} (f(z_i^{(L)}) - y_i) \cdot f'(z_i^{(L)}) \cdot \delta_{i,j} \quad \delta_{i,j} \text{ denotes the Kronecker delta} \\ &= (f(z_j^{(L)}) - y_j) \cdot f'(z_j^{(L)}) \\ &= (a_j^{(L)} - y_j) \cdot f'(z_j^{(L)}) \end{aligned}$$

Thus we have proven equation **BP1**.

We proceed to prove equation **BP2**. To this end we compute $\varepsilon_j^{(l)}$ for $l < L$. This time, we need the chain rule of multivariate calculus. As a reminder, the chain rule in multivariate calculus works as follows: Assume that the functions

$$f : \mathbb{R}^k \rightarrow \mathbb{R} \quad \text{and} \quad g : \mathbb{R}^n \rightarrow \mathbb{R}^k.$$

are differentiable³. If the function $h : \mathbb{R}^n \rightarrow \mathbb{R}$ is defined as

$$h(\mathbf{x}) := f(g(\mathbf{x})) \quad \text{for all } \mathbf{x} \in \mathbb{R}^n,$$

then the partial derivative of h with respect to x_j satisfies

$$\frac{\partial h}{\partial x_j} = \sum_{i=1}^k \frac{\partial f}{\partial y_i} \cdot \frac{\partial g_i}{\partial x_j}$$

We proceed to prove the equation (**BP2**). We have

³If I had written this text in German, I would have said that f and g are “total differenzierbar”.

$$\begin{aligned}
\varepsilon_j^{(l)} &= \frac{\partial C_{\mathbf{x},\mathbf{y}}}{\partial z_j^{(l)}} \\
&= \sum_{i=1}^{m^{(l+1)}} \frac{\partial C_{\mathbf{x},\mathbf{y}}}{\partial z_i^{(l+1)}} \cdot \frac{\partial z_i^{(l+1)}}{\partial z_j^{(l)}} \quad \text{using the chain rule of multivariate calculus} \\
&= \sum_{i=1}^{m^{(l+1)}} \varepsilon_i^{(l+1)} \cdot \frac{\partial z_i^{(l+1)}}{\partial z_j^{(l)}} \quad \text{using the definition of } \varepsilon_i^{(l+1)}
\end{aligned}$$

In order to proceed, we have to remember the definition of $z_i^{(l+1)}$. We have

$$z_i^{(l+1)} = \left(\sum_{k=1}^{m^{(l)}} w_{i,k}^{(l+1)} \cdot f(z_k^{(l)}) \right) + b_i^{(l+1)}$$

Therefore, the partial derivatives $\frac{\partial z_i^{(l+1)}}{\partial z_j^{(l)}}$ can be computed as follows:

$$\begin{aligned}
\frac{\partial z_i^{(l+1)}}{\partial z_j^{(l)}} &= \sum_{k=1}^{m^{(l)}} w_{i,k}^{(l+1)} \cdot f'(z_k^{(l)}) \cdot \frac{\partial z_k^{(l)}}{\partial z_j^{(l)}} \\
&= \sum_{k=1}^{m^{(l)}} w_{i,k}^{(l+1)} \cdot f'(z_k^{(l)}) \cdot \delta_{k,j} \\
&= w_{i,j}^{(l+1)} \cdot f'(z_j^{(l)})
\end{aligned}$$

If we substitute this expression back into the result we got for $\varepsilon_j^{(l)}$ we have shown the following:

$$\begin{aligned}
\varepsilon_j^{(l)} &= \sum_{i=1}^{m^{(l+1)}} \varepsilon_i^{(l+1)} \cdot \frac{\partial z_i^{(l+1)}}{\partial z_j^{(l)}} \\
&= \sum_{i=1}^{m^{(l+1)}} \varepsilon_i^{(l+1)} \cdot w_{i,j}^{(l+1)} \cdot f'(z_j^{(l)}) \\
&= \sum_{i=1}^{m^{(l+1)}} w_{i,j}^{(l+1)} \cdot \varepsilon_i^{(l+1)} \cdot f'(z_j^{(l)})
\end{aligned}$$

Therefore, we have now proven equation (BP2).

We prove equation (BP4) next. According to the chain rule we have

$$\frac{\partial C_{\mathbf{x},\mathbf{y}}}{\partial w_{j,k}^{(l)}} = \frac{\partial C_{\mathbf{x},\mathbf{y}}}{\partial z_j^{(l)}} \cdot \frac{\partial z_j^{(l)}}{\partial w_{j,k}^{(l)}}$$

Now by definition of $\varepsilon_j^{(l)}$, the first factor on the right hand side of this equation is equal to $\varepsilon_j^{(l)}$:

$$\varepsilon_j^{(l)} = \frac{\partial C_{\mathbf{x},\mathbf{y}}}{\partial z_j^{(l)}}.$$

In order to proceed, we need to evaluate the partial derivative $\frac{\partial z_j^{(l)}}{\partial w_{j,k}^{(l)}}$. The term $z_j^{(l)}$ has been defined as follows:

$$z_j^{(l)} = \left(\sum_{i=1}^{m^{(l)}} w_{j,i}^{(l)} \cdot f(z_i^{(l-1)}) \right) + b_j^{(l)}.$$

Hence we have

$$\begin{aligned} \frac{\partial z_j^{(l)}}{\partial w_{j,k}^{(l)}} &= \sum_{i=1}^{m^{(l)}} \frac{\partial w_{j,i}^{(l)}}{\partial w_{j,k}^{(l)}} \cdot f(z_i^{(l-1)}) \\ &= \sum_{i=1}^{m^{(l)}} \delta_{i,k} \cdot f(z_i^{(l-1)}) \\ &= f(z_k^{(l-1)}) = a_k^{(l-1)}. \end{aligned}$$

Combining these equations we arrive at

$$\frac{\partial C_{\mathbf{x},\mathbf{y}}}{\partial w_{j,k}^{(l)}} = \varepsilon_j^{(l)} \cdot a_k^{(l-1)}.$$

Therefore, equation (BP4) has been verified.

Exercise 25: Prove equation (BP3). ◇

8.3 Stochastic Gradient Descent

The equations describing backpropagation describe the gradient of the cost function for a single training example $\langle \mathbf{x}, \mathbf{y} \rangle$. However, when we train a neural network, we need to take all training examples into account. If we have n training examples

$$\langle \mathbf{x}^{(1)}, \mathbf{y}^{(1)} \rangle, \dots, \langle \mathbf{x}^{(n)}, \mathbf{y}^{(n)} \rangle,$$

then the quadratic error cost function has been previously defined as the sum

$$C(W^{(2)}, \dots, W^{(L)}, \mathbf{b}^{(2)}, \dots, \mathbf{b}^{(L)}; \mathbf{x}^{(1)}, \mathbf{y}^{(1)}, \dots, \mathbf{x}^{(n)}, \mathbf{y}^{(n)}) := \frac{1}{2 \cdot n} \cdot \sum_{i=1}^n \left(\mathbf{o}(\mathbf{x}^{(i)}) - \mathbf{y}^{(i)} \right)^2.$$

In practical applications of neural networks, the number n of training examples is usually big. For example, when we later develop a neural network to classify handwritten digits, we will have 50,000 training examples. More ambitious projects that use neural networks to recognize objects in images use millions of training examples. For example [ImageNet](#), which is a set of labelled images collected for the purpose of training neural networks for image recognition, contains about 14 million images. When we compute the gradient of the quadratic error function with respect to a weight matrix $W^{(l)}$ or a bias vector $b^{(l)}$ we have to compute the sums

$$\frac{1}{2 \cdot n} \cdot \sum_{i=1}^n \frac{\partial C_{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}}}{\partial w_{j,k}^{(l)}} \quad \text{and} \quad \frac{1}{2 \cdot n} \cdot \sum_{i=1}^n \frac{\partial C_{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}}}{\partial b_j^{(l)}}$$

over all training examples in order to perform a single step of gradient descent. If n is large, this is very costly w.r.t. the necessary computational resources. In order to reduce the computational costs, we note that these sums can be regarded as computing average values. In [stochastic gradient descent](#), we approximate these sums by randomly choosing a small subset of the training examples. In order to formulate this approximation in a convenient notation, let us assume that instead of using all n training examples, we just use the first m training examples. Then we approximate the sums shown above as follows:

$$\frac{1}{2 \cdot n} \cdot \sum_{i=1}^n \frac{\partial C_{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}}}{\partial w_{j,k}^{(l)}} \approx \frac{1}{2 \cdot m} \cdot \sum_{i=1}^m \frac{\partial C_{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}}}{\partial w_{j,k}^{(l)}} \quad \text{and} \quad \frac{1}{2 \cdot n} \cdot \sum_{i=1}^n \frac{\partial C_{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}}}{\partial b_j^{(l)}} \approx \frac{1}{2 \cdot m} \cdot \sum_{i=1}^m \frac{\partial C_{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}}}{\partial b_j^{(l)}},$$

i.e. we approximate these sums by the average value of their first m training examples. Of course, in general we will not choose the first m training examples but rather we will choose m **random** training examples. The randomness of this choice is the reason this algorithm is called **stochastic** gradient descent. It turns out that if we take care that eventually all training examples are used during gradient descent, then the approximations given above can speed up the learning of neural networks substantially.

8.4 Implementation

Next, we will take a look at a neural network that is able to recognize handwritten digits. The **MNIST database of handwritten digits** contains 70 000 images of handwritten digits. These images have a size of 28×28 pixels. Figure 8.3 shows the first 18 images.

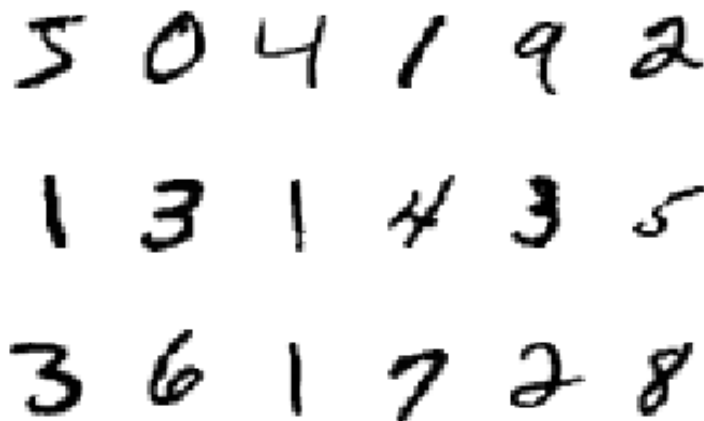


Figure 8.3: The first 18 images of the MNIST dataset.

The 70,000 images are divided into three groups:

1. The first group contains 50,000 images and is designated as the training set.
2. The second group contains 10,000 images and is designated as the validation set.
3. The last group contains 10,000 images and is designated as the test set.

We will use the first 50 000 images to train a neural network, while the last group of 10 000 images will be used to check the accuracy of the trained network. As we do not use any regularization, we will not use the validation set. As a matter of convenience, the images have been converted into one large **pickled** zip files. You can download this file at the following address:

<https://github.com/karlstroetmann/Artificial-Intelligence/raw/master/Python/mnist.pkl.gz>

Next, we describe a *Python* program that loads these images, trains a neural network on it, and finally evaluates the accuracy of the network. The program is shown in the Figures 8.4, 8.5, 8.6, and 8.7 on the following pages.

```
import gzip
import pickle
import numpy          as np
import matplotlib.pyplot as plt
import random

def vectorized_result(d):
    e = np.zeros((10, 1), dtype=np.float32)
    e[d] = 1.0
    return e

def load_data():
    with gzip.open('mnist.pkl.gz', 'rb') as f:
        train, validate, test = pickle.load(f, encoding="latin1")
        training_inputs = [np.reshape(x, (784, 1)) for x in train[0]]
        training_results = [vectorized_result(y) for y in train[1]]
        training_data = list(zip(training_inputs, training_results))
        test_inputs = [np.reshape(x, (784, 1)) for x in test[0]]
        test_data = list(zip(test_inputs, test[1]))
        return training_data, test_data

training_data, test_data = load_data()
```

Figure 8.4: Code to load the image files.

The code on Figure 8.4 on page 219 shows the code to load the image files. We discuss it line by line.

1. Since the images have been compressed as a `.gz` file, we need the module `gzip` to uncompress the file.
2. The format that has been used to store the images is called `pickle`. This is a binary format that can be used to [serialize](#) *Python* objects into binary strings. These strings can then be stored as files and later be used to restore the corresponding *Python* objects. In order to read pickled objects, we import the module `pickle`.
3. The images of the handwritten digits that we are going to import have a size of 28×28 pixels. In this program, the images are stored as `numpy` arrays of size $28 \cdot 28 = 784$.
4. In order to be able to display these images, we import `matplotlib`.
5. Every image of a handwritten character is associated with a number $d \in \{0, \dots, 9\}$. We need to transform these numbers into the expected output of our neural network. This neural network will have 10 output nodes corresponding to these digits. The k -th output node will be 1 if the neural network has recognized the digit k , while all other output nodes will be 0.

The function `vectorized_result(d)` takes a digit $d \in \{0, \dots, 9\}$ and returns a `numpy` array `e` of shape $(10, 1)$ such that $e[d][0] = 1$ and $e[j][0] = 0$ for $j \neq d$. For example, we have

```
vectorized_result(2) = array([[0.],
                             [0.],
                             [1.],
                             [0.],
                             [0.],
                             [0.],
                             [0.],
                             [0.],
                             [0.],
                             [0.]], dtype=float32)
```

For reasons of efficiency we will only use [single precision floating point numbers](#).

6. The function `load_data` reads the file `mnist.pkl.gz`, uncompresses it, and returns two lists:
 - (a) `training_data` stores the first 50 000 images. The images are stored as pairs (x, y) : x is a `numpy` array of shape $(784, 1)$ and y is a `numpy` array of shape $(10, 1)$.
 - (b) `test_data` holds 10 000 images. `test_data` is a list containing 10,000 pairs of the form $(\mathbf{x}, y)$, where $\mathbf{x}$ is a 784-dimensional `numpy` array containing the input image, and $y \in \{0, \dots, 9\}$ is the corresponding digit value.

Note that the formats for training data and test data are different. For the training data $\mathbf{y}$ is a vector, but for the test data y is a number.

```
def rndMatrix(rows, cols):
    return np.random.randn(rows, cols) / np.sqrt(cols)

def sigmoid(x):
    return 1.0 / (1.0 + np.exp(-x))

def sigmoid_prime(x):
    s = sigmoid(x)
    return s * (1 - s)
```

Figure 8.5: The constructor of the class `network`.

Figure 8.5 on page 220 shows the implementation of three utility functions.

1. The function `rndMatrix(r, c)` creates a matrix of shape (r, c) that is filled with random numbers. These numbers have a Gaussian distribution with mean 0 and variance $1/c$. This function is used to initialize the weight matrices. It is important that not all weights are initialized to the same number, for otherwise they would stay the same and then the different neurons would effectively all calculate the same feature instead of different features. It is also important that the weights are not too big for otherwise the associated neurons would [saturate](#) and training would be very slow.

2. The function `sigmoid( $x$ )` computes the sigmoid function. If x is a number, `sigmoid( $x$ )` computes

$$S(x) := \frac{1}{1 + \exp(-x)}$$

If $\mathbf{x}$ is a vector of the form $\mathbf{x} = (x_1, \dots, x_n)^\top$, we have

$$S(\mathbf{x}) = (S(x_1), \dots, S(x_n))^\top.$$

3. The function `sigmoid_prime( $x$ )` computes the derivative of the sigmoid function. The implementation is based on the equation:

$$S'(x) = S(x) \cdot (1 - S(x))$$

where x can either be a number or an array .

Figure 8.6 shows the first part of the class `Network`. This class represents a neural network with one hidden layer.

1. The member variable `mInputSize` specifies the number of input nodes. The neural network for the recognition of handwritten digits has 784 inputs. These inputs are the grey values of the 28×28 pixels that constitute the image of the handwritten digit.
2. The member variable `mHiddenSize`, specifies the number of neurons in the hidden layer. We assume that there is only one hidden layer. I have experimented with 30 neurons, 40 neurons, 60 neurons, and 100 neurons.
 - (a) For 30 neurons, the trained neural network achieved an accuracy of 94.8%.
 - (b) For 60 neurons, the network achieved an accuracy of 96.1%.
 - (c) If there are 100 neurons in the hidden layer, the network achieved an accuracy of 97.8%.
For 100 neurons, the number of weights in the hidden layer is $784 \cdot 100 = 78\,400$. Therefore, the number of weights is greater than the number of training examples. Hence, we should really use **regularization** in order to prevent over-fitting and increase the accuracy of the network.
3. The argument `mOutputSize` specifies the number of output neurons. For the neural network recognizing handwritten digits this number is 10 since there is an output neuron for every digit.
4. Besides storing the topology of the neural network, the class `Network` stores the biases and weights of all the neurons. The weights are initialized as random numbers.
 - (a) `mBiasesH` stores the bias vector of the hidden layer.
 - (b) `mBiasesO` stores the bias vector of the output layer.
 - (c) `mWeightsH` stores the weight matrix $W^{(2)}$, which specifies the weights connecting the input layer with the hidden layer.
 - (d) `mWeightsO` stores the weight matrix $W^{(3)}$, which specifies the weights connecting the hidden layer with the output layer.
5. The function `feedforward` receives an image $\mathbf{x}$ of a digit that is stored as a vector of shape $(784, 1)$ and computes the output of the neural network for this image. The code is a straightforward implementation of the equations (FF1v) and (FF2v). These equations are repeated here for convenience:

$$(a) \quad \mathbf{a}^{(1)}(\mathbf{x}) = \mathbf{x} \tag{FF1v}$$

```

class Network(object):
    def __init__(self, hiddenSize):
        self.mInputSize = 28 * 28
        self.mHiddenSize = hiddenSize
        self.mOutputSize = 10
        self.mBiasesH = np.zeros((self.mHiddenSize, 1))
        self.mBiasesO = np.zeros((self.mOutputSize, 1))
        self.mWeightsH = rndMatrix(self.mHiddenSize, self.mInputSize)
        self.mWeightsO = rndMatrix(self.mOutputSize, self.mHiddenSize)

    def feedforward(self, x):
        AH = sigmoid(self.mWeightsH @ x + self.mBiasesH)
        AO = sigmoid(self.mWeightsO @ AH + self.mBiasesO)
        return AO

    def sgd(self, training_data, epochs, mbs, eta, test_data):
        n_test = len(test_data)
        n = len(training_data)
        for j in range(epochs):
            random.shuffle(training_data)
            mini_batches = [training_data[k : k+mbs] for k in range(0, n, mbs)]
            for mini_batch in mini_batches:
                self.update_mini_batch(mini_batch, eta)
            print('Epoch %2d: %d / %d' % (j, self.evaluate(test_data), n_test))

    def update_mini_batch(self, mini_batch, eta):
        nabla_BH = np.zeros((self.mHiddenSize, 1))
        nabla_BO = np.zeros((self.mOutputSize, 1))
        nabla_WH = np.zeros((self.mHiddenSize, self.mInputSize))
        nabla_WO = np.zeros((self.mOutputSize, self.mHiddenSize))
        for x, y in mini_batch:
            dltNbl_BH, dltNbl_BO, dltNbl_WH, dltNbl_WO = self.backprop(x, y)
            nabla_BH += dltNbl_BH
            nabla_BO += dltNbl_BO
            nabla_WH += dltNbl_WH
            nabla_WO += dltNbl_WO
        alpha = eta / len(mini_batch)
        self.mBiasesH -= alpha * nabla_BH
        self.mBiasesO -= alpha * nabla_BO
        self.mWeightsH -= alpha * nabla_WH
        self.mWeightsO -= alpha * nabla_WO

```

Figure 8.6: The class Network, part I.

$$(b) \mathbf{a}^{(l)}(\mathbf{x}) = S\left(W^{(l)} \cdot \mathbf{a}^{(l-1)}(\mathbf{x}) + \mathbf{b}^{(l)}\right) \quad \text{for all } l \in \{2, \dots, L\}. \quad (\text{FF2v})$$

6. The method `sgd` implements [stochastic gradient descent](#). It receives 5 arguments.

- (a) `training_data` is a list of pairs of the form $(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})$. Here, $\mathbf{x}^{(i)}$ is a vector of dimension 784. This vector contains the pixels of an image showing one of the handwritten digits from the training set. $\mathbf{y}^{(i)}$ is the [one-hot encoding](#) of the digit that is shown in the image $\mathbf{x}^{(i)}$.
- (b) `epochs` is the number of iterations of gradient descent.
- (c) `mbs` is the size of the mini-batches that are used in stochastic gradient descent. I have achieved the fastest learning when I have used a mini-batch size of 10. Using a mini-batch size of 20 was slightly slower, but this parameter seems not to be critical.
- (d) `eta` is the learning rate.
- (e) `test_data` is the list of test data. These data are only used to check the accuracy after every epoch, they are not used to determine the weights or biases.

The implementation of stochastic gradient descent executes a `for`-loop that runs `epoch` number of times. At the beginning of each iteration, the training data are shuffled randomly. Next, the data is chopped up into chunks of size `mbs`. These chunks are called [mini-batches](#). The inner `for`-loop iterates over all mini-batches and executes one step of gradient descent that only uses the data from the given mini-batch. At the end of each iteration of the outer `for`-loop, the accuracy of the current version of the neural net is printed.

7. The method `update_mini_batch` performs one step of gradient descent for the data from one mini-batch. It receives two arguments.
 - (a) `mini_batch` is the list of training data that constitute one mini-batch.
 - (b) `eta` is the [learning rate](#).

The implementation of `update_mini_batch` works as follows:

- (a) First, we initialize the vectors `nabla_BH`, `nabla_BO` and the matrices `nabla_WH`, `nabla_WO` to contain only zeros.
 - (a) `nabla_BH` will store the gradient of the bias vector of the hidden layer.
 - (b) `nabla_BO` will store the gradient of the bias vector of the output layer.
 - (c) `nabla_WH` will store the gradient of the weight matrix of the hidden layer.
 - (d) `nabla_WO` will store the gradient of the weight matrix of the output layer.
- (b) Next, we iterate of all training examples in the mini-batch and for every training example $[\mathbf{x}, \mathbf{y}]$ we compute the contribution of this training example to the gradients of the cost function C , i.e. we compute

$$\nabla_{\mathbf{b}^{(l)}} C_{\mathbf{x}, \mathbf{y}} \quad \text{and} \quad \nabla_{W^{(l)}} C_{\mathbf{x}, \mathbf{y}}$$
 for the hidden layer and the output layer. These gradients are computed by the function `backprop`.
- (c) Finally, the bias vectors and the weight matrices are updated according to the learning rate and the computed gradients.

The method `backprop` that is shown in Figure 8.7 computes the gradients of the bias vectors and the weight matrices with respect to a single training example $\langle \mathbf{x}, \mathbf{y} \rangle$. The implementation of `backprop` proceeds as follows:

1. First, the vector `ZH` is computed according to the formula

$$\mathbf{z}^{(2)} = W^{(2)} \cdot \mathbf{x} + \mathbf{b}^{(2)}.$$

Here, $W^{(2)}$ is the weight matrix of the hidden layer that is stored in `mWeightsH`, while $\mathbf{b}^{(2)}$ is the bias vector of the hidden layer. This vector is stored in `mBiasesH`.


```

def backprop(self, x, y):
    # feedforward pass
    ZH = self.mWeightsH @ x + self.mBiasesH
    AH = sigmoid(ZH)
    ZO = self.mWeightsO @ AH + self.mBiasesO
    AO = sigmoid(ZO)
    # backwards pass, output layer
    epsilon0 = (AO - y) # * sigmoid_prime(ZO)
    nabla_B0 = epsilon0
    nabla_W0 = epsilon0 @ AH.transpose()
    # backwards pass, hidden layer
    epsilonH = (self.mWeightsO.transpose() @ epsilon0) * sigmoid_prime(ZH)
    nabla_BH = epsilonH
    nabla_WH = epsilonH @ x.transpose()
    return (nabla_BH, nabla_B0, nabla_WH, nabla_W0)

def evaluate(self, test_data):
    test_results = \
        [(np.argmax(self.feedforward(x)), y) for (x, y) in test_data]
    return sum(int(y1 == y2) for (y1, y2) in test_results)
# end of class Network

net = Network(40)
net.sgd(training_data, 60, 10, 0.1, test_data)

```

Figure 8.7: The class `Network`, part II.

2. The activation of the neurons in the hidden layer `AH` is computed by applying the sigmoid function to the vector $\mathbf{z}^{(2)}$.

3. Next, the vector `ZH` is computed according to the formula

$$\mathbf{z}^{(3)} = \mathbf{W}^{(3)} \cdot \mathbf{x} + \mathbf{b}^{(3)}.$$

Here, $\mathbf{W}^{(3)}$ is the weight matrix of the output layer that is stored in `mWeightsO`, while $\mathbf{b}^{(3)}$ is the bias vector of the output layer. This vector is stored in `mBiasesO`.

4. The activation of the neurons in the output layer `AO` is computed by applying the sigmoid function to the vector $\mathbf{z}^{(3)}$.

These four steps constitute the forward pass of backpropagation.

5. Next, the error in the output layer `epsilon0` is computed using the backpropagation equation (BP1v)

$$\epsilon^{(3)} = (\mathbf{a}^{(3)} - \mathbf{y}) \odot S'(\mathbf{z}^{(3)}).$$

6. According to equation (BP3v), the gradient of the cost function with respect to the bias vector of the output layer is given as

$$\nabla_{\mathbf{b}^{(3)}} C_{\mathbf{x}, \mathbf{y}} = \epsilon^{(3)}.$$

This gradient is stored in the variable `nabla_B0`.

7. According to equation (BP4v), the gradient of the cost function with respect to the weight matrix of the output layer is given as

$$\nabla_{W^{(3)}} C_{\mathbf{x}, \mathbf{y}} = \boldsymbol{\epsilon}^{(3)} \cdot (\mathbf{a}^{(2)})^\top.$$

This gradient is stored in the variable `nablaa.W0`.

8. Next, the error in the hidden layer `epsilonH` is computed using the backpropagation equation (BP2v)

$$\boldsymbol{\epsilon}^{(2)} = \left((W^{(3)})^\top \cdot \boldsymbol{\epsilon}^{(3)} \right) \odot S'(\mathbf{z}^{(2)}).$$

9. Finally, the gradients of the cost function with respect to the bias vector and the weight matrix of the hidden layer are computed. This is completely analogous to the computation of the corresponding gradients of the output layer.
10. The method `evaluate` is used to evaluate the accuracy of the neural network on the test data.
11. Finally, we see how the computation can be started by creating an object of class `Network` and then calling the method `sgd`.

The program discussed in this section is available as a jupyter notebook at the following link

[Digit-Recognition-NN.ipynb](#).

The file is located in the directory `Python/Chapter-07` in my [github repository](#) for this lecture. There is also a vectorized version of this notebook available at

[Digit-Recognition-NN-vectorized.ipynb](#).

This version is about four times faster than the version discussed in this section.

Exercise 26:

- Modify the notebook [Digit-Recognition-NN.ipynb](#) so that it uses the `ReLU` function instead of the sigmoid function as the activation function for the hidden layer.
- Develop a notebook that constructs 5 neural networks trained for digit recognition. Implement a final classifier that utilizes a majority voting [ensemble strategy](#) based on the outputs of these networks.

Chapter 9

Automatic Differentiation

All modern libraries for neural networks, i.e. [TensorFlow](#) and [PyTorch](#) make heavy use of [automatic differentiation](#). [Automatic differentiation](#) is a technique for computing the gradient of a function that does neither rely on numeric approximation nor does it force us to compute symbolic derivatives manually. In fact, the technique is one of the major methodological breakthroughs in machine learning in particular and science and engineering in general in recent years. Although the idea was first published in 1964 by R. E. Wengert [[Wen64](#)], it has only been widely understood and accepted in recent years, cf. Baydin et. al. [[BPRS18](#)].

There are two modes of automatic differentiation: [Forward mode](#) and [reverse mode](#). Forward mode is quite inefficient when the number n of input variables is big. The reason is that forward mode needs to traverse the [computational graph](#) $n + 1$ times (once to compute the values and n times to compute the partial derivatives), while reverse mode needs to traverse the computational graph just twice. Hence for big values of n only reverse mode automatic differentiation is a viable option. We proceed to define the crucial notion of a computational graph.

Definition 46 (Computational Graph) Assume a set $\{x_1, \dots, x_k\}$ of [input variables](#) is given. A [computational graph](#) G is a list of [computational nodes](#). There are four types of computational nodes:

1. A [variable node](#) is a tuple of length 1 of the form

$$\langle x, \rangle$$

where x is a variable from the set $\{x_1, \dots, x_k\}$. This node represents the given input variable.

2. A [constant node](#) is a pair of the form

$$\langle n, r \rangle$$

where n is the [name](#) of the node and r is a floating point number. This node is interpreted as the assignment

$$n := r.$$

The name n is a string that can be understood as the name of an auxiliary variable.

3. A [unary node](#) is a tuple of the form

$$\langle n, f, a \rangle$$

where, again, n is the [name](#) of the node that is used as an auxiliary variable. f is a unary function symbol from a set of unary functions. In our examples f will be a member of the set

$$\{\text{sqrt}, \text{exp}, \text{ln}, \text{sin}, \text{cos}, \text{arctan}\}$$

and a is the name of another node occurring in the list. This node is interpreted as the assignment

$$n := f(a).$$

4. A **binary node** is a tuple of the form

$$\langle n, o, a_1, a_2 \rangle$$

where n is the **name** of the node, o is a binary operator from the set

$$\{+, -, *, /\}$$

and a_1 and a_2 are the names of computational nodes.

This node is interpreted as the assignment

$$n := a_1 \circ a_2.$$

As in the previous case, the name n is a string that can be understood as the name of an auxiliary variable.

The **output variable** of G is the name of the variable that is defined in the last node. ◇

```
CG = [ ('x1', ),
        ('x2', ),
        ('v1', '+', 'x1', 'x2'),
        ('v2', '-', 'x1', 'x2'),
        ('v3', 'sin', 'v1'),
        ('v4', 'cos', 'v2'),
        ('v5', '*', 'v3', 'v4'),
        ('v6', '*', 'v1', 'v2'),
        ('y', '+', 'v5', 'v6')
      ]
```

Figure 9.1: A Computational Graph for $\sin(x_1 + x_2) \cdot \cos(x_1 - x_2) + (x_1 + x_2) \cdot (x_1 - x_2)$.

Example: Figure 9.1 shows a computational graph for the expression

$$\sin(x_1 + x_2) \cdot \cos(x_1 - x_2) + (x_1 + x_2) \cdot (x_1 - x_2).$$

This computational graph uses the input variables x_1 and x_2 . Figure 9.2 shows a rendering of this computational graph. ◇

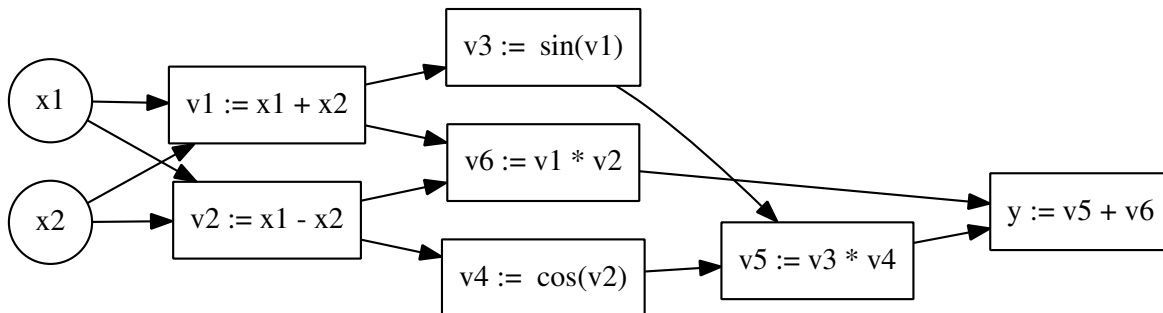


Figure 9.2: Rendering of the computation graph shown in Figure 9.1.

Definition 47 (admissible) A computational graph G is **admissible** if for every node of the form

$$\langle n, o, a_1, a_2 \rangle$$

that occurs in the list G there are nodes labelled with a_1 and a_2 that occur in the list G before this node and for every node of the form

$$\langle n, f, a \rangle$$

that occurs in the list G there is a node labelled with a that occurs in the list G before this node. If a computational graph is admissible, the nodes can be evaluated in the same order as they are listed in G . $\diamond$

```
def eval_graph(CG, Values):
    for node in CG:
        match node:
            case (v, ):
                pass
            case (v, r):
                Values[v] = r
            case (v, '+', a1, a2):
                Values[v] = Values[a1] + Values[a2]
            case (v, '-', a1, a2):
                Values[v] = Values[a1] - Values[a2]
            case (v, '*', a1, a2):
                Values[v] = Values[a1] * Values[a2]
            case (v, '/', a1, a2):
                Values[v] = Values[a1] / Values[a2]
            case (v, 'sqrt', a):
                Values[v] = math.sqrt(Values[a])
            case (v, 'exp', a):
                Values[v] = math.exp(Values[a])
            case (v, 'log', a):
                Values[v] = math.log(Values[a])
            case (v, 'sin', a):
                Values[v] = math.sin(Values[a])
            case (v, 'cos', a):
                Values[v] = math.cos(Values[a])
            case (v, 'atan', a):
                Values[v] = math.atan(Values[a])
    return Values['y']
```

Figure 9.3: A function that evaluates a computational graph.

In order to **evaluate** an admissible computational graph G that contains n input variables, we will assume that the first n nodes are labelled with the variables $x_1, \dots, x_n$ and that the output variable of G is y . Furthermore, we need a dictionary **Values** that assigns a value to each of the variables $x_1, \dots, x_n$. Then the function `eval_graph` that is shown in Figure 9.3 can be used to evaluate the nodes of the computational graph CG one by one. The idea is that initially the dictionary **Values** maps all variables to floating point values. Then the nodes of the computational graph are evaluated one by

one. For example, if a node of the form

$$\langle v, '+', a_1, a_2 \rangle$$

has to be evaluated, then we can assume that the nodes that are labelled with a_1 and a_2 have already been evaluated and that their values are stored in the dictionary `Values`. These values are added and the resulting value is stored under the key v in the dictionary `Values`.

In the following we will assume that all computational graphs are admissible. The crucial definition in the theory of reverse mode automatic differentiation is the notion of an [adjoint](#), which will be given later after we have defined the notion of a [parent](#) of a node.

```
def parents(CG):
    Parents = {}
    for node in CG:
        match node:
            case (p, _, a):
                add_to_dictionary(Parents, a, p)
            case (p, _, a1, a2):
                add_to_dictionary(Parents, a1, p)
                add_to_dictionary(Parents, a2, p)
    return Parents

def add_to_dictionary(D, key, value):
    if key in D:
        D[key] |= { value }
    else:
        D[key] = { value }

def node_dictionary(CG):
    D = {}
    for node in CG:
        name = node[0]
        D[name] = node
    return D
```

Figure 9.4: Auxiliary functions.

Definition 48 (Parent) If G is a computational graph and $\langle v, o, a_1, a_2 \rangle$ is a node in G , then v is a parent of the nodes that are labelled with a_1 and a_2 . Furthermore, if $\langle v, f, a \rangle$ is a node in G , then v is a parent of the node that is labelled with a . $\diamond$

Figure 9.4 shows the implementation of the function `parents` that can be used to compute the parents of a node. It also contains the auxiliary function `node_dictionary` that takes a computational graph `CG` as its argument and returns a dictionary associating every node with its name.

Definition 49 (Adjoint) Assume G is a computational graph such that the output variable is y . If v is any node in G , then the [adjoint](#) of v , which is written as $\bar{v}$, is defined as the partial derivative of the output variable y w.r.t. v , i.e.

$$\bar{v} := \frac{\partial y}{\partial v}. \quad \diamond$$

The next theorem is an immediate consequence of the [multivariable chain rule](#).

Theorem 50 Assume v is a node of a computational graph G and that $p_1, \dots, p_k$ are all the parents of this node in G . Then the adjoint $\bar{v}$ of the node v is given as

$$\bar{v} = \frac{\partial y}{\partial v} = \sum_{i=1}^k \frac{\partial y}{\partial p_i} \cdot \frac{\partial p_i}{\partial v} = \sum_{i=1}^k \bar{p}_i \cdot \frac{\partial p_i}{\partial v}.$$

Example: To keep things simple, assume that the variables $\mathbf{x}_1$ and $\mathbf{x}_2$ that are shown in the computational graph in Figure 9.2 are both initialized with the value $\pi/4$. Before the adjoints can be computed, we have to compute the values associated with the nodes. These are as follows:

1. $\mathbf{v}_1 = \pi/2$,
2. $\mathbf{v}_2 = 0$,
3. $\mathbf{v}_3 = 1$,
4. $\mathbf{v}_4 = 1$,
5. $\mathbf{v}_5 = 1$,
6. $\mathbf{v}_6 = 0$,
7. $\mathbf{y} = 1$.

Next, we compute the adjoints.

1. $\bar{\mathbf{y}} = \frac{\partial \mathbf{y}}{\partial \mathbf{y}} = 1$,
2. $\bar{\mathbf{v}}_6 = \frac{\partial \mathbf{y}}{\partial \mathbf{v}_6} = 1$,
3. $\bar{\mathbf{v}}_5 = \frac{\partial \mathbf{y}}{\partial \mathbf{v}_5} = 1$.
4. $\bar{\mathbf{v}}_4 = \frac{\partial \mathbf{y}}{\partial \mathbf{v}_5} \cdot \frac{\partial \mathbf{v}_5}{\partial \mathbf{v}_4} = \bar{\mathbf{v}}_5 \cdot \mathbf{v}_3 = 1 \cdot 1 = 1$.
5. $\bar{\mathbf{v}}_3 = \frac{\partial \mathbf{y}}{\partial \mathbf{v}_5} \cdot \frac{\partial \mathbf{v}_5}{\partial \mathbf{v}_3} = \bar{\mathbf{v}}_5 \cdot \mathbf{v}_4 = 1 \cdot 1 = 1$.
6. $\bar{\mathbf{v}}_2 = \frac{\partial \mathbf{y}}{\partial \mathbf{v}_6} \cdot \frac{\partial \mathbf{v}_6}{\partial \mathbf{v}_2} + \frac{\partial \mathbf{y}}{\partial \mathbf{v}_4} \cdot \frac{\partial \mathbf{v}_4}{\partial \mathbf{v}_2} = \bar{\mathbf{v}}_6 \cdot \mathbf{v}_1 - \bar{\mathbf{v}}_4 \cdot \sin(\mathbf{v}_2) = 1 \cdot \pi/2 - 1 \cdot \sin(0) = \pi/2 - 1 \cdot 0 = \pi/2$.
7. $\bar{\mathbf{v}}_1 = \frac{\partial \mathbf{y}}{\partial \mathbf{v}_6} \cdot \frac{\partial \mathbf{v}_6}{\partial \mathbf{v}_1} + \frac{\partial \mathbf{y}}{\partial \mathbf{v}_3} \cdot \frac{\partial \mathbf{v}_3}{\partial \mathbf{v}_1} = \bar{\mathbf{v}}_6 \cdot \mathbf{v}_2 + \bar{\mathbf{v}}_3 \cdot \cos(\mathbf{v}_1) = 1 \cdot 0 + 1 \cdot \cos(\pi/2) = 0 + 0 = 0$.
8. $\bar{\mathbf{x}}_1 = \frac{\partial \mathbf{y}}{\partial \mathbf{v}_1} \cdot \frac{\partial \mathbf{v}_1}{\partial \mathbf{x}_1} + \frac{\partial \mathbf{y}}{\partial \mathbf{v}_2} \cdot \frac{\partial \mathbf{v}_2}{\partial \mathbf{x}_1} = \bar{\mathbf{v}}_1 \cdot 1 + \bar{\mathbf{v}}_2 \cdot 1 = 0 \cdot 1 + \pi/2 \cdot 1 = \pi/2$.
9. $\bar{\mathbf{x}}_2 = \frac{\partial \mathbf{y}}{\partial \mathbf{v}_1} \cdot \frac{\partial \mathbf{v}_1}{\partial \mathbf{x}_2} + \frac{\partial \mathbf{y}}{\partial \mathbf{v}_2} \cdot \frac{\partial \mathbf{v}_2}{\partial \mathbf{x}_2} = \bar{\mathbf{v}}_1 \cdot 1 + \bar{\mathbf{v}}_2 \cdot (-1) = 0 \cdot 1 + \pi/2 \cdot (-1) = -\pi/2$.

Hence we have shown the following:

$$\frac{\partial y}{\partial \mathbf{x}_1} \left(\frac{\pi}{4}, \frac{\pi}{4} \right) = \frac{\pi}{2} \quad \text{and} \quad \frac{\partial y}{\partial \mathbf{x}_2} \left(\frac{\pi}{4}, \frac{\pi}{4} \right) = -\frac{\pi}{2}.$$

Note that we have found the exact partial derivatives for a specific point, namely for the arguments $\mathbf{x}_1 = \pi/4$ and $\mathbf{x}_2 = \pi/4$. Automatic differentiation is not symbolic differentiation and hence is not able to derive general formulas but rather computes values for specific arguments. However, these values are not numerical approximations but are, instead, exact. $\diamond$

Of course, we do not want to perform computations like the following ourselves. The function `partial_derivative` shown in Figure 9.5 takes a computational `Node` and computes the partial derivative of this node with respect to the given argument `arg`. The last argument `Values` is a dictionary containing the values that are associated with the different nodes.


```

def partial_derivative(Node, arg, Values):
    match Node:
        case n, '+', a1, a2:
            if arg == a1 == a2:
                return 2
            if arg == a1 or arg == a2:
                return 1
        case n, '-', a1, a2:
            if arg == a1 == a2:
                return 0
            if arg == a1:
                return 1
            if arg == a2:
                return -1
        case n, '*', a1, a2:
            if arg == a1 == a2:
                return 2 * Values[a1]
            if arg == a1:
                return Values[a2]
            if arg == a2:
                return Values[a1]
        case n, '/', a1, a2:
            if arg == a1 == a2:
                return 0
            if arg == a1:
                return 1 / Values[a2]
            if arg == a2:
                return -Values[a1] / Values[a2] ** 2
        case n, 'sqrt', a:
            return 0.5 / math.sqrt(Values[a])
        case n, 'exp', a:
            return math.exp(Values[a])
        case n, 'log', a:
            return math.log(Values[a])
        case n, 'sin', a:
            return math.cos(Values[a])
        case n, 'cos', a:
            return -math.sin(Values[a])
        case n, 'atan', a:
            return 1 / (1 + Values[a]**2)

```

Figure 9.5: Computing the partial derivative of a node.

The function `adjoints` shown in Figure 9.6 computes the adjoints of a given computational graph. It needs a dictionary `Values` that maps the variables $x_1, \dots, x_n$ to their values. It returns a dictionary that associates all node names with their adjoints.

```

def adjoints(CG, Values):
    eval_graph(CG, Values)
    NodeDict = node_dictionary(CG)
    Parents = parents(CG)
    n = len(CG)
    Adjoints = {}
    Adjoints['y'] = 1
    for k in range(2, n+1):
        Node = CG[-k]
        name = Node[0]
        result = 0
        for parent_name in Parents[name]:
            parent_node = NodeDict[parent_name]
            pd = partial_derivative(parent_node, name, Values)
            result += Adjoints[parent_name] * pd
        Adjoints[name] = result
    return Adjoints

```

Figure 9.6: Computing the adjoints of a computational graph.

9.1 The Library autograd

The library `autograd` implements the theory shown in the previous section. We will introduce this library via a couple of simple examples.

```

import autograd as ag
import autograd.numpy as np

def f(x):
    return x * np.exp(x)

fs = ag.grad(f)

print(fs(1.0))

```

Figure 9.7: A simple example demonstrating ‘autograd’.

Figure 9.7 on page 233 shows a simple example of the usage of this library.

1. In line 1 we import the library `autograd` and introduce the abbreviation `ag`.
2. Next, we have to import the autograd version of the library `numpy`. This version offers most, but not all features of `numpy`. We have to use this version because `autograd` works by creating a computational graph behind the scene. If we would use the standard version of `numpy`, which is implemented outside of *Python* in the programming language *C*, `autograd` would not be able to create this computational graph.
3. Next we define the function $f := x \mapsto x \cdot \exp(x)$. According to the *product rule*, the derivative

of f is given as

$$\frac{df}{dx} = \exp(x) + x \cdot \exp(x).$$

4. Line 7 shows how we can implement the derivative of `f` without any knowledge of mathematical analysis. This can be done by calling the function `grad` from the library `autograd` and supplying the function `f` as its argument.
5. The function `fs` that is generated by `autograd` can then be called just like any other Python function.

The previous example isn't too surprising. After all, we can do similar things using the library `SymPy`, which is a Python library for doing symbolic mathematics. The real magic of `autograd` starts to happen when we take the derivative of a *Python* function that uses control structures like `while`-loops or `if`-statements. We proceed to give an example.

```
def mySqrt(x):
    root = x
    eps = 2.0e-15
    while abs(x - root * root) > eps:
        root = 0.5 * (root + x / root)
    return root

mySqrtGrad = ag.grad(mySqrt)
```

Figure 9.8: The Babylonian method to compute the square root.

The program shown in Figure 9.8 on page 234 shows an implementation of the *Babylonian method* for computing square roots. The function `mySqrt` defines the sequence $(r_n)_{n \in \mathbb{N}}$ as

- (a) $r_0 = \frac{1}{2} \cdot x$,
- (b) $r_{n+1} = \frac{1}{2} \cdot \left(r_n + \frac{x}{r_n}\right)$ for all $n \in \mathbb{N}$.

It can be shown that this sequence converges quadratically to the square root of x , i.e. we have:

$$\lim_{n \rightarrow \infty} r_n = \sqrt{x}$$

We can compute the derivative of the function `mySqrt` by just calling `ag.grad(mySqrt)`. However, when doing this we discover a limitation of `autograd`: The derivative of the square root function is known to be

$$\frac{df}{dx} = \frac{1}{2 \cdot \sqrt{x}}.$$

When we evaluate `mySqrtGrad` this function returns the same values as the expression given above, with one exception. If $x = 1$, then `mySqrtGrad` returns 1, although the derivative is $\frac{1}{2}$. To understand what is going on let us investigate what happens when `mySqrt(1)` is computed.

1. `x` is set to 1 in line 1.
2. `root` is set to 1 in line 2.

3. Therefore, in line 4 the expression `x - root * root` yields 0 and the `while`-loop is not executed.
4. Finally, `root`, which is equal to `x` is returned.

Effectively, for the argument 1, the computational graph produced by `mySqrt` is the same as the computational graph of the identity function $\text{id}(x) = x$. Hence, the derivative computed by `mySqrtGrad` for $x = 1$ is equal to the derivative of the identity function, which is 1. There is an easy fix to solve this problem: We just have to make sure that the `while`-loop is executed at least once. Figure 9.9 on page 235 shows the resulting implementation.

```
def mySqrt(x):
    root = x
    eps = 2.0e-15
    while True:
        root = 0.5 * (root + x / root)
        if abs(x - root * root) < eps:
            return root
```

Figure 9.9: A version of `sqrt` that is correctly differentiated by `autograd`.

Bibliography

- [Alp20] Ethem Alpaydın. *Introduction to Machine Learning*. The MIT Press, Cambridge, Massachusetts, fourth edition, 2020.
- [BH69] Arthur Earl Bryson, Jr. and Yu-Chi Ho. *Applied Optimal Control: Optimization, Estimation, and Control*. Blaisdell Pub., Waltham, Massachusetts, 1969.
- [BN98] Franz Baader and Tobias Nipkow. *Term Rewriting and All That*. Cambridge University Press, 1998.
- [BPRS18] Atılım Güneş Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. Automatic differentiation in machine learning: a survey. *Journal of Machine Learning Research*, 18:1–43, 2018.
- [CV95] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995.
- [Dec03] Rina Dechter. *Constraint Processing*. Morgan Kaufmann, 2003.
- [HNR68] Peter Hart, Nils Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics SSC4*, 4(2):100–107, 1968.
- [HNR72] Peter Hart, Nils Nilsson, and Bertram Raphael. Correction to “A formal basis for the heuristic determination of minimum cost paths”. *SIGART Newsletter*, 37:28–29, 1972.
- [Jam86] Bill James. *The Bill James Baseball Abstract*. Ballantine Books, New York, 1986.
- [JWHT14] Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. *An Introduction to Statistical Learning: with Applications in R*. Springer, 2014.
- [KB70] Donald E. Knuth and Peter B. Bendix. Simple word problems in universal algebras. In J. Leech, editor, *Computational Problems in Abstract Algebra*, pages 263–297. Pergamon Press, 1970.
- [KM75] Donald E. Knuth and Ronald W. Moore. An analysis of alpha-beta pruning. *Artificial Intelligence*, 6(4):293–326, 1975.
- [Kor85] Richard Korf. Depth-first iterative-deepening: An optimal admissible tree search. *Artificial Intelligence*, 27:97–109, 1985.
- [Kow17] Alexandre Kowalczyk. *Support Vector Machines Succinctly*. Syncfusion, 2017.
- [Lew03] Michael Lewis. *Moneyball: The Art of Winning an Unfair Game*. W. W. Norton & Company, New York, 2003.

- [MC82] Tony A. Marsland and Murray Campbell. Parallel search of strongly ordered game trees. *ACM Computing Surveys*, 14(4):533–551, 1982.
- [MM82] Alberto Martelli and Ugo Montanari. An efficient unification algorithm. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 4(2):258–282, 1982.
- [Nie19] Michael A. Nielsen. *Neural Networks and Deep Learning*. Determination Press, 2019.
- [RHW86] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning internal representations by error propagation. In David E. Rumelhart and James L. McClelland, editors, *Parallel Distributed Processing: Explorations in the Microstructure of Cognition, Vol. 1*, pages 318–362. MIT Press, 1986.
- [RN20] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson Education Limited, 4rd edition, 2020.
- [Sam59] Arthur L. Samuel. Some studies in machine learning using the game of checkers. *IBM Journal*, 3(3):535–554, 1959.
- [SHM⁺16] David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of go with deep neural networks and tree search. *Nature*, 529:484, 2016.
- [Sny05] Jan A. Snyman. *Practical Mathematical Optimization: An Introduction to Basic Optimization Theory and Classical and New Gradient-Based Algorithms*. Springer Publishing, 2005.
- [Wen64] R.E. Wengert. A simple automatic derivative evaluation program. *Communications of the ACM*, 7(8):463–464, 1964.

List of Figures

1.1	Bash commands to set up an Anaconda environment for Python.	6
2.1	Start state of the missionaries-and-infidels problem.	9
2.2	The missionary and cannibals problem coded as a search problem.	10
2.3	A graphical representation of the missionaries and cannibals puzzle.	12
2.4	The 3×3 sliding puzzle.	12
2.5	The 3×3 sliding puzzle.	14
2.6	Breadth first search.	16
2.7	The function <code>path_to</code>	17
2.8	A solution of the missionaries and cannibals puzzle.	17
2.9	A queue based implementation of breadth first search.	18
2.10	The depth first search algorithm.	19
2.11	A recursive implementation of depth first search.	21
2.12	Iterative deepening implemented in <i>Python</i>	22
2.13	Example for possible paths in a graph	24
2.14	Example of space usage of conventional and bidirectional-BFS	25
2.15	Bidirectional breadth first search.	25
2.16	The function <code>bfs_one_step</code>	26
2.17	Combining two paths.	27
2.18	Explanation of the inequality $h(s_1) \leq 1 + h(s_2)$	28
2.19	The Manhattan distance between two states.	30
2.20	The best first search algorithm.	31
2.21	The A* search algorithm.	33
2.22	Bidirectional A* search.	35
2.23	A start state and a goal state for the 4×4 sliding puzzle.	36
2.24	Iterative deepening A* search.	36
2.25	A solution of the eight queens puzzle.	39
3.1	A map of Australia.	44
3.2	A solution of the eight queens puzzle.	46
3.3	A partial solution of the eight queens puzzle.	47
3.4	The n queens problem formulated as a CSP.	47
3.5	Solving a CSP via brute force search.	52
3.6	Auxiliary functions for brute force search.	53
3.7	A backtracking CSP solver.	54
3.8	A backtracking CSP solver: The function <code>backtrack_search</code>	55
3.9	The definition of the function <code>is_consistent</code>	55
3.10	The function <code>collectVars</code>	56
3.11	Constraint Propagation.	58

3.12	Implementation of <code>solve_unary</code> .	59
3.13	Implementation of <code>backtrack_search</code> .	60
3.14	Finding a most constrained variable.	60
3.15	Constraint Propagation.	61
3.16	A cryptarithmic puzzle	63
3.17	Formulating “SEND + MORE = MONEY” as a CSP.	63
3.18	Consistency maintenance in <i>Python</i> .	65
3.19	The implementation of <code>exists_value</code> .	67
3.20	A constraint solver with consistency checking as a preprocessing step.	68
3.21	A constraint solver using local search.	70
3.22	Implementation of local search.	71
3.23	The function <code>numConflicts</code> .	72
3.24	Solving a simple text problem with Z3.	74
3.25	Solving a simple text problem.	75
3.26	The moves of a knight, courtesy of chess.com .	78
3.27	The Knight’s Tour: Computing the constraints.	79
3.28	The function <code>solve</code> .	80
4.1	A <i>Python</i> implementation of tic-tac-toe.	84
4.2	Tic-Tac-Toe implemented by a bitboard.	87
4.3	The Minimax algorithm.	89
4.4	Memoization.	90
4.5	The function <code>play_game</code> .	91
4.6	Example game tree showing α - β -Pruning. (Original: Wikipedia .)	92
4.7	α - β -Pruning.	95
4.8	Implementation of the function <code>evaluate</code> .	99
4.9	Cached implementation of the functions <code>alphaBetaMax</code> and <code>alphaBetaMin</code> .	100
4.10	Progressive Deepening	100
4.11	Depth-limited α - β -pruning.	103
5.1	Confluence	114
5.2	Local Confluence	115
5.3	The Proof of Newman’s Lemma.	118
6.1	The head of the file <code>cars.csv</code> .	133
6.2	Simple Linear Regression	134
6.3	Calling the function <code>simple_linear_regression</code> .	135
6.4	General linear regression.	142
6.5	Salary table for civil servants in Germany.	144
6.6	Salaries for civil servants w.r.t. to pay grade.	144
6.7	Data entry for German Civil Servant Salaries.	145
6.8	Implementing Simple Linear Regression.	145
6.9	Implementing Polynomial Regression (Degree 2).	146
6.10	Attempting to match salaries with a linear and a quadratic model.	147
6.11	Data loading and preprocessing.	148
6.12	Sorting features by correlation importance.	149
6.13	Creating a small training set to induce overfitting.	150
6.14	Iteratively training models with increasing complexity.	150
6.15	The effect of overfitting.	151

7.1	The function $x \mapsto \sin(x) - \frac{1}{2} \cdot x^2$.	157
7.2	The gradient ascent algorithm.	160
7.3	The sigmoid function.	161
7.4	Results of an exam.	166
7.5	An implementation of logistic regression.	167
7.6	The function <code>logisticRegression</code> .	170
7.7	Probability of passing an exam versus hours of studying.	171
7.8	Results of an exam given hours of study and IQ.	172
7.9	Probability of passing an exam versus hours of studying.	173
7.10	Logistic Regression using SciKit-Learn	173
7.11	Probability of passing an exam versus hours of studying.	175
7.12	Some fake data.	177
7.13	Fake data with linear decision boundary.	178
7.14	A script for second order logistic regression.	179
7.15	Elliptical decision boundary for fake data.	180
7.16	Polynomial Logistic Regression.	180
7.17	Fake data with a decision boundary of fourth order.	181
7.18	Fake data with a decision boundary of order 14.	182
7.19	Fake data with a decision boundary of order 14, regularized.	183
7.20	A Naive Bayes Classifier for Spam Detection: Part I	186
7.21	A Naive Bayes Classifier for Spam Detection: Part II	187
7.22	A Naive Bayes Classifier for Spam Detection: Part III	188
7.23	A Naive Bayes Classifier for Spam Detection: Part IIII	189
7.24	Three points to separate.	190
7.25	Three points separated by logistic regression.	191
7.26	Points separated by logistic regression.	192
7.27	Maximizing $f(x, y) = xy$ subject to $x^2 + y^2 = 2$. At point A (3rd quadrant), the level curve cuts through the constraint; the gradients are not parallel. At point B (1st quadrant), the level curve is tangent to the constraint; the gradients are parallel.	193
7.28	Points separated by a support vector machine.	199
7.29	SVM Initialization.	202
7.30	The training method <code>fit</code> .	203
7.31	Computing the bias b .	204
7.32	The method <code>predict</code> .	205
8.1	A neural network with topology $[3, 6, 4, 2]$.	208
8.2	A neural network with topology $[8, 12, 8, 6, 3]$.	209
8.3	The first 18 images of the MNIST dataset.	218
8.4	Code to load the image files.	219
8.5	The constructor of the class <code>network</code> .	220
8.6	The class <code>Network</code> , part I.	222
8.7	The class <code>Network</code> , part II.	224
9.1	A Computational Graph for $\sin(x_1 + x_2) \cdot \cos(x_1 - x_2) + (x_1 + x_2) \cdot (x_1 - x_2)$.	227
9.2	Rendering of the computation graph shown in Figure 9.1.	227
9.3	A function that evaluates a computational graph.	228
9.4	Auxiliary functions.	229
9.5	Computing the partial derivative of a node.	232
9.6	Computing the adjoints of a computational graph.	233

9.7	A simple example demonstrating ‘autograd’.	233
9.8	The Babylonian method to compute the square root.	234
9.9	A version of <code>sqrt</code> that is correctly differentiated by <code>autograd</code> .	235

Index

- E -variety, 108
- R^2 , 132
- Σ -equation, 106
- Σ -term, 106
- $\mathcal{T}_\Sigma$, 106
- $s \doteq t$, 121
- `next_states`, 8
- 15 puzzle, 12
- 8 puzzle, 12
- 8 queens puzzle, 45

- accuracy, 154
- activation function, 207
- activation vector, 209
- adjoint, 229
- admissible heuristic, 28
- admissible, computational graph, 228
- algorithm of Martelli and Montanari, 121
- AlphaGo, 82
- arg max function, 157

- backpropagation, 211, 213
- backpropagation equations, 213, 214
- backtracking, 53
- bias, 207
- bias term, 154
- bias trick, 138
- bias vector, 208
- bidirectional search, 24
- binary classification, 163
- binary feature, 153
- bitboard, 86
- blind search, 28
- branching factor, 24
- breadth first search, 15

- centrally symmetric, 161
- chain rule, 214
- chain rule, multivariate calculus, 215
- classification problem, 153

- composition $\sigma\tau$, 120
- Composition of Substitutions, 120
- computational graph, 226
- computational node, 226
- conflict, 71
- connected variables, 64
- consistency maintenance, 64
- consistent heuristic, 28
- constant feature, 164
- constraint propagation, 57
- constraint satisfaction problem, 43
- corrected sample standard deviation, 131
- critical pair, 124

- declarative programming, 7
- deep blue, 82
- deep neural network, 207
- dependent variable, 130
- depth first search, 19
- design matrix, 138
- double-ended queue, 19
- doubly linked list, 19

- edge, 8
- eight queens puzzle, 45
- error in layer l , 212

- feature, 153, 155
- feature matrix, 138, 156
- feed forward equations, 210
- feed forward neural network, 207
- frequency, 154
- fully connected, 208
- function symbol, 105

- game, 82
- general linear regression, 137
- goal state, 8
- gradient, 139
- gradient ascent, 157
- gradient descent, 211

- Gram matrix, [203](#)
- Hadamard product, [212](#)
- heuristic, [28](#)
- hidden layer, [207](#)
- higher order features, [143](#)
- homogenization, [138](#)
- IDA* search, [35](#)
- independent variable, [130](#)
- input dimension, [208](#)
- input layer, [207](#)
- input node, [207](#)
- iterative deepening, [22](#)
- Ke Jie, [82](#)
- kernel function, [198](#)
- kernel trick, [198](#)
- Lagrange multipliers, [193](#)
- likelihood, [164](#)
- linear model, [130](#)
- linear regression, [130](#)
- local search, [69](#)
- log-likelihood, [164](#)
- logistic function, [161](#)
- logistic regression, [160](#)
- logit function, [163](#)
- machine learning, definition, [4](#)
- map coloring, [43](#)
- maximum likelihood estimation, [164](#)
- mean squared error, [131](#), [137](#)
- memoization, [89](#), [90](#)
- minimax algorithm, [87](#)
- missionaries and cannibals, [8](#)
- MNIST database of handwritten digits, [218](#)
- most constrained variable heuristic, [58](#)
- MSE, [137](#)
- node, [8](#)
- normal equation, [140](#), [141](#)
- number of layers, [207](#)
- one hot encoding, [148](#)
- optimal solution, [17](#)
- optimistic heuristic, [28](#)
- output dimension, [208](#)
- output layer, [207](#)
- partial variable assignment, [43](#)
- path, [8](#)
- Pearson correlation coefficient, [131](#)
- Pearson's r , [131](#)
- pickle, [218](#)
- Pluribus, [82](#)
- proportion of explained variance, [132](#)
- proportion of unexplained variance, [132](#)
- quadratic error cost function, [210](#)
- rectified linear unit, [207](#)
- ReLU function, [207](#)
- residual sum of squares, [132](#)
- RSS, [132](#)
- sample correlation coefficient, [131](#)
- sample covariance, [131](#)
- sample mean value, [131](#)
- sample standard deviation, [131](#)
- sample variance, [132](#)
- search heuristic, [28](#)
- search problem, [7](#)
- search problem, solution, [8](#)
- sigmoid function, [161](#)
- signature, [105](#)
- simple linear regression, [130](#)
- slack variables, [200](#)
- sliding puzzle, [7](#), [11](#)
- soft margin SVM, [200](#)
- solution of a CSP, [43](#)
- solution, minimal, [8](#)
- solution, of a search problem, [8](#)
- start state, [8](#)
- state, [7](#)
- Stirling's approximation of the factorial, [24](#)
- stochastic gradient descent, [217](#)
- sudoku, [80](#)
- supervised learning, [155](#)
- support vector, [197](#)
- symbolic AI, [4](#)
- syntactical equation, [121](#)
- system of syntactical equations, [121](#)
- term, [106](#)
- terminal state, [83](#)
- tic-tac-toe, [84](#)
- topology, [207](#)
- total sum of squares, [132](#)
- training set, [153](#)
- transfinite induction, [116](#)

transposition operator, [130](#)
TSS, [132](#)
two person games, [83](#)

unifier, [121](#)
uninformed search, [28](#)
utility of a state, [87](#)

value of a state, [87](#)
variable, [105](#)
variable assignment, [43](#), [107](#)

weight matrix, [208](#)
weight of NN, [208](#)

zebra puzzle, [56](#)
zero sum game, [83](#)